



GamepadOS

Your phone is the controller

Wi-Fi or USB

Gyro steering

2.5 ms wired

Free

THE COMPLETE GUIDE

From first principles to expert — how every part of your phone-as-gamepad system actually works, with real code.

Android + C++

Python + ViGEm

GRX encryption

Website + Updater

iOS port

Shipped set: Android 1.3.0 (versionCode 22) · PC server 1.1.16

A ground-up field guide to the GamepadOS codebase · 14 chapters, 5 parts.

Contents

A Language Primer	6
Why One Project Uses Six Languages	7
JavaScript and TypeScript — the Controller's Brain	8
JavaScript Again — but on the Server (Node.js)	9
Kotlin — the Android Shell	9
C++ — the Speed Layer	9
Python — the PC Server	10
Swift — the iPhone Port	11
The Web Skin: HTML and CSS	11
The Configuration Languages	12
Shell: Bash and PowerShell — Running the Build	12
Quick Reference — Every Language at a Glance	13
 Part I · The Big Picture & Foundations	 15
The Big Picture — What GamepadOS Actually Is	17
What GamepadOS Actually Does	18
The Cast of Components	19
The Repo Layout — What Lives Where	21
The Wire Contract — 20 Bytes That Both Sides Must Agree On	22
The Journey of a Single Button Press	23
Current Shipped Versions and Where Truth Lives	26
Foundations — Networks, Bytes, and Virtual Gamepads	28
What a Network Is — Addresses, Ports, and Your LAN	29
TCP vs UDP — Why the Input Path Uses UDP	30
Bits, Bytes, Endianness, and the 20-Byte Packet	32
Client and Server — Who Talks to Whom	35
What a Gamepad Is to Windows — XInput, Drivers, and ViGEm	37
The App's Skin and Muscle — WebView, Native C++, JNI, and the APK	39
 Part II · Inside the Phone	 43
The Controller Screen — Inside App.tsx	45
From Web Page to Game Controller	46

React in Ten Minutes: State, Refs, and Effects	48
The View Machine: Switching Screens	50
Widgets on an SVG Canvas	51
The Gyro Toggle and the Top-Edge Indicator	55
Speaking to Kotlin: The Bridge and the 20-Byte Packet	58
Building and Shipping the UI into the APK	60
The Android Shell — Kotlin, WebView, and the Bridge	63
The Big Picture: A Native Shell Around a Web Brain	64
Waking Up: onCreate, Fullscreen, and a Screen That Never Sleeps	65
The Layer Cake: SurfaceView, Camera, and the WebView	67
The AndroidBridge: How JavaScript Calls Kotlin	68
Into C++: JNI and the Native Engine	70
Feeling and Sensing: Gyro Steering and Haptics	72
Permissions, Packaging, and the Build	74
The C++ Engine — gamepad-engine.cpp	78
Why a C++ Engine Sits at the Bottom of the Stack	79
The JNI Bridge — How Kotlin Calls C++	80
The 20-Byte Packet — Every Byte Accounted For	82
The Send Loop — Awake Only When It Must Be	84
Sockets, Discovery, Reconnect, and Wired USB	88
ACKs, Latency, Rumble, and the GRX Encryption Hook	91
Building One Engine for Every Phone	93
Part III · The Protocol, the Server & Encryption	96
Discovery, Pairing, and the Wire Protocol	98
Two Machines Finding Each Other	99
One Socket, Many Kinds of Message	102
The 20-Byte Input Packet, Byte for Byte	105
The Handshake in Motion	107
Staying Alive, and Saying Goodbye Without a Word	110
When Packets Vanish	112
Wrapping the Whole Thing: GRX and the Full Catalogue	112
The PC Server — server.py from Socket to Virtual Pad	115
What server.py Is, and How It Wakes Up	116
The Wire Format: Parsing 20 Bytes with struct	118

Opening the Sockets: Ports, UDP, and the Firewall	120
The Virtual Pad: vgamepad, Pads, and apply_inputs	122
Threads and the Three Transports	124
The Window, the Updater, and Feedback	127
Packaging: PyInstaller, the Installer, and driver/	129
GRX — The Encrypted Input Layer	132
Why you encrypt controller input at all	133
Symmetric, asymmetric, and the X25519 key exchange	134
AES-GCM, nonces, and why reuse is fatal	135
Turning a shared secret into keys: HKDF and the PSK	137
The GRX handshake, message by message	138
The sealed packet and the replay window	141
Three implementations, one wire — and where GRX actually stands	144
Part IV · Performance & Alternate Transports	146
Latency Engineering — Chasing Milliseconds	148
The Finger-to-Game Journey	149
Where the Milliseconds Hide	151
The Native Send Loop: From Fixed Poll to Event-Driven	152
Socket-Level Tricks: TOS, connect(), Non-Blocking Sends	154
The PC Side: Draining to Newest, Priority, Timer Resolution	156
Measuring Latency Honestly	158
Expert Trade-offs — Chasing the Last Millisecond	160
USB Mode — The AOA Transport	162
What AOA Actually Is	163
The Handshake: Talking a Phone Into Accessory Mode	164
Re-Enumeration and Opening the Pipes	166
The Same 20 Bytes, a Different Pipe	167
The Android Side: Answering the Attach	168
Wiring It Into the Server, Setup, and Real Status	170
Part V · Shipping, the Platform, Ports & Lessons	173
Releases, Installers, and the Updater	175
Why Versioning Is the Hardest Easy Problem	176
The Two Products, and Their Version Fields	176

The PC Server's Version — and the Rule Written Beside It	178
How the Server Checks for an Update	179
Downloading Safely — Never Execute Garbage	180
The Website Side — Where "Latest" Is Decided	181
Publishing a Release — the Admin Ritual	182
One Manual Publish Script, and the Archive	183
The Whole Loop, End to End	184
The Website — Downloads, Feedback, Tickets, Admin	186
What the Website Is For	187
How an Express Server Works	188
CORS — Who Is Allowed to Call the API	189
The Database, Through Prisma	189
Authentication — How Admin Login Works	191
The Ticketing System — from Form to Inbox	193
The Inbound-Email Webhook — Replies Become Messages	194
The Download Counter	196
The Admin Portal and Deployment	196
The iOS and Mac Port	198
The Goal — the Same Controller, on an iPhone	199
The Port Strategy — Keep the Web UI, Rebuild the Native Shell	199
The Host View Controller	200
The Bridge Shim — Faking a Synchronous API	201
GRX in CryptoKit — the Same Bytes, a Different Library	203
The Build System and Deliberate Gaps	203
What's Done, and What Still Needs a Mac	205
The Mac Server — the Genuinely Hard Part	205
War Stories — Incidents, Dead Ends, and Lessons	207
Why a Chapter of Disasters	208
War Story 1 — The Antigravity Incident	208
War Story 2 — The Rust Server That Dropped Every Packet	211
The Lessons, Distilled	212
Reading the Logs — a Practical Skill	213

FIELD PRIMER

A Language Primer

the tongues this system speaks



Why One Project Uses Six Languages

Open the GamepadOS folders and you will find files ending in `.tsx`, `.kt`, `.cpp`, `.py`, `.swift`, `.js`, plus a scattering of `.yaml`, `.prisma`, `.iss`, and `.kts`. That can look intimidating, as if you need to "know six languages." You do not. Each language is here for one reason, doing one job it happens to be best at, and this primer exists so that the moment you meet any of them in a later chapter, you already know what it is and why it showed up.

IN PLAIN TERMS

A codebase is like a workshop. You do not build a whole cabinet with a single tool — you reach for a saw to cut, a drill to make holes, a brush to finish. Programming languages are tools like that.

GamepadOS reaches for a fast, low-level tool (C++) on the one job that must be blisteringly quick, a friendly UI tool (TypeScript/React) for the buttons you touch, an easy glue tool (Python) for the PC side, and so on. The variety is not complexity for its own sake — it is the right tool for each job.

There is one distinction worth learning before anything else, because it explains *how* each language actually runs.

DEFINITION

Compiled language — one that is translated ahead of time into raw machine instructions (or an intermediate form) by a program called a *compiler*, producing a standalone artifact you then run. C++, Kotlin, and Swift are compiled. Compiling catches many mistakes before the program ever runs, and the result is fast.

DEFINITION

Interpreted language — one that is read and executed line by line, at run time, by another program called an *interpreter* — there is no separate "build the machine code" step you wait on. JavaScript and Python are interpreted. This makes them quick to write and change, at some cost in raw speed.

DEFINITION

Declarative format — not a language that "does" things at all, but one that *describes* things — a list of facts a tool then acts on. YAML, JSON, the Prisma schema, and the Inno Setup script are declarative: they say *what* you want (these database tables, this installer, this build config), and some other program makes it so.

NOTE

Keep that three-way split in mind — **compiled** (C++, Kotlin, Swift), **interpreted** (JavaScript/TypeScript, Python), **declarative** (YAML, JSON, Prisma, Gradle, Inno Setup). Every file in the project is one of these three, and knowing which tells you how it becomes real.

JavaScript and TypeScript — the Controller's Brain

The on-screen controller — every button, stick, and menu you touch — is written in **TypeScript** using **React**, and it lives in `apps/controller-ui` (the heart of it is `App.tsx`). To understand that sentence you need three nested ideas.

DEFINITION

JavaScript (JS) — the programming language that runs inside every web browser. It is *interpreted* and was built to make web pages interactive — respond to taps, change what's on screen, talk to servers. Your controller UI is, underneath everything, a web page, so JavaScript is what makes it move.

DEFINITION

TypeScript (TS) — JavaScript with *type labels* added. A "type" is a note that says "this value is a number" or "this is text." TypeScript checks those labels before the app ships and catches whole classes of mistakes (passing text where a number was expected) that plain JavaScript would only discover by crashing later. The labels are stripped out before the app runs — the phone runs plain JavaScript.

DEFINITION

React / JSX / TSX — React is a toolkit for building user interfaces out of reusable pieces called *components*. JSX is the special syntax that lets you write HTML-like tags directly inside JavaScript; when the file also uses TypeScript, the extension is `.tsx`. So `App.tsx` means "a React component file, written in TypeScript."

Here is what all three look like together — a tiny button component:

teaching example

TypeScript · React

```
1 type ButtonProps = { label: string; pressed: boolean };
2
3 function PadButton({ label, pressed }: ButtonProps) {
4   // JSX: HTML-like tags returned straight from a function
5   return <div className={pressed ? "btn held" : "btn"}>{label}</div>;
6 }
```

The `type ButtonProps = { label: string; pressed: boolean }` line is the TypeScript part — it declares that this component expects a piece of text called `label` and a true/false value called `pressed`. The function is a React *component*: give it those inputs and it returns some JSX (`<div>...</div>`) describing what to draw. The `pressed ? "btn held" : "btn"` swaps the styling depending on whether the button is held. When you read Chapter 3, every widget is a variation on this shape.

Why here. The UI needs to be drawn once and run on Android, and later reused *unchanged* on iPhone. A web UI (HTML/CSS/JS) is the one thing that runs identically inside both an Android WebView and an iPhone WKWebView — which is exactly why the iOS port could keep this code byte-for-byte. TypeScript adds the safety net; React keeps thousands of lines of button logic organized.

JavaScript Again — but on the Server (Node.js)

The support website's backend is *also* JavaScript, but running outside any browser, on a server, through a program called **Node.js**. Same language, completely different setting.

DEFINITION

Node.js — a program that runs JavaScript on a server (or your PC) instead of in a browser. It lets you use one language — JavaScript — for both the web page and the machine that answers its requests. The website's `server.js` is a Node.js program.

Why here. Using JavaScript on the server too means the whole website (front and back) is one language, which is simpler to maintain for one person. You will see it in Chapter 12, written with a helper toolkit called Express.

Kotlin — the Android Shell

The Android app that hosts your web UI is written in **Kotlin** (`MainActivity.kt`, in `apps/android-client`).

DEFINITION

Kotlin — the modern, official language for building Android apps. It is *compiled* (into a form the phone's runtime executes) and is known for being safe and concise. It is the language Google recommends for Android, which is why the native shell — the part that opens the WebView, reads the gyroscope, and buzzes the vibration motor — is written in it.

teaching example

Kotlin

```
1 // A Kotlin function: 'fun', a typed parameter, string templating with $
2 fun greet(name: String) {
3     val message = "Hello, $name"    // 'val' = a value that won't change
4     println(message)
5 }
```

`fun` starts a function; `name: String` says the parameter is text (Kotlin, like TypeScript, is typed); `val` declares a value that cannot be reassigned; and `"Hello, $name"` shows Kotlin's string templating, where `$name` is replaced by the variable's contents. Chapter 4 walks the real `MainActivity.kt`, which is denser but built from these same pieces.

Why here. To put a web page full-screen on Android and connect it to the phone's camera, sensors, and vibration, you need a real Android app — and Kotlin is the native language for that. It is the thin but essential "shell" around the web UI.

C++ — the Speed Layer

Buried inside the Android app is a small block of C++ (`gamepad-engine.cpp`). It exists purely for speed, on the one path where speed is everything: packing each input into bytes and firing it across the network.

DEFINITION

C++ — a low-level, *compiled* language that runs about as close to the bare metal as software gets — extremely fast, with precise control over memory. That control is also why it is harder and less forgiving than Kotlin or Python. GamepadOS uses it for exactly one thing: the network hot-path that must add as little delay and jitter as possible.

DEFINITION

NDK / JNI — the NDK (Native Development Kit) is the toolkit that lets an Android app include C++ alongside its Kotlin. JNI (Java Native Interface) is the bridge that lets Kotlin *call* those C++ functions. You will see JNI function names like `Java_com_gamepad_client_MainActivity_initNetworkNative` — that long name is how Kotlin and C++ find each other.

teaching example

C++

```
1 // C++: a fixed-layout struct, exactly the kind used for the 20-byte packet
2 #pragma pack(push, 1)
3 struct Sample { uint64_t timestamp; uint16_t buttons; };
4 #pragma pack(pop)
```

`struct` groups fields into one record; `uint64_t` and `uint16_t` are exact-size integers (8 bytes and 2 bytes) — that precision is why C++ is used to lay out the wire packet down to the byte. The `#pragma pack(1)` tells the compiler not to add hidden padding, so the struct is exactly the size you counted. Chapter 5 is entirely this file.

Why here. Kotlin and JavaScript occasionally pause to clean up memory ("garbage collection"), and those pauses cause tiny, unpredictable delays — poison for a real-time controller. C++ has no such pauses, so the send loop runs on a high-priority C++ thread and the input reaches your PC with the least possible lag.

Python — the PC Server

The program on your Windows PC that receives the packets and impersonates an Xbox controller is written in **Python** (`server.py`, in `apps/pc-server`).

DEFINITION

Python — a highly readable, *interpreted*, general-purpose language, famous for being easy to write. GamepadOS uses it for the PC server because it makes the networking, the virtual-gamepad calls, and the desktop window quick to build — and because it can be packaged into a single `.exe` so users never need Python installed.

teaching example

Python

```
1 # Python: no type labels required, indentation defines blocks
2 def to_axis(byte_value):
3     # map a 0..255 byte to a -1.0..+1.0 stick position
4     return max(-1.0, min(1.0, (byte_value - 128) / 127.0))
```

Notice there are no curly braces — Python uses *indentation* to show what's inside the function. There are also no type labels (they're optional in Python). This function converts one packet byte into a joystick position, the kind of small, clear routine Python excels at. Chapter 7 walks the real `server.py`.

Why here. The PC side is not latency-critical in the same way the phone's send loop is (it mostly receives and forwards), so the priority is development speed and rich libraries — exactly Python's strengths. A library called `vgamepad` gives it one-line access to the virtual-controller driver.

Swift — the iPhone Port

The in-progress iPhone version is written in **Swift** (`apps/ios-client/Sources/*.swift`).

DEFINITION

Swift — Apple's modern, *compiled* language for iPhone, iPad, and Mac apps. In the iOS port it replaces *both* the Kotlin shell and the C++ engine from Android — Swift is fast enough to do the networking itself — while the web UI is reused unchanged. Apple's built-in `CryptoKit` gives Swift the exact encryption pieces the GRX layer needs.

teaching example

Swift

```
1 // Swift: 'let' is a constant, types can be inferred or written explicitly
2 let port: UInt16 = 7777
3 func neutralStick() -> UInt8 { return 128 } // center value
```

`let` declares a constant (Swift's version of Kotlin's `val`); `UInt16` and `UInt8` are exact-size integers like C++ uses, which matters because Swift must produce the *same bytes* on the wire as the Android app. Chapter 13 covers the port and why the protocol constants are untouchable.

Why here. iPhones do not run Kotlin or Android C++, so the native shell had to be rebuilt — and Swift is the native language for Apple devices. The clever part is how little had to be rewritten: only the shell, not the UI.

The Web Skin: HTML and CSS

Wherever there is a web page — the controller UI and the website's pages — two non-programming languages describe how it looks.

DEFINITION

HTML — the language that describes the *structure* of a web page — "here is a heading, here is a button, here is an image." It is declarative: it lays out content, it does not compute.

DEFINITION

CSS — the language that describes the *style* of a web page — colors, sizes, spacing, animations. Also declarative. When Chapter 3 talks about a button's color changing when pressed, CSS is doing that styling; JavaScript just flips which CSS class applies.

The Configuration Languages

A lot of files in the project are pure description — they contain no logic, they just tell some tool how to do its job. You will meet several, and they are all easy once you know what each configures.

DEFINITION

JSON — a plain-text format for structured data — lists and labelled values, written with `{ }` and `[ ]`. It is everywhere: `package.json` lists a project's dependencies, the update *manifest* the PC server fetches is JSON, and the phone-PC pairing data is JSON-like. If you can read a shopping list with sub-lists, you can read JSON.

DEFINITION

YAML — a data format like JSON but written with indentation instead of braces, designed to be easy for humans to read and edit. The iOS project's build description, `project.yml`, is YAML — it lists the app's settings, permissions, and version in a clean, indented form.

DEFINITION

Prisma schema (`.prisma`) — a declarative description of the website's database — each `model` block defines one table and its columns. From this one file, a tool generates all the code that reads and writes the database. Chapter 12 reads it directly.

DEFINITION

Gradle Kotlin DSL (`.kts`) — the Android build configuration, written in a Kotlin-flavored form. `build.gradle.kts` is where the app's `versionCode`, `versionName`, signing key, and CPU targets are set — the file the release process edits every time. It is Kotlin syntax used declaratively to configure a build.

DEFINITION

Inno Setup script (`.iss`) — the recipe for the Windows installer. It says what to install, where, and what commands to run (install the driver, add firewall rules, launch the app). The `.iss` is read by the Inno Setup tool, which produces `GamepadServer-Setup.exe`. Chapter 11 shows it.

DEFINITION

CMake — the build description for the C++ engine (`CMakeLists.txt`). It tells the compiler how to turn `gamepad-engine.cpp` into a library for each phone CPU type. You mostly just need to know it is "the thing that builds the C++."

Shell: Bash and PowerShell — Running the Build

Finally, the commands that *build and ship* everything are written for a command line — a text prompt where you type instructions to your computer.

DEFINITION

Shell / command line — a text interface where you type commands (like `npm run build`) and the computer runs them. **PowerShell** is the shell on Windows; **Bash** is the shell on Mac and Linux. The build and release steps in `RELEASE.md` are shell commands — how you turn source code into an actual APK or `.exe`.

teaching example

Bash

```
1 # A shell command: run the UI build, then copy the result into the app
2 npm run build
3 robocopy dist ..\android-client\app\src\main\assets\dist /MIR
```

These are not part of the app — they are instructions *you* run to assemble it. `npm run build` compiles the React UI; `robocopy` copies the result into the Android project. Chapter 11 uses commands like these to produce a release.

Quick Reference — Every Language at a Glance

Keep this table handy; every code block in the book shows its language in the caption's top-right corner, and this says what each one is and where it lives.

Language / format	What it is	Runs how	Where in GamepadOS
TypeScript / React (<code>.tsx</code>)	typed JavaScript + UI toolkit	interpreted (in the WebView)	the controller UI — <code>App.tsx</code>
JavaScript (<code>.js</code>)	web + server scripting	interpreted (Node.js)	the website backend — <code>server.js</code>
Kotlin (<code>.kt</code>)	Android's native language	compiled	the Android shell — <code>MainActivity.kt</code>
C++ (<code>.cpp</code>)	low-level speed	compiled (via NDK)	the engine — <code>gamepad-engine.cpp</code>
Python (<code>.py</code>)	readable general-purpose	interpreted (or frozen <code>.exe</code>)	the PC server — <code>server.py</code>
Swift (<code>.swift</code>)	Apple's native language	compiled	the iOS port
HTML / CSS	page structure + style	declarative	every web page
JSON	structured data	declarative	manifests, <code>package.json</code>
YAML (<code>.yaml</code>)	human-friendly data	declarative	the iOS build — <code>project.yaml</code>
Prisma (<code>.prisma</code>)	database description	declarative	the website's data model
Gradle KTS (<code>.kts</code>)	Android build config	declarative (Kotlin)	<code>build.gradle.kts</code>
Inno Setup (<code>.iss</code>)	Windows installer recipe	declarative	the PC installer

Bash / PowerShell

build & release
commands

run at the command line

`RELEASE.md` steps

NOTE

You will never need all of these in your head at once. When a chapter shows a `.py` file, it is the PC server; a `.tsx` file is the controller UI; a `.kt` file is the Android shell; a `.cpp` file is the engine. That mapping — file extension to "which part of the system" — is most of what you need, and this primer is here to flip back to whenever a new one appears.

PART I

The Big Picture & Foundations

CHAPTER 1

The Big Picture — What GamepadOS Actually Is





Figure 1.1 — The end-to-end journey of one button press, phone to game.

What GamepadOS Actually Does

Before any code, hold the whole thing in your head as a single sentence: **GamepadOS turns your Android phone into a wireless Xbox controller for your Windows PC, with no extra hardware.** You open the app on your phone, point its camera at a QR code on your PC screen, and from that moment every tap, every stick drag, every tilt of the phone shows up inside your PC games exactly as if a real Xbox 360 controller were plugged into a USB port.

DEFINITION

Game controller (gamepad) — the handheld input device with two thumbsticks, a D-pad, and face buttons that consoles use. A PC game expects to read one of these; your phone is pretending to be one.

IN PLAIN TERMS

Imagine a TV remote that, instead of talking to one TV, could impersonate *any* remote. You teach it to speak "TV language" and the TV never realizes it isn't the original remote. Your phone does exactly this: the PC game asks Windows "is a controller plugged in?", Windows says "yes, an Xbox 360 pad", and that pad is really your phone broadcasting over Wi-Fi.

The magic word is **impersonation**. The game never knows a phone is involved. That is deliberate — it means GamepadOS works with *every* PC game that supports a normal controller, without the game needing to be modified, patched, or even aware GamepadOS exists. You did not write a plugin for any specific game. You wrote a system that manufactures a fake-but-real controller at the operating-system level.

DEFINITION

Wi-Fi / LAN — your home wireless network. Both the phone and the PC are joined to the same network, so they can send each other small chunks of data directly. LAN means Local Area Network — the little network inside your home, as opposed to the wider internet.

There is also a wired mode (USB tethering) for people who want the absolute lowest delay, and — this is the whole point of the project — the phone-to-PC stream is **encrypted** so nobody else on your Wi-Fi can hijack your inputs. That encryption layer is called GRX, and you will meet it shortly.

The Cast of Components

GamepadOS is not one program. It is a small constellation of programs that each own one job. Here is the entire cast, top to bottom.

1. The controller UI (the screen you touch). A web app written in TypeScript/React that draws the on-screen buttons, sticks, and triggers. It lives at [apps/controller-ui](#). This is the part users actually see and tap.

DEFINITION

React / TypeScript — React is a toolkit for building user interfaces out of reusable pieces. TypeScript is JavaScript with type labels that catch mistakes before the app runs. Together they draw the controller buttons and react to your touches.

2. The Android shell (the app wrapper). A thin Android app written in Kotlin, at [apps/android-client](#). Its whole job is to host the web UI in a full-screen **WebView** and wire it to the phone's hardware — the camera (for QR scanning), the gyroscope (for tilt-to-steer), the vibration motor, and the network.

DEFINITION

WebView — a web browser with no address bar, embedded inside a native app. It lets an Android app display a web page as if it were the app's own screen. Your controller UI is that web page.

DEFINITION

Kotlin — the modern programming language for Android apps. The shell that boots the WebView and talks to the phone's sensors is written in Kotlin.

3. The C++ engine (the fast part). Inside the Android app is a small block of C++ code, `gamepad-engine.cpp`, that does the time-critical work: packing each input into bytes and firing it across the network on a high-priority real-time thread. It exists because C++ can do this with less delay and less jitter than Kotlin can.

DEFINITION

C++ / NDK — C++ is a low-level, very fast programming language. The NDK (Native Development Kit) is the toolkit that lets an Android app include C++ code alongside its Kotlin. GamepadOS uses it purely for speed on the network hot-path.

4. The PC server (the impersonator). A Python program, `apps/pc-server/server.py`, that runs on your Windows PC. It listens for the phone's packets, decodes them, and drives a **virtual Xbox 360 controller** through a Windows driver called **ViGEmBus**.

DEFINITION

ViGEmBus — a free Windows driver that lets an ordinary program create a fake game controller that Windows treats as real hardware. GamepadOS's Python server uses it to "plug in" the phone as an Xbox 360 pad.

DEFINITION

Python — a readable, general-purpose programming language. The PC server is a single Python file; it is shipped to users as a compiled `.exe` so they don't need Python installed.

5. GRX — the encrypted input layer. Not a separate program but a security layer woven through the C++ engine, a Kotlin crypto file, and the Python server. It wraps every input in authenticated encryption so an eavesdropper on your Wi-Fi can neither read nor forge your controller inputs.

6. The support website + admin portal. At `F:/hlooo/website`, a public site (the download page, setup guide, contact form) plus an Express backend with an admin portal where you, the owner, publish new releases, read support tickets, and watch the download counter. This is also the server that the in-app updater phones home to.

DEFINITION

Express / backend — Express is a framework for building web servers in JavaScript. The "backend" is the always-on program (hosted on a service called Railway) that answers the website's requests, stores tickets, and serves the app downloads.

7. The release + updater pipeline. The discipline (documented in `RELEASE.md`) of bumping version numbers, building the APK and the PC `.exe`, and activating them in the admin portal so the in-app updater offers them to existing users.

8. The in-progress iOS port. Under `apps/ios-client` and a newer `apps/GamepadOS-iOS`, a from-scratch iPhone version of the controller, authored on Windows but not yet compiled (it needs a Mac with Xcode). There is also an experimental `apps/GamepadServer-linux` — a Linux flavour of the PC server. Both are newer than the July-4 session handoff and remain unshipped exploration.

The Repo Layout — What Lives Where

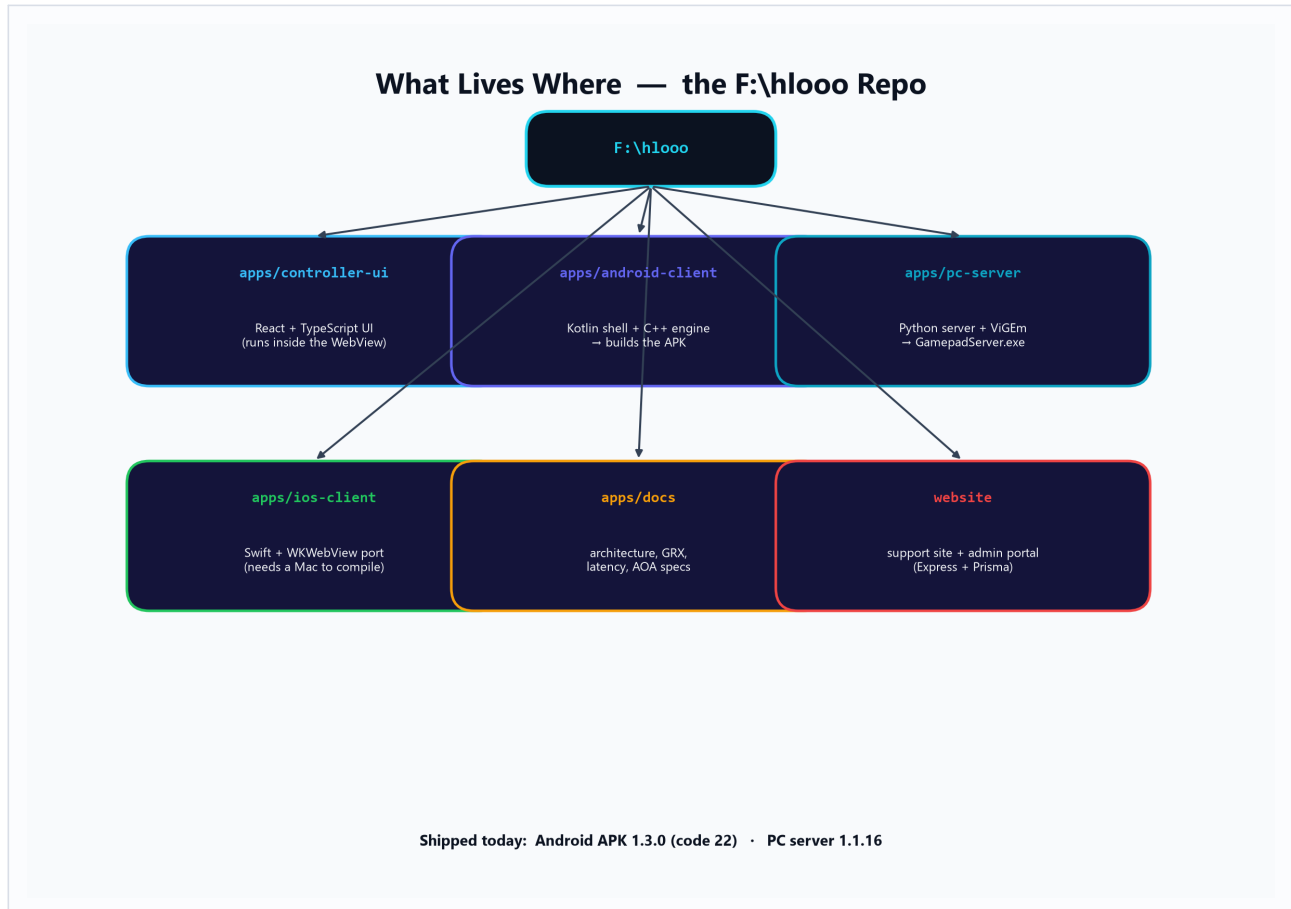


Figure 1.2 — What lives where under F:\hlooo.

Everything lives under `F:\hlooo`. Note that `F:` is a Google Drive mount, and the project *moved* here from an older path (`F:/flexsquares/hlooo`) — so any absolute path in an old note may be stale.

WATCH OUT

The `ARCHITECTURE.md` doc still draws the old top-level layout (`App Interface Design/`, `android-client/`, `pc-server/` directly under the repo root). That is out of date. The **actual** shipped layout nests everything under `apps/`, and the React UI folder was renamed from `App Interface Design` to `controller-ui`. When a doc and the filesystem disagree, the filesystem wins — always confirm with a directory listing.

Path

What it is

<code>apps/controller-ui/</code>	React/TypeScript controller UI (the <code>App.tsx</code> monolith + components)
<code>apps/android-client/</code>	Kotlin Android shell + <code>cpp/gamepad-engine.cpp</code> C++ engine
<code>apps/pc-server/</code>	Python <code>server.py</code> + GRX crypto + installer + PyInstaller spec
<code>apps/ios-client/</code> , <code>apps/GamepadOS-iOS/</code>	in-progress iPhone port (needs a Mac to compile)
<code>apps/GamepadServer-linux/</code>	experimental Linux server port
<code>apps/docs/</code>	architecture, GRX protocol, low-latency, AOA setup docs
<code>website/</code>	public site (<code>frontend/</code>) + Express backend + admin portal (<code>backend/</code>)
<code>releases-archive/</code>	frozen, byte-identical copies of every shipped build (rollback point)
<code>RELEASE.md</code> , <code>SESSION_HANDOFF_*.md</code>	the release checklist and the "read me first" session state

DEFINITION

Monolith — a single large source file that holds an entire subsystem instead of splitting it across many files. `App.tsx` is a monolith — the whole controller UI, its state, and its networking live in one ~3800-line file, on purpose, so the tightly-coupled data flow is visible in one place.

The Wire Contract — 20 Bytes That Both Sides Must Agree On

Everything the phone tells the PC per moment is squeezed into exactly **20 bytes**. This is the single most important shared agreement in the whole system: the phone writes these 20 bytes in one exact order, and the PC reads them back in the identical order. Get one byte offset wrong and the A button becomes the X button, or the left stick drifts. The C++ side pins the layout down as a packed struct:

`apps/android-client/app/src/main/cpp/gamepad-engine.cpp` · lines 51-65

C++

```

1 #pragma pack(push, 1)
2 struct GamepadPayload {
3     uint64_t timestamp; // 8 bytes
4     uint16_t buttons; // 2 bytes
5     uint8_t leftTrigger; // 1 byte
6     uint8_t rightTrigger; // 1 byte
7     uint8_t leftStickX; // 1 byte
8     uint8_t leftStickY; // 1 byte
9     uint8_t rightStickX; // 1 byte
10    uint8_t rightStickY; // 1 byte
11    uint32_t authToken; // 4 bytes

```

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · (cont.) C++
12 };
13 #pragma pack(push, 1)
14
15 static_assert(sizeof(GamepadPayload) == 20, "FATAL: Payload struct is not exactly 20 bytes!");
```

Walk through it. `#pragma pack(push, 1)` tells the compiler "do not insert any hidden padding between these fields" — normally a compiler pads structs for speed, which would silently change the byte count. `timestamp` (8 bytes) is the milliseconds-since-epoch clock reading, used to throw away packets that arrive out of order. `buttons` (2 bytes) is a **bitmask**: 16 on/off switches packed into 16 bits, one per button. Then the two triggers and four stick axes are one byte each (0–255). `authToken` (4 bytes) was the old cleartext password — you will see below that it is now vestigial. Finally `static_assert` is a compile-time tripwire: if the struct ever comes out to anything other than 20 bytes, the build *fails* rather than shipping a broken wire format.

DEFINITION

Byte / bitmask — a byte is 8 bits, and a bit is a single 0-or-1 switch. A bitmask packs many yes/no facts into one number: bit 0 of `buttons` means "A pressed", bit 1 means "B pressed", and so on. Reading a bit is asking `buttons & (1 << n)`.

DEFINITION

Little-endian — the order bytes are written for multi-byte numbers. Little-endian puts the least-significant byte first. Both phone and PC agree on little-endian (< in Python's format string) so the numbers reconstruct correctly.

The PC's matching half is one Python line: `PAYLOAD_FORMAT = '<Q H B B B B B I'` — `Q` is the 8-byte timestamp, `H` the 2-byte buttons, six `B` bytes for triggers and sticks, and `I` the 4-byte token. `<` locks in little-endian. That format string is the literal mirror of the C++ struct above.

The 16 button bits are assigned: bit 0 = A, 1 = B, 2 = X, 3 = Y, 4 = LB, 5 = RB, 6 = Start, 7 = Back, 8 = L3, 9 = R3, 10–13 = D-pad up/down/left/right, and **bit 14 = the Xbox Guide button** (□). That last one matters: the `ARCHITECTURE.md` bit table stops at bit 13 and never mentions bit 14, but the shipped server clearly handles it. Code wins.

The Journey of a Single Button Press

Now trace one thumb-tap all the way to the game reacting. This is the spine of the whole product.

Step 1 — the touch. Your thumb lands on the A button drawn by the React UI. `App.tsx` records it in an in-memory "held" set. Sixty times a second (configurable up to 1000 Hz), a timer fires `sendGamepadTelemetry`, which builds a fresh 20-byte buffer: it writes the current time, folds every held button into the `buttons` bitmask, rescales the triggers, applies gyroscope tilt and a circular deadzone to the sticks, and converts each axis to a 0–255 byte centered on **128**.

```
apps/controller-ui/src/app/App.tsx · lines 968-974 TypeScript
1      // 5. Auth Token (32-bit uint, little-endian). The native UDP TX layer OWNS this
2      //      field — it overwrites offset 16 with the real pairing-key hash before every
3      //      send (gamepad-engine.cpp). On the USB-debug WebSocket path (which bypasses
4      //      native) the server doesn't validate the token and treats loopback as
5      //      authorized, so write 0 (the off-LAN/USB-accepted value) instead of a fake
6      //      constant — keeps the JS frame honest and avoids shipping a bogus token.
7      view.setUint32(16, 0, true);
```

Here the JavaScript writes **zero** into the auth-token field at byte offset 16 — not the `0xABCD1234` constant the old architecture doc claims. Two things changed: the native C++ layer stamps the real pairing-key hash there just before sending, and once GRX encryption is active the whole field is meaningless anyway (the encryption authenticates the packet). This is a clean example of the doc lagging the code — the shipped truth is `setUint32(16, 0, true)`.

Step 2 — over the JSI bridge into C++. `App.tsx` hands the finished buffer to `window.sendGamepadPacket(buffer)`, a function the Kotlin shell injected into the WebView. That call crosses the bridge from web-world into the native C++ engine, which copies the bytes into its own `GamepadPayload` struct and queues them for its real-time send thread.

DEFINITION

JSI / JNI bridge — a wired-up doorway between two languages. JSI lets the JavaScript UI call a native function; JNI (Java Native Interface) lets Kotlin and C++ call each other. The 20-byte buffer travels JavaScript → Kotlin → C++ through these doorways.

Step 3 — the send decision. The C++ thread does not blindly transmit 500 times a second. It decides whether *this* tick is worth a packet:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 288-295 C++
1      // fast. The counter is (re)armed below once we confirm a successful send.
2      bool redundant = (redundancyRemaining > 0);
3      bool doSend    = changed || heartbeat || redundant;
4
5      networkPayload.timestamp = htobe64(rawTimestamp);
6      networkPayload.buttons = htobe16(networkPayload.buttons);
7      networkPayload.authToken = htobe32(expectedHash);
8      // Note: uint8_t analog values do not require endianness swapping.
```

`doSend` is true only if something `changed` since last tick, or it is time for a periodic `heartbeat` keep-alive, or a `redundant` copy is owed (GamepadOS sends a couple of duplicate packets after a change so a single lost packet over Wi-Fi doesn't drop your input). This gating is a core latency-and-heat optimization: an idle controller barely transmits. Notice `authToken = htobe32(expectedHash)` — this is the native layer stamping the real pairing hash the JS comment promised, converted to little-endian.

Step 4 — encryption (GRX). If a paired encrypted session is live, the raw 20 bytes are *not* sent as-is. The C++ engine calls up into Kotlin to seal them:

apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 309-319

C++

```

1      if (doSend && grxReady.load() && g_mid_seal && g_activity) {
2          JNIEnv* e = grxEnv();
3          if (e) {
4              jbyteArray in = e->NewByteArray((jsize)sizeof(GamepadPayload));
5              if (in) {
6                  e->SetByteArrayRegion(in, 0, (jsize)sizeof(GamepadPayload), (const jbyte*)&networkPaylo
ad);
7
8                  jbyteArray wire = (jbyteArray)e->CallObjectMethod(g_activity, g_mid_seal, in);
9                  if (wire) {
10                     jsize wn = e->GetArrayLength(wire);
11                     if (wn > 0 && wn <= (jsize)sizeof(grxBuf)) {
12                         e->GetByteArrayRegion(wire, 0, wn, (jbyte*)grxBuf);

```

This hands the 20-byte plaintext to a Kotlin `seal` method (AES-128-GCM authenticated encryption) and gets back a **41-byte wire frame**: `version (1) | counter_low32 (4) | ciphertext (20) | tag (16)`. The 16-byte tag is a cryptographic seal — the PC rejects the packet if a single bit was altered, and a strictly-increasing counter makes replaying an old packet impossible. Crucially, the seal is gated on `doSend`: sealing on every idle poll was a real bug (~500 wasted encryptions/second) that this `&&` fixed.

DEFINITION

GRX / AEAD — GRX (GamepadOS Realtime eXchange) is the project's encrypted transport. AEAD (Authenticated Encryption with Associated Data) both hides the contents *and* proves they weren't tampered with. GRX replaced the old plain `authToken`, which anyone on the Wi-Fi could read and forge.

Step 5 — across the network. The C++ thread pushes the frame out a **non-blocking UDP socket** to the PC's IP address on port 7777 (the server falls back to 7778+ if 7777 is taken).

DEFINITION

UDP / port — UDP is a fire-and-forget way to send small packets — no waiting for confirmation, which is exactly right for real-time input where a fresh packet is always better than a re-sent stale one. A port (here 7777) is a numbered mailbox on the PC so the packet reaches the right program.

Step 6 — the PC decodes and impersonates. `server.py` receives the frame, decrypts and replay-checks it (GRX), drains its socket to keep only the *newest* packet (never process a backlog — that would add lag), and applies the inputs to the virtual pad:

apps/pc-server/server.py · lines 983-1000

Python

```

1      gamepad.left_joystick_float(x_value_float=_axis(ls_x), y_value_float=-_axis(ls_y))
2      gamepad.right_joystick_float(x_value_float=_axis(rs_x), y_value_float=-_axis(rs_y))
3
4      XB = vg.XUSB_BUTTON
5      if buttons & (1 << 0): gamepad.press_button(button=XB.XUSB_GAMEPAD_A)
6      if buttons & (1 << 1): gamepad.press_button(button=XB.XUSB_GAMEPAD_B)
7      if buttons & (1 << 2): gamepad.press_button(button=XB.XUSB_GAMEPAD_X)
8      if buttons & (1 << 3): gamepad.press_button(button=XB.XUSB_GAMEPAD_Y)
9      if buttons & (1 << 4): gamepad.press_button(button=XB.XUSB_GAMEPAD_LEFT_SHOULDER)

```

```
apps/pc-server/server.py · (cont.) Python
10         if buttons & (1 << 5): gamepad.press_button(button=XB.XUSB_GAMEPAD_RIGHT_SHOULDER)
11         if buttons & (1 << 6): gamepad.press_button(button=XB.XUSB_GAMEPAD_START)
12         if buttons & (1 << 7): gamepad.press_button(button=XB.XUSB_GAMEPAD_BACK)
13         if buttons & (1 << 8): gamepad.press_button(button=XB.XUSB_GAMEPAD_LEFT_THUMB)
14         if buttons & (1 << 9): gamepad.press_button(button=XB.XUSB_GAMEPAD_RIGHT_THUMB)
15         if buttons & (1 << 10): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_UP)
16         if buttons & (1 << 11): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_DOWN)
17         if buttons & (1 << 12): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_LEFT)
18         if buttons & (1 << 13): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_RIGHT)
```

`gamepad` is a `VX360Gamepad` — a **virtual Xbox 360 pad**, created via ViGEm. Each `if buttons & (1 << n)` checks one bit of the mask and, if set, presses the matching Xbox button. `_axis(b)` converts a 0–255 byte to the -1.0...+1.0 float XInput wants using `(b - 128) / 127.0`, so 128 is dead center, and the Y axes are negated because screen-down and stick-up point opposite ways. After the last line (`gamepad.update()`), Windows sees the pad's new state and the game reacts — end to end, typically in a few milliseconds.

WATCH OUT

`ARCHITECTURE.md` repeatedly says the virtual pad is a **DualShock 4**. The shipped server creates a `VX360Gamepad` and presses `XUSB_*` (Xbox 360 / XInput) buttons — there is a stray leftover "Init DualShock 4" comment, but the live code is Xbox 360. Likewise the doc says stick center is 127; the code centers on 128 on both ends. Code wins on both counts.

Current Shipped Versions and Where Truth Lives

Two numbers define "the current product," and each has exactly one source-of-truth file:

Component	Version	Source of truth
Android APK	1.3.0 , versionCode 22	<code>apps/android-client/app/build.gradle.kts</code>
PC server	1.1.16	<code>apps/pc-server/server.py</code> (<code>APP_VERSION = "1.1.16"</code>)

DEFINITION

versionCode — an integer Android bumps every release (here 22). Humans read `versionName` ("1.3.0"); Android and the updater compare the integer `versionCode` to decide whether a newer build exists. `APP_VERSION` plays the same role for the PC server.

DEFINITION

APK — the single installable file for an Android app (`GamepadOS-1.3.0.apk`). The in-app updater downloads a newer APK and hands it to Android's installer; it only works if every release is signed with the same key.

The updater loop ties the whole system together: each running app and server periodically asks the website backend `GET /api/version`. If the version it reports is higher than what's running, the user is

offered an update. That is why `RELEASE.md` is emphatic about bumping the version in *every* spot and then hitting **Register & Activate** in the admin portal — until you activate, `/api/version` keeps serving the old set and nobody gets the new build. As of the July-4 handoff, APK 1.3.0 and PC 1.1.16 were built, signed, installed on your phone, and archived byte-identically in `releases-archive/`, with the portal activation still pending on you (it needs owner access and the `RELEASE_KEY`).

NOTE

When this book cites a byte offset, a port, or a constant, it was read from the source file, not from a doc — because you have already seen three places (DS4-vs-Xbox, 127-vs-128, `0xABCD1234`-vs-0) where a friendly doc drifted from the shipped reality. The habit to build: trust the file the compiler reads, and treat prose docs as hints.

CHAPTER 2

Foundations — Networks, Bytes, and Virtual Gamepads

2

What a Network Is — Addresses, Ports, and Your LAN

Before a single button press can travel from your phone to your PC, both machines have to be able to *find* each other and *reach* each other. That is what a network does.

DEFINITION

Network — A group of computers wired or wirelessly connected so they can exchange data. Your phone, your PC, and your Wi-Fi router form one small network.

DEFINITION

LAN (Local Area Network) — A network confined to one physical place — your home. Everything joined to the same router is on the same LAN and can talk directly, without going out to the internet.

DEFINITION

Wi-Fi — The wireless radio technology your phone and PC use to join the LAN. It is just the *medium* — the invisible cable — over which network data travels.

IN PLAIN TERMS

Think of your LAN as an apartment building and the router as the front desk. Every device that moves in gets a room number so mail can be delivered to it. To send someone a note you need two things: which room they are in, and which mail-slot on their door the note is for. Those two things are the *IP address* and the *port*.

DEFINITION

IP address — The room number of a device on the network, written as four numbers like **192.168.1.42**. On your home LAN these usually start with **192.168.** and are handed out by the router.

DEFINITION

Port — A numbered slot *within* one device, from 0 to 65535, that lets many programs share one IP address without their messages getting mixed up. The IP gets the message to the right machine; the port gets it to the right *program* on that machine.

Your GamepadOS PC server picks a port and then *listens* on it — it tells the operating system "any network message that arrives for this port number, hand it to me." Here is exactly how it does that:

apps/pc-server/server.py · lines 658-673

Python

```
1 def bind_server_socket(base_port=7777, max_attempts=10):
2     for port in range(base_port, base_port + max_attempts):
3         try:
4             sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```

apps/pc-server/server.py · (cont.) Python
5         sock.setblocking(False)
6         # Stop a vanished client from killing the recv thread (see helper).
7         suppress_udp_connreset(sock)
8         # Low-latency socket tuning:
9         #   - Large RX buffer so a burst of 1000Hz packets is never dropped
10        #   while the thread is briefly busy (we still drain to newest).
11        #   - Modest TX buffer so ACK sends never block on a full kernel buffer.
12        #   - IP_TOS DSCP-EF to match the client's QoS tagging symmetrically.
13        try:
14            sock.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 1 << 20) # 1 MB
15        except OSError:
16            pass

```

The server tries port `7777` first (`base_port=7777`) and, if something else is already using it, walks upward through `7778`, `7779` ... up to ten ports total (`max_attempts=10`). For each candidate it creates a socket, marks it non-blocking, and then — a few lines further down, at `sock.bind(('0.0.0.0', port))` — claims that port. The address `0.0.0.0` is a wildcard meaning "listen on *every* network interface this PC has," so the phone can reach the server whether it arrives over Wi-Fi or over the USB tether adapter. The `SO_RCVBUF` line asks the operating system for a one-megabyte receive buffer (`1 << 20` is the number 1 shifted left 20 bit positions, which equals 1048576) so that if a thousand packets a second briefly pile up, none are silently thrown away before the program can read them.

NOTE

When the server later shows you a QR code, it encodes exactly `{ip},{port},{key}` — the IP address it found for itself, the port it successfully bound (usually `7777`), and an auth key. The phone reads all three from one camera scan. That is how the phone learns *where* to send.

DEFINITION

Socket — The software object that represents one endpoint of a network conversation — think of it as the actual mail-slot the program holds open. `socket.socket(...)` in the code above is the act of opening one.

TCP vs UDP — Why the Input Path Uses UDP

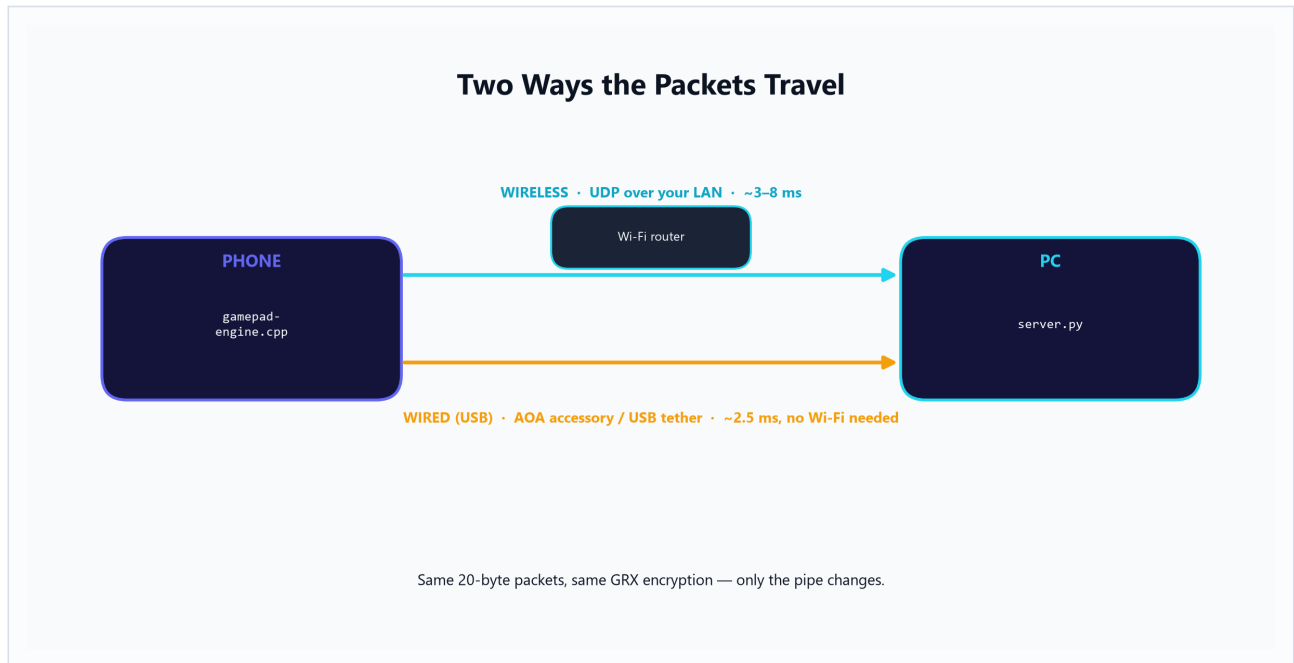


Figure 2.1 — The two ways packets travel: Wi-Fi (UDP) and wired USB.

When two programs talk over a network they must agree on a *transport* — the set of rules for how messages are chopped up, addressed, and delivered. There are two you will meet everywhere, and GamepadOS deliberately chooses one of them for the input stream.

DEFINITION

TCP (Transmission Control Protocol) — A reliable, ordered transport. It guarantees every byte arrives, in order, retransmitting anything lost. The price is waiting: if one message goes missing, everything behind it stalls until the lost piece is re-sent.

DEFINITION

UDP (User Datagram Protocol) — A fire-and-forget transport. Each message is a self-contained *datagram* thrown at the destination with no guarantee of arrival, order, or acknowledgement. It is faster and has no waiting, but you must tolerate the occasional lost message.

DEFINITION

Datagram / packet — One self-contained chunk of data sent across the network in a single shot. For GamepadOS this is the 20-byte controller snapshot you will meet in the next section.

DEFINITION

Head-of-line blocking — The stall TCP suffers when one lost packet freezes every later packet behind it, even though those later ones already arrived. For real-time input this is fatal — a single hiccup would freeze your controller.

IN PLAIN TERMS

TCP is a registered letter: the postman waits for a signature, and if letter #3 gets lost he will not deliver #4, #5, or #6 until #3 is re-sent and signed for — in order, guaranteed, but slow. UDP is shouting the current joystick position across the room: if one shout is missed, who cares, another is coming in one millisecond with a *newer* position anyway.

That last sentence is the whole argument. Your phone sends a fresh, complete controller state up to 1000 times per second. If one datagram is dropped, the *next* one — already on its way — carries newer, better information. Re-sending the stale one would be worse than useless. The project's own design notes state this directly:

- *"TCP adds head-of-line blocking. For real-time input, a dropped packet is better than a delayed one. The server only processes the latest packet anyway."*

You can see the choice baked into the code on both ends. The PC creates its socket with `socket.SOCK_DGRAM` (the flag that means UDP) in the excerpt above. The phone's C++ engine does the identical thing: `udpSocket = socket(AF_INET, SOCK_DGRAM, 0);`. `AF_INET` selects IPv4 addressing (the `192.168.x.x` style), and `SOCK_DGRAM` selects the datagram — UDP — transport. Had either side wanted TCP it would have written `SOCK_STREAM` instead.

The server leans all the way into "only the latest matters." Every loop it *drains* the socket, reading every queued datagram but keeping only the newest per phone, so old inputs can never accumulate into lag. In the receive loop the comment says it plainly: *"Drain all waiting datagrams ... input frames keep the NEWEST per source IP."*

WATCH OUT

UDP's freedom has a sharp edge on Windows: `sendto()` succeeds locally even when nothing is listening on the other end. A rising "packets sent" counter therefore does *not* prove the PC received anything. That is why GamepadOS measures a real acknowledgement (an "ACK" the server echoes back) to decide whether the link is truly alive — a point the C++ engine calls out explicitly.

Bits, Bytes, Endianness, and the 20-Byte Packet

Now that a datagram can travel, what is *inside* it? To answer that you need to know how computers store the smallest pieces of information.

DEFINITION

Bit — The atom of digital information: a single 0 or 1. One on/off switch.

DEFINITION

Byte — Eight bits grouped together. One byte can hold 256 distinct values (0 through 255). A trigger pressed "37% of the way" is stored as a single byte; a stick axis is one byte where 128 is roughly center.

DEFINITION

Bitmask — A single number whose individual *bits* each stand for a separate yes/no flag. GamepadOS packs the pressed/not-pressed state of fourteen buttons into one 2-byte number this way, instead of wasting a whole byte per button.

The controller state that flies across the network is a rigid, fixed-size structure agreed upon by both machines to the exact byte. The phone's C++ engine defines it like this:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 51-65 C++
1 #pragma pack(push, 1)
2 struct GamepadPayload {
3     uint64_t timestamp; // 8 bytes
4     uint16_t buttons;    // 2 bytes
5     uint8_t leftTrigger; // 1 byte
6     uint8_t rightTrigger; // 1 byte
7     uint8_t leftStickX;  // 1 byte
8     uint8_t leftStickY;  // 1 byte
9     uint8_t rightStickX; // 1 byte
10    uint8_t rightStickY; // 1 byte
11    uint32_t authToken;  // 4 bytes
12 };
13 #pragma pack(pop)
14
15 static_assert(sizeof(GamepadPayload) == 20, "FATAL: Payload struct is not exactly 20 bytes!");
```

Read the sizes down the right margin and add them: $8 + 2 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 4 = \mathbf{20 \text{ bytes exactly}}$. The names ending in a number tell you how many bits each field holds: `uint64_t` is an *unsigned 64-bit integer* (8 bytes, holding the millisecond timestamp), `uint16_t` is 16 bits (the 2-byte button bitmask), `uint8_t` is a single byte (each trigger and each stick axis), and `uint32_t` is 4 bytes (the auth token).

DEFINITION

Struct — A named bundle of fields laid out one after another in memory — a template for a fixed record. The C++ `struct GamepadPayload` is the packet layout.

DEFINITION

Struct packing / padding — Compilers normally insert invisible gap bytes between fields so each lands on a tidy memory boundary — "padding." `#pragma pack(push, 1)` forbids that, forcing the fields to sit flush with no gaps so the struct is exactly 20 bytes and not, say, 24.

DEFINITION

`static_assert` — A compile-time check. If `sizeof(GamepadPayload)` were ever anything but 20, the app would *refuse to compile* with that FATAL message — a safety net guaranteeing the phone and PC can never silently disagree about the packet size.

The receiving PC must decode those same 20 bytes back into numbers. It describes the identical layout using Python's `struct` module and a compact format string:

apps/pc-server/server.py · lines 114-125

Python

```

1 # 16-Byte Payload Structure:
2 # [8 bytes uint64] Timestamp
3 # [2 bytes uint16] Buttons Bitmask
4 # [1 byte uint8] Left Trigger
5 # [1 byte uint8] Right Trigger
6 # [1 byte uint8] Left Stick X
7 # [1 byte uint8] Left Stick Y
8 # [1 byte uint8] Right Stick X
9 # [1 byte uint8] Right Stick Y
10 # [4 bytes uint32] Auth Token
11 PAYLOAD_FORMAT = '<Q H B B B B B B I'
12 PAYLOAD_SIZE = struct.calcsize(PAYLOAD_FORMAT)

```

The format string '`<Q H B B B B B B I`' is a precise mirror of the C++ struct. Each letter is one field: `Q` is an 8-byte unsigned integer (the timestamp), `H` is a 2-byte unsigned integer (the buttons), each `B` is a single byte (triggers and stick axes), and `I` is a 4-byte unsigned integer (the auth token). `struct.calcsize(PAYLOAD_FORMAT)` counts that up to 20 and stores it in `PAYLOAD_SIZE`, which the receive loop later uses to reject any datagram that is not exactly the right length.

WATCH OUT

Look at the very first comment line: it says "**16-Byte Payload Structure**" — but the fields listed under it sum to 20, the format string decodes to 20, and the C++ side `static_assert`s on 20. The comment is stale; the `code` is 20 bytes. This is your first concrete example of a rule that holds for the rest of this book: when a comment or an older document disagrees with the running code, the code wins. Trust `PAYLOAD_FORMAT` and the `static_assert`, not the "16-Byte" label.

That `<` at the front of the format string is doing quiet but critical work.

DEFINITION

Endianness — The order in which the bytes of a multi-byte number are laid out — smallest-byte-first ("little-endian") or largest-byte-first ("big-endian"). Two machines that disagree will read each other's numbers scrambled.

The `<` means *little-endian*, and it must match the phone. Android devices are always little-endian, which is why the phone's engine can treat its byte-order conversion helpers as do-nothing identity operations — the comment in the C++ file even notes "Android is always LE — these are identity ops." Both ends agree on little-endian, so a timestamp written on the phone reads back as the same number on the PC.

Finally, the *buttons* field earns its name as a bitmask. The server tests it one bit at a time to decide which buttons are held:

```

apps/pc-server/server.py · lines 975-1000 Python
1     gamepad = sess.pad
2     with GAMEPAD_LOCK:
3         reset_gamepad_state(gamepad)
4         gamepad.left_trigger(value=lt)
5         gamepad.right_trigger(value=rt)
6
7     def _axis(b):
8         return max(-1.0, min(1.0, (b - 128) / 127.0))
9     gamepad.left_joystick_float(x_value_float=_axis(ls_x), y_value_float=-_axis(ls_y))
10    gamepad.right_joystick_float(x_value_float=_axis(rs_x), y_value_float=-_axis(rs_y))
11
12    XB = vg.XUSB_BUTTON
13    if buttons & (1 << 0): gamepad.press_button(button=XB.XUSB_GAMEPAD_A)
14    if buttons & (1 << 1): gamepad.press_button(button=XB.XUSB_GAMEPAD_B)
15    if buttons & (1 << 2): gamepad.press_button(button=XB.XUSB_GAMEPAD_X)
16    if buttons & (1 << 3): gamepad.press_button(button=XB.XUSB_GAMEPAD_Y)
17    if buttons & (1 << 4): gamepad.press_button(button=XB.XUSB_GAMEPAD_LEFT_SHOULDER)
18    if buttons & (1 << 5): gamepad.press_button(button=XB.XUSB_GAMEPAD_RIGHT_SHOULDER)
19    if buttons & (1 << 6): gamepad.press_button(button=XB.XUSB_GAMEPAD_START)
20    if buttons & (1 << 7): gamepad.press_button(button=XB.XUSB_GAMEPAD_BACK)
21    if buttons & (1 << 8): gamepad.press_button(button=XB.XUSB_GAMEPAD_LEFT_THUMB)
22    if buttons & (1 << 9): gamepad.press_button(button=XB.XUSB_GAMEPAD_RIGHT_THUMB)
23    if buttons & (1 << 10): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_UP)
24    if buttons & (1 << 11): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_DOWN)
25    if buttons & (1 << 12): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_LEFT)
26    if buttons & (1 << 13): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_RIGHT)

```

The expression `1 << 0` is the number 1 with its single 1-bit shifted left by zero positions — it is just `...0001`, testing the lowest bit. `1 << 1` is `...0010`, `1 << 2` is `...0100`, and so on: each shift walks the test one bit higher. The `&` is a *bitwise AND*: `buttons & (1 << 0)` is nonzero only when bit 0 of the received `buttons` number is set. So bit 0 means A, bit 1 means B, bit 4 means the left shoulder, bit 10 means D-pad up — exactly the assignment the phone used when it built the mask. The two `_axis` conversions turn a raw stick byte (0–255, center 128) into the `-1.0-to-+1.0` float that a real gamepad reports; note the `-` in front of the Y axis, which flips screen coordinates (down-is-positive) into gamepad coordinates (up-is-positive).

EXPERT DETAIL

The doc's *Button Bit Assignments* table stops at bit 13 (D-pad Right), but the live code continues to **bit 14 = the Guide/Xbox button** (`XUSB_GAMEPAD_GUIDE`), forwarded on every transport so the Xbox overlay opens. Another case of code out-running the older doc.

Client and Server — Who Talks to Whom

Every networked conversation has two roles, and it pays to be precise about which machine plays which.

DEFINITION

Server — The program that waits, listening on a known address and port for others to connect. Your `server.py` on the Windows PC is the server: it binds `0.0.0.0:7777` and waits.

DEFINITION

Client — The program that initiates the conversation by reaching out to the server's address. Your phone app is the client: it reads the server's IP and port from the QR code and starts firing datagrams at them.

IN PLAIN TERMS

The server is a shopkeeper who unlocks the shop each morning and waits behind the counter at a fixed address. The client is a customer who has to know that address to walk in. The shop does not go looking for customers; customers come to the shop. That is why the *PC* shows the QR code (advertising its address) and the *phone* scans it (learning where to go).

The first contact is a tiny handshake. The phone, not yet knowing whether anyone is home, *broadcasts* its packets to the whole LAN. The moment the server receives a valid one it replies with a short acknowledgement, and from then on the phone stops broadcasting and sends only to that one PC — "locking onto the PC's unicast IP," in the engine's words.

DEFINITION

Broadcast — A datagram addressed to *every* device on the LAN at once, used before the phone knows exactly which machine is the server.

DEFINITION

Unicast — A datagram addressed to exactly one recipient. After the handshake the phone switches to unicast and even `connect()`s its socket to that peer, so the kernel caches the route and each send skips a repeated address lookup — a small latency win the C++ code makes deliberately.

DEFINITION

ACK (acknowledgement) — The short reply the server echoes back on each packet. Because it carries the phone's own send-timestamp, the phone can subtract to compute a true round-trip latency instead of guessing.

Why does all of this happen over Wi-Fi in the first place, and what does that cost you? Because your phone and PC share a LAN, packets never leave your home — they hop phone → router → PC, a path of a few meters. But Wi-Fi is a *shared radio*, and radios introduce jitter: the signal contends with neighbors, microwaves, and the router's own power-saving naps. The project's latency guide quantifies the trade-offs:

Setup	Latency
USB tethering (wired-quality)	~6–12 ms

Tuned 5 GHz Wi-Fi (this app)	~10–18 ms
Typical 5 GHz home Wi-Fi	~20–35 ms
2.4 GHz / congested	50–120 ms+

The single most effective escape from Wi-Fi's variability is to stop using it. Plug the phone into the PC with a USB cable, turn on USB tethering, and Windows creates a "Remote NDIS" network adapter — a private wired subnet between just the two devices. The app's UDP path is *unchanged*; the datagrams simply flow over copper instead of radio, and jitter collapses to wired levels. This is why the guide recommends USB tethering for competitive play — same code, different physical wire.

What a Gamepad Is to Windows — XInput, Drivers, and ViGEm



Figure 2.2 — The Android app in four layers.

The server has now decoded a controller state. But nothing on the PC is actually a gamepad — there is no plastic controller plugged in. So how does a *game* end up feeling a button press? This is where the operating system's idea of a device comes in.

DEFINITION

Driver — The software translator that lets the operating system talk to a piece of hardware. When you plug in a real Xbox controller, its driver is what turns USB electrical signals into "the A button is down" for Windows to hand to games.

DEFINITION

XInput — Microsoft's standard programming interface for Xbox-style controllers on Windows. Games that "support a controller" almost always mean they read XInput. It defines the exact set of inputs: A/B/X/Y, two shoulders, two triggers, two clickable sticks, a D-pad, Start/Back, and Guide.

DEFINITION

Virtual device — A device that exists purely in software but which the operating system treats as real hardware. GamepadOS creates a *virtual* Xbox 360 controller — Windows and every game believe a physical pad is plugged in, even though the "pad" is your `server.py` process.

The magic that fabricates that fake-but-real controller is a driver named ViGEMBus, driven through a Python wrapper.

DEFINITION

ViGEMBus — The "Virtual Gamepad Emulation Bus" — a Windows driver that can spawn virtual Xbox 360 and DualShock controllers on demand. It is the piece that must be installed on the PC (the server bundles and silently installs its MSI on first run).

DEFINITION

vgamepad — The Python library that talks to the ViGEMBus driver, letting `server.py` create a virtual pad and set its buttons and sticks with simple method calls. It is the first line of `requirements.txt`.

You already saw vgamepad in action in the bitmask code above — `gamepad.press_button(...)`, `gamepad.left_trigger(...)`, `gamepad.left_joystick_float(...)`. The pad itself is created with a single call, `pad = vg.VX360Gamepad()`, and every change is committed to the driver with `gamepad.update()`. The flow is: decode the 20 bytes → set each button, trigger, and stick on the virtual pad → call `update()` once → ViGEMBus reports the new state to Windows → the game reacts, none the wiser that the "controller" is a phone across the room.

WATCH OUT

The `ARCHITECTURE.md` document repeatedly says the server creates a "Virtual **DualShock 4** Controller" and maps `vgamepad.DS4_BUTTONS`. The running code does no such thing — it calls `vg.VX360Gamepad()` and `vg.XUSB_BUTTON(XUSB_GAMEPAD_A, XUSB_GAMEPAD_LEFT_SHOULDER, ...)`, which is an **Xbox 360 / XInput** pad, not a DualShock 4. The code wins: your virtual controller is an Xbox 360 pad. This matters in practice, because a few games treat XInput and DualShock pads differently.

EXPERT DETAIL

vgamepad connects to ViGEmBus *at import time* — the module runs `VBUS = VBus()` on load, so a bare `import vgamepad` on a PC without the driver raises `VIGEM_ERROR_BUS_NOT_FOUND` before `main()` can run. The server works around this with `_load_vgamepad()`: it tries the import, and if the driver is missing it silently runs the bundled `ViGEmBusSetup` MSI (one UAC prompt) and then polls the import up to ~40 times, re-initializing the module each attempt until the freshly installed driver registers. That is how a fresh PC self-installs "everything needed" from the single `GamepadServer.exe`.

The App's Skin and Muscle — WebView, Native C++, JNI, and the APK

The last foundations are about the phone app itself. It is built from two very different materials welded together: a web page for the *skin* (all the buttons you see and touch) and hand-written native code for the *muscle* (the part that shovels packets onto the network as fast as physically possible).

DEFINITION

WebView — A full web browser engine embedded inside a native app, with no address bar or tabs — just a rectangle that renders HTML, CSS, and JavaScript. The entire GamepadOS controller UI is a web page (`App.tsx`, built with React) displayed inside one.

DEFINITION

React / TypeScript — React is a toolkit for building user interfaces out of reusable components; TypeScript is JavaScript with type labels that catch mistakes before the app runs. The controller's on-screen buttons, sticks, and menus are all React/TypeScript.

Here is where the Android app hands its screen over to that web page:

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 311-333 Kotlin

1      val webViewInstance = android.webkit.WebView(this).apply {
2          setBackgroundColor(Color.TRANSPARENT)
3          setLayerType(View.LAYER_TYPE_HARDWARE, null) // Hardware acceleration for ultra-low latency rendering
4
5          settings.apply {
6              javaScriptEnabled = true
7              domStorageEnabled = true
8              allowFileAccess = true
9              allowContentAccess = true
10             databaseEnabled = true
11             mediaPlaybackRequiresUserGesture = false
12             cacheMode = android.webkit.WebSettings.LOAD_NO_CACHE
13
14             // Allow loading ES modules over file:/// protocol to bypass CORS policy blocks
15             allowUniversalAccessFromFileURLs = true
16             allowFileAccessFromFileURLs = true
17             // Allow the WebView to open ws://127.0.0.1 (USB-debugging transport)
18             mixedContentMode = android.webkit.WebSettings.MIXED_CONTENT_ALWAYS_ALLOW
19         }
20     }
```

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · (cont.)
```

Kotlin

```
20         clearCache(true)
21
22         // JSI-like JS-to-Java WebView Bridge
23         addJavascriptInterface(object {
```

This Kotlin creates the WebView, makes its background transparent (so native graphics can show through behind the UI), and turns on hardware acceleration for smooth high-refresh rendering. `javascriptEnabled = true` lets the React app run; `domStorageEnabled = true` is what allows your custom pad layouts to persist in `localStorage`. Elsewhere the app points this WebView at `file:///android_asset/dist/index.html` — the *compiled* React app, bundled inside the APK, loaded straight from local storage rather than from the internet. The final `addJavascriptInterface(...)` line is the beginning of the bridge that lets the web page call down into native code.

DEFINITION

Kotlin — The modern programming language for Android apps; it is what `MainActivity.kt` is written in, forming the thin native shell that hosts the WebView.

DEFINITION

Native code — Code compiled directly to the phone's processor instructions, rather than run inside an interpreter or a browser. It runs faster and with more predictable timing than JavaScript or Kotlin.

DEFINITION

C++ / NDK — C++ is a low-level language that compiles to native machine code; the NDK (Native Development Kit) is Android's toolset for including C++ in an app. The performance-critical UDP send loop (`gamepad-engine.cpp`) is C++ precisely so it can run at real-time priority without the pauses that managed languages suffer.

Why bother with two languages? Because the language that draws the pretty buttons is the wrong language for hitting a 1000-per-second network deadline. JavaScript and Kotlin both run under a *garbage collector* — an automatic memory janitor that occasionally freezes the program for a few milliseconds to clean up. A freeze like that mid-stream shows up as input jitter. C++ has no garbage collector, so the send loop runs on a dedicated real-time thread with steady timing. The design notes put it bluntly: *"Java/Kotlin have GC pauses that cause packet jitter. The C++ thread bypasses this entirely."*

The bridge between the web/Kotlin world and the C++ world is called JNI.

DEFINITION

JNI (Java Native Interface) — The standard mechanism by which Java/Kotlin code calls into C/C++ code and back. Functions on the C++ side are given long names encoding the Kotlin class they belong to, so the two halves can find each other.


```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 611-629 C++

1  extern "C" JNIEXPORT void JNICALL
2  Java_com_gamepad_client_MainActivity_injectNativePayload(JNIEnv* env, jobject this, jbyteArray data) {
3      jsize len = env->GetArrayLength(data);
4      if (len == sizeof(GamepadPayload)) {
5          // FIX 5: single-copy JNI. GetByteArrayRegion copies the 20 bytes straight
6          // into currentPayload (no Get/Release pin pair, no intermediate buffer +
7          // extra memcpy). Done under payloadMutex so it can't tear against the TX
8          // thread's read. The 20-byte length guard above is preserved.
9          {
10             std::lock_guard<std::mutex> lock(payloadMutex);
11             env->GetByteArrayRegion(data, 0, sizeof(GamepadPayload), reinterpret_cast<jbyte*>(&currentPaylo
ad));
12             payloadDirty = true;
13         }
14         // Wake the TX thread NOW so this input transmits immediately rather than on
15         // the next poll tick. Notify after releasing the lock so the woken thread
16         // doesn't immediately block on a mutex we still hold.
17         payloadCv.notify_one();
18     }
19 }
```

The function name `Java_com_gamepad_client_MainActivity_injectNativePayload` is JNI's naming convention: it means "the native implementation of the `injectNativePayload` method on the `MainActivity` class in package `com.gamepad.client`." On the Kotlin side there is a matching one-line declaration, `private external fun injectNativePayload(data: ByteArray)`, where `external` means "the body lives in native code." When the React app has built its 20 bytes, they travel down through Kotlin into this C++ function. It first checks the array is exactly `sizeof(GamepadPayload)` (20) — rejecting anything malformed — then copies those bytes straight into `currentPayload` under a mutex lock so the sending thread cannot read a half-written packet, and finally calls `payloadCv.notify_one()` to *wake the send thread immediately* rather than waiting for its next scheduled tick. That native library only becomes callable because of one line at the bottom of `MainActivity.kt`: `System.loadLibrary("gamepad_engine")`, which loads the compiled C++ into the app at startup.

Finally, how does all of this — React, Kotlin, C++ — become one installable thing on your phone?

DEFINITION

Compiling — Translating human-readable source code into the machine instructions (or bytecode) a computer can execute. The React app is compiled into `dist/`, the C++ into a native `.so` library, the Kotlin into Android bytecode.

DEFINITION

APK (Android Package) — The single `.apk` file that is an Android app — a zip archive bundling the compiled code, the web assets under `assets/dist/`, images, and a manifest. Installing the APK on your phone installs the whole controller.

The build order follows those definitions exactly: first `npm run build` compiles the React UI and its output is copied into `android-client/app/src/main/assets/dist/`; then Android Studio compiles the Kotlin and the C++ engine and zips everything — web assets included — into one APK you install. Behind the current shipped app is server version `APP_VERSION = "1.1.16"` on the PC side, and each release requires the phone APK, the PC exe, and the update manifest to agree on their versions or the

in-app updater loops. With these foundations — addresses and ports, UDP datagrams, the 20-byte packed struct, client/server roles, ViGEm's virtual XInput pad, and the WebView-plus-native-C++ app wrapped into an APK — every later chapter now has the vocabulary it needs.

PART II

Inside the Phone

CHAPTER 3

The Controller Screen — Inside App.tsx

3

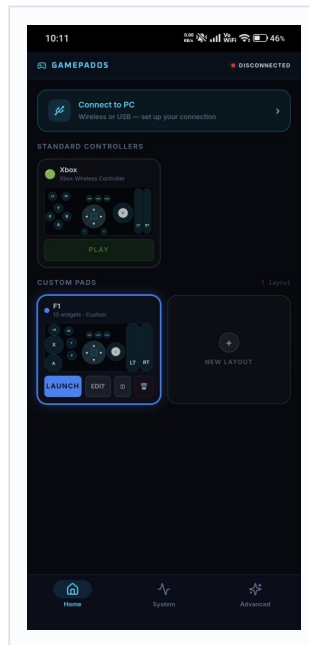


Figure 3.1 — The home screen: the standard Xbox pad plus your saved custom layouts.

From Web Page to Game Controller

When you press the A button on your phone, you are not tapping a native Android button. You are tapping a colored circle drawn on a *web page* — the same kind of page a browser shows — that happens to be running full-screen inside your Android app. Everything you see on the controller screen is HTML, CSS, and JavaScript, authored as a small program in the folder `apps/controller-ui`.

DEFINITION

WebView — a full web browser with no address bar, embedded inside a native app. Your Kotlin app opens one, points it at your web page, and the page fills the whole screen — the user never knows it is "a browser."

DEFINITION

React — a JavaScript library for building user interfaces out of reusable pieces called components. You describe *what* the screen should look like for a given set of data, and React figures out the minimal changes needed to make the real screen match.

DEFINITION

TypeScript — JavaScript with type labels bolted on (`x: number`, `id: string`). The labels are checked before shipping and then stripped out; the phone runs plain JavaScript. Files end in `.tsx` (TypeScript + markup) or `.ts`.

DEFINITION

Vite — the build tool that turns your many `.tsx` source files into one shippable bundle, and also runs a live dev server while you edit. Pronounced "veet."

IN PLAIN TERMS

Think of the controller screen as a poster you design on a big sheet of paper. React is the artist who redraws only the parts that changed. TypeScript is the proofreader who checks your labels before printing. Vite is the print shop that flattens your many draft pages into one finished poster. The WebView is the picture frame bolted to the wall (the Android app) that the poster lives inside.

The whole thing starts from one tiny file. When the WebView loads the page, the browser reads `index.html`, which contains an empty `<div id="root"></div>` and a single script tag pointing at `main.tsx`. That script is the ignition switch.

apps/controller-ui/src/main.tsx · lines 1-6

TypeScript · React

```
1 import { createRoot } from "react-dom/client";
2 import App from "../app/App.tsx";
3 import "../styles/index.css";
4
5 createRoot(document.getElementById("root")!).render(<App />);
```

`document.getElementById("root")` grabs that empty `<div>` from the HTML. `createRoot(...)` tells React "this box is yours to control." `.render(<App />)` hands React your entire application — the `App` component imported from `../app/App.tsx` — and says "draw it in here." From this single line, React builds the whole controller. The `import "../styles/index.css"` pulls in the global stylesheet. That is the *entire* boot sequence: one DOM box, one root, one component.

DEFINITION

DOM (Document Object Model) — the live, in-memory tree of everything on the page — every box, circle, and line of text is a "node." JavaScript changes the DOM; the browser repaints. `getElementById` searches that tree by the element's `id`.

DEFINITION

component — a JavaScript function that returns markup. `App` is a component; so are `Btn`, `Dpad`, and `ControllerScreen`. Components can contain other components, forming a tree.

The project's shape is described by `package.json`, which lists every third-party library the UI depends on and the commands to build it.

apps/controller-ui/package.json · lines 5-14

JSON

```

1  "type": "module",
2  "scripts": {
3    "build": "vite build",
4    "dev": "vite"
5  },
6  "dependencies": {
7    "@capacitor/core": "^8.3.4",
8    "@emotion/react": "11.14.0",
9    "@emotion/styled": "11.14.1",
10   "@mui/icons-material": "7.3.5",

```

Two scripts matter: `npm run dev` starts Vite's live server (edit a file, the phone updates instantly); `npm run build` produces the final bundle. The dependencies include `@capacitor/core` — the toolkit that wraps the web build into an Android app — plus UI libraries like MUI, Radix, and `framer-motion` for animation. Further down (not shown), `peerDependencies` pins `react 18.3.1` and `react-dom 18.3.1`, and `devDependencies` pins `vite 6.3.5` and the Capacitor Android tooling at `^8.3.4`. These exact versions are what actually ship; if an older note says otherwise, the file wins.

React in Ten Minutes: State, Refs, and Effects

Before you can read a single widget, you need three ideas: **state**, **refs**, and **effects**. React calls these "hooks" because they hook your component into React's machinery.

DEFINITION

hook — a special function whose name starts with `use` (`useState`, `useRef`, `useEffect`). Hooks let a plain function-component remember things and react to changes. They must be called at the top of the component, never inside a loop or `if`.

DEFINITION

state — a value the component *remembers between redraws*, and which, when changed, tells React to redraw. "Is the A button held? Which screen are we on?" are state.

Here is the smallest possible example of `useState`, so you have the pattern in your head:

teaching example

TypeScript · React

```

1  function Counter() {
2    const [count, setCount] = useState(0); // count starts at 0
3    return <button onClick={() => setCount(count + 1)}>{count}</button>;
4  }

```

`useState(0)` returns a pair: the current value (`count`) and a setter (`setCount`). Reading `count` gives you the number; calling `setCount(count + 1)` both stores the new number *and* schedules React to redraw the button so it shows the new total. You never edit `count` directly — you always go through the setter, because the setter is what triggers the redraw.

DEFINITION

ref — a box that holds a value *without* triggering a redraw when it changes. Created with `useRef`. Use it for things the screen doesn't need to reflect instantly — or for values you must read/write faster than React can redraw.

DEFINITION

effect — a piece of code that runs *after* React draws, created with `useEffect`. Use it to talk to the outside world: start a timer, subscribe to the sensor, clean up when the component goes away.

Your controller uses all three in a single, revealing custom hook: `useHeld`, which tracks which buttons are currently pressed. It is the beating heart of the "lowest latency possible" design.

```
apps/controller-ui/src/app/App.tsx · lines 206-227 TypeScript · React

1  function useHeld(onChange?: () => void) {
2    const [held, setHeld] = useState<Set<BtnId>>(new Set());
3    const heldRef = useRef<Set<BtnId>>>(held);
4    const onChangeRef = useRef(onChange);
5    useEffect(() => { onChangeRef.current = onChange; });
6    const dn = useCallback((id: BtnId) => {
7      heldRef.current = new Set([...heldRef.current, id]);
8      if (onChangeRef.current) onChangeRef.current();
9      setHeld(new Set(heldRef.current));
10   }, []);
11   const up = useCallback((id: BtnId) => {
12     const n = new Set(heldRef.current); n.delete(id); heldRef.current = n;
13     if (onChangeRef.current) onChangeRef.current();
14     setHeld(new Set(heldRef.current));
15   }, []);
16   const clear = useCallback(() => {
17     heldRef.current = new Set();
18     if (onChangeRef.current) onChangeRef.current();
19     setHeld(new Set());
20   }, []);
21   return { held, heldRef, dn, up, clear };
22 }
```

Look at `dn` (button "down"). It does three things in a deliberate order: (1) it updates `heldRef.current` — the *ref* copy — so the live truth changes **synchronously, right now**; (2) it fires `onChangeRef.current()`, which sends the network packet immediately from that fresh ref; (3) *then* it calls `setHeld(...)` to update the *state* copy, which is only used to repaint the button's highlight. The insight: the packet goes out before React has redrawn anything. The screen catching up (green glow) can wait 16 milliseconds; the *game* cannot. This "ref-then-send, state-last" pattern repeats for every input on the controller.

DEFINITION

`useCallback` — wraps a function so React hands back the *same* function instance on every redraw instead of a fresh one. This keeps child components from needlessly re-rendering. The `[]` at the end means "this function never needs rebuilding."

DEFINITION

Set — a JavaScript collection that holds each value at most once. **held** is a **Set** of button IDs; a button is pressed if its ID is in the set. **BtnId** is the union type from **types.ts** — `"A" | "B" | "X" | ...` — so TypeScript rejects a typo like `"Aa"`.

NOTE

The **onChangeRef** dance (a ref holding the callback, refreshed every render by that bare **useEffect**) exists so **dn/up** can stay "frozen" with `[]` yet always call the *newest* send function. It is a standard trick for "a stable function that calls a changing function."

The View Machine: Switching Screens

The top-level **App** component is a state machine with four screens. It remembers which one is showing and swaps between them.

DEFINITION

view — which full-screen page is active. **types.ts** defines `type View = "scanner" | "dashboard" | "controller" | "editor"`. The app opens on `"dashboard"`.

The render simply asks "which view?" and mounts the matching screen. Here is the controller branch:

```
apps/controller-ui/src/app/App.tsx · lines 4214-4232 TypeScript · React

1      {view === "controller" && (
2        <motion.div
3          key="controller"
4          initial={{ opacity: 0, scale: 0.98 }}
5          animate={{ opacity: 1, scale: 1 }}
6          exit={{ opacity: 0, scale: 0.98 }}
7          transition={{ duration: 0.34, ease: [0.16, 1, 0.3, 1] }}
8          style={{ willChange: "transform, opacity", backfaceVisibility: "hidden" }}
9          className="absolute inset-0 z-20"
10         >
11         <ControllerScreen
12           onBack={() => navigateTo("dashboard")}
13           isActive={view === "controller"}
14           premium={premium} setPremium={setPremium}
15           credits={credits} setCredits={setCredits}
16           activeMapping={activeMapping}
17           customPad={activeCustomPad}
18           controllerLayout={activeCustomPad ? "standard" : controllerLayout}
19           posOverride={posOverrides[selectedPresetId]}
```

`{view === "controller" && (...)}` is a plain JavaScript boolean gate: the **ControllerScreen** only exists in the tree when **view** equals `"controller"`. **motion.div** (from **framer-motion**) fades and scales the screen in and out. `isActive={view === "controller"}` is important — the controller uses that flag to start/stop its 16 ms send loop, so a hidden controller isn't secretly firing packets. Everything below (**activeMapping**,

`customPad`, the gyro settings) is passed *down* as props, the one-directional way React shares data from parent to child.

DEFINITION

prop — an input handed to a component, like a function argument. `<ControllerScreen isActive={...} />` passes `isActive` as a prop. Data flows down via props; changes flow up via callback props like `onBack`.

Switching views isn't instant, because the phone must physically *rotate* — the dashboard is portrait, the controller is landscape. `navigateTo` drops a black "curtain," rotates the device, swaps the view underneath, then lifts the curtain:

```
apps/controller-ui/src/app/App.tsx · lines 3927-3949 TypeScript · React

1  function navigateTo(target: View) {
2    if ((view === "dashboard" && target === "controller") || (view === "controller" && target === "dashboar
   d")) {
3      runCurtainSequence(() => {
4        const bridge = (window as any).AndroidBridge;
5        if (bridge && bridge.setScreenOrientation) {
6          try {
7            bridge.setScreenOrientation(target === "controller" ? "landscape" : "portrait");
8          } catch (e) { console.error(e); }
9        }
10       setView(target);
11       viewRef.current = target;
12     });
13   } else {
14     setView(target);
15     viewRef.current = target;
16     const bridge = (window as any).AndroidBridge;
17     if (bridge && bridge.setScreenOrientation) {
18       try {
19         bridge.setScreenOrientation(target === "controller" ? "landscape" : "portrait");
20       } catch (e) {}
21     }
22   }
23 }
```

`(window as any).AndroidBridge` is your first sight of the *bridge* — the object Kotlin injects into the page so JavaScript can call native code. `bridge.setScreenOrientation("landscape")` reaches out of the web world and rotates the actual Android activity. `runCurtainSequence` wraps the swap so the ugly physical rotation happens *behind* an opaque black overlay; `setView(target)` changes the state that the render above reads. Note the mirrored `viewRef.current = target` — a ref copy of the view, kept so the Android hardware-back handler can read the current screen without waiting for a redraw (the same ref-then-state idea again).

Widgets on an SVG Canvas

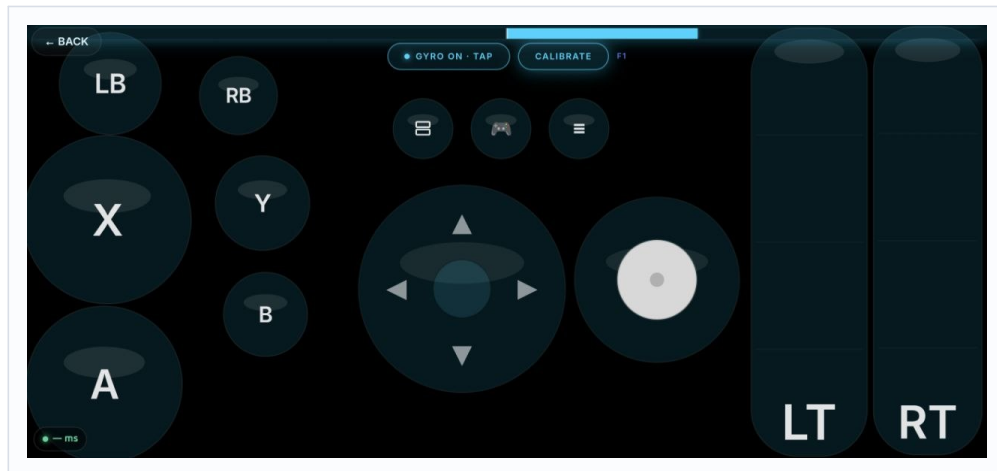


Figure 3.2 — A custom pad in play (landscape).

Every button, stick, trigger, and d-pad is drawn as shapes inside one big `<svg>` element with a fixed coordinate system.

DEFINITION

SVG (Scalable Vector Graphics) — a way to draw shapes — circles, rectangles, paths — using math instead of pixels, so they stay razor-sharp at any size. `<circle cx cy r>` is a circle at center (cx, cy) with radius r.

DEFINITION

viewBox — the SVG's internal coordinate grid. The controller uses `viewBox="0 0 1280 570"` — a fixed **1280**×**570** design space. A button at x=160 sits at the same relative spot on a tiny phone or a tablet, because SVG scales the whole grid uniformly to fit the screen. This exact 1280 width must match the pad editor's canvas so what you design is what you play (WYSIWYG).

The building blocks live in `components/Widgets.tsx`: `Btn`, `Dpad`, `RightStick`, and `TriggerPill`. Start with the plain button and its touch handling, because multitouch is the subtle part.

DEFINITION

pointer event — a unified touch/mouse/stylus event. `onPointerDown` fires when a finger lands, `onPointerUp` when it lifts, `onPointerCancel` when the system steals the touch (a call comes in). Each touch carries a `pointerId` identifying *that specific finger*.

DEFINITION

multitouch — tracking several fingers at once. A gamepad demands it — you hold a stick with your left thumb while tapping A with your right. Every widget must claim only *its own* finger and ignore the others.

DEFINITION

pointer capture — `setPointerCapture(pointerId)` tells the browser "route every future move/up for this finger to *me*, even if it slides outside my shape." Without it, dragging your thumb off a button would silently drop the release and the button would stick.

apps/controller-ui/src/app/components/Widgets.tsx · lines 94-117

TypeScript · React

```

1  <g style={{ cursor: "pointer", touchAction: "none" }}
2    onPointerDown={(e) => {
3      dn(id);
4      spawn(e);
5      try {
6        (e.currentTarget as Element).setPointerCapture(e.pointerId);
7      } catch (err) {
8        console.warn("Btn: Pointer capture failed", err);
9      }
10   }}
11   onPointerUp={(e) => {
12     try {
13       (e.currentTarget as Element).releasePointerCapture(e.pointerId);
14     } catch (err) {}
15     up(id);
16     releaseHaptic();
17   }}
18   onPointerCancel={(e) => {
19     try {
20       (e.currentTarget as Element).releasePointerCapture(e.pointerId);
21     } catch (err) {}
22     up(id);
23   }}
24   onLostPointerCapture={() => up(id)}>

```

On `onPointerDown`: `dn(id)` marks the button held (fires the packet), `spawn(e)` starts the water-ripple animation, and `setPointerCapture` locks this finger to this button. On `onPointerUp` and `onPointerCancel`: release the capture and call `up(id)`. The extra `onLostPointerCapture={() => up(id)}` is a safety net — if the OS yanks the capture for any reason, the button still un-presses instead of freezing "on." `touchAction: "none"` disables the browser's own scroll/zoom gestures so a fast tap-and-slide never gets hijacked into a page scroll. Each finger's `pointerId` keeps this button's press independent of every other widget.

The analog stick is where real geometry appears. `useStick` converts your finger's raw screen position into a clamped (`x`, `y`) offset from the stick's center.

DEFINITION

deflection — how far the stick knob is pushed from center, and in which direction. A real stick can't leave its socket; yours can't leave a circle of radius `maxR` (here **60** units).

```
apps/controller-ui/src/app/App.tsx · lines 264-286 TypeScript · React

1  const onMove = useCallback((e: React.PointerEvent) => {
2    if (!active.current || e.pointerId !== pid.current) return;
3    const p = toSVG(e.clientX, e.clientY);
4    const dx = p.x - centerRef.current.x, dy = p.y - centerRef.current.y;
5    const dist = Math.sqrt(dx * dx + dy * dy);
6    const r = Math.min(dist, maxR), a = Math.atan2(dy, dx);
7    const newX = Math.cos(a) * r;
8    const newY = Math.sin(a) * r;
9
10   // subtle "edge" tick — fire once when the knob first hits full deflection,
11   // re-arm only after it eases back in. Mirrors a real stick's hard stop.
12   const onRim = dist >= maxR - 0.5;
13   if (onRim && !atRim.current) playHaptic("tick");
14   atRim.current = onRim;
15
16   posRef.current = { x: newX, y: newY };
17
18   if (knobRef.current) {
19     knobRef.current.style.transform = `translate(${newX}px, ${newY}px)`;
20   }
21
22   if (onChangeRef.current) onChangeRef.current();
23 }, [maxR]);
```

The guard `e.pointerId !== pid.current` is multitouch discipline: this stick only listens to the finger that first grabbed it (`pid.current`), so a stray second thumb can never yank it. `dx`, `dy` is the finger's offset from center; `dist =  $\sqrt{dx^2 + dy^2}$` is the straight-line distance (Pythagoras). `Math.min(dist, maxR)` clamps that distance to the socket radius, and `Math.atan2(dy, dx)` gives the angle. Multiplying `cos(angle)` and `sin(angle)` by the clamped radius rebuilds the knob position *on the rim* when you push past the edge — so the knob glides around the circle instead of escaping it. The result is stored in `posRef` (read by the packet builder) and drawn by nudging the knob's CSS `transform`. When the knob first touches the rim, a light "tick" haptic fires once, re-armed only after it eases back — mimicking a real stick's hard stop.

DEFINITION

toSVG — a helper that converts a finger's screen pixel coordinates into the SVG's 1280×570 coordinate space, using the browser's `getScreenCTM()` transform matrix. Without it, a touch measured in phone pixels wouldn't line up with a button placed in design units.

The d-pad is one circular widget that decodes *eight* directions (including diagonals) from where your thumb sits, using angles:

```
apps/controller-ui/src/app/components/Widgets.tsx · lines 215-225 TypeScript · React

1  const dirsAt = (px: number, py: number): Set<string> => {
2    const out = new Set<string>();
3    const dx = px - cx, dy = py - cy;
4    if (Math.hypot(dx, dy) < inner) return out;           // dead center → no direction
```

```
apps/controller-ui/src/app/components/Widgets.tsx · (cont.) TypeScript · React

5   const a = Math.atan2(dy, dx) * 180 / Math.PI;           // -180..180 (0=right, 90=down)
6   if (Math.abs(a) <= 67.5) out.add(idFor.right);
7   if (Math.abs(a) >= 112.5) out.add(idFor.left);
8   if (a >= 22.5 && a <= 157.5) out.add(idFor.down);
9   if (a <= -22.5 && a >= -157.5) out.add(idFor.up);
10  return out;
11  };
```

`Math.hypot(dx, dy) < inner` returns *no* direction when your thumb rests near dead-center (a built-in deadzone). Otherwise it computes the thumb's angle in degrees and lights up each cardinal whose band it falls in. The bands deliberately **overlap** at the corners: "right" covers $\pm 67.5^\circ$, "down" covers 22.5° – 157.5° , so at 45° *both* right and down turn on — a true diagonal — and sliding your thumb rolls smoothly from one direction into the next. It returns a `Set` because up to two directions can be active together.

The trigger pill (`TriggerPill`) is analog: how far *up* the pill your finger sits sets the pull from 0 to 1, exactly like squeezing a real trigger.

```
apps/controller-ui/src/app/components/Widgets.tsx · lines 375-381 TypeScript · React

1   const fillAt = (e: React.PointerEvent): number => {
2     const svg = svgRef.current; if (!svg) return 1;
3     const pt = svg.createSVGPoint(); pt.x = e.clientX; pt.y = e.clientY;
4     const m = svg.getScreenCTM(); if (!m) return 1;
5     const p = pt.matrixTransform(m.inverse());
6     return Math.max(0, Math.min(1, (y + h - p.y) / h));
7   };
```

`(y + h - p.y) / h` measures the finger's distance from the *bottom* of the pill, divided by the pill height — so touching the bottom gives 0, the top gives 1, and `Math.max(0, Math.min(1, ...))` clamps anything outside. When a pad is saved with `analogTrigger: false` ("Normal" mode in the editor), the widget passes `digital={true}` and a tap jumps straight to 1.0 instead of feathering — a trigger that behaves like a plain button. That single field on `CustomBtnDef` (defined in `types.ts`) is what flips the behavior.

EXPERT DETAIL

`TriggerPill`, `Dpad`, and `useStick` each carry their own `pid` ref (the owning `pointerId`) and each call `setPointerCapture` on `pointerDown`. That per-widget ownership is what makes genuine N-finger play work in a `WebView`: the browser fans every pointer event to whichever `<g>` captured that specific finger, and each widget's `if (e.pointerId !== pid.current) return` gate discards the rest. There is no global touch dispatcher — ownership is distributed to the leaves.

The Gyro Toggle and the Top-Edge Indicator

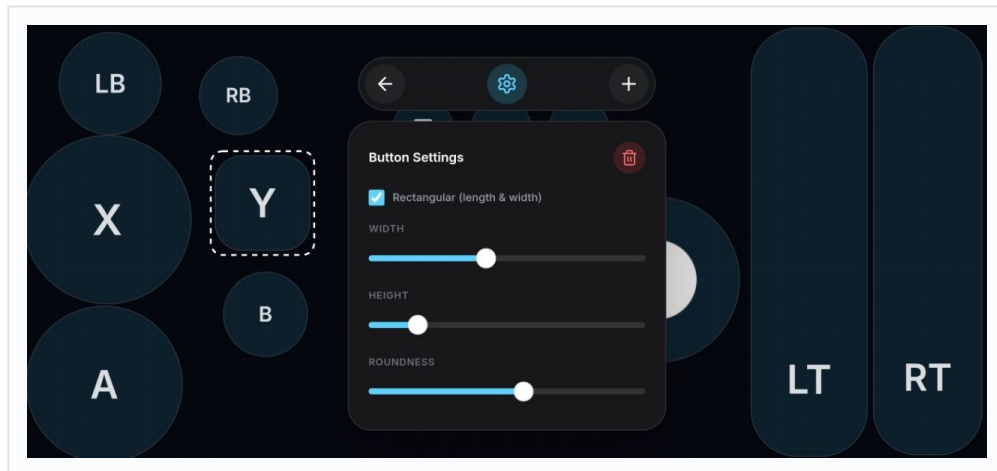


Figure 3.3 — Editing a button's shape and size in the layout builder.

Gyro steering lets you tilt the phone to steer (racing) or aim (3D look). The reading comes from native code, is smoothed there, and is injected into the stick values. On screen, its only footprint is a **toggle button** and a thin **indicator bar pinned to the very top edge**.

DEFINITION

gyroscope — the sensor that reports the phone's tilt angle. Your `useGyro` hook reads it 120 times a second via `bridge.getGyroscopeDataJson()`, where `data.nx/data.ny` are the tilt angles in degrees, already filtered by the native "i€" filter.

The indicator has two loops, deliberately split: a **data loop** every 8 ms (120 Hz) that reads the sensor and feeds the game value, and a **render loop** tied to `requestAnimationFrame` that only slides the on-screen bar. Here is the render loop:

```
apps/controller-ui/src/app/App.tsx · lines 575-589 TypeScript · React

1  const render = (ts: number) => {
2    const dt = Math.min((ts - lastTs) / 1000, 0.1); // clamp after a stall
3    lastTs = ts;
4    const k = 1 - Math.exp(-SMOOTH_RATE * dt);
5    dispBx += (targetBx - dispBx) * k;
6    dispBy += (targetBy - dispBy) * k;
7    if (Math.abs(targetBx - dispBx) < 0.004) dispBx = targetBx; // snap the tail
8    if (Math.abs(targetBy - dispBy) < 0.004) dispBy = targetBy;
9    // scaleX (GPU-composited, no layout reflow) = smoothest update.
10   if (leftBarRef.current) leftBarRef.current.style.transform = `scaleX(${dispBx < 0 ? -dispBx : 0})`
11   ;
12   if (rightBarRef.current) rightBarRef.current.style.transform = `scaleX(${dispBx >= 0 ? dispBx : 0})`
13   ;
14   // 3D-mode dot: translate ±16px within the 44px box (X mirrors the bar sign; Y up = look up).
15   if (dotRef.current) dotRef.current.style.transform = `translate(${-dispBx * 16}px, ${-dispBy * 16}p
x)`;
16   reqId = requestAnimationFrame(render);
17   };
18 }
```


DEFINITION

`requestAnimationFrame` — asks the browser to run your function once, right before the next screen repaint (about 60 times/sec). Chaining it (the last line re-requests itself) makes a smooth animation loop synced to the display.

DEFINITION

`scaleX` — a CSS transform that stretches an element horizontally. `scaleX(0)` collapses it to nothing; `scaleX(0.5)` is half-width. Because scaling is a GPU-composited transform, it repaints *without* recalculating page layout — the cheapest possible animation.

`dispBx += (targetBx - dispBx) * k` is an exponential glide: each frame the displayed value moves a fraction k of the way toward the target, so the bar eases in instead of snapping. $k = 1 - \exp(-\text{SMOOTH_RATE} * dt)$ (with `SMOOTH_RATE = 36`) makes that glide *frame-rate independent* — it feels identical on a 60 Hz and a 120 Hz phone, fixing an older bug where the bar converged twice as fast on 120 Hz screens. Then the payoff: the left half's width is driven by `scaleX(dispBx < 0 ? -dispBx : 0)` and the right half by the positive case. Tilt left, the left bar grows from center; tilt right, the right bar grows. In 3D mode the same value instead `translates` a dot within a 44px box.

Now the indicator's *look*, which is a precise, user-mandated design. It sits in its own layer at the very top edge, **behind** the buttons, edge-to-edge, exactly **6.5px** thick, with **no glow**:

```
apps/controller-ui/src/app/App.tsx · lines 1250-1273 TypeScript · React

1      {gyroMode === "racing" && (
2        <div className="w-full overflow-hidden flex relative transition-opacity duration-200"
3          style={{
4            // Edge-to-edge like 1.2.9, but 65% of its thickness (10px → 6.5px).
5            height: "6.5px",
6            background: gyroOn ? "rgba(0,212,255,0.14)" : "rgba(255,255,255,0.12)",
7            opacity: gyroOn ? 1 : 0.7,
8          }}>
9          /* center tick */
10         <div className="absolute left-1/2 top-0 bottom-0 w-[2px] -translate-x-1/2 bg-white/50 z-10" />
11         <div className="flex-1 overflow-hidden">
12           /* left half — scales from the RIGHT edge (toward center) */
13           <div ref={gyroTilt.leftBarRef} className="h-full w-full"
14             style={{ transform: "scaleX(0)", transformOrigin: "right center", willChange: "transform"
15             },
16             background: "#00d4ff" }} />
17         </div>
18         <div className="flex-1 overflow-hidden">
19           /* right half — scales from the LEFT edge (toward center) */
20           <div ref={gyroTilt.rightBarRef} className="h-full w-full"
21             style={{ transform: "scaleX(0)", transformOrigin: "left center", willChange: "transform"
22             },
23             background: "#00d4ff" }} />
24         </div>
25       </div>
26     )}
```

The container is `height: "6.5px"` and full width. Its two halves start at `scaleX(0)` — invisible — and the render loop grows them. Crucially, the left half's `transformOrigin: "right center"` means it scales *from the right edge inward toward the center tick*; the right half uses `"left center"` so it grows the other way.

The `refs` (`leftBarRef`, `rightBarRef`) are the handles the render loop writes to directly, bypassing React for buttery updates. The whole layer is placed at `zIndex: 0`, while the button canvas is at `z-[5]` and the toggle HUD at `z-10`. So the moving bar is *behind* the buttons — which is exactly why every widget in `Widgets.tsx` draws an opaque black silhouette (`fill="#000"`) underneath its translucent body: without it, the cyan bar would shine through the ~88%-transparent button fills. There are no `boxShadow/blur` glows on the indicator for the same reason — a glow would bleed through the buttons and make the bar look like it was floating on top.

WATCH OUT

This design is exact and load-bearing: **6.5px** thick (65% of the old 10px), edge-to-edge, **no glow**, **z-0 behind the buttons**, `scaleX` driving the fills, growing from the outer edges toward the center tick. Do not re-center it, re-glow it, or move it above the buttons — earlier versions did each of those and each looked wrong against the semi-transparent widget fills.

The toggle itself is a normal `<button>` in the HUD layer that calls `setGyroOn(!gyroOn)` and, when on, shows a CALIBRATE button that snapshots the current tilt as "neutral." Because the HUD is `pointer-events-none` (so it never blocks the gamepad), the toggle re-enables pointer events on itself and calls `e.stopPropagation()` so a tap on it is never swallowed by the pad underneath.

Speaking to Kotlin: The Bridge and the 20-Byte Packet

The UI and the native Android layer talk through two channels. First, a **bridge object** Kotlin injects onto `window` as `AndroidBridge`, whose methods JavaScript calls — `setScreenOrientation`, `getGyroscopeDataJson`, `getNetworkTelemetryJson`, `triggerRumble`, `getSafeAreaTop`, `exitApp`. Second, the reverse: Kotlin calls *back into* JavaScript by invoking functions the UI parks on `window`, like `window.handleAndroidBack`, `window.onRumblePacket`, and `window.onUsbTetherChanged`.

DEFINITION

JS bridge — a shared object that lets the web page and the native app call each other's code. Kotlin's `WebView.addJavascriptInterface(...)` exposes `AndroidBridge`; the UI reads it as `(window as any).AndroidBridge`. The `as any` tells TypeScript "trust me, this exists at runtime" because native injects it.

The most important call is the one that sends your inputs to the PC: `window.sendGamepadPacket(buffer)`, a fast native function. Everything you touch is packed into a single **20-byte** binary message. The button layout is a bitmask:

apps/controller-ui/src/app/App.tsx · lines 329-345

TypeScript · React

```
1 const BTN_MAP: Record<string, number> = {
2   A: 1 << 0,
3   B: 1 << 1,
4   X: 1 << 2,
5   Y: 1 << 3,
6   LB: 1 << 4,
```

apps/controller-ui/src/app/App.tsx · (cont.)

TypeScript · React

```

7   RB: 1 << 5,
8   menu: 1 << 6,
9   view: 1 << 7,
10  lstick: 1 << 8,
11  rstick: 1 << 9,
12  up: 1 << 10,
13  down: 1 << 11,
14  left: 1 << 12,
15  right: 1 << 13,
16  home: 1 << 14, // → DS4 PS/Guide button (decoded server-side)
17 };

```

DEFINITION

bitmask — packing many yes/no flags into the bits of one number. $1 \ll 0$ is bit 0 (value 1), $1 \ll 5$ is bit 5 (value 32). To say "A and RB are held," you OR their bits together: $1 \mid 32 = 33$. The PC reads each bit back out. Sixteen bits hold all fifteen buttons with room to spare.

Now the packet builder. It writes each field at an exact byte offset into a reused 20-byte buffer:

apps/controller-ui/src/app/App.tsx · lines 851-869

TypeScript · React

```

1   const sendGamepadTelemetry = useCallback(() => {
2     const buffer = packetRef.current!.buffer;
3     const view = packetRef.current!.view;
4
5     // 1. Timestamp (64-bit uint, little-endian)
6     const now = Date.now();
7     view.setBigUint64(0, BigInt(now), true);
8
9     // 2. Button Bitmask (16-bit uint, little-endian)
10    // Read live held state from refs (updated synchronously on pointerdown/up)
11    // so both immediate sends and the keep-alive interval see current values.
12    let bitmask = 0;
13    const heldCustomSet = heldCustomRef.current;
14
15    heldCustomSet.forEach(id => {
16      const canonical = uidBitMap[id] ?? id;
17      const bit = BTN_MAP[canonical];
18      if (bit) bitmask |= bit;
19    });

```

DEFINITION

ArrayBuffer / DataView — an **ArrayBuffer** is a fixed block of raw bytes; a **DataView** is a lens for reading/writing typed values (8/16/32/64-bit) at chosen offsets inside it. `view.setBigUint64(0, ..., true)` writes an 8-byte number starting at byte 0.

DEFINITION

little-endian — the byte order where the least-significant byte comes first. The `true` argument to every `set...` call selects little-endian, which must match what the PC server expects.

The exact wire format is: bytes **0–7** the timestamp, bytes **8–9** the button bitmask, byte **10** LT (0–255), byte **11** RT, byte **12** left-stick X, byte **13** left-stick Y, byte **14** right-stick X, byte **15** right-stick Y, and bytes **16–19** a 32-bit auth token. The builder reads live values from *refs* (`heldCustomRef`, `ltFillRef`, `rstick.posRef`) — never from React state — so a press packed here is always current. `uidBitMap[id] ?? id` translates a custom widget's unique id into its canonical button name before looking up the bit.

Bytes	Field	Written by
0–7	timestamp (ms)	<code>setBigUint64(0, ..., true)</code>
8–9	button bitmask	<code>setUint16(8, ..., true)</code>
10 / 11	LT / RT	<code>setUint8(10/11, ...)</code>
12–15	LS X, LS Y, RS X, RS Y	<code>setUint8(12–15, ...)</code>
16–19	auth token	<code>setUint32(16, 0, true)</code>

The analog fields get real math. Triggers apply a 15% "hair-trigger" rescale (`activeThreshold = 0.85`, so `finalLtFill = ltFill / 0.85` clamped to 1) then map 0–1 onto 0–255. Sticks are centered on **128** via `axisByte(norm) = 128 + norm*127`, so a resting stick sends exactly 128 and the PC sees no phantom drift. Interestingly, the token at bytes 16–19 is written as **0** here — a comment explains the native UDP layer *overwrites* offset 16 with the real pairing-key hash before every send, so the JS deliberately ships an honest zero rather than a fake constant. Finally the send: if the USB-debug WebSocket is open, `usbWS.send(buffer)`; otherwise `window.sendGamepadPacket(buffer)` (the native JSI/UDP path). A `setInterval(..., 16)` heartbeat re-sends the current state ~60×/sec so a single dropped UDP packet self-heals.

NOTE

Rumble flows the *other* direction. The PC sends a rumble packet; native code calls `window.onRumblePacket(left, right, ...)`, and the UI responds with `bridge.triggerRumble(...)` to buzz the phone. Over Wi-Fi the UI polls `bridge.getRumbleState()` ("seq:left:right") at ~30 Hz and fires only when the sequence advances.

Building and Shipping the UI into the APK

You have four screens, dozens of components, and a hundred npm libraries. The phone must receive *one* self-contained web page. That flattening is Vite's job, configured in `vite.config.ts`:

apps/controller-ui/vite.config.ts · lines 20-30

TypeScript

```

1 export default defineConfig(({command}) => ({
2   base: command === 'serve' ? '/' : './',
3   plugins: [
4     figmaAssetResolver(),
5     // Only use viteSingleFile for production builds, not dev server
6     ...(command === 'build' ? [viteSingleFile()] : []),
7     // The React and Tailwind plugins are both required for Make, even if
8     // Tailwind is not being actively used - do not remove them
9     react(),
10    tailwindcss(),
11  ],

```

DEFINITION

bundler — a tool that follows every `import` from `main.tsx` outward, gathers all the code and CSS, strips the TypeScript types, and emits browser-ready output. Vite is the bundler.

DEFINITION

`viteSingleFile` — a plugin that inlines *all* JavaScript and CSS directly into `index.html`, producing a single file with no external requests. The conditional `command === 'build'` means it runs only for `npm run build`, never the dev server.

`base: command === 'serve' ? '/' : './'` uses absolute paths while developing but *relative* paths (`./`) in the build — essential, because inside the APK the page loads from a local file, not a web root. During development, the `server` block (not shown) binds Vite to `host: '0.0.0.0'` on port **5173**, so your phone on the same Wi-Fi can load the live-reloading page straight off your PC. For release, `npm run build` emits everything into a `dist/` folder as one inlined `index.html`.

That `dist/` folder is the handoff point to Android, defined in `capacitor.config.ts`. Capacitor's config sets `webDir: 'dist'`, `appId: 'app.controller.ui'`, and `appName: 'controller-ui'`. The Capacitor CLI copies `dist/` into the Android project's assets so the built-in WebView loads it at startup — the same page you developed, now frozen inside the APK.

DEFINITION

Capacitor — the native runtime (from the Ionic team) that wraps a web build into a real iOS/Android app. It provides the WebView host and the plumbing for native plugins. Your `AndroidBridge` and `sendGamepadPacket` are supplied by *custom* native code layered on top of that host.

IN PLAIN TERMS

Building for release is like mailing a pop-up book. In your workshop (dev server) the pages are loose and you re-glue them constantly. To ship, `viteSingleFile` presses the whole book into one sheet (`index.html`), Capacitor slips that sheet into the picture frame (the Android WebView), and the frame is nailed into the APK. The reader opens the app and the book pops up — no internet, no loose pages.

So the full life of a button press: your thumb lands on an SVG `<circle>`, `onPointerDown` captures the finger and calls `dn(id)`, `dn` flips a bit in `heldRef` and immediately runs `sendGamepadTelemetry`, which packs 20 bytes and calls `window.sendGamepadPacket` across the bridge into native code, which encrypts and fires it over UDP to your PC — all before React has repainted the button's green glow. Every layer in this chapter exists to keep that path as short as physically possible.

CHAPTER 4

The Android Shell — Kotlin, WebView, and the Bridge

4



Figure 4.1 — Web UI on WebView on native engine on OS transport.

The Big Picture: A Native Shell Around a Web Brain

Open the GamepadOS app on your phone and you see one seamless controller. Under the hood, though, you built a *hybrid* app: a thin native Android layer written in Kotlin that owns the window, the sensors, the vibration motor, the camera, and the network engine — and a web app (your React UI) that lives inside it and draws every button you touch.

DEFINITION

Kotlin — The modern programming language Google recommends for Android apps. It compiles to the same bytecode as Java and can call Java, C++, and Android system services directly.

DEFINITION

APK — Android Package — the single `.apk` file that contains your compiled code, images, web assets, and native libraries. Installing an app means copying and registering an APK.

DEFINITION

WebView — A built-in Android component that renders web pages (HTML/CSS/JavaScript) inside a native app, using the same engine as Chrome — but embedded in *your* window, under *your* control.

DEFINITION

Activity — Android's word for "one screen of an app." An Activity owns a window, receives lifecycle callbacks (created, resumed, paused, destroyed), and hosts your views. GamepadOS has exactly one: `MainActivity`.

IN PLAIN TERMS

Think of the app as a theater. Kotlin built the building: the walls (window), the electricity (sensors, vibration, networking), and the stage machinery (the C++ engine in the basement). The React web app is the actor performing on stage. The audience (you) only watches the actor — but every light, sound, and trapdoor the actor triggers is real building infrastructure, requested through an intercom called `AndroidBridge`.

Three layers cooperate on every frame of gameplay: the **WebView** (JavaScript UI) reads your touches, the **Kotlin shell** (`MainActivity.kt`, about 1,570 lines) brokers requests and owns hardware, and the **C++ engine** (`gamepad-engine.cpp`, covered in its own chapter) blasts 20-byte input packets over the network a thousand times per second. This chapter is about the middle layer.

Before diving into `MainActivity`, meet the other Kotlin class the manifest names — the Application class:

```
apps/android-client/app/src/main/java/com/gamepad/client/GamepadApplication.kt · lines 5-11 Kotlin
1 class GamepadApplication : Application() {
2     override fun onCreate() {
3         super.onCreate()
4         // (Charge-bypass failsafe removed in v1.0 – was Shizuku-only. The default
5         //   uncaught-exception handler is left untouched.)
6     }
7 }
```

This is deliberately almost empty. An `Application` object is created once per process, before any Activity, and is the traditional home for global initialization. Yours used to install a charge-bypass failsafe driven by Shizuku (a root-like privilege tool), but that entire feature was cut for v1.0, leaving just a call to the parent class. It still must exist because `AndroidManifest.xml` declares `android:name=".GamepadApplication"` — deleting the class without editing the manifest would crash the app at launch.

Waking Up: onCreate, Fullscreen, and a Screen That Never Sleeps

DEFINITION

Lifecycle — The fixed sequence of callbacks Android fires as your Activity is born, shown, hidden, and killed: `onCreate` → `onResume` → `onPause` → `onDestroy` (plus a few others). You never call these yourself; the system does.

Everything starts in `onCreate`. The very first things your app does are about the *window* — the physical rectangle of glass — because a game controller has unusual demands: it must never sleep, never show status bars mid-game, and should render as fast as the panel allows.

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 228-249 Kotlin
1      if (android.os.Build.VERSION.SDK_INT >= android.os.Build.VERSION_CODES.P) {
2          window.attributes.layoutInDisplayCutoutMode = android.view.WindowManager.LayoutParams.LAYOUT_IN
   _DISPLAY_CUTOUT_MODE_SHORT_EDGES
3      }
4
5      // 1. OLED Blackout & Thermal Optimization
6      window.addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)
7      window.decorView.setBackgroundColor(Color.BLACK)
8      WindowCompat.setDecorFitsSystemWindows(window, false)
9
10     // Render at the display's MAX refresh rate (e.g. 120Hz) instead of the default
11     // 60Hz, so the WebView UI + animations can be smooth on high-refresh phones.
12     // Picks the highest-rate supported mode; harmless/no-op on 60Hz-only devices.
13     try {
14         val disp = if (android.os.Build.VERSION.SDK_INT >= android.os.Build.VERSION_CODES.R) display
15                     else @Suppress("DEPRECATION") windowManager.defaultDisplay
16         disp?.supportedModes?.maxByOrNull { it.refreshRate }?.let { best ->
17             val lp = window.attributes
18             lp.preferredDisplayModeId = best.modeId
19             lp.preferredRefreshRate = best.refreshRate
20             window.attributes = lp
21         }
22     } catch (e: Exception) {}
```

Four decisions live here. First, on Android 9+ (`VERSION_CODES.P`) the layout is allowed to extend into the camera notch (`SHORT_EDGES`), so the UI is truly edge-to-edge. Second, `FLAG_KEEP_SCREEN_ON` tells the system "while this window is visible, never dim or lock" — this is your wake lock, and because it is a window flag rather than a manually acquired lock, Android releases it automatically the moment the app leaves the screen (no leaked battery drain). Third, the root view is painted pure black: on an OLED panel, black pixels are literally *off*, saving power and heat during long sessions. Fourth, the code scans every display mode the panel supports, picks the one with the highest refresh rate (120 Hz on your POCO), and requests it — so the WebView animates at panel speed instead of the 60 Hz default.

DEFINITION

Wake lock — A request to the operating system to keep hardware (screen, CPU, Wi-Fi radio) awake when it would normally power down. GamepadOS uses three kinds: the screen flag above, a Wi-Fi low-latency lock, and a multicast lock.

DEFINITION

Immersive mode — A fullscreen state where Android hides the status bar and navigation bar entirely; a swipe from the edge shows them temporarily. GamepadOS enters it via `hideSystemUI()` when a session connects.

`onCreate` also registers a back-button handler with a subtle rule: it first checks whether the on-screen keyboard is open (via `WindowInsetsCompat.Type.ime()`); if so, back just hides the keyboard. Otherwise it forwards the event *into JavaScript* — `window.handleAndroidBack()` — so the React app can pop its own navigation stack, falling back to `AndroidBridge.exitApp()` when there is nothing left to pop. The native layer never guesses what "back" means; the web brain decides.

`hideSystemUI()` (called on connect and whenever the orientation flips to landscape) combines the legacy `SYSTEM_UI_FLAG_IMMERSIVE_STICKY` flag set with the modern `WindowInsetsControllerCompat.hide(...)` API, and sets `BEHAVIOR_SHOW_TRANSIENT_BARS_BY_SWIPE` so the bars only peek back on a deliberate swipe. `showSystemUI()` reverses it when you return to the portrait dashboard.

NOTE

The manifest sets `android:windowSoftInputMode="adjustNothing"`, meaning the window never resizes or pans when a keyboard opens. That keeps the immersive layout rock-solid; text entry instead happens in a *native* dialog (`showTextInput`), completely outside the WebView.

The Layer Cake: SurfaceView, Camera, and the WebView

`setupHybridLayout()` stacks three views inside one `FrameLayout`, bottom to top: a transparent `SurfaceView` (a raw pixel canvas the C++ engine can draw to directly, via `initGameplaySurface`), a `CameraX PreviewView` (invisible until you scan a QR code), and the `WebView` on top.

DEFINITION

SurfaceView — A special Android view backed by a raw graphics surface that non-Java code (like your C++ engine) can render into directly, bypassing the normal UI pipeline.

The `WebView` is where the configuration gets opinionated:

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 311-329 Kotlin
1      val webViewInstance = android.webkit.WebView(this).apply {
2          setBackgroundColor(Color.TRANSPARENT)
3          setLayerType(View.LAYER_TYPE_HARDWARE, null) // Hardware acceleration for ultra-low latency rendering
4
5          settings.apply {
6              javaScriptEnabled = true
7              domStorageEnabled = true
8              allowFileAccess = true
9              allowContentAccess = true
10             databaseEnabled = true

```

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · (cont.) Kotlin
11      mediaPlaybackRequiresUserGesture = false
12      cacheMode = android.webkit.WebSettings.LOAD_NO_CACHE
13
14      // Allow loading ES modules over file:/// protocol to bypass CORS policy blocks
15      allowUniversalAccessFromFileURLs = true
16      allowFileAccessFromFileURLs = true
17      // Allow the WebView to open ws://127.0.0.1 (USB-debugging transport)
18      mixedContentMode = android.webkit.WebSettings.MIXED_CONTENT_ALWAYS_ALLOW
19  }
```

The transparent background lets the SurfaceView show through beneath the UI. `LAYER_TYPE_HARDWARE` forces GPU compositing. JavaScript and DOM storage are enabled because the React app is the product. `LOAD_NO_CACHE` guarantees a fresh UI after every update. The two `FileURLs` switches exist because the production UI is loaded from `file:///android_asset/dist/index.html` — bundled files, not a web server — and browsers normally forbid `file:///` pages from importing modules or fetching anything (a security rule called CORS). Finally, `MIXED_CONTENT_ALWAYS_ALLOW` lets the page open a plain `ws://127.0.0.1` WebSocket, the USB-debugging transport.

WATCH OUT

`allowUniversalAccessFromFileURLs = true` makes this WebView *privileged*: any page loaded into it can read local files and call every bridge method. That is exactly why two guardrails exist. Chrome DevTools debugging (`setWebContentsDebuggingEnabled`) is wrapped in `if (BuildConfig.DEBUG)`, and the Vite dev-server probe (which checks `http://192.168.1.39:5173` with an 800 ms timeout and hot-reloads from it) also runs only in debug builds. A release APK loads *only* the bundled assets — otherwise a hostile machine on your LAN answering at that address could inject its own page into a WebView that can call `connectToPC`.

One more piece of layout plumbing: an insets listener measures the status bar, navigation bar, and notch in real pixels, converts them to density-independent pixels (defaulting to 44 top / 24 bottom when nothing is reported), and pushes them into CSS as `--android-safe-top`, `--android-safe-bottom`, `--android-safe-left`, and `--android-safe-right`. Your React styles read those variables so buttons never hide under a notch. The same values are re-injected on every page load in `onPageFinished`, alongside a tiny injected script that defines `window.sendGamepadPacket` — the JS-side half of the bridge you are about to meet.

The AndroidBridge: How JavaScript Calls Kotlin

DEFINITION

@JavascriptInterface — An annotation that marks a Kotlin/Java method as callable from JavaScript inside a WebView. `addJavascriptInterface(object, "AndroidBridge")` publishes all such methods on a global JS object named `AndroidBridge`.

The bridge is a single anonymous Kotlin object registered under the name `"AndroidBridge"`. From the React app's point of view, `window.AndroidBridge.triggerHaptic(20)` is just a function call; under the hood Android marshals it onto a WebView thread and runs your Kotlin. Around thirty methods are exposed. The hottest one carries controller input in the fallback (pre-native) path:

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 343-351 Kotlin
1      @android.webkit.JavascriptInterface
2      fun sendGamepadPacketNative(base64Data: String) {
3          try {
4              val bytes = android.util.Base64.decode(base64Data, android.util.Base64.NO_WRAP)
5              injectNativePayload(bytes)
6          } catch (e: Exception) {
7              android.util.Log.e("GamepadEngine", "Error decoding JSI gamepad telemetry packet",
8                  e)
9          }
10     }
```

DEFINITION

Base64 — A way to encode raw bytes as plain text (letters, digits, +, /) so they can travel through string-only channels. The bridge can only pass strings, so JS packs the 20-byte input frame into Base64 and Kotlin decodes it back.

The injected `window.sendGamepadPacket` helper converts a JS `ArrayBuffer` to Base64, this method decodes it, and `injectNativePayload` hands the bytes to C++. That is the whole input path when the native engine's own touch pipeline isn't driving.

The most consequential bridge method is `connectToPC(ip, port, key)` — the JS pairing flow calls it after a QR scan or a discovery hit:

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 382-398 Kotlin
1      if (isTransitioning.compareAndSet(false, true)) {
2          try {
3              isEngineRunning = true
4              serverIp = ip
5              serverPort = port
6              serverKey = key
7              // GRX (dormant unless GRX_ENABLED): derive PSK from the pairing
8              // key + start the handshake. start() must run AFTER the socket is
9              // up (initNetworkNative) when wiring — harmless while disabled.
10             grx = if (GRX_ENABLED && key != "usb") {
11                 try { GrxClient(GrxCrypto.pskFromPairingKey(key), GrxCrypto.GRX_LTID) {
12                     b -> nativeGrxSendRaw(b) } }
13                 catch (e: Exception) { null }
14             } else null
15             // Classify the link for the dashboard badge by the ACTUAL
16             // active interface (works on every OEM/subnet), not a
17             // hardcoded IP prefix. Explicit "usb" connect mode also counts.
18             connectionType = if (key == "usb" || isUsbTetherActiveInternal()) "wired" else "wireless"
```

DEFINITION

Atomic compare-and-set — An operation that checks a value and changes it in one indivisible step, so two threads can never both "win." `isTransitioning.compareAndSet(false, true)` means: if no session is starting or live, claim the slot; otherwise do nothing.

That guard is the scar tissue of a real incident. JavaScript retry loops call `connectToPC` repeatedly before the first acknowledgment arrives; an older version tore the session down and rebuilt it on every call,

causing connect/disconnect flapping (this is the exact behavior the "Antigravity" incident broke). Now a second call while connected is a harmless no-op — the flag is only released by `stopEngine()`, `cleanupSessionAndReturn()`, or `onDestroy()`. Inside the guard the method caches the server address, builds a `GrxClient` (unless the key is the literal string `"usb"`), classifies the link as `"wired"` or `"wireless"` by inspecting real network interfaces, then acquires two radio locks, calls `initNetworkNative(ip, port, key)` to start the C++ engine, starts the GRX handshake, and enters immersive mode.

The two locks deserve names. A `MulticastLock ("GamepadOSBroadcastLock")` lets the app receive UDP broadcast/multicast packets, which many phones filter to save power — discovery needs it. A `WifiLock ("GamepadOSLowLatencyLock")` is the single biggest anti-jitter win in the whole app: it pins the Wi-Fi radio in `WIFI_MODE_FULL_LOW_LATENCY` (Android 10+, falling back to `WIFI_MODE_FULL_HIGH_PERF` on older phones), preventing the radio from napping between your tiny packets — naps that otherwise show up as 10–40 ms latency spikes.

Data flows the other way too, in two styles. **Polling:** JS periodically calls getters like `getNetworkTelemetryJson()`, which reports `packetCount`, a fixed `hz` of 1000, measured round-trip `latency`, and `linkAlive` — true only when the PC's last acknowledgment arrived within 2,500 ms (packet count alone can't prove a connection, because UDP sends succeed even with nobody listening). `getRumbleState()` returns `"seq:left:right"` unpacked from one native 64-bit value (sequence in the high bits, motor strengths in the low two bytes); JS fires the motor only when `seq` changes. `getGyroscopeDataJson()` returns filtered tilt plus an `age` field — milliseconds since the sensor event — so the UI can *prove* its readout is fresh. **Push:** Kotlin calls `webView?.evaluateJavascript(...)` to invoke JS globals such as `window.onQRScanned(payload)`, `window.onUsbTetherChanged(true)` (from a watcher that polls the USB interface every 1,500 ms), `window.onSessionExited()`, and the updater's `window.__onUpdateProgress(pct)`.

Bridge method	Direction	Purpose
<code>sendGamepadPacketNative(b64)</code>	JS → Kotlin → C++	Inject a 20-byte input frame
<code>connectToPC(ip, port, key)</code>	JS → Kotlin	Start engine + GRX + locks
<code>getNetworkTelemetryJson()</code>	JS polls Kotlin	Latency, linkAlive, packet count
<code>evaluateJavascript("window.onQRScanned...")</code>	Kotlin → JS	Push events into React

EXPERT DETAIL

Every string Kotlin interpolates into `evaluateJavascript` is a code-injection surface. The QR path escapes backslashes *first*, then quotes, then newlines — in that order — before building `window.onQRScanned('...')`; `showTextInput` and `postUpdateStatus` do the same. Reversing the order (quotes before backslashes) would let a payload containing `\'` break out of the JS string literal.

Into C++: JNI and the Native Engine

DEFINITION

JNI — Java Native Interface — the mechanism that lets Kotlin/Java call functions compiled from C or C++ (and lets C++ call back up). Kotlin declares a function `external`; the implementation lives in a native library loaded at runtime.

The native library is loaded once, when the class is first touched, by `init { System.loadLibrary("gamepad_engine") }` in MainActivity's companion object. The name matches `project("gamepad_engine")` in `src/main/cpp/CMakeLists.txt`, which compiles a single file, `gamepad-engine.cpp`, and links the `android` and `log` system libraries. Here is the entire Kotlin-facing surface of the engine:

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 40-50 Kotlin
1  private external fun initGameplaySurface(surface: Surface, width: Int, height: Int)
2  private external fun destroyGameplaySurface()
3  private external fun injectNativePayload(data: ByteArray)
4  private external fun initNetworkNative(ip: String, port: Int, key: String)
5  private external fun stopNetworkNative()
6  // AOA direct-USB: hand the opened accessory's raw fd to the native engine.
7  private external fun initAccessoryNative(fd: Int)
8  private external fun getNativePacketCount(): Long
9  private external fun getNativeLatencyMs(): Float
10 private external fun getMsSinceLastAck(): Long
11 private external fun getNativeRumble(): Long
```

Read these as the engine's control panel: give it a drawing surface (wired to `surfaceChanged`, torn down in `surfaceDestroyed`), feed it input bytes, start or stop the UDP network loop, or — for the AOA wired path — hand it a raw file descriptor. The four getters are how all telemetry escapes C++ without callbacks: cheap, thread-safe reads of counters the hot loop maintains.

DEFINITION

File descriptor (fd) — A small integer the operating system uses as a handle to an open file, socket, or device. `initAccessoryNative(fd)` passes the opened USB accessory's handle so C++ can `read()/write()` it directly.

DEFINITION

AOA — Android Open Accessory — a USB protocol where the PC declares itself an accessory to the phone. When the PC server performs the AOA handshake, Android launches your app with a `USB_ACCESSORY_ATTACHED` intent; the identity strings (manufacturer `GamepadOS`, model `GamepadController`, version `1.0` in `res/xml/accessory_filter.xml`) must match what `apps/pc-server/aoa_transport.py` sends, or Android routes the accessory elsewhere. `openAccessoryAndStart` then calls `pfd.detachFd()` — transferring ownership of the descriptor to C++, which closes it in `stopNetworkNative`.

Two further `external` functions belong to GRX: `nativeGrxSendRaw(bytes)` (Kotlin → C++: transmit raw bytes on the engine's UDP socket) and `nativeSetGrxReady(ready)` (flip the C++ hot path into seal-everything mode). And uniquely, C++ calls *up* into Kotlin: the transmit path calls

`grxSeal(frame20)` on every frame, and the receive path calls `onGrxControl(frame)` whenever a packet's first byte is `0xE1`, `0xE2`, or `0xE3` — the GRX control-frame markers. The object that answers these calls:

```
apps/android-client/app/src/main/java/com/gamepad/client/GrxCliient.kt · lines 50-63 Kotlin
1  fun onServerMessage(frame: ByteArray): Boolean {
2      if (frame.isEmpty()) return false
3      val t = frame[0].toInt() and 0xFF
4      if (t == GrxCrypto.T_SHELL0 && !session.established) {
5          val confirm = session.handleServerHello(frame)
6          if (confirm != null) send(confirm) // handshake complete; input may now flow
7          return true // consumed regardless (don't treat as data)
8      }
9      return false
10 }
11
12 /** Seal a 20-byte input frame -> 41-byte GRX wire frame, or null if not yet established. */
13 fun seal(frame20: ByteArray): ByteArray? =
14     if (session.established) session.seal(frame20) else null
```

`GrxCliient` is a small conductor around `GrxCrypto.ClientSession` (the actual math lives in the GRX chapter). `start()` sends a `CLIENT_HELLO` through the `send` callback — which is `nativeGrxSendRaw`, so handshake packets ride the engine's existing socket. When the server's `SERVER_HELLO` arrives, `onServerMessage` verifies it, auto-sends `CLIENT_CONFIRM`, and flips `established`; `MainActivity.onGrxControl` then calls `nativeSetGrxReady(true)`, and from that moment the C++ hot path routes every 20-byte frame through `seal()`, growing it to a 41-byte encrypted wire frame. Against an old, GRX-unaware server the handshake simply never completes and the legacy cleartext path keeps working — graceful fallback with `GRX_ENABLED = true` shipped live in 1.3.0.

Feeling and Sensing: Gyro Steering and Haptics

DEFINITION

Sensor fusion — Combining multiple physical sensors (gyroscope for rotation speed, accelerometer for gravity direction) into one clean estimate of the phone's orientation. Android exposes the fused result as a "rotation vector" sensor.

Steering does not read the raw gyroscope. `MainActivity` registers for `TYPE_GAME_ROTATION_VECTOR` (gyro + accelerometer, no magnetometer, so no compass drift), falling back to `TYPE_ROTATION_VECTOR` on devices without it. Registration happens in `onResume` at `SENSOR_DELAY_FASTEST` — up to 200 Hz — and, crucially, events are delivered on a dedicated `HandlerThread` named "gyro-sensor" so the fusion math never competes with WebView UI work on the main thread (that contention was a real source of jitter). `onPause` unregisters everything. Each event runs this pipeline:

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 1246-1270 Kotlin
1  // Fused attitude → rotation matrix (device→world).
2  android.hardware.SensorManager.getRotationMatrixFromVector(fusionRotMatrix, event.values)
3
4  // Remap into a consistent LANDSCAPE-VIEW frame so the zero + axes are identical in
```



```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · (cont.) Kotlin
5      // either landscape orientation (app is SENSOR_LANDSCAPE → ROTATION_90 or _270).
6      val (axisX, axisY) = if (currentDisplayRotation() == android.view.Surface.ROTATION_270)
7          android.hardware.SensorManager.AXIS_MINUS_Y to android.hardware.SensorManager.AXIS_X
8      else
9          android.hardware.SensorManager.AXIS_Y to android.hardware.SensorManager.AXIS_MINUS_X
10     android.hardware.SensorManager.remapCoordinateSystem(fusionRotMatrix, axisX, axisY, fusionRemapped)
11
12     // World-up in the remapped view frame (3rd row). Level phone (screen vertical,
13     // facing you) → up ≈ (0, 1, 0).
14     val ux = fusionRemapped[6]
15     val uy = fusionRemapped[7]
16     val uz = fusionRemapped[8].coerceIn(-1f, 1f)
17     // ROLL = angle of up WITHIN the screen plane = left/right steering. Both legs
18     // (ux,uy) stay large in play holds → stable; ratio is preserved as you lean
19     // forward/back → decoupled from pitch; no gimbal-lock except screen-flat.
20     val rollDeg = STEER_SIGN * Math.toDegrees(Math.atan2(ux.toDouble(), uy.toDouble()))
21     // PITCH = how far up tips OUT of the screen plane = forward/back lean (3D look Y).
22     val pitchDeg = PITCH_SIGN * Math.toDegrees(Math.asin(uz.toDouble()))
23     fusedRollDeg = rollFilter.filter(rollDeg, event.timestamp).toFloat()
24     fusedPitchDeg = pitchFilter.filter(pitchDeg, event.timestamp).toFloat()
25     lastSensorEventNs = event.timestamp
```

Step by step: the fused orientation becomes a 3×3 rotation matrix; the matrix is remapped so "up" means the same thing whether you rotated the phone left or right into landscape (`ROTATION_90` vs `ROTATION_270` get mirrored axis pairs); the third row of the remapped matrix — indices 6, 7, 8 — is the world's up direction expressed in screen coordinates. Roll (steering) is the angle of that up-vector *within* the screen plane, computed with `atan2(ux, uy)`; pitch (forward/back lean, used for 3D-mode look) is how far up tips *out* of the plane, via `asin(uz)`. Because both legs of the `atan2` stay large in any realistic grip, steering is stable at any hold angle — flat on a table, reclined, or vertical — with the only degenerate pose being screen-facing-straight-up, which nobody plays in. `STEER_SIGN` (+1) and `PITCH_SIGN` (−1) are one-line polarity switches if an axis ever reads backwards on a new device.

DEFINITION

One Euro filter — An adaptive smoothing filter (Casiez, Roussel & Vogel, CHI 2012) whose strength changes with signal speed: strong smoothing when your hand is still (kills jitter), almost none during fast motion (no lag). A fixed filter must pick one or the other.

Both axes pass through a `OneEuroFilter` tuned at `minCutoff = 2.8`, `beta = 0.5` — a deliberately latency-biased tune. The results land in `@Volatile` fields so the UI thread's bridge reads (`getGyroscopeDataJson`) are safe without locks.

Haptics is the other physical channel, and it is capability-tiered because phone vibration motors differ wildly:

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 687-702 Kotlin
1      } else if (amplitude in 1..254) {
2          // No amplitude control (e.g. Moto G84): every custom amplitude silently
3          // collapses to FULL strength, so weak rumble feels as strong as strong
4          // rumble — and under sustained rumble the back-to-back full pulses read
5          // as one continuous buzz. Emulate intensity with a duty-cycle (PWM)
6          // waveform: on-time per ~16ms cycle is proportional to the requested
```

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · (cont.) Kotlin
7          // amplitude, so a low value pulses briefly (gentle) and a high value
8          // stays on longer (strong).
9          val period = 16L
10         val on = (period * amplitude / 255).coerceIn(1, period)
11         val off = (period - on).coerceAtLeast(0)
12         val cycles = (durationMs / period).toInt().coerceIn(1, 8)
13         val timings = LongArray(cycles * 2 + 1) { i ->
14             if (i == 0) 0L else if (i % 2 == 1) on else off
15         }
16         vibrator.vibrate(android.os.VibrationEffect.createWaveform(timings, -1))
```

DEFINITION

PWM — Pulse-width modulation — faking a "50% strength" output from an on/off-only device by switching it on for 50% of each tiny cycle. Here the cycle is 16 ms and the on-time scales with the requested amplitude (1–255).

This branch of `doVibrate` handles motors *without* amplitude control (your Moto G84 test device): such motors silently play every custom amplitude at full blast, so game rumble at strength 40 would feel identical to 255. The fix builds an alternating on/off waveform — at most 8 cycles — whose duty cycle encodes intensity. Motors *with* amplitude control (your POCO) take the simpler branch above: a single `createOneShot` with the real amplitude. On top of this sits `playHaptic(event)`, the semantic layer: button presses map to the OEM-tuned `EFFECT_CLICK`, releases to `EFFECT_TICK`, trigger pulls to `EFFECT_HEAVY_CLICK`, played with `USAGE_TOUCH` attributes on Android 13+ so they scale with the user's system haptic-strength setting. And there is one more device-specific trick: on no-amplitude motors, a press-click would be *cancelled* by the release-tick ~45 ms later (the motor plays one effect at a time), so press-like events instead play an atomic double-pulse waveform `longArrayOf(0, 16, 30, 16)` that cannot be superseded mid-play.

Permissions, Packaging, and the Build

DEFINITION

Manifest — `AndroidManifest.xml` — the app's identity card. It declares the package name, every permission, every Activity, and hardware features, all read by the system at install time.

DEFINITION

Permission — A capability the app must declare (and sometimes ask the user for at runtime) before the OS allows it. Undeclared calls throw `SecurityException`.

```
apps/android-client/app/src/main/AndroidManifest.xml · lines 4-18 XML
1     <uses-permission android:name="android.permission.INTERNET" />
2     <uses-permission android:name="android.permission.CAMERA" />
3     <uses-permission android:name="android.permission.VIBRATE" />
4     <uses-permission android:name="android.permission.HIGH_SAMPLING_RATE_SENSORS" />
5     <uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />
```

apps/android-client/app/src/main/AndroidManifest.xml · (cont.)

XML

```
6     <uses-permission android:name="android.permission.CHANGE_WIFI_MULTICAST_STATE" />
7     <uses-permission android:name="android.permission.WAKE_LOCK" />
8     <uses-permission android:name="android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS" />
9     <!-- In-app self-update: download the new APK and hand it to the system installer. -->
10    <uses-permission android:name="android.permission.REQUEST_INSTALL_PACKAGES" />
11
12    <uses-feature android:name="android.hardware.camera" android:required="false" />
13    <uses-feature android:name="android.hardware.sensor.gyroscope" android:required="false" />
14    <!-- AOA direct-USB transport (optional – Wi-Fi/tether still work without it). -->
15    <uses-feature android:name="android.hardware.usb.accessory" android:required="false" />
```

Each line earns its place: INTERNET for the UDP engine and updater downloads; CAMERA for QR pairing (the only *runtime* permission — the user sees a dialog, handled by `onRequestPermissionsResult`); VIBRATE for all haptics; HIGH_SAMPLING_RATE_SENSORS because Android 12+ caps undeclared apps at 200 Hz sensor delivery — exactly the rate your steering wants; ACCESS_WIFI_STATE for the band/RSSI diagnostics in `getWifiInfoJson()`; CHANGE_WIFI_MULTICAST_STATE for the MulticastLock; WAKE_LOCK for the WifiLock; REQUEST_IGNORE_BATTERY_OPTIMIZATIONS so `requestBatteryExemption()` can open the whitelist dialog (the top fix for MIUI/HyperOS radio throttling); and REQUEST_INSTALL_PACKAGES for the one-click updater, which downloads the new APK to `cacheDir/updates/update.apk` in 64 KiB chunks, verifies its SHA-256 against the manifest hash, and hands it to the installer via the declared `FileProvider` (`content://` URIs are mandatory on Android 7+; `file://` is blocked). The three `uses-feature` entries are all `required="false"` so the Play Store never hides the app from a phone lacking a camera, gyroscope, or USB accessory support — the app degrades gracefully instead. `android:usesCleartextTraffic="true"` is required because everything on the LAN — discovery, dev server, WebSocket — is plain HTTP/UDP, not HTTPS.

Now the build recipe:

apps/android-client/app/build.gradle.kts · lines 19-42

Kotlin

```
1     defaultConfig {
2         applicationId = "com.gamepad.client"
3         minSdk = 24
4         targetSdk = 34
5         versionCode = 22
6         versionName = "1.3.0"
7
8         ndk {
9             // Broad device coverage (universal APK for sideload + in-app updater):
10            // arm64-v8a – all modern phones/tablets (primary)
11            // armeabi-v7a – older / budget 32-bit ARM phones
12            // x86_64 – Android emulators, Chromebooks, x86 tablets
13            // (32-bit x86 omitted: effectively dead, only adds size.)
14            abiFilters.add("arm64-v8a")
15            abiFilters.add("armeabi-v7a")
16            abiFilters.add("x86_64")
17        }
18
19        externalNativeBuild {
```

apps/android-client/app/build.gradle.kts · (cont.)

Kotlin

```

20         cmake {
21             cppFlags += listOf("-O3", "-flto", "-ffast-math")
22         }
23     }
24 }

```

DEFINITION

Gradle — The build system that compiles Kotlin, runs the NDK/CMake native build, packages assets, and signs the APK. `build.gradle.kts` is its Kotlin-syntax configuration file.

DEFINITION

ABI — Application Binary Interface — the CPU architecture a native library is compiled for. Kotlin bytecode runs anywhere, but `libgamepad_engine.so` must be compiled per architecture.

DEFINITION

versionCode / versionName — The integer Android compares to decide "is this an upgrade?" (22) and the human-readable string users see (1.3.0). Every release must increment the code.

`minSdk = 24` means Android 7.0 is the floor (that is why the code is littered with `SDK_INT` checks for features from 26, 29, 30, 31, and 33); `targetSdk/compileSdk 34` means it is built against Android 14 rules. The `abiFilters` block is the "multi-ABI" decision: rather than shipping separate per-architecture APK *splits*, you build **one universal APK** containing all three engine builds — arm64-v8a, armeabi-v7a, and x86_64 — because sideloading and the in-app updater need a single file that installs anywhere. (If any older note describes ABI *splits*, the code wins: it is `abiFilters` in one APK.) The C++ flags `-O3 -flto -ffast-math` compile the engine at maximum optimization. Elsewhere in the file: the release build type enables `isMinifyEnabled` and `isShrinkResources` (ProGuard shrinking — dead code and unused resources stripped, guided by `proguard-rules.pro`), and signing reads `release.keystore` with the password, alias, and key password pulled from `local.properties` — so no secret ever sits in the repo.

DEFINITION

Keystore / signing — Every APK must be cryptographically signed. Android only installs an update if its signature matches the installed app's, so `release.keystore` is your identity — lose it and you can never update existing installs.

Dependencies are lean by design: AndroidX core/appcompat/material, four CameraX 1.3.1 artifacts for the QR scanner, ZXing core 3.5.3 for offline QR decoding (chosen over ML Kit's ~4.9 MB bundle), and Google Tink 1.13.0 supplying X25519 + HKDF for the GRX handshake.

Finally, the one-command build:

apps/android-client/build_apk.bat · lines 1-7

bat

```
1 @echo off
2 REM Build tools now live in F:\hlooo\tools\ (moved out of apps\).
3 set "JAVA_HOME=F:\hlooo\tools\jdk\jdk-17.0.19+10"
4 set "ANDROID_HOME=F:\Android\Sdk"
5 set "PATH=%JAVA_HOME%\bin;%PATH%"
6 echo Starting Gradle build...
7 F:\hlooo\tools\gradle-8.5\bin\gradle.bat clean assembleRelease
```

The script is deliberately self-contained: it points `JAVA_HOME` at the JDK 17.0.19+10 bundled in `F:\hlooo\tools\` (Gradle 8.5 and AGP require Java 17), points `ANDROID_HOME` at the SDK on `F:\Android\Sdk`, then runs the repo's own Gradle 8.5 with `clean assembleRelease`. `clean` deletes previous outputs so nothing stale leaks into a release; `assembleRelease` compiles Kotlin, drives CMake 3.22.1 across all three ABIs, packages `app/src/main/assets/dist` (the pre-built React bundle) into the APK, runs ProGuard, and signs with the release keystore. The finished, installable file lands at `app/build/outputs/apk/release/app-release.apk` — the exact artifact (versionCode 22, versionName 1.3.0) that your release pipeline hashes, publishes, and that `beginApkUpdate` later downloads onto users' phones.

NOTE

The React UI must be built *before* the APK: Gradle does not run Vite. The web bundle is copied into `assets/dist` ahead of time, and `assembleRelease` just packages whatever is sitting there — a stale `dist` silently ships a stale UI.

CHAPTER 5

The C++ Engine — gamepad-engine.cpp

5

Why a C++ Engine Sits at the Bottom of the Stack

Every time your thumb slides across the phone screen, a chain of software wakes up: your React UI notices the touch, packs the new controller state into a tiny block of memory, and hands it down to a piece of code written in C++ whose only job is to shove that block onto the network and out to your PC. That C++ code is one file — `gamepad-engine.cpp` — and this chapter is about everything it does.

The natural question is: why C++ at all? Your UI is already written in TypeScript, and the Android shell around it is Kotlin. Why drop down to a harder, older, more dangerous language for the last few inches of the journey?

DEFINITION

Latency — The delay between a cause and its effect — here, the time between your finger moving and the game reacting. Measured in milliseconds (ms); lower is better. A wired Xbox pad is about 4-8 ms.

DEFINITION

Garbage collection (GC) — An automatic memory-cleanup process that languages like Java, Kotlin, and JavaScript run periodically. It is convenient, but it can *pause your whole program* for a few milliseconds at unpredictable moments to tidy up unused memory.

The reason is that GC pause. If the code sending your input were written in Kotlin, then every so often — with no warning — the runtime would freeze your send loop for a few milliseconds to collect garbage. In a word processor nobody notices. In a game controller pushing packets a thousand times a second, that freeze shows up as a stutter in your aim. The project's own design notes say this outright: *"Java/Kotlin have GC pauses that cause packet jitter. The C++ thread bypasses this entirely."*

DEFINITION

NDK (Native Development Kit) — Android's toolkit for writing parts of an app in C or C++ that compile to raw machine code, instead of running on the Java/Kotlin virtual machine. "Native" means it runs directly on the phone's processor.

DEFINITION

UDP (User Datagram Protocol) — A way of sending small packets of data over a network with no guarantee they arrive, in order, or at all — but with almost no overhead. The opposite of TCP, which guarantees delivery but adds delay.

IN PLAIN TERMS

Think of your whole app as a mail room. The React UI is the person who writes the letter. Kotlin is the clerk who walks it downstairs. But the actual mail carrier — the one who has to sprint out the door the instant a letter lands, rain or shine, a thousand times a second — that is the C++ engine. You want your fastest, most single-minded runner in that job, not the clerk who occasionally stops to reorganize the whole filing cabinet.

The engine is compiled into a shared library — a `.so` file — that the Kotlin side loads at startup. The build recipe for it is astonishingly small:

```
apps/android-client/app/src/main/cpp/CMakeLists.txt · lines 1-11 CMake

1 cmake_minimum_required(VERSION 3.22.1)
2 project("gamepad_engine")
3
4 add_library(${CMAKE_PROJECT_NAME} SHARED gamepad-engine.cpp)
5
6 # Strictly link Android Input and Logging libraries
7 target_link_libraries(${CMAKE_PROJECT_NAME}
8     android
9     log
10     # Oboe or AAudio goes here later when we add the Rx Rumble Thread
11 )
```

DEFINITION

CMake — A build-system generator. You describe *what* to build in a `CMakeLists.txt` file, and CMake figures out the exact compiler commands for each target platform.

This file says three things. `project("gamepad_engine")` names the library, which is why the Kotlin side later calls `System.loadLibrary("gamepad_engine")`. The `add_library(... SHARED gamepad-engine.cpp)` line compiles your one source file into a *shared* library (a `.so` loaded at runtime, versus a static one baked into the app binary). And `target_link_libraries` wires in exactly two Android system libraries: `android` (for platform APIs) and `log` (so your `LOGI/LOGE` messages show up in logcat). The comment about "Oboe or AAudio" is a leftover note — no audio library is actually linked, because rumble ended up handled a different way, as you will see.

The JNI Bridge — How Kotlin Calls C++

Kotlin and C++ do not speak the same language natively. The translator between them is JNI.

DEFINITION

JNI (Java Native Interface) — The standard mechanism that lets Java/Kotlin code call C/C++ functions and vice-versa. It defines a strict naming convention so the runtime can find the right native function for a given Kotlin method.

On the Kotlin side, you declare a method as `external` — meaning "the body of this lives in native code, go find it." Here are the engine's entry points, exactly as the Android shell declares them:

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 40-50 Kotlin
1  private external fun initGameplaySurface(surface: Surface, width: Int, height: Int)
2  private external fun destroyGameplaySurface()
3  private external fun injectNativePayload(data: ByteArray)
4  private external fun initNetworkNative(ip: String, port: Int, key: String)
5  private external fun stopNetworkNative()
6  // AOA direct-USB: hand the opened accessory's raw fd to the native engine.
7  private external fun initAccessoryNative(fd: Int)
8  private external fun getNativePacketCount(): Long
9  private external fun getNativeLatencyMs(): Float
10 private external fun getMsSinceLastAck(): Long
11 private external fun getNativeRumble(): Long
```

Each of these has a matching C++ function with a very specific name. The rule: `Java_` + the fully-qualified class name with dots turned into underscores + `_` + the method name. So `MainActivity.injectNativePayload` becomes the C++ function `Java_com_gamepad_client_MainActivity_injectNativePayload`. The runtime uses that name to link the two sides. This is the single most important function in the file — it is how a fresh controller state gets from the UI into the engine:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 611-629 C++
1  extern "C" JNIEXPORT void JNICALL
2  Java_com_gamepad_client_MainActivity_injectNativePayload(JNIEnv* env, jobject this, jbyteArray data) {
3      jsize len = env->GetArrayLength(data);
4      if (len == sizeof(GamepadPayload)) {
5          // FIX 5: single-copy JNI. GetByteArrayRegion copies the 20 bytes straight
6          // into currentPayload (no Get/Release pin pair, no intermediate buffer +
7          // extra memcopy). Done under payloadMutex so it can't tear against the TX
8          // thread's read. The 20-byte length guard above is preserved.
9          {
10             std::lock_guard<std::mutex> lock(payloadMutex);
11             env->GetByteArrayRegion(data, 0, sizeof(GamepadPayload), reinterpret_cast<jbyte*>(&currentPaylo
12             ad));
13             payloadDirty = true;
14         }
15         // Wake the TX thread NOW so this input transmits immediately rather than on
16         // the next poll tick. Notify after releasing the lock so the woken thread
17         // doesn't immediately block on a mutex we still hold.
18         payloadCv.notify_one();
19     }
20 }
```

Walking through this: `env` and `this` are JNI plumbing (a handle to the Java runtime, and the calling object). The real argument is `data`, a Java byte array. The function first checks that the array is exactly the size of a `GamepadPayload` (20 bytes) and silently ignores anything else — a cheap guard against a malformed packet. Then, holding a lock (`payloadMutex`) so the send thread cannot read half-written data, it copies the 20 bytes *straight* into the global `currentPayload` struct with `GetByteArrayRegion`. It sets `payloadDirty = true` to signal "there is new input," and finally calls `payloadCv.notify_one()` to *wake the send thread this instant* rather than let it sleep out its timer. That wake-immediately design is the core of the engine's low latency, and we will return to it.

DEFINITION

Mutex — Short for "mutual exclusion." A lock that only one thread can hold at a time, used to stop two threads from reading and writing the same data simultaneously and corrupting it.

WATCH OUT

The architecture doc calls this bridge function `sendGamepadPacket` and describes a `window.sendGamepadPacket(buffer)` call. The actual code tells a longer story. The JavaScript UI calls `sendGamepadPacketNative(base64Data)` on the Kotlin bridge; Kotlin base64-decodes it and calls `injectNativePayload(bytes)`; and *that* is what maps to the C++ function above. When the doc and the code disagree, the code wins: the native entry point is named `injectNativePayload`, not `sendGamepadPacket`.

EXPERT DETAIL

`JNI_OnLoad` at the top of the file caches the `JavaVM*` into a global `g_vm` the moment the library loads. This matters because the C++ send thread sometimes needs to call *back up* into Kotlin (for encryption), and to do that from a non-Java thread it must first attach itself to the VM via `AttachCurrentThread`. Caching the VM pointer once is what makes those up-calls possible without re-plumbing it on every call.

The 20-Byte Packet — Every Byte Accounted For

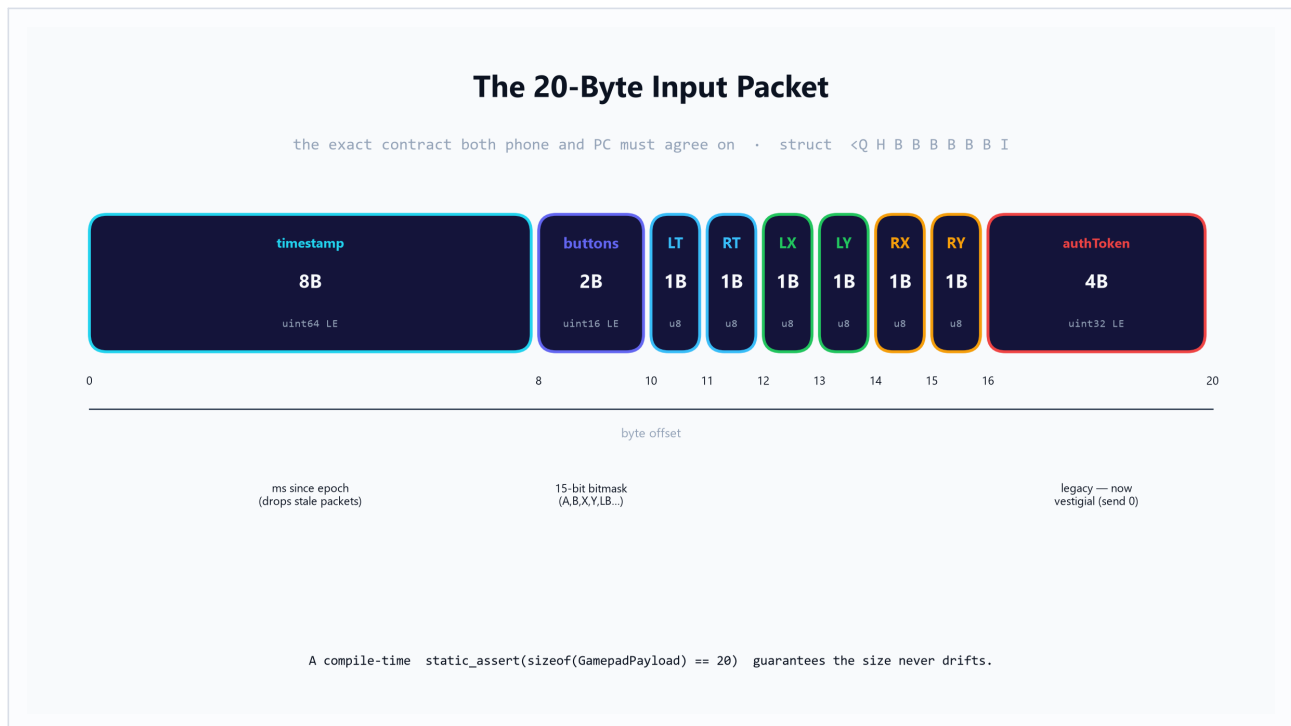


Figure 5.1 — The 20-byte input packet, every field and offset.

The heart of the whole system is a shared agreement: the phone and the PC must lay out the controller state in memory *identically*, byte for byte, or the PC will read garbage. That agreement is a C struct,

deliberately frozen at exactly 20 bytes:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 51-65 C++

1  #pragma pack(push, 1)
2  struct GamepadPayload {
3      uint64_t timestamp; // 8 bytes
4      uint16_t buttons; // 2 bytes
5      uint8_t leftTrigger; // 1 byte
6      uint8_t rightTrigger; // 1 byte
7      uint8_t leftStickX; // 1 byte
8      uint8_t leftStickY; // 1 byte
9      uint8_t rightStickX; // 1 byte
10     uint8_t rightStickY; // 1 byte
11     uint32_t authToken; // 4 bytes
12 };
13 #pragma pack(pop)
14
15 static_assert(sizeof(GamepadPayload) == 20, "FATAL: Payload struct is not exactly 20 bytes!");
```

DEFINITION

Struct — A bundle of related variables stored back-to-back in memory as one unit. Reading the struct's raw bytes gives you a fixed-layout binary record.

DEFINITION

`#pragma pack(1)` — A compiler instruction meaning "do not insert any hidden padding between these fields." Compilers normally pad structs for alignment, which would silently change the byte layout; this forbids that.

DEFINITION

`static_assert` — A check performed at *compile time*, not while running. If the condition is false, the program refuses to compile. Here it guarantees the struct is exactly 20 bytes; if a future edit accidentally changed the size, the build breaks instead of shipping a broken wire format.

Here is what every byte means, by offset:

Offset	Size	Field	Meaning
0	8 B	<code>timestamp</code>	send-time stamp, used for ordering and latency
8	2 B	<code>buttons</code>	16-bit bitmask, one bit per button
10	2 B	<code>leftTrigger, rightTrigger</code>	0 = released, 255 = fully pressed
12	4 B	four stick axes	0..255, 128 = centered
16	4 B	<code>authToken</code>	the QR-issued key, or 0 for USB

DEFINITION

Bitmask — A single number where each individual bit (0 or 1) is a separate on/off flag. The `buttons` field packs 14 buttons into 16 bits: bit 0 is A/Cross, bit 1 is B/Circle, and so on up through bit 13 (D-Pad Right). Pressing A and B at once sets bits 0 and 1, giving the number 3.

The `leftStickX = 128` center value is not arbitrary — it is a contract shared across three codebases. The UI computes `128 + normalized * 127`, the engine pre-seeds the struct to 128 on startup, and the Python server decodes with `(byte - 128) / 127`. Get that constant wrong anywhere and the sticks drift.

The PC side confirms the layout with a matching unpack string, `'<Q H B B B B B B I'` — little-endian `uint64`, `uint16`, six `uint8`s, and a `uint32`, which sum to exactly 20 bytes. The `<` prefix pins the byte order:

DEFINITION

Endianness — The order in which the bytes of a multi-byte number are stored. *Little-endian* puts the least-significant byte first. Android phones and x86 PCs are both little-endian, so they agree — but the engine still makes the assumption explicit with `htole64/htole32/htole16` helper macros (defined as no-op identity functions, since the platform is already little-endian) so the intent is documented in the code.

Two subtle facts about specific fields deserve a spotlight. First, the `authToken`:

DEFINITION

authToken — A 4-byte secret shared via the QR code. The phone stamps it into every packet; the PC rejects any packet whose token does not match, so a random device on the same Wi-Fi cannot inject input. A token of 0 means "unauthenticated" (used by the wired USB path).

That token is derived when the connection is set up. `initNetworkNative` parses the hex key string from the QR into `expectedHash` with `expectedHash = (uint32_t)strtoul(key, NULL, 16);`, and the send loop stamps `expectedHash` into `authToken` on every outgoing packet. The doc mentions a default of `0xABCD1234`, but the real value is whatever the QR encoded.

WATCH OUT

The second surprise: the doc says the `timestamp` is "ms since epoch." The code contradicts this. The UI does write a millisecond timestamp, but the send thread *overwrites* it just before transmission with a monotonic nanosecond clock reading (`clock_gettime(CLOCK_MONOTONIC, ...)`). The PC confirms this — its stale-packet guard treats a backward jump of more than `1_000_000_000` (one second, in nanoseconds) as a clock reset. So on the wire, `timestamp` is monotonic nanoseconds, not epoch milliseconds. Code wins.

The Send Loop — Awake Only When It Must Be

All the sending happens on one dedicated thread, `txThreadLoop`. The old design for a thread like this would be a *busy poll*: wake up every couple of milliseconds, check if anything changed, send if so, sleep, repeat — a thousand times a second whether or not you touched the screen. That is exactly what this

engine deliberately *stopped* doing, because it kept the phone's cores spun up and made it run hot.

But before it does anything, the thread tries to make itself high-priority so the operating system wakes it on time even under load:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 223-245 C++

1  {
2      struct sched_param sp;
3      memset(&sp, 0, sizeof(sp));
4      // Use a low RT priority: just above normal, well clear of audio/driver
5      // threads. sched_get_priority_min gives the floor for SCHED_FIFO.
6      int rtMin = sched_get_priority_min(SCHED_FIFO);
7      sp.sched_priority = (rtMin > 0) ? rtMin + 1 : 1;
8      int rc = pthread_setschedparam(pthread_self(), SCHED_FIFO, &sp);
9      if (rc != 0) {
10         // SCHED_FIFO denied (typical on Android without CAP_SYS_NICE).
11         // Fall back to the strongest CFS niceness we can get. setpriority
12         // returns -1 on failure but -1 is also a valid niceness, so clear
13         // errno first and check it afterwards.
14         errno = 0;
15         if (setpriority(PRIO_PROCESS, 0, -19) != 0 && errno != 0) {
16             LOGI("TX thread priority bump unavailable (SCHED_FIFO rc=%d, setpriority errno=%d); running
17 at default.", rc, errno);
18         } else {
19             LOGI("TX thread niced to high CFS priority (SCHED_FIFO denied rc=%d).", rc);
20         }
21     } else {
22         LOGI("TX thread elevated to SCHED_FIFO prio %d.", sp.sched_priority);
23     }
24 }
```

DEFINITION

SCHED_FIFO — A "real-time" scheduling policy on Linux/Android. A thread using it runs ahead of ordinary threads and is not preempted by them — ideal for a latency-critical loop. But it usually requires a special capability (**CAP_SYS_NICE**) that normal Android apps do not have.

DEFINITION

nice value — A number from -20 (highest priority) to +19 (lowest) that biases the ordinary scheduler. A more-negative value means "schedule me sooner."

This block asks for a *low* real-time priority via **SCHED_FIFO** — `rtMin + 1`, just barely above normal, so it never fights with audio or driver threads. On stock Android that request almost always fails with a permission error, so the code *gracefully degrades*: it falls back to `setpriority(..., -19)`, the strongest ordinary priority it can get. The comment about clearing `errno` first is a genuine trap — `setpriority` returns -1 on failure, but -19 through +19 are also valid return values, so you cannot use the return value alone to detect failure; you must zero `errno` and check it afterward. Every step is best-effort: it never crashes and never blocks if the bump is denied.

NOTE

An older version of this thread also *pinned* itself to the phone's big performance cores. That was removed on purpose — forcing the big cores awake 1000 times a second was the main cause of high CPU and phone heat. The code comment is emphatic: "do NOT re-add it."

Now the loop runs. On each pass it snapshots the latest input, then decides whether it actually needs to transmit:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 270-290 C++
1      uint8_t curInput[8];
2      memcpy(curInput, &networkPayload.buttons, 8);
3      struct timespec now;
4      clock_gettime(CLOCK_MONOTONIC, &now);
5      uint64_t rawTimestamp = (uint64_t)now.tv_sec * 1000000000LL + now.tv_nsec;
6      bool changed = (memcmp(curInput, lastSentInput, 8) != 0);
7      // Adaptive keep-alive: ~30Hz normally (the thermal/CPU win noted above), but
8      // ~60Hz while rumble is active. The PC only emits fresh RMB in reply to an
9      // inbound frame, so when the pad is held steady during rumble a 30Hz uplink
10     // caps rumble-state updates at ~33ms; 60Hz halves that. Reverts to 30Hz the
11     // moment rumble clears (rumbleLeft/Right are the last values the PC sent us).
12     bool rumbleActive = (rumbleLeft.load(std::memory_order_relaxed) != 0) ||
13                         (rumbleRight.load(std::memory_order_relaxed) != 0);
14     uint64_t heartbeatNs = rumbleActive ? 16000000ULL : 33000000ULL;
15     bool heartbeat = (rawTimestamp - lastSendNs) >= heartbeatNs; // adaptive keep-alive
16     // FIX 1: a fresh change arms CHANGED_PACKET_REDUNDANCY back-to-back sends.
17     // While that counter is positive we keep re-sending (redundant) even when
18     // the input hasn't changed again, so a single dropped datagram recovers
19     // fast. The counter is (re)armed below once we confirm a successful send.
20     bool redundant = (redundancyRemaining > 0);
21     bool doSend = changed || heartbeat || redundant;
```

This is the decision engine. It builds an 8-byte "fingerprint" of the current input — the **buttons**, triggers, and stick axes, which sit contiguously in the struct — and compares it to what it last sent. **changed** is true only if the input actually differs. **heartbeat** is a keep-alive timer: even with the pad held perfectly still, send *something* every 33 ms (about 30 Hz) so the link stays warm and the PC knows you are still there. That heartbeat speeds up to 16 ms (60 Hz) while rumble is active, because the PC only sends fresh rumble in reply to an inbound packet, so a faster uplink means snappier rumble updates. **redundant** covers loss recovery. The final **doSend** is true if *any* of the three fire. This is why the phone stopped running hot: when you hold still and nothing is rumbling, it drops from ~1000 packets/second to ~30.

DEFINITION

Keep-alive (heartbeat) — A small packet sent at a steady low rate purely to prove the connection is still alive, even when there is no real data to send.

DEFINITION

Redundancy — Deliberately sending the same changed packet a few times (`CHANGED_PACKET_REDUNDANCY = 3`) back-to-back. Since UDP can silently drop a packet, sending three copies means one lost datagram still gets your button-press through within a couple of milliseconds instead of waiting up to 33 ms for the next change.

When `doSend` is true, the actual transmission is a "fire and forget" — it never blocks waiting for the network:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 331-348 C++
1      bool haveTransport = accessoryMode ? (accessoryFd != -1) : (udpSocket != -1);
2      if (haveTransport && doSend) {
3          ssize_t sent;
4          if (accessoryMode) {
5              // AOA: write the 20-byte frame straight to the accessory fd (raw USB
6              // bulk → PC). No addressing/auth handshake — USB is point-to-point.
7              sent = write(accessoryFd, &networkPayload, sizeof(GamepadPayload));
8          } else if (socketConnected) {
9              // FIX 2: socket is connect()ed to the locked unicast peer, so the
10             // kernel route is cached — send() skips the per-call route lookup
11             // and address copy that sendto() repeats every time.
12             sent = send(udpSocket, outBuf, outLen, MSG_DONTWAIT);
13          } else {
14              // Broadcast discovery (or not yet locked on): keep the explicit
15              // destination so the datagram still reaches the broadcast address.
16              sent = sendto(udpSocket, outBuf, outLen, MSG_DONTWAIT,
17                           (struct sockaddr*)&serverAddr, sizeof(serverAddr));
18          }
19      }
```

DEFINITION

MSG_DONTWAIT — A flag telling the send call "if the buffer is full, fail immediately instead of blocking." This guarantees the send thread never stalls.

Three transports, one loop. If `accessoryMode` is on, the frame is `write()`-ten straight to a USB file descriptor (the wired path — more on that shortly). Otherwise it goes over UDP, and there are two sub-cases. Once the engine has locked onto the PC's real address it uses `send()` on a *connected* socket, which lets the kernel cache the route and skip the per-call address lookup. Before that lock happens — during discovery, when it is broadcasting to the whole subnet — it uses `sendto()` with an explicit destination. Either way, `outBuf/outLen` point either at the raw 20-byte struct or (when encryption is active) at the sealed frame.

The loop's final act is what makes it event-driven rather than a busy poll:

apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 589-595

C++

```

1      {
2          std::unique_lock<std::mutex> lock(payloadMutex);
3          if (!payloadDirty && isRunning) {
4              payloadCv.wait_for(lock, std::chrono::nanoseconds(RX_POLL_NS),
5                               [] { return payloadDirty || !isRunning.load(); });
6          }
7      }

```

DEFINITION

Condition variable — A thread-coordination tool that lets a thread *sleep* until another thread signals it. Here `payloadCv.wait_for` puts the send thread to sleep for up to `RX_POLL_NS` (2 ms) — unless `injectNativePayload` calls `notify_one` first, in which case it wakes instantly.

This is the payoff of the whole design. When you touch the screen, `injectNativePayload` sets `payloadDirty` and rings the bell; the send thread wakes in microseconds and your input goes out immediately. When nothing is happening, the thread sleeps out its 2 ms timer, which just keeps incoming ACKs and rumble draining and the 30 Hz heartbeat ticking. Fresh input pays *zero* polling delay; idle costs almost no CPU. The lambda predicate (`return payloadDirty || !isRunning`) also guards against a "lost wakeup" — input that arrives in the tiny window between the snapshot and the wait.

Sockets, Discovery, Reconnect, and Wired USB

The UDP socket itself is created inside `initNetworkNative`, and its configuration is where a lot of the latency tuning lives:

apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 686-701

C++

```

1      udpSocket = socket(AF_INET, SOCK_DGRAM, 0);
2      if (udpSocket != -1) {
3          int flags = fcntl(udpSocket, F_GETFL, 0);
4          fcntl(udpSocket, F_SETFL, flags | O_NONBLOCK);
5          int broadcastEnable = 1;
6          setsockopt(udpSocket, SOL_SOCKET, SO_BROADCAST, &broadcastEnable, sizeof(broadcastEnable));
7          applyLowLatencyTos(udpSocket);
8      }
9      // FIX 2: start unconnected — the hot path uses sendto() during broadcast
10     // discovery and only switches to send() after we connect() to the locked peer.
11     socketConnected = false;
12     connectedPeer.s_addr = 0;
13     memset(&serverAddr, 0, sizeof(serverAddr));
14     serverAddr.sin_family = AF_INET;
15     serverAddr.sin_port = htons(port);
16     inet_pton(AF_INET, ip, &serverAddr.sin_addr);

```


DEFINITION

Socket — A software endpoint for network communication — one end of a pipe your program can send bytes into and receive bytes out of. `SOCK_DGRAM` requests a UDP (datagram) socket; `AF_INET` means IPv4.

DEFINITION

Non-blocking (`O_NONBLOCK`) — A socket mode where reads and sends return immediately if there is nothing to do, instead of freezing the thread until data arrives. Essential for the never-stall send loop.

The socket is made non-blocking, broadcast is enabled (so the phone can shout "who's out there?" to the whole subnet before it knows the PC's exact address), and `applyLowLatencyTos` tags it as high-priority traffic. Then the destination address is built from the `ip` and `port` the QR code provided — the port is *not* hardcoded in the engine; it arrives at connection time. Note `socketConnected = false`: the socket starts in discovery mode using `sendto`, and only later "locks on."

That priority tagging is worth seeing:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 135-149 C++

1 static void applyLowLatencyTos(int sock) {
2     if (sock < 0) return;
3     int tos = 0xB8; // DSCP EF (Expedited Forwarding) << 2 | ECN 0
4     setsockopt(sock, IPPROTO_IP, IP_TOS, &tos, sizeof(tos));
5     // Android/Linux Wi-Fi WMM access-category selection is driven by skb->priority
6     // (SO_PRIORITY), which many Wi-Fi HALs honour even when they ignore IP_TOS/DSCP.
7     // prio 6 (TC_PRIO_INTERACTIVE) maps to WMM AC_VO — the same high-priority queue the
8     // WIFI_MODE_FULL_LOW_LATENCY lock targets, so they compound and cut uplink queuing
9     // jitter on busy APs. Best-effort: prio <= 6 is allowed unprivileged; failure is
10    // harmless (older kernels may not define SO_PRIORITY).
11    #ifdef SO_PRIORITY
12        int prio = 6;
13        setsockopt(sock, SOL_SOCKET, SO_PRIORITY, &prio, sizeof(prio));
14    #endif
15 }
```

DEFINITION

DSCP / IP_TOS — A field in every IP packet's header that says "this is high-priority, expedite me." `0xB8` is the DSCP code for *Expedited Forwarding* — the class routers use for voice/interactive traffic. A WMM-capable Wi-Fi router will move these packets ahead of bulk downloads.

DEFINITION

SO_PRIORITY — A Linux socket setting that stamps a priority directly onto the outgoing packet buffer. Many phone Wi-Fi chips honour this even when they ignore the DSCP header, mapping priority 6 to the "voice" queue — the highest-priority Wi-Fi lane. Setting *both* means one of them almost always works.

Together these two settings ask the network to treat your gamepad packets like a phone call rather than a file download, cutting the jitter that a busy access point would otherwise add. Both are best-effort: if the kernel does not support them, the failure is silently harmless.

Now, how does the engine go from "broadcasting to everyone" to "locked onto my PC"? When the PC receives a valid packet it replies with an **ACK**. The engine, on seeing that ACK arrive from a real unicast address while it is still broadcasting, switches its destination to that address and then *connects* the socket to it, so future sends use the faster **send()** path. And if the network drops out entirely, it rebuilds the socket with exponential backoff:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 556-581 C++

1      if (!accessoryMode && udpSocket == -1 && isRunning) {
2          int backoff = 50; // ms
3          while (isRunning && udpSocket == -1) {
4              udpSocket = socket(AF_INET, SOCK_DGRAM, 0);
5              if (udpSocket != -1) {
6                  int flags = fcntl(udpSocket, F_GETFL, 0);
7                  fcntl(udpSocket, F_SETFL, flags | O_NONBLOCK);
8                  int broadcastEnable = 1;
9                  setsockopt(udpSocket, SOL_SOCKET, SO_BROADCAST, &broadcastEnable, sizeof(broadcastEnable));
10             }
11             applyLowLatencyTos(udpSocket);
12             // FIX 2: a fresh socket is unconnected. Force re-discovery via
13             // sendto() until we re-lock + re-connect() to the unicast peer.
14             socketConnected = false;
15             connectedPeer.s_addr = 0;
16             LOGI("Socket successfully recovered.");
17             break;
18         }
19         LOGE("Socket recovery failed. Retrying in %d ms...", backoff);
20         // Split ms into sec + nsec: tv_nsec must be < 1e9, and at the
21         // 1000ms cap the old {0, backoff*1e6} hit exactly 1e9 → EINVAL →
22         // nanosleep returned immediately → 100% CPU busy-spin.
23         struct timespec backoffTime = {backoff / 1000, (long)(backoff % 1000) * 1000000L};
24         nanosleep(&backoffTime, NULL);
25         backoff = std::min(backoff * 2, 1000);
26     }
}
```

DEFINITION

Exponential backoff — A retry strategy that doubles the wait between attempts (50 ms, 100, 200, 400, 800, capped at 1000) so a persistently-failing operation does not hammer the system.

This block only runs after an earlier send failed with a fatal network error (**ENETUNREACH**, **ENETDOWN**, etc.), which closes the socket and sets **udpSocket = -1**. It then loops trying to recreate the socket, waiting **backoff** ms between tries and doubling each time up to a one-second cap. The comment records a real bug that was fixed: splitting the delay into seconds and nanoseconds matters because a **tv_nsec** of exactly one billion is invalid and made **nanosleep** return instantly, turning the retry into a 100%-CPU busy-spin. When the socket comes back, it resets to discovery mode (**socketConnected = false**) to re-find the PC.

NOTE

There is a deliberate *absence* here. `stopNetworkNative` sends **no** "goodbye" packet. The comment explains why: a cleartext teardown packet would let any device on the Wi-Fi spoof your IP and kill your session with three bytes. Instead the engine just goes quiet and lets the server's idle watchdog clean up the virtual pad after ~3 seconds. That is a security trade-off, chosen on purpose.

EXPERT DETAIL

The wired path (`accessoryMode`) uses Android Open Accessory — `initAccessoryNative` takes a raw file descriptor from `UsbManager.openAccessory(...).detachFd()` and the send loop `write()`s the same 20-byte frames to it instead of a socket. Same event loop, same redundancy, same priority — just a different pipe. Because USB is point-to-point there is no addressing, no auth token (`expectedHash = 0`), and no broadcast-lock dance.

ACKs, Latency, Rumble, and the GRX Encryption Hook

Every trip through the loop also *drains the receive queue*, regardless of whether it sent anything, so incoming replies are handled at the full 2 ms rate. The most interesting reply is the ACK, because it carries the raw material for a true latency measurement:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 452-474 C++

1      {
2          struct timespec ackNow;
3          clock_gettime(CLOCK_MONOTONIC, &ackNow);
4          uint64_t ackNs = (uint64_t)ackNow.tv_sec * 1000000000LL + ackNow.tv_nsec;
5          lastAckMonoNs.store(ackNs, std::memory_order_relaxed); // link-alive heartbeat
6
7          // TRUE round-trip: the PC echoed back the exact send-time we
8          // stamped into this packet's `timestamp` field. RTT = now -
9          // that echoed time. (Old code compared against the LAST send,
10         // which had just been overwritten → it measured ~microseconds
11         // between two adjacent C++ lines, not the network. Fixed.)
12         if (rec >= 11) {
13             uint64_t echoed = 0;
14             std::memcpy(&echoed, rxBuffer + 3, 8); // little-endian on Android
15             if (echoed != 0 && ackNs > echoed) {
16                 float rttMs = (float)(ackNs - echoed) / 1000000.0f;
17                 if (rttMs >= 0.0f && rttMs < 1000.0f) { // sanity clamp
18                     float prev = latencyMsAtomic.load(std::memory_order_relaxed);
19                     float smoothed = (prev <= 0.0f) ? rttMs : (prev * 0.8f + rttMs * 0.2f);
20                     latencyMsAtomic.store(smoothed, std::memory_order_relaxed);
21                 }
22             }
23         }
```

DEFINITION

ACK (acknowledgment) — A tiny reply the PC sends back — the three bytes "ACK" followed by the 8-byte timestamp it read out of the packet it just processed. It proves the PC received and acted on your input.

DEFINITION

Round-trip time (RTT) — The total time for a packet to travel to the PC and its reply to travel back — the real-world latency number shown in your app's telemetry.

The trick is beautiful in its simplicity. The engine stamped a nanosecond send-time into the packet's `timestamp`; the PC copies that exact value into its ACK (`b"ACK" + struct.pack("<Q", timestamp)`); when the ACK returns, the engine subtracts the echoed time from *now* to get the true round trip. An earlier version compared against the *last* send time, which had already been overwritten, so it was really measuring microseconds between two adjacent lines of C++ — a meaningless near-zero. The fix was to echo the timestamp through the PC. The result is smoothed with an exponential moving average (`prev*0.8 + rtt*0.2`) so the HUD number does not jitter, and clamped to a sane 0-1000 ms range. Separately, `lastAckMonoNs` records *when* the last ACK arrived — the only trustworthy "the PC is actually there" signal, since a UDP `sendto` succeeds locally even if nothing is listening.

The same RX drain also handles rumble coming back from the PC:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 424-434 C++
1         if (rec >= 5 && rxBuffer[0] == 'R' && rxBuffer[1] == 'M' && rxBuffer[2] == 'B') {
2             uint32_t dst = ntohl(serverAddr.sin_addr.s_addr);
3             bool dstBroadcast = (dst == INADDR_BROADCAST) || ((dst & 0xFF) == 0xFF);
4             if (!dstBroadcast && fromAddr.sin_addr.s_addr != serverAddr.sin_addr.s_addr) {
5                 continue; // rumble from an unexpected host - ignore it
6             }
7             rumbleLeft.store((uint8_t)rxBuffer[3], std::memory_order_relaxed);
8             rumbleRight.store((uint8_t)rxBuffer[4], std::memory_order_relaxed);
9             rumbleSeq.fetch_add(1, std::memory_order_relaxed);
10            continue; // keep draining the queue; rumble handled
11        }
```

DEFINITION

Rumble (force feedback) — The vibration a game sends back to the controller. The PC packs it as "RMB" plus two bytes — the large (low-frequency) and small (high-frequency) motor strengths, each 0-255.

When a 5-byte RMB datagram arrives, the engine first checks it came from the locked PC address (so no other LAN host can buzz your phone), then stores the two motor values and *bumps a sequence counter*. It does not vibrate directly. Instead the JavaScript layer polls `getNativeRumble()` — which packs the sequence in the high bits and the two motor bytes in the low bits of one `jlong` — and fires the vibrator only when the sequence changes, so your on/off toggle and intensity slider still apply. This RX handling runs every 2 ms, not just when sending, precisely so rumble and the link-alive signal are never delayed by a steady-held pad.

Finally, when the encrypted transport is live, the outgoing frame gets sealed by calling *up* into Kotlin:

```

1      if (doSend && grxReady.load() && g_mid_seal && g_activity) {
2          JNIEnv* e = grxEnv();
3          if (e) {
4              jbyteArray in = e->NewByteArray((jsize)sizeof(GamepadPayload));
5              if (in) {
6                  e->SetByteArrayRegion(in, 0, (jsize)sizeof(GamepadPayload), (const jbyte*)&networkPaylo
ad);
7
8                  jbyteArray wire = (jbyteArray)e->CallObjectMethod(g_activity, g_mid_seal, in);
9                  if (wire) {
10                     jsize wn = e->GetArrayLength(wire);
11                     if (wn > 0 && wn <= (jsize)sizeof(grxBuf)) {
12                         e->GetByteArrayRegion(wire, 0, wn, (jbyte*)grxBuf);
13                         outBuf = grxBuf;
14                         outLen = (size_t)wn;
15                     }
16                     e->DeleteLocalRef(wire);
17                 }
18                 e->DeleteLocalRef(in);
19             }
20         }

```

DEFINITION

GRX — The project's encrypted-input layer. Once a handshake establishes, the plain 20-byte frame is "sealed" (encrypted and authenticated) into a 41-byte wire frame before it leaves the phone. The engine detects handshake control frames (first byte `0xE1/0xE2/0xE3`) on the RX path and hands them up to Kotlin's `onGrxControl`.

This calls the Kotlin `grxSeal(byte[]) -> byte[]` method through JNI, turning the 20-byte payload into an encrypted frame that `outBuf/outLen` then point at for the send. The critical guard is `doSend &&`: sealing is expensive (a JNI up-call plus AES-GCM plus heap allocations), so it runs *only* on ticks that actually transmit — not on every idle 2 ms poll, which an earlier version did, wastefully sealing ~500 frames/second whose output was thrown away. When `grxReady` is false (no GRX server, or pre-handshake), this whole block is skipped and the output is byte-for-byte the legacy cleartext path.

WATCH OUT

The architecture doc's section on the engine describes "gyroscope sensor fusion" living in native code, feeding `nativeGyroRightStickX/Y`. The code contradicts this. A comment in `gamepad-engine.cpp` states plainly that "the old native sensor-fusion path and its state were removed: gyro steering is implemented in Kotlin — `onSensorChanged` → rotation vector + 1€ filter — and reaches this engine inside the regular 20-byte payload." So the engine does *no* gyro math today; tilt arrives already baked into the stick bytes. Code wins.

Building One Engine for Every Phone

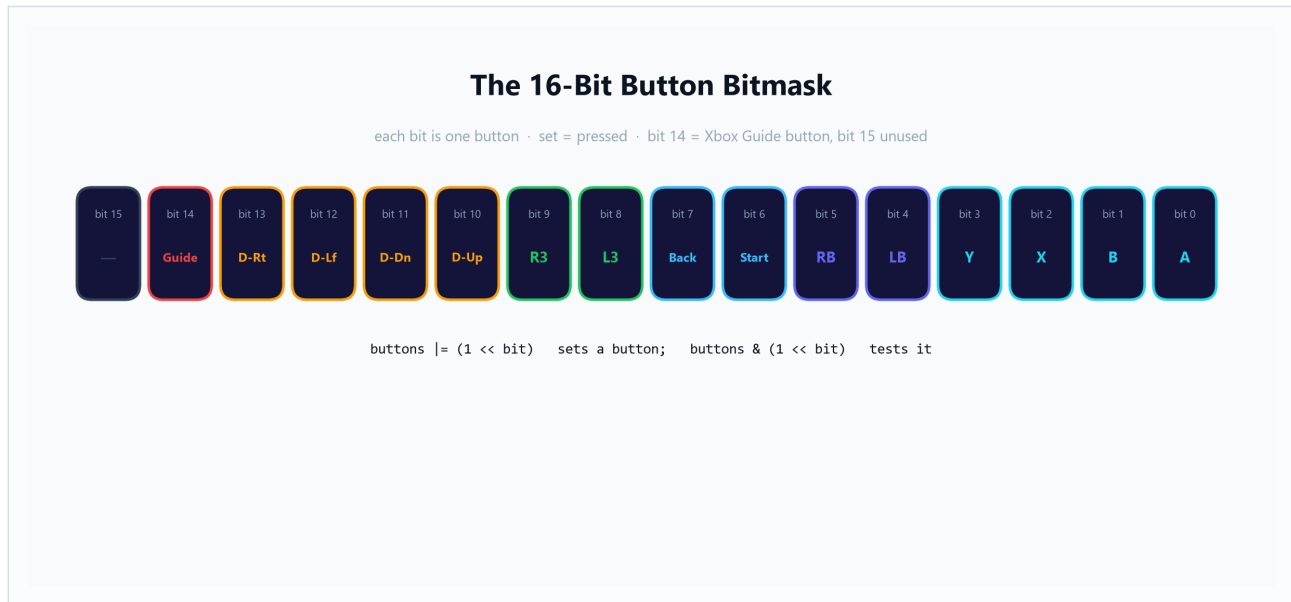


Figure 5.2 — The 16-bit button bitmask.

A phone's processor speaks one specific machine language, and there are several in the wild. The engine must be compiled separately for each.

DEFINITION

ABI (Application Binary Interface) — The exact machine-code format and calling conventions for a given processor family. `arm64-v8a` is 64-bit ARM (modern phones), `armeabi-v7a` is 32-bit ARM (older/budget phones), and `x86_64` is 64-bit Intel/AMD (emulators, Chromebooks). A `.so` built for one ABI will not run on another.

CMake compiles the source, but *which* ABIs to build for is set in the Gradle config:

```
apps/android-client/app/build.gradle.kts · lines 26-41 Kotlin

1      ndk {
2          // Broad device coverage (universal APK for sideload + in-app updater):
3          // arm64-v8a — all modern phones/tablets (primary)
4          // armeabi-v7a — older / budget 32-bit ARM phones
5          // x86_64 — Android emulators, Chromebooks, x86 tablets
6          // (32-bit x86 omitted: effectively dead, only adds size.)
7          abiFilters.add("arm64-v8a")
8          abiFilters.add("armeabi-v7a")
9          abiFilters.add("x86_64")
10     }
11
12     externalNativeBuild {
13         cmake {
14             cppFlags += listOf("-O3", "-fllto", "-ffast-math")
15         }
16     }
```

The `abiFilters` list tells the build to produce three copies of `libgamepad_engine.so`, one per ABI, and bundle all three into a single universal APK — so the same downloadable file installs on a flagship, a budget phone, or an emulator, with 32-bit x86 deliberately omitted to save size. The `cppFlags` are the compiler's optimization dials: `-O3` is maximum speed optimization, `-flto` is link-time optimization (letting the compiler optimize across the whole program at link time), and `-ffast-math` relaxes strict floating-point rules for speed — safe here because the engine's only float work is the latency smoothing.

DEFINITION

`-flto` (**link-time optimization**) — An optimization that defers some work to the linking stage so the compiler can see and optimize the entire program at once, rather than one file at a time.

This is `versionCode = 22, versionName = "1.3.0"` — the current shipped engine. At runtime, Android picks the right `.so` for the device's ABI automatically, and the Kotlin `init { System.loadLibrary("gamepad_engine") }` block loads it — closing the loop back to the `project("gamepad_engine")` name you saw in the very first code block. From that moment, every `external fun` in Kotlin has a real C++ function behind it, and your finger movements have their fast, single-minded mail carrier.

PART III

**The Protocol, the Server &
Encryption**

CHAPTER 6

Discovery, Pairing, and the Wire Protocol

6

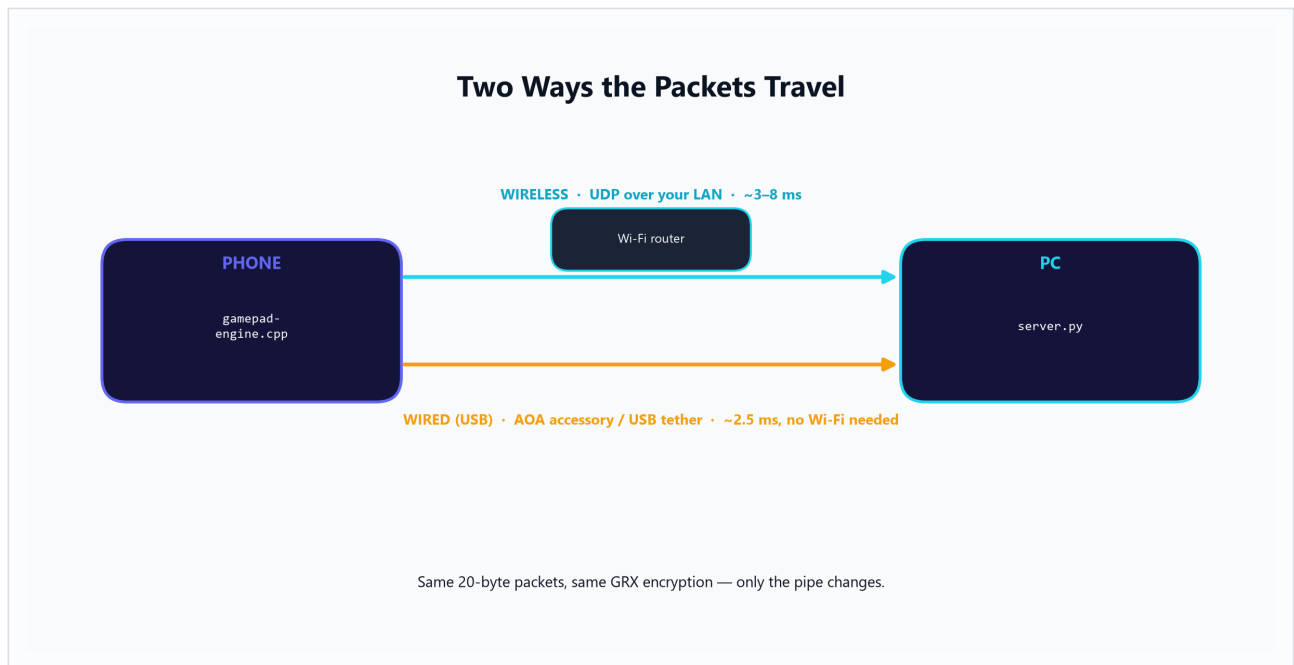


Figure 6.1 — Discovery and input over Wi-Fi (UDP) or USB.

Two Machines Finding Each Other

Before your phone can move a thumbstick in a game, two separate computers — the phone in your hand and the PC across the room — have to agree that they are talking to *each other* and nobody else. They are not plugged together (usually), they have never met, and the Wi-Fi they share might have a dozen other devices on it. This section is about that first handshake of introductions.

DEFINITION

LAN — Local Area Network — the little private network your home router creates. Every device on your Wi-Fi (phone, PC, TV) is on the same LAN and can send messages directly to one another.

DEFINITION

IP address — A number like `192.168.1.42` that identifies one device on a network, the way a street address identifies one house. Two devices on the same LAN reach each other by IP.

DEFINITION

Port — A numbered "mailbox slot" on a single device. One PC has thousands of ports; a program listens on a specific one so the operating system knows which program a message is for. Your server listens on port `7777`.

IN PLAIN TERMS

Think of the PC as a house. The IP address is the house's street address; the port is which mailbox on the porch. To deliver a letter you need both. Your phone needs to learn the PC's address *and* mailbox number before it can send anything — and that is exactly what the QR code hands over.

The QR code is a three-field business card

When your server starts, it prints one short line of text into a QR code and shows it on screen. That text is the entire introduction.

DEFINITION

QR code — A square barcode that encodes a short piece of text. A phone camera can read it instantly, which is far more reliable than typing an IP address by hand.

DEFINITION

Pairing key — A short secret shared only between this PC and the phone that scans it, used to prove later packets really came from the paired phone.

apps/pc-server/server.py · lines 1922-1930

Python

```

1     key = _load_or_create_key()
2     expected_hash = int(key, 16)
3
4     # Generate QR code as PIL image
5     csv_str = f"{ip},{port},{key}"
6     qr = qrcode.QRCode(box_size=6, border=2)
7     qr.add_data(csv_str)
8     qr.make(fit=True)
9     qr_pil = qr.make_image(fill_color="black", back_color="white").get_image()

```

The QR content is dead simple: three fields separated by commas — *ip,port,key*, for example *192.168.1.42,7777,a1b2c3d4*. The *ip* is the PC's LAN address (chosen carefully by *get_lan_ip()*, which prefers a real *192.168.x* address over a VPN tunnel). The *port* is whichever port the server managed to bind. The *key* is an 8-character hexadecimal string.

DEFINITION

Hexadecimal (hex) — Base-16 numbers using digits 0-9 and letters a-f. Each hex character is 4 bits, so 8 hex characters = 32 bits = 4 bytes. *int(key, 16)* converts that text into the actual number the server will compare against.

The line *expected_hash = int(key, 16)* turns the 8-character key into a 32-bit number the server keeps in memory. Every input packet the phone sends will carry that same number, and the server checks it — that is how it tells your phone apart from a stranger's phone on the same Wi-Fi.

The key survives a restart

An important detail hides in *_load_or_create_key*: the key is *persistent*, not regenerated each launch.

apps/pc-server/server.py · lines 1901-1920

Python

```

1  def _load_or_create_key():
2      base = os.environ.get("LOCALAPPDATA") or os.path.expanduser("~")
3      kdir = os.path.join(base, "GamepadServer")
4      kfile = os.path.join(kdir, "pairing_key.txt")
5      try:
6          with open(kfile, "r") as f:
7              k = f.read().strip().lower()
8              if len(k) == 8:
9                  int(k, 16) # validates hex
10                 return k
11     except Exception:
12         pass
13     k = secrets.token_hex(4)
14     try:
15         os.makedirs(kdir, exist_ok=True)
16         with open(kfile, "w") as f:
17             f.write(k)
18     except Exception:
19         pass # disk issue → session-only key (old behavior), still works
20     return k

```

The function looks for `pairing_key.txt` under `%LOCALAPPDATA%\GamepadServer`. If a valid 8-hex-character key is there, it reuses it; otherwise it generates a fresh one with `secrets.token_hex(4)` (4 random bytes → 8 hex chars) and writes it to disk. `secrets` is a cryptographically strong random source, not the ordinary predictable one.

NOTE

This persistence fixes a real bug: an earlier build made a *new* random key on every launch, which silently invalidated the phone's saved pairing and forced a QR re-scan after every server restart. Now the same PC always shows the same key, so you only re-scan if the PC's IP changes.

The phone side: parsing the business card

When the phone's camera decodes the QR, the Android layer hands the raw string to the web UI, which parses it. The parser is deliberately forgiving about format.

apps/controller-ui/src/app/components/Dialogs.tsx · lines 266-288

TypeScript

```

1  function parsePairingPayload(raw: string): { ip: string; port: number; key: string } | null {
2      const payload = (raw || "").trim().replace(/[\r\n]+/g, "");
3      let ip = "", port = 7777, key = "";
4      try {
5          if (payload.includes(",")) {
6              const p = payload.split(",");
7              ip = (p[0] || "").trim();
8              port = parseInt((p[1] || "").trim(), 10) || 7777;
9              key = (p[2] || "").trim();
10         } else if (payload.startsWith("http") || payload.startsWith("gamepad")) {

```

```
apps/controller-ui/src/app/components/Dialogs.tsx · (cont.) TypeScript
11     const url = new URL(payload);
12     ip = url.hostname;
13     port = parseInt(url.port, 10) || 7777;
14     key = url.searchParams.get("key") || url.pathname.replace(/\\/g, "");
15   } else if (payload.startsWith("{")) {
16     const d = JSON.parse(payload);
17     ip = (d.ip || d.host || "").trim();
18     port = parseInt(d.port, 10) || 7777;
19     key = (d.key || d.password || "").trim();
20   } else {
21     const p = payload.split(":");
22     ip = (p[0] || "").trim();
23     if (p.length > 1) port = parseInt((p[1] || "").trim(), 10) || 7777;
```

Your server always emits the comma form, so the first branch (`payload.includes(",")`) is the one that fires: split on commas, take field 0 as `ip`, field 1 as `port` (defaulting to 7777 if it can't parse), field 2 as `key`. The other branches (URL, JSON, `host:port`) exist so the app can also accept codes from other tools — a robustness cushion. If nothing usable comes out, the caller shows *"That QR code isn't a valid server code"* and keeps scanning rather than dialing a wrong address. Once parsed, it calls `bridge.connectToPC(parsed.ip, parsed.port, parsed.key)`, which is the doorway into the native engine.

One Socket, Many Kinds of Message

A reader coming from other remote-control apps might expect *two* connections: a reliable "control channel" to negotiate the session and a fast "input channel" for the actual joystick data. GamepadOS does **not** work that way. Over Wi-Fi there is a single UDP flow, and every kind of message — inputs, acknowledgements, rumble, even the encryption handshake — travels on that one socket, told apart by looking at the message's first bytes.

DEFINITION

UDP — User Datagram Protocol. A way to send small standalone messages ("datagrams") across a network with no setup and no delivery guarantee — fire and forget. Fast, but a lost message is simply gone.

DEFINITION

TCP — The reliable alternative to UDP: it guarantees delivery and order, at the cost of waiting and re-sending. GamepadOS avoids TCP for input because a *late* joystick reading is worse than a *missing* one.

DEFINITION

Datagram / packet — One self-contained UDP message. In this system the input datagram is exactly 20 bytes.

The port numbers, exactly

The server binds a UDP socket, and it does so with a fallback range in case **7777** is taken.

DEFINITION

Socket — A software endpoint your program opens to send and receive network messages — one end of the pipe.

apps/pc-server/server.py · lines 685-689

Python

```
1 sock.bind(('0.0.0.0', port))
2 return sock, port
3 except OSError:
4     continue
5 raise Exception("Could not bind to any port.")
```

The enclosing loop is `def bind_server_socket(base_port=7777, max_attempts=10):` it tries **7777**, then **7778**, and so on up to **7786**, binding the first free one. `bind(('0.0.0.0', port))` means "listen on this port on *every* network interface of this PC" — Wi-Fi, Ethernet, USB tether, all of them. The Windows Firewall rule the server installs opens the whole **7777-7786** range plus a program-scoped rule, so wherever it lands, the phone's packets get through.

There is a second, separate port with the *same number*: `WS_PORT = 7777`, a TCP WebSocket bound to **127.0.0.1** for the USB-debugging transport. Because one is UDP and the other is TCP, and TCP **7777** is bound only to loopback, there is no clash.

Transport	Protocol	Address:Port	Who dials whom
Wi-Fi / USB-tether	UDP	0.0.0.0:7777 (7778-7786 fallback)	phone → PC
USB-debugging	TCP WebSocket	127.0.0.1:7777 (via adb reverse)	phone → PC (over cable)
AOA direct-USB	raw USB bulk	no IP at all	point-to-point

DEFINITION

WebSocket — A long-lived two-way message channel built on top of TCP, originally for browsers. Here the phone opens `ws://127.0.0.1:7777` and streams the same 20-byte packets through it.

DEFINITION

adb reverse — An Android debugging trick that tunnels a TCP port from the phone back to the PC over the USB cable, so **127.0.0.1:7777** *on the phone* actually reaches the PC.

DEFINITION

AOA (Android Open Accessory) — A mode where the phone acts as a raw USB device and the PC reads/writes bytes directly to it — the lowest-latency wired path, ~1-2 ms, with no IP stack involved.

DEFINITION

Loopback — The special address `127.0.0.1` meaning "this same machine." Traffic to it never leaves the device.

Broadcast vs. a direct address

Over Wi-Fi the QR gives the phone the PC's exact IP, so the phone can send straight there. But the USB-tether path is trickier: the phone doesn't know the PC's tether IP in advance, so it *broadcasts*.

DEFINITION

Unicast — A message aimed at one specific IP address — a letter to one house.

DEFINITION

Broadcast — A message aimed at *every* device on a subnet at once — a flyer to the whole street. The limited broadcast address is `255.255.255.255`.

DEFINITION

Directed broadcast — A broadcast aimed at one specific subnet, e.g. `192.168.42.255`, which reaches every host whose address starts `192.168.42`.

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 143-160 Kotlin

1     private fun usbBroadcastAddress(): String? {
2         try {
3             val ifaces = java.net.NetworkInterface.getNetworkInterfaces() ?: return null
4             for (el in ifaces) {
5                 val n = el.name?.lowercase() ?: continue
6                 if ((n.contains("rndis") || n.contains("usb")) && el.isUp) {
7                     for (ia in el.interfaceAddresses) {
8                         val a = ia.address
9                         val b = ia.broadcast
10                        if (a is java.net.Inet4Address && !a.isLoopbackAddress && b is java.net.Inet4Address
11                    s) {
12                            return b.hostAddress
13                        }
14                    }
15                }
16            } catch (e: Exception) { e.printStackTrace() }
17            return null
18        }
19    }
```

This walks the phone's network interfaces, finds the USB-tether one (its name contains `rndis` or `usb`), and returns that interface's *directed* broadcast address (like `192.168.42.255`). Why not just use `255.255.255.255`? Because a limited broadcast only leaves the phone through its default route — the Wi-Fi — so it would never travel down the USB cable. A directed subnet broadcast is routed out the interface that owns that subnet, which is the USB link. The phone blasts its 20-byte packets to that broadcast address until the PC answers; the answer tells the phone the PC's real unicast IP, and from then on it aims there. That lock-on is Section 4.

The 20-Byte Input Packet, Byte for Byte



Figure 6.2 — The input packet on the wire.

This is the single most important contract in the entire project: the exact shape of one input message. Both ends must agree on it to the byte, or the PC reads garbage. It is defined once in C++ and once in Python, and they must match.

DEFINITION

Struct — A fixed layout describing how several values are packed into a run of bytes, in a defined order and size.

DEFINITION

Byte offset — How many bytes from the start of the packet a field begins. Offset 0 is the very first byte.

apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 51-65

C++

```
1 #pragma pack(push, 1)
2 struct GamepadPayload {
3     uint64_t timestamp; // 8 bytes
4     uint16_t buttons;    // 2 bytes
5     uint8_t leftTrigger; // 1 byte
6     uint8_t rightTrigger; // 1 byte
7     uint8_t leftStickX;  // 1 byte
8     uint8_t leftStickY;  // 1 byte
9     uint8_t rightStickX; // 1 byte
```

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · (cont.) C++

10     uint8_t rightStickY; // 1 byte
11     uint32_t authToken; // 4 bytes
12 };
13 #pragma pack(pop)
14
15 static_assert(sizeof(GamepadPayload) == 20, "FATAL: Payload struct is not exactly 20 bytes!");
```

DEFINITION

`#pragma pack(1)` — A compiler instruction meaning "do not insert hidden padding between fields." Normally a compiler aligns an 8-byte field to an 8-byte boundary, which would push this struct past 20 bytes. `pack(1)` forces the fields flush against each other.

DEFINITION

`static_assert` — A compile-time check. If `sizeof(GamepadPayload)` is ever not exactly 20, the app *refuses to build* with that error message — a safety net so a stray field can never silently break the wire format.

Add up the sizes: $8 + 2 + 1 \times 6 + 4 = 20$. The Python side must decode the identical layout, which it does with a `struct` format string. In `server.py`: `PAYLOAD_FORMAT = '<Q H B B B B B I'` and `PAYLOAD_SIZE = struct.calcsize(PAYLOAD_FORMAT)` (which computes to 20). The `<` means little-endian; `Q` is the 8-byte timestamp, `H` the 2-byte buttons, six `B` bytes for triggers and sticks, and `I` the 4-byte auth token.

DEFINITION

Little-endian — A convention for storing multi-byte numbers with the *least* significant byte first. Both Android and Windows-x86 are little-endian, so the two ends agree without conversion.

WATCH OUT

The comment header above `PAYLOAD_FORMAT` in `server.py` reads "16-Byte Payload Structure" — that label is stale. The fields it lists total 20 bytes and `PAYLOAD_SIZE` really is 20. Trust the code, not the header.

Here is the authoritative byte map:

Offset	Size	Field	Meaning
0	8	timestamp	ordering / RTT (see below)
8	2	buttons	bitmask, bits 0-14
10	1	leftTrigger	0 released ... 255 full
11	1	rightTrigger	0 released ... 255 full
12	4	LSX, LSY, RSX, RSY	0...128(center)...255 each
16	4	authToken	pairing-key hash (or 0)

DEFINITION

Bitmask — A single number where each *bit* is an independent on/off flag. The 16-bit `buttons` field packs up to 16 buttons: bit 0 = A, bit 1 = B, ... bit 13 = D-Pad Right, and **bit 14 = the Guide/Xbox button**. (The `ARCHITECTURE.md` bit table stops at bit 13; the code carries bit 14 as Guide — the code wins.)

Who actually fills in the bytes

Here is a subtle truth that contradicts the older architecture notes. The web UI *does* write a timestamp and an auth token — but the native layer **overwrites both** just before sending.

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 292-295 C++
1      networkPayload.timestamp = htobe64(rawTimestamp);
2      networkPayload.buttons = htobe16(networkPayload.buttons);
3      networkPayload.authToken = htobe32(expectedHash);
4      // Note: uint8_t analog values do not require endianness swapping.
```

Just above these lines, `rawTimestamp` is computed from `clock_gettime(CLOCK_MONOTONIC, ...)` in **nanoseconds**, not from a wall clock. So the timestamp on the wire is a monotonic nanosecond counter, and `authToken` is set to `expectedHash` — the number parsed from the QR key in `initNetworkNative`. The web UI, meanwhile, writes `Date.now()` (milliseconds since 1970) at offset 0 and a literal `0` at offset 16 (`App.tsx: view.setUint32(16, 0, true)`). Both of those JS values are *discarded* by the native overwrite before the packet leaves the phone over UDP.

NOTE

This is a direct contradiction of `ARCHITECTURE.md`, which claims the timestamp is "ms since epoch" and the token is `0xABCD1234` written by JS. On the real UDP wire the timestamp is monotonic nanoseconds and the token is the QR key hash. The code is the source of truth. (The JS-written `0` token still matters on the USB-debug WebSocket path, which bypasses native — there the server treats loopback as authorized, so `0` is the correct honest value.)

The Handshake in Motion



Figure 6.3 — The GRX handshake wrapping the base protocol.

Now watch a connection come alive, step by step. There is no formal SYN/ACK negotiation like TCP; the "handshake" is simply the phone sending input and the PC answering, after which the phone locks its aim.

When `connectToPC` runs, it eventually calls into native `initNetworkNative`, which prepares the socket and starts the send thread.

DEFINITION

Non-blocking socket — A socket that never makes the thread wait. If there's nothing to receive, the call returns immediately instead of freezing — essential for a tight real-time loop.

apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 686-701

C++

```

1  udpSocket = socket(AF_INET, SOCK_DGRAM, 0);
2  if (udpSocket != -1) {
3      int flags = fcntl(udpSocket, F_GETFL, 0);
4      fcntl(udpSocket, F_SETFL, flags | O_NONBLOCK);
5      int broadcastEnable = 1;

```

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · (cont.) C++
6      setsockopt(udpSocket, SOL_SOCKET, SO_BROADCAST, &broadcastEnable, sizeof(broadcastEnable));
7      applyLowLatencyTos(udpSocket);
8  }
9  // FIX 2: start unconnected — the hot path uses sendto() during broadcast
10 // discovery and only switches to send() after we connect() to the locked peer.
11 socketConnected = false;
12 connectedPeer.s_addr = 0;
13 memset(&serverAddr, 0, sizeof(serverAddr));
14 serverAddr.sin_family = AF_INET;
15 serverAddr.sin_port = htons(port);
16 inet_pton(AF_INET, ip, &serverAddr.sin_addr);
```

It creates a UDP socket (`SOCK_DGRAM`), makes it non-blocking (`O_NONBLOCK`), enables `SO_BROADCAST` (so it may send to a broadcast address for the USB case), tags it as low-latency traffic, and records the destination `serverAddr` from the scanned `ip` and `port`. It starts *unconnected* on purpose: while the destination might still be a broadcast address, it uses `sendto()`. Then the 1000 Hz TX thread launches and packets start flowing.

On the PC, each arriving packet reaches `handle_frame`, which unpacks it and checks the token.

```
apps/pc-server/server.py · lines 1322-1336 Python
1      timestamp, buttons, lt, rt, ls_x, ls_y, rs_x, rs_y, auth_token = \
2          struct.unpack(PAYLOAD_FORMAT, data)
3
4      if not encrypted:
5          # Legacy cleartext auth policy (subnet-agnostic):
6          # - matching token always accepted (QR pairing);
7          # - token 0 accepted ONLY off our primary LAN (USB-tether/loopback).
8          # USB tether is checked explicitly: when the phone's mobile data is
9          # the PC's internet, the default route (and thus lan_ip) can BE the
10         # tether adapter, which made the off-LAN test reject the tether
11         # client and USB tethering silently never connected.
12         if auth_token != expected_hash:
13             if not (auth_token == 0 and
14                     (is_offlan_client(ip, lan_ip) or is_usb_tether_client(ip))):
15                 return
```

`struct.unpack` reverses the exact 20-byte layout into nine Python values. Then the auth rule: a packet whose `auth_token` matches `expected_hash` (the QR key) is always accepted. A packet with token `0` is accepted *only* if it arrives from off the main Wi-Fi LAN — a USB tether or loopback, where by definition only your own PC-and-phone pair live. Any other mismatch simply *returns*, dropping the packet. This is what stops a random device on your Wi-Fi from driving your game.

If the packet passes, the server drives the virtual pad and then answers with an acknowledgement: `sock.sendto(b"ACK" + struct.pack("<Q", timestamp), addr)`. That reply is 11 bytes — the three letters `ACK` plus the *same* 8-byte timestamp echoed back.

DEFINITION

ACK — Acknowledgement — a tiny reply proving the message was received and processed.

DEFINITION

RTT — Round-Trip Time — how long a packet takes to go phone→PC→phone. Measured by subtracting the echoed send-time from "now."

Back on the phone, receiving that ACK does two crucial jobs: it proves the PC is really there, and it lets the phone stop broadcasting and lock onto the PC's unicast address.

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 481-488 C++
1      uint32_t curDest = ntohl(serverAddr.sin_addr.s_addr);
2      bool curIsBroadcast = (curDest == INADDR_BROADCAST) || ((curDest & 0xFF) == 0xFF);
3      if (curIsBroadcast) {
4          serverAddr.sin_addr = fromAddr.sin_addr;
5          char ipStr[INET_ADDRSTRLEN];
6          inet_ntop(AF_INET, &serverAddr.sin_addr, ipStr, sizeof(ipStr));
7          LOGI("Received ACK from PC. Locked destination IP to unicast: %s", ipStr);
8      }
```

If the phone was aiming at a broadcast address (255.255.255.255 or anything ending in .255), it copies the *source* address of the ACK — the PC's true unicast IP — into `serverAddr`. From that moment it sends only to the PC. Shortly after, it `connect()`s the socket to that peer so the kernel caches the route and the hot path can use the cheaper `send()` instead of `sendto()`. That is the whole "pairing" dance: send, get ACK, lock on.

EXPERT DETAIL

The ACK also carries the RTT payload. Because the echoed timestamp is the phone's own `CLOCK_MONOTONIC` nanosecond stamp, the phone computes $RTT = now - echoed$ against the *same* clock, then exponentially smooths it ($prev * 0.8 + rtt * 0.2$). The link is judged "alive" only if an ACK arrived recently (`lastAckMonoNs`), because UDP `sendto()` succeeds locally even when nothing is listening — a rising send count is not proof of a connection.

Staying Alive, and Saying Goodbye Without a Word

Once connected, the phone does *not* blast 1000 identical packets per second. It sends only when something changes, plus a slow heartbeat to keep the link warm.

DEFINITION

Heartbeat / keep-alive — A periodic "still here" packet sent even when nothing changed, so the other side knows the link is live and any dropped packet gets a fresh chance.

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 275-290 C++
1      bool changed = (memcmp(curInput, lastSentInput, 8) != 0);
2      // Adaptive keep-alive: ~30Hz normally (the thermal/CPU win noted above), but
3      // ~60Hz while rumble is active. The PC only emits fresh RMB in reply to an
4      // inbound frame, so when the pad is held steady during rumble a 30Hz uplink
5      // caps rumble-state updates at ~33ms; 60Hz halves that. Reverts to 30Hz the
6      // moment rumble clears (rumbleLeft/Right are the last values the PC sent us).
```

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · (cont.) C++

7     bool rumbleActive = (rumbleLeft.load(std::memory_order_relaxed) != 0) ||
8                          (rumbleRight.load(std::memory_order_relaxed) != 0);
9     uint64_t heartbeatNs = rumbleActive ? 16000000ULL : 33000000ULL;
10    bool heartbeat = (rawTimestamp - lastSendNs) >= heartbeatNs; // adaptive keep-alive
11    // FIX 1: a fresh change arms CHANGED_PACKET_REDUNDANCY back-to-back sends.
12    // While that counter is positive we keep re-sending (redundant) even when
13    // the input hasn't changed again, so a single dropped datagram recovers
14    // fast. The counter is (re)armed below once we confirm a successful send.
15    bool redundant = (redundancyRemaining > 0);
16    bool doSend    = changed || heartbeat || redundant;
```

The thread compares the current 8 input bytes (buttons through sticks) with the last-sent 8. It transmits if any of three things is true: the input **changed**, a **heartbeat** interval elapsed (33 ms $\approx$ 30 Hz normally, 16 ms $\approx$ 60 Hz during rumble), or a **redundant** resend is still owed. When input changes, **CHANGED_PACKET_REDUNDANCY = 3** back-to-back copies are queued so a single lost datagram is recovered within a couple of 2 ms polls instead of waiting for the next change. Holding the stick perfectly still costs only $\sim$ 30 packets a second — that is the fix that stopped the phone from cooking itself at 1000 Hz.

There is no disconnect packet — on purpose

When you close the app, the phone does **not** tell the PC. `stopNetworkNative` just closes the socket, with an explicit comment explaining why: a cleartext, unauthenticated "BYE" packet would let any device on the LAN spoof your source IP and kill your session with 3 bytes. Instead, the PC notices the silence itself, via an idle watchdog.

DEFINITION

Watchdog — Background code that watches for "nothing happening for too long" and cleans up when a timeout passes.

```
apps/pc-server/server.py · lines 866-875 Python

1     def idle_tick(self, now):
2         """Reset briefly-quiet pads to neutral (anti-stuck) and drop long-silent
3         ones so the virtual controller disappears from Windows on disconnect."""
4         with self.lock:
5             drop = []
6             for key, s in self.sessions.items():
7                 gap = now - s.last_seen
8                 if gap > 3.0:
9                     drop.append(key)
10                elif gap > 0.5 and not s.is_neutral:
```

`idle_tick` runs about once a second. For each active controller session it measures **gap**, the seconds since the last packet. A gap over **0.5 s** snaps that pad back to neutral (so a button held at the instant of a drop doesn't stick down forever). A gap over **3.0 s** drops the session entirely, unplugging the virtual controller from Windows. So a clean disconnect just *happens* — quietly, within $\sim$ 3 seconds, no packet required.

DEFINITION

Exponential backoff — A retry strategy that waits longer after each failure (50 ms, 100, 200, ...) up to a cap, to avoid hammering. On the phone, if the UDP socket dies, `txThreadLoop` rebuilds it with backoff from 50 ms up to 1000 ms, then resumes discovery — the reconnect path.

When Packets Vanish

UDP makes no promises: a datagram can be lost, duplicated, or arrive out of order. The protocol is built to shrug all three off, because for real-time input a stale reading is worse than a missing one.

IN PLAIN TERMS

Imagine shouting joystick positions across a noisy room. If one shout is lost, don't repeat it — just shout the *newest* position, which is what matters. If two shouts arrive out of order, ignore the older one. That is exactly the PC's strategy.

First, **loss**: handled by redundancy (three copies on change) and the heartbeat, so a single drop self-heals. Second, **duplicates**: harmless, because the server skips re-applying an identical report (`apply_inputs` compares against `sess.last_report` and does nothing if unchanged) — a duplicate causes no IOCTL to the driver. Third, **out-of-order**: rejected by timestamp.

apps/pc-server/server.py · lines 1360-1367

Python

```
1      # Per-session ordering: drop genuinely out-of-order packets within one stream.
2      if timestamp < sess.last_ts:
3          if (sess.last_ts - timestamp) > 1_000_000_000:
4              sess.last_ts = timestamp    # clock reset → accept
5          else:
6              return
7      sess.last_ts = timestamp
8      sess.last_seen = now
```

Each session remembers the highest timestamp it has seen (`sess.last_ts`). A packet whose timestamp is *older* is a straggler and is dropped with `return`. The one exception is a jump larger than `1_000_000_000` — one second in nanoseconds — which means the phone's monotonic clock reset (a fresh connection), so the server accepts it and re-bases. This is why the phone's monotonic-nanosecond timestamp matters: it gives every packet a strictly increasing sequence number for free.

WATCH OUT

Also note the server *drains* all queued datagrams each wake-up and keeps only the **newest per source IP** (`latest[ip] = (data, addr)`) before processing. If a burst piles up, older ones in the burst are discarded unread — again, freshest-wins. A stale ACK is likewise never sent for a rejected frame, so the unicast lock and liveness signal only ever advance on in-order input.

Wrapping the Whole Thing: GRX and the Full Catalogue

Everything so far is the *legacy cleartext* protocol: the 20-byte packet with a 32-bit token. That token is replayable — anyone who captures one packet can resend it. GRX is the encryption layer that wraps the base protocol to fix that, and it rides the *same* UDP socket.

DEFINITION

AEAD / AES-GCM — Authenticated Encryption with Associated Data. It both hides the contents *and* stamps them with a tag that proves they weren't forged or altered. GRX uses AES-128-GCM.

DEFINITION

Handshake (crypto) — A short exchange at connect time where both sides agree on secret keys, here in one round trip (1-RTT).

DEFINITION

PSK — Pre-Shared Key — a secret both ends already hold. GRX derives its 32-byte PSK from the same QR pairing key, so no new pairing step is needed.

The clever part is *multiplexing*: GRX handshake control frames and data frames share the input socket, distinguished by their first byte.

apps/pc-server/grx_session.py · lines 6-10

Python

```
1 Wire framing (first byte discriminates; data frames always start with VERSION=1):
2 0xE1 CLIENT_HELLO : 0xE1 | client_eph_pub(32) | lp(long_term_id)
3 0xE2 SERVER_HELLO : 0xE2 | server_eph_pub(32) | server_confirm(32)
4 0xE3 CLIENT_CONFIRM : 0xE3 | client_confirm(32)
5 0x01 DATA : GRX data frame (see grx_crypto, 41 B)
```

When the phone connects with GRX enabled, it sends **0xE1 CLIENT_HELLO** carrying a fresh ephemeral X25519 public key. The server replies **0xE2 SERVER_HELLO** (its own public key plus a confirmation MAC), the phone replies **0xE3 CLIENT_CONFIRM**, and both sides now share keys derived by HKDF from the ECDH secret *and* the QR-derived PSK. After that, each 20-byte input becomes a **41-byte** data frame (**WIRE_LEN = HEADER_LEN + PLAINTEXT_LEN + TAG_LEN = 5 + 20 + 16**), starting with byte **0x01**. On the C++ side these control frames are recognized by their **0xE1/0xE2/0xE3** first byte and handed up to Kotlin; the ACK ('A') and rumble ('R') replies can never collide because their first bytes differ.

WATCH OUT

Telling handshake frames from input frames by first byte alone is dangerous, and there was a real bug here. A 20-byte cleartext input begins with a little-endian timestamp whose low byte sweeps **0-255**, so ~1 in 128 input packets accidentally started with **0xE1/0xE2/0xE3** and got misrouted into the handshake path — where unpacking 20 bytes as a 37-byte structure raised an error and *killed the UDP thread*. The fix is `is_handshake`, which requires both the right first byte **and** a minimum length (`{0xE1: 34, 0xE2: 65, 0xE3: 33}`), so a 20-byte input can never masquerade as a handshake.

Here is the complete message catalogue for the base + GRX protocol on the wire:

Message	Direction	First byte(s)	Size	Purpose
---------	-----------	---------------	------	---------

Input (cleartext)	phone → PC	timestamp	20 B	joystick/button state
GRX data	phone → PC	<code>0x01</code>	41 B	encrypted input
CLIENT_HELLO	phone → PC	<code>0xE1</code>	≥34 B	start handshake
SERVER_HELLO	PC → phone	<code>0xE2</code>	65 B	reply + confirm
CLIENT_CONFIRM	phone → PC	<code>0xE3</code>	33 B	finish handshake
ACK	PC → phone	<code>ACK</code>	11 B	receipt + echoed timestamp
RMB (rumble)	PC → phone	<code>RMB</code>	5 B	large + small motor

Versioning and compatibility

The base 20-byte protocol has **no version field at all** — compatibility is guaranteed purely by the struct layout matching, enforced by the C++ `static_assert(... == 20 ...)` and the Python `struct` format. Add a field on one side without the other, and the sizes diverge and packets are ignored. Newer capabilities (like button bit 14 = Guide) are added *within* the existing 20 bytes so old and new stay wire-compatible. GRX, by contrast, carries an explicit `VERSION = 1` byte in every frame and inside its authenticated AAD, so an incompatible GRX build is rejected rather than misread.

Separately, the *application* version — `APP_VERSION = "1.1.16"` in `server.py` — governs the update pipeline, not the wire protocol. The server fetches a manifest from `supportportal.gamepad.space/api/version` and compares versions numerically via `_parse_version` (so `1.2.10 > 1.2.9`). That is about shipping new binaries, and is fully independent of whether two already-running builds can exchange packets.

NOTE

The `GRX_REQUIRED = False` flag in `server.py` still permits the legacy cleartext path today; flipping it to `True` (once the Android GRX build fully ships) hard-disables the replayable token path and closes the downgrade hole for good. Until then, the two protocols coexist on the one socket, each recognized by frame length and first byte.

CHAPTER 7

The PC Server — `server.py` from Socket to Virtual Pad

7

What server.py Is, and How It Wakes Up

Everything in this chapter lives in one file: `apps/pc-server/server.py`, about 1,992 lines of Python. It is the program that runs on your Windows PC while you play. Its whole job is to sit quietly, wait for tiny messages from your phone, and pretend to Windows that a real Xbox controller is plugged into a USB port. When you press A on the phone, this program makes Windows believe an A button clicked on a physical pad.

DEFINITION

Python — A programming language that runs one line at a time, top to bottom, with no separate "compile" step during development. You run a `.py` file directly and it executes.

DEFINITION

server — A program that waits for other programs (here, your phone app) to contact it, then responds. It does not go looking for the phone; the phone comes to it.

IN PLAIN TERMS

Think of `server.py` as a hotel front desk. It does not chase guests down the street. It stands at the desk (a network port), and when a guest (your phone) walks up and hands over a slip of paper (a 20-byte packet), the desk reads the slip and does exactly what it says — press this button, tilt this stick — then hands back a receipt (an ACK).

The very first thing a Python file does is pull in the tools it needs. That is what the top ten lines do.

apps/pc-server/server.py · lines 1-10

Python

```
1 import socket
2 import selectors
3 import struct
4 import re
5 import qrcode
6 import sys
7 import threading
8 import time
9 import os
10 import secrets
```

DEFINITION

import — A line that loads another chunk of code (a "module") so you can use it. `import socket` makes all of the networking functions available under the name `socket`.

Each of these is a capability you will meet later in the chapter. `socket` opens network connections. `selectors` lets one thread watch a socket efficiently. `struct` turns raw bytes into numbers and back. `qrcode` draws the pairing QR image. `sys` and `os` talk to the operating system. `threading` runs several jobs

at once. `time` measures moments. `secrets` generates a cryptographically strong random pairing key. The first six are third-party or standard-library helpers; `struct`, `socket`, and `threading` are the load-bearing three you will see over and over.

Immediately after the imports, the file does something defensive. In a windowed build there is no console attached, so `sys.stdout` can be `None`; lines 12-15 point `stdout` and `stderr` at `os.devnull` so any stray `print()` never crashes the program by writing to a missing pipe.

WATCH OUT

This matters for "logging". Nearly all diagnostics in `server.py` are plain `print()` calls. In the packaged `.exe` (a windowed build), `stdout` is redirected to the null device — so those prints go nowhere. Real, persistent diagnostics are written to actual files next to the exe: `update_check.log`, `GamepadServer_error.log`, and `_grxserver.log`. When you run `server.py` from source in a terminal, the prints appear; in the shipped app they are silent by design.

One server, and only one

If you double-click the app twice you must not get two servers fighting over the same network port. The guard is a Windows "mutex" — a named flag the operating system lets only one process own at a time.

apps/pc-server/server.py · lines 17-29

Python

```
1 def _acquire_single_instance():
2     """Ensure only ONE server runs per user session."""
3     if not sys.platform.startswith("win"):
4         return True
5     try:
6         import ctypes
7         ERROR_ALREADY_EXISTS = 183
8         h = ctypes.windll.kernel32.CreateMutexW(None, False, "RemoteGamepadServerSingleton")
9         if not h or ctypes.windll.kernel32.GetLastError() == ERROR_ALREADY_EXISTS:
10            return None
11        return h
12    except Exception:
13        return True
```

DEFINITION

mutex — Short for "mutual exclusion" — a named token the OS hands to exactly one owner. If a second process asks for the same name, it is told "already exists" instead of getting its own.

DEFINITION

ctypes — A Python module that lets Python call raw Windows system functions directly, as if it were a C program. `ctypes.windll.kernel32.CreateMutexW(...)` reaches into Windows itself.

This calls `CreateMutexW` with the fixed name `"RemoteGamepadServerSingleton"`. If Windows reports error 183 (`ERROR_ALREADY_EXISTS`), another copy is already running, so the function returns `None`; the code just below (lines 31-42) then pops up a friendly "already running" message and exits. Windows automatically releases the mutex when the owning process dies, so a crash never permanently locks you

out.

NOTE

A subtle detail: this function is *defined twice* — once here at line 17 and again, with a longer docstring, at line 598. But `_singleton = _acquire_single_instance()` runs at line 31, before the second definition exists. So the version that actually executes is this one, lines 17-29. The second copy is never called. If code and comments ever seem to disagree, trust the line that runs.

Loading the driver bridge without crashing

The single most fragile moment of startup is `import vgamepad`. That library connects to the ViGEmBus driver *the instant it is imported* — and on a brand-new PC that driver is not installed yet, so a plain import would crash before your `main()` ever runs.

apps/pc-server/server.py · lines 82-98

Python

```

1 def _load_vgamepad():
2     try:
3         return importlib.import_module("vgamepad")
4     except Exception as err:
5         if "BUS_NOT_FOUND" not in str(err) and "vigem" not in str(err).lower():
6             raise # a genuinely unrelated import error – surface it as-is
7
8     install_vigembus_driver() # bundled ViGEmBus MSI + one UAC prompt
9     # The MSI installs asynchronously after the prompt, so poll the import until
10    # the driver registers (up to ~40s), re-running module init each attempt.
11    for _ in range(40):
12        for _m in [k for k in list(sys.modules) if k.startswith("vgamepad")]:
13            del sys.modules[_m]
14        try:
15            return importlib.import_module("vgamepad")
16        except Exception:
17            time.sleep(1)

```

DEFINITION

ViGEmBus — A free, signed Windows driver (by "nefarius") that lets user programs create fake game controllers. Without it there is no virtual pad. `vgamepad` is the Python library that talks to it.

The function tries the import. If it fails with a message mentioning `BUS_NOT_FOUND` or `vigem`, it knows the driver is missing, calls `install_vigembus_driver()` (which runs an MSI bundled *inside* your own exe), then loops up to 40 times — deleting the half-loaded module from `sys.modules` and re-importing once per second — until the freshly-installed driver answers. This is why your app can install "everything it needs" on a fresh PC with no separate download. The result is stored in the module-level name `vg` at line 112, and every virtual-pad call in the file goes through `vg`.

The Wire Format: Parsing 20 Bytes with struct

Your phone does not send words like "button A pressed". It sends 20 raw bytes, tightly packed, arranged in a fixed order both sides agree on. The agreement is one line.

apps/pc-server/server.py · lines 114-125

Python

```

1 # 16-Byte Payload Structure:
2 # [8 bytes uint64] Timestamp
3 # [2 bytes uint16] Buttons Bitmask
4 # [1 byte uint8] Left Trigger
5 # [1 byte uint8] Right Trigger
6 # [1 byte uint8] Left Stick X
7 # [1 byte uint8] Left Stick Y
8 # [1 byte uint8] Right Stick X
9 # [1 byte uint8] Right Stick Y
10 # [4 bytes uint32] Auth Token
11 PAYLOAD_FORMAT = '<Q H B B B B B I'
12 PAYLOAD_SIZE = struct.calcsize(PAYLOAD_FORMAT)

```

DEFINITION

struct — A Python module that converts between raw bytes and Python numbers using a "format string". It is the translator between the phone's byte stream and usable values.

DEFINITION

byte — The smallest addressable unit of data, holding a number from 0 to 255. Twenty of them make up one input packet.

WATCH OUT

The comment says "16-Byte" but that is stale. Add the field sizes it lists: $8 + 2 + 1 + 1 + 1 + 1 + 1 + 1 + 4 = 20$ **bytes**. And `struct.calcsize('<Q H B B B B B I')` computes 20 at runtime, which is what `PAYLOAD_SIZE` actually equals. The code wins: every packet is exactly 20 bytes, and the whole file checks `len(data) == PAYLOAD_SIZE`. The "16" in the comment header is simply wrong.

Reading the format string left to right: `<` means little-endian (least significant byte first) and, crucially, forces *standard* sizes with *no padding* between fields — so the 20 bytes are exactly as listed, no surprise gaps. `Q` is an unsigned 64-bit integer (the 8-byte timestamp, offsets 0-7). `H` is an unsigned 16-bit integer (the 2-byte button bitmask, offsets 8-9). The six `B`s are single unsigned bytes (offsets 10-15): left trigger, right trigger, then left-stick X/Y and right-stick X/Y. `I` is an unsigned 32-bit integer (the 4-byte auth token, offsets 16-19).

Field	Type	Bytes	Offset
Timestamp	uint64	8	0-7
Buttons bitmask	uint16	2	8-9
LT, RT, LSX, LSY, RSX, RSY	uint8 each	6	10-15
Auth token	uint32	4	16-19

DEFINITION

little-endian — A byte-ordering where the smallest-value byte comes first. The number 0x0102 is stored as 02 01. Both phone and PC must agree, and the < in the format string is that agreement.

DEFINITION

bitmask — A single number whose individual binary bits each mean "on" or "off" for a different button. One 16-bit number can carry 15 button states at once (bits 0 through 14).

To turn 20 received bytes back into numbers you call `struct.unpack(PAYLOAD_FORMAT, data)`, which returns a tuple of nine values in that exact order. You will see that exact call in every one of the three transports later. This one format string is the single source of truth the phone code and the PC code must both obey — change it on one side without the other and every button becomes garbage.

Opening the Sockets: Ports, UDP, and the Firewall

Now the server needs an address on the network where the phone can reach it. That address is a "socket" bound to a "port".

DEFINITION

socket — A software endpoint for network communication — like a mailbox the OS gives your program. You write into it to send and read from it to receive.

DEFINITION

port — A numbered slot on your PC (0-65535) so multiple programs can share one network card without mixing up each other's mail. Your server lives on port 7777.

DEFINITION

UDP — User Datagram Protocol. It fires small independent packets with no handshake and no guarantee of delivery or order — fast, but "fire and forget". Perfect for game input, where the newest packet matters and a lost one is simply replaced 16 milliseconds later.

apps/pc-server/server.py · lines 658-686

Python

```

1 def bind_server_socket(base_port=7777, max_attempts=10):
2     for port in range(base_port, base_port + max_attempts):
3         try:
4             sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
5             sock.setblocking(False)
6             # Stop a vanished client from killing the recv thread (see helper).
7             suppress_udp_connreset(sock)
8             # Low-latency socket tuning:
9             # - Large RX buffer so a burst of 1000Hz packets is never dropped
10            #   while the thread is briefly busy (we still drain to newest).
11            # - Modest TX buffer so ACK sends never block on a full kernel buffer.
12            # - IP_TOS DSCP-EF to match the client's QoS tagging symmetrically.
```



```
apps/pc-server/server.py · (cont.) Python
13         try:
14             sock.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 1 << 20) # 1 MB
15         except OSError:
16             pass
17         try:
18             sock.setsockopt(socket.SOL_SOCKET, socket.SO_SNDBUF, 1 << 20) # 1 MB ACK headroom
19         except OSError:
20             pass
21         try:
22             # Match the phone's outbound tagging (DSCP-EF, Expedited Forwarding)
23             # so our ACKs land in the same WMM/QoS class as inbound packets.
24             IPTOS_DSCP_EF = 0xB8
25             sock.setsockopt(socket.IPPROTO_IP, socket.IP_TOS, IPTOS_DSCP_EF)
26         except OSError:
27             pass
28         sock.bind(('0.0.0.0', port))
29         return sock, port
```

`socket.socket(socket.AF_INET, socket.SOCK_DGRAM)` creates an IPv4 UDP socket. `setblocking(False)` makes reads return instantly instead of freezing the thread when no packet is waiting — essential for the tight loop later. The three `setsockopt` calls tune it: a 1 MB receive buffer (`1 << 20` is one bit shifted 20 places = 1,048,576) so a burst of packets is never dropped while the thread is busy; a 1 MB send buffer so ACKs never stall; and `IP_TOS = 0xB8` (DSCP Expedited Forwarding) to tag outgoing packets as high-priority on Wi-Fi. Finally `sock.bind(('0.0.0.0', port))` claims the port on *all* network interfaces (`0.0.0.0` means "any adapter"). The `for` loop tries 7777 first and, if it is taken, walks 7778, 7779, up to 7786 — ten attempts — before giving up.

NOTE

There is also a WebSocket transport (USB-debug mode) that uses TCP on port 7777 (`WS_PORT = 7777`). UDP 7777 and TCP 7777 do not collide: they are different protocols, and the OS tracks them separately. So one port number, 7777, serves both the Wi-Fi path (UDP) and the wired WebSocket path (TCP).

The firewall problem

Binding a socket is not enough. Windows Firewall silently drops inbound packets from your phone unless a rule allows them — the single most common cause of "the phone says connecting but nothing happens", worst on Public networks. The `ensure_firewall_rule()` function (line 479) writes a tiny `.bat` file that runs `netsh` to delete stale rules and add a *program-scoped* allow rule — "let this exe receive on any port, any profile, UDP and TCP". A program rule is bulletproof because it covers the 7778+ fallback automatically. If you are already an administrator it applies silently; otherwise it triggers one UAC prompt, and if you decline, private networks usually still work.

DEFINITION

firewall — The part of Windows that decides which network packets are allowed in. By default it blocks unsolicited inbound traffic, which includes your phone's very first packet.

The pairing QR

The phone needs to know your PC's IP address, the port, and the pairing key. That trio is encoded into a QR code the phone scans. In `main()` (lines 1926-1930) the server builds the string `f"{ip},{port},{key}"` — three comma-separated values like `192.168.1.5,7777,a3f19b02` — and feeds it to `qrcode.QRCode(box_size=6, border=2)` to render the black-on-white image the window displays. `get_lan_ip()` (line 432) picks the *right* IP: it scores candidates so a real `192.168.x` home-LAN address beats a VPN tunnel's `10.x` or a VirtualBox `192.168.56.x`, which is what keeps QR pairing working even with Surfshark connected. The `key` is an 8-hex-character pairing secret generated by `secrets.token_hex(4)` and persisted in `%LOCALAPPDATA%\GamepadServer\pairing_key.txt` so the same PC always shows the same key — you only re-scan when the IP changes. On the server side, `expected_hash = int(key, 16)` turns that hex key into the 32-bit number every cleartext packet's auth-token field must match.

The Virtual Pad: vgamepad, Pads, and apply_inputs

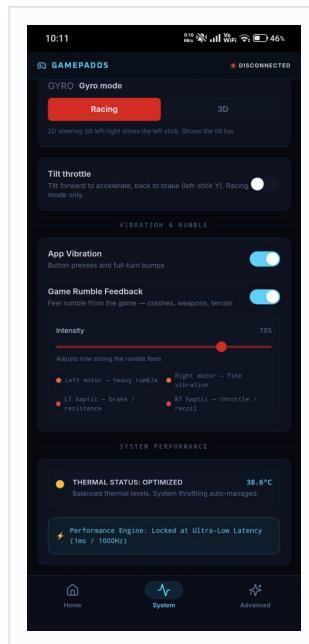


Figure 7.1 — Rumble the server sends back, surfaced in the app.

Now the payoff. Decoded numbers must become an actual Xbox controller Windows can see. That controller is a `VX360Gamepad` object from `vgamepad`.

DEFINITION

virtual gamepad — A software-only controller that ViGEmBus injects into Windows so games read it exactly like a real USB Xbox 360 pad. `vg.VX360Gamepad()` creates one.

Because you can connect up to four phones, the server keeps a `PadManager` that owns one virtual pad per phone. Each connected phone gets its *own* controller so their inputs never merge — session keyed by the phone's source IP (or `"usb:N"` / `"aoa"` for the wired transports), capped at `MAX_PADS = 4` (XInput's limit).

apps/pc-server/server.py · lines 834-841

Python

```

1         try:
2             with GAMEPAD_LOCK:
3                 pad = vg.VX360Gamepad()
4                 reset_gamepad_state(pad)
5                 pad.update()
6         except Exception:
7             return None
8         s = PadSession(pad)

```

DEFINITION

GAMEPAD_LOCK — A `threading.Lock` that guarantees only one thread writes to a virtual pad at a time. Since the UDP thread, the WebSocket thread, and the idle watchdog can all touch pads, every pad write is wrapped in `with GAMEPAD_LOCK:` so they never collide.

This creates a fresh `VX360Gamepad`, resets it to true neutral (`reset_gamepad_state` centers both sticks on exactly 0.0 so Windows never reads a phantom nudge), calls `pad.update()` to push that neutral state to the driver, and wraps it in a `PadSession` — a small object (defined with `__slots__` for memory efficiency at line 761) that remembers this pad's last report, timestamps, address, and rumble values. If pad creation throws, it returns `None` and the caller declines the connection.

Sticks and triggers

The heart of the translation is `apply_inputs()`. First the analog axes.

apps/pc-server/server.py · lines 963-984

Python

```

1     sess.is_neutral = False
2
3     def _snap(v):
4         return 128 if abs(v - 128) <= 4 else v
5     ls_x = _snap(ls_x); ls_y = _snap(ls_y)
6     rs_x = _snap(rs_x); rs_y = _snap(rs_y)
7
8     report = (buttons, lt, rt, ls_x, ls_y, rs_x, rs_y)
9     if report == sess.last_report:
10         return
11     sess.last_report = report
12
13     gamepad = sess.pad
14     with GAMEPAD_LOCK:
15         reset_gamepad_state(gamepad)
16         gamepad.left_trigger(value=lt)
17         gamepad.right_trigger(value=rt)
18
19     def _axis(b):
20         return max(-1.0, min(1.0, (b - 128) / 127.0))
21     gamepad.left_joystick_float(x_value_float=_axis(ls_x), y_value_float=-_axis(ls_y))
22     gamepad.right_joystick_float(x_value_float=_axis(rs_x), y_value_float=-_axis(rs_y))

```

The phone sends each stick axis as a byte 0-255 where 128 is centered. `_snap` applies a tiny deadzone: any value within 4 of 128 is forced to exactly 128, killing jitter from a resting thumb. Then a dedup check — if the whole `report` tuple is identical to the last one applied, it returns early and skips the driver write entirely (no change means no IOCTL, saving work). `_axis` converts a byte to the -1.0...+1.0 float the pad wants: `(b - 128) / 127.0`, clamped so it never exceeds the legal range. Note the Y axes are negated (`-_axis(...)`) because screen-down and stick-down point opposite ways. Triggers are simpler: `lt` and `rt` are already 0-255, exactly what `left_trigger/right_trigger` expect, so they pass straight through.

Buttons: reading the bitmask

The 16-bit button number is decoded bit by bit.

```
apps/pc-server/server.py · lines 986-1006 Python
1      XB = vg.XUSB_BUTTON
2      if buttons & (1 << 0): gamepad.press_button(button=XB.XUSB_GAMEPAD_A)
3      if buttons & (1 << 1): gamepad.press_button(button=XB.XUSB_GAMEPAD_B)
4      if buttons & (1 << 2): gamepad.press_button(button=XB.XUSB_GAMEPAD_X)
5      if buttons & (1 << 3): gamepad.press_button(button=XB.XUSB_GAMEPAD_Y)
6      if buttons & (1 << 4): gamepad.press_button(button=XB.XUSB_GAMEPAD_LEFT_SHOULDER)
7      if buttons & (1 << 5): gamepad.press_button(button=XB.XUSB_GAMEPAD_RIGHT_SHOULDER)
8      if buttons & (1 << 6): gamepad.press_button(button=XB.XUSB_GAMEPAD_START)
9      if buttons & (1 << 7): gamepad.press_button(button=XB.XUSB_GAMEPAD_BACK)
10     if buttons & (1 << 8): gamepad.press_button(button=XB.XUSB_GAMEPAD_LEFT_THUMB)
11     if buttons & (1 << 9): gamepad.press_button(button=XB.XUSB_GAMEPAD_RIGHT_THUMB)
12     if buttons & (1 << 10): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_UP)
13     if buttons & (1 << 11): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_DOWN)
14     if buttons & (1 << 12): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_LEFT)
15     if buttons & (1 << 13): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_RIGHT)
16     # Guide (bit 14, phone's ): forwarded on every transport so the Xbox
17     # overlay works. An old Windows build reset the RNDIS tether adapter when
18     # the Game Bar opened; retested clean on Win11 26200 (2026-07-03).
19     # allow_guide (default True) is kept as a per-session escape hatch.
20     if buttons & (1 << 14) and sess.allow_guide:
21         gamepad.press_button(button=XB.XUSB_GAMEPAD_GUIDE)
```

DEFINITION

bitwise AND (&) — An operator that checks bits. `buttons & (1 << 3)` isolates bit 3: `1 << 3` is the number 8 (binary `1000`), and the AND is non-zero only if bit 3 of `buttons` is set. That is how one number carries 15 independent yes/no button states.

Each line tests one bit and, if set, presses the matching Xbox button. The mapping is fixed: bit 0 = A, 1 = B, 2 = X, 3 = Y, 4 = left shoulder, 5 = right shoulder, 6 = Start, 7 = Back, 8 = left thumb-click, 9 = right thumb-click, 10-13 = D-pad up/down/left/right, and bit 14 = Guide (the Xbox button, gated by `sess.allow_guide`). Because `reset_gamepad_state` cleared everything at the top of the lock block, any button whose bit is *not* set simply stays released. The final `gamepad.update()` (line 1008) flushes this whole snapshot to the driver in one atomic write.

Threads and the Three Transports

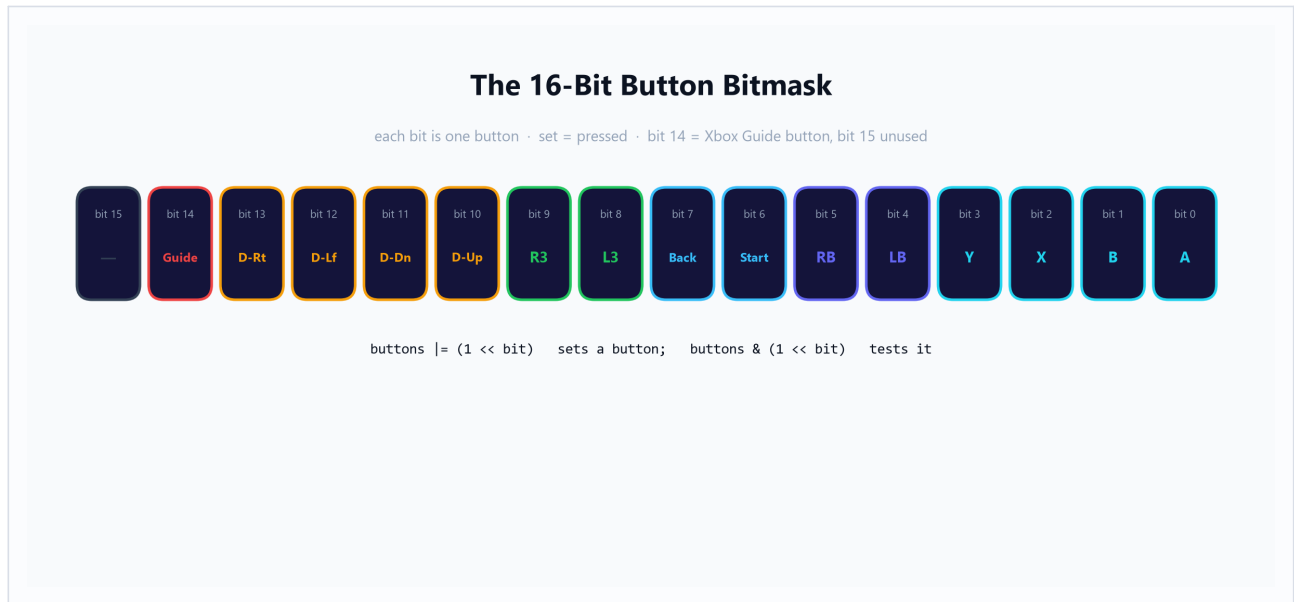


Figure 7.2 — The button bits the server maps to XUSB buttons.

A gamepad server must do several things at once: listen for Wi-Fi packets, serve the wired WebSocket, watch for a direct-USB accessory, and keep the window responsive. Python does this with threads.

DEFINITION

thread — An independent line of execution inside one program. Multiple threads run seemingly at the same time, sharing the program's memory. Here, one thread listens on UDP while another draws the window.

DEFINITION

daemon thread — A background thread that does not keep the program alive. When the main window closes, all daemon threads die with it automatically — no manual cleanup needed.

The `main()` function launches them right after binding the socket.

apps/pc-server/server.py · lines 1933-1947

Python

```

1  udp_thread = threading.Thread(
2      target=run_udp_loop,
3      args=(sock, padmgr, expected_hash, ip, key),    # key -> GRX PSK (encrypted path)
4      daemon=True
5  )
6  udp_thread.start()
7
8  # USB-debugging wired transport: keep `adb reverse` alive and serve the
9  # WebSocket bridge on localhost. Wi-Fi/tether (UDP) keeps working alongside it.
10 start_adb_reverse_watcher(WS_PORT)
11 threading.Thread(target=start_ws_bridge, args=(padmgr, WS_PORT), daemon=True).start()
12
13 # AOA direct-USB transport (lowest-latency wired path, ~1-2 ms). Coexists with

```

apps/pc-server/server.py · (cont.)

Python

```

14     # UDP/WS; no-ops if pyusb/libusb or a WinUSB-bound accessory isn't present.
15     threading.Thread(target=start_aoa_bridge, args=(padmgr,), daemon=True).start()

```

Four background jobs start here, all daemon: `run_udp_loop` (Wi-Fi and USB-tether, the main path), `start_adb_reverse_watcher` (keeps `adb reverse tcp:7777 tcp:7777` alive so a USB-debugging phone's localhost reaches you), `start_ws_bridge` (the WebSocket server on TCP 127.0.0.1:7777), and `start_aoa_bridge` (Android Open Accessory, raw-USB, ~1-2 ms latency). After these, `main()` calls `run_gui(...)` on the *main* thread, which blocks until you close the window. The three transports coexist — the phone uses whichever it is on, and the server applies whatever arrives.

The UDP receive loop

`run_udp_loop` (line 1298) is where Wi-Fi input actually lands. It uses a `selectors` object to wait efficiently on the non-blocking socket, then *drains* every waiting datagram in a burst and keeps only the newest per source IP (`latest[ip] = (data, addr)`), so a momentary stall never makes you replay stale input. Datagrams are sorted by length: 41 bytes is a GRX-encrypted frame, exactly `PAYLOAD_SIZE` (20) is a legacy cleartext frame.

DEFINITION

selectors — A standard-library module that asks the OS "tell me when this socket has data" instead of busy-waiting. `sel.select(timeout=0.2)` sleeps up to 0.2 s, waking the instant a packet arrives.

For each accepted 20-byte frame, `handle_frame` unpacks it, checks the auth token (a matching token passes; token 0 passes only from USB-tether or loopback), calls `apply_inputs`, and then replies.

apps/pc-server/server.py · lines 1377-1386

Python

```

1         try:
2             sock.sendto(b"ACK" + struct.pack("<Q", timestamp), addr)
3         except Exception:
4             pass
5
6         with telemetry.lock:
7             telemetry.transport = "Wi-Fi"
8             telemetry.client_ip = ip
9             telemetry.last_packet_time = last_packet_time
10            telemetry.packet_count += 1

```

DEFINITION

ACK — An acknowledgement — a receipt sent back so the phone knows its packet arrived. Here it is the literal bytes `ACK` followed by the original 8-byte timestamp packed with `struct.pack("<Q", timestamp)`.

The ACK echoes the phone's own send-time back, so the phone can subtract it from the current time to measure true round-trip latency. Crucially, the ACK is sent *after* `apply_inputs`, so the pad update (what the game feels) is never delayed by this network syscall. Then, under `telemetry.lock`, it records the

transport name, client IP, and a running packet count — the numbers the GUI reads to show "Connected devices". `telemetry` is a shared `ServerTelemetry` object (line 281) protected by its own lock because both the network threads and the GUI thread read and write it.

Rumble: feedback back to the phone

Force-feedback flows the opposite direction — from the game, through the pad, back to the phone that should buzz. When a pad is created, it registers a callback (`_cb`, line 844) that ViGEm calls whenever the game emits rumble; the callback stashes the two motor strengths onto the `PadSession`. The UDP loop then ships them back inside `handle_frame`.

```
apps/pc-server/server.py · lines 1351-1358 Python
1      r1 = sess.rumble_large & 0xFF
2      rs = sess.rumble_small & 0xFF
3      if (r1 or rs) or sess.last_rmb_sent != (0, 0):
4          try:
5              sock.sendto(b"RMB" + bytes([r1, rs]), addr)
6          except Exception:
7              pass
8          sess.last_rmb_sent = (r1, rs)
```

DEFINITION

rumble / force-feedback — The vibration a game triggers. An Xbox pad has two motors — a large low-frequency one and a small high-frequency one. Their strengths (0-255 each) are `rumble_large` and `rumble_small`.

The message is the literal bytes `RMB` followed by the two motor values. `& 0xFF` clamps each to a single byte. The condition sends while either motor is non-zero (to sustain the buzz) plus one final all-zero message (to stop it), and skips the send otherwise so idle play generates no rumble traffic. The wired transports use the identical `b"RMB"` framing — the WebSocket bridge sends it every 50 ms in an `async_rumble()` task (line 1173), and AOA sends it over the raw USB endpoint (line 1263). Each phone vibrates only for its own pad because the callback closure captured that specific session.

EXPERT DETAIL

The per-pad rumble callback creates a reference cycle: `pad` → `cmp_func` → `_cb` closure → `session` → `pad`. Plain reference counting can never break it, so `PadManager._free` (line 893) explicitly unregisters the notification and nulls `pad.cmp_func` and `s.pad`, forcing the ViGEm target to unplug *immediately* rather than lingering as a ghost pad that squats an XInput slot and pushes the next real phone to player 2.

The Window, the Updater, and Feedback

There is no system-tray icon — the UI is a small Tkinter window (`run_gui`, line 1510) that shows the QR, a live "Connected devices: N" count refreshed every 400 ms, the copyable pairing key, and update/feedback controls. `console=False` in the build means no console window ever appears.

DEFINITION

Tkinter — Python's built-in windowing toolkit. `tk.Tk()` makes the window; `tk.Label`, `tk.Button`, and `tk.Entry` are the widgets inside it; `root.mainloop()` runs the event loop that keeps it responsive.

The version check

On launch, and whenever you click "Check for updates", the server fetches a manifest and compares versions. The current version is a plain constant.

apps/pc-server/server.py · lines 133-140

Python

```

1 APP_VERSION = "1.1.16"
2 # Public manifest served by the backend ({"pc": {"version","url","notes"}, ...}).
3 # Overridable via env var so a backend move never needs a code change + reship.
4 # NOTE 2026-07-02: the previous hardcoded host (admin.gamepad.space) never served
5 # this endpoint – set GAMEPAD_UPDATE_URL or fix the default to the live backend.
6 UPDATE_MANIFEST_URL = os.environ.get(
7     "GAMEPAD_UPDATE_URL",
8     "https://supportportal.gamepad.space/api/version")

```

The shipped version is **1.1.16**. The manifest URL defaults to `https://supportportal.gamepad.space/api/version` but is overridable with the `GAMEPAD_UPDATE_URL` environment variable — so if the backend ever moves you never have to recompile. An earlier default (`admin.gamepad.space`) never served this endpoint; the code now points at the live host, and the comment records that fix.

WATCH OUT

The release rule (from RELEASE.md) is that `APP_VERSION` here, `AppVersion` + `VersionInfoVersion` in `installer/GamepadServer.iss`, and the version registered in the admin Releases panel must *all* match the build you ship. If the exe's baked version and the manifest's advertised version disagree, the in-app updater loops forever: install → relaunch → "update available" again. All three currently read 1.1.16.

apps/pc-server/server.py · lines 174-186

Python

```

1 req = urllib.request.Request(
2     UPDATE_MANIFEST_URL,
3     headers={"User-Agent": "GamepadServer/%s" % APP_VERSION})
4 with urllib.request.urlopen(req, timeout=timeout) as resp:
5     data = json.loads(resp.read().decode("utf-8"))
6 pc = data.get("pc", {}) or {}
7 latest = str(pc.get("version", "")).strip()
8 url = pc.get("url") or ""
9 notes = pc.get("notes") or ""
10 available = bool(latest) and (_parse_version(latest) > _parse_version(APP_VERSION))
11 return {"available": available, "latest": latest, "url": url,
12         "notes": notes, "sha256": str(pc.get("sha256") or ""),
13         "error": None, "kind": None}

```


DEFINITION

JSON — A text format for structured data — nested lists and key/value maps. The manifest is JSON: a "pc" object holding `version`, `url`, `notes`, and `sha256`. `json.loads` parses that text into a Python dictionary.

This downloads the manifest with `urllib.request` (pure standard library, so it works inside the frozen exe with no extra bundling), parses the JSON, reads the `pc` section, and computes `available` by comparing versions *numerically* via `_parse_version` — which splits "1.2.10" into (1, 2, 10) so 1.2.10 correctly outranks 1.2.9 (string comparison would get that wrong). The whole function is wrapped so it *never* raises: on any failure it returns a kind of `offline/server/timeout/parse` so the UI can show a specific message, and logs to `update_check.log`. When an update exists, "Download & install update" runs `download_update` (streams the installer, verifies the SHA-256 and the MZ exe header so a corrupt or HTML-error download is never run) then `launch_installer_elevated` (runs the Inno installer `/SILENT`, which closes the running server and relaunches the new build). The "Send Feedback" dialog posts a JSON ticket to `https://supportportal.gamepad.space/api/support/ticket` tagged `source="pc"`.

Packaging: PyInstaller, the Installer, and driver/

A user cannot run `server.py` directly — they have no Python. So two tools turn it into a single double-clickable `.exe` and a friendly setup wizard.

PyInstaller and the .spec

DEFINITION

PyInstaller — A tool that bundles a Python script plus the Python interpreter and every library into one self-contained `.exe`. A `.spec` file is its recipe.

apps/pc-server/GamepadServer.spec · lines 16-35

Python

```
1  datas=[
2      ('app_icon.png', '.'),
3      ('C:\\Users\\akhil\\AppData\\Roaming\\Python\\Python314\\site-packages\\vgamepad', 'vgamepad'),
4      # adb + its DLLs so the exe can run `adb reverse` self-contained (USB-debugging mode).
5      # Tools now live in F:\\hlooo\\tools\\ (moved out of apps\\); path is relative to this spec.
6      ('..\\..\\tools\\platform-tools\\adb.exe', '.'),
7      ('..\\..\\tools\\platform-tools\\AdbWinApi.dll', '.'),
8      ('..\\..\\tools\\platform-tools\\AdbWinUsbApi.dll', '.'),
9      # libusb-1.0 backend for the AOA direct-USB transport (pyusb needs it).
10     ('C:\\Users\\akhil\\AppData\\Roaming\\Python\\Python314\\site-packages\\libusb\\_platform\\windows\\
\\x86_64\\libusb-1.0.dll', '.'),
11 ],
12     hiddenimports=ws_imports + ['vgamepad', 'aoa_transport', 'usb', 'usb.backend.libusb1',
13                                 'grx_crypto', 'grx_session', 'cryptography'],
14     hookspath=[],
15     hooksconfig={},
16     runtime_hooks=[],
17     excludes=[],
18     noarchive=False,
19     optimize=0,
20 )
```

DEFINITION

datas / hiddenimports — **datas** is the list of extra *files* to embed (icons, the ViGEmBus MSI that ships inside `vgamepad`, `adb.exe`, `libusb-1.0.dll`). **hiddenimports** names modules PyInstaller cannot detect automatically because they are imported lazily or by string — here the whole `websockets` submodule set, plus `aoa_transport`, `grx_crypto`, `grx_session`, and `cryptography`.

The **datas** list bundles `app_icon.png`, the entire `vgamepad` package (which carries the ViGEmBus MSI used by the self-install), `adb.exe` and its two DLLs (so USB-debugging works with nothing extra installed), and `libusb-1.0.dll` for the AOA transport. **hiddenimports** forces in modules that PyInstaller's static scan would miss. At runtime the bundled files are unpacked to a temp folder and found via `sys._MEIPASS` (that is what `get_resource_path` at line 44 reads). The `EXE(...)` block below sets `name='GamepadServer'`, `console=False` (windowed, no terminal), `icon=['app_icon.png']`, and `upx=False` — UPX compression is deliberately off because packed exes are often quarantined by antivirus and slow to launch.

The Inno Setup installer

The `.exe` alone works, but the polished experience is `GamepadServer-Setup.exe`, produced by Inno Setup from `installer/GamepadServer.iss`.

DEFINITION

Inno Setup — A free tool that builds a Windows setup wizard from a script (`.iss`). It installs to Program Files, creates shortcuts, and can run commands during install.

apps/pc-server/installer/GamepadServer.iss · lines 72-90

Inno Setup

```

1 [Run]
2 ; 1) ViGEmBus driver – silent, no reboot. Skipped if the driver service already exists.
3 Filename: "msiexec.exe"; Parameters: "/i ""{tmp}\ViGEmBusSetup_x64.msi"" /qn /norestart"; \
4   StatusMsg: "Installing controller driver (ViGEmBus)..."; Flags: runhidden waituntilterminated; \
5   Check: Is64BitInstallMode
6 Filename: "msiexec.exe"; Parameters: "/i ""{tmp}\ViGEmBusSetup_x86.msi"" /qn /norestart"; \
7   StatusMsg: "Installing controller driver (ViGEmBus)..."; Flags: runhidden waituntilterminated; \
8   Check: not Is64BitInstallMode
9 ; 2) Firewall: delete any stale copies, then add program-scoped allow rules (UDP+TCP, any profile).
10 Filename: "{sys}\netsh.exe"; Parameters: "advfirewall firewall delete rule name=""Gamepad Server (UDP)""";
11   Flags: runhidden
12 Filename: "{sys}\netsh.exe"; Parameters: "advfirewall firewall delete rule name=""Gamepad Server (TCP)""";
13   Flags: runhidden
14 Filename: "{sys}\netsh.exe"; Parameters: "advfirewall firewall delete rule name=""RemoteGamepad Server UDP""";
15   Flags: runhidden
16 Filename: "{sys}\netsh.exe"; Parameters: "advfirewall firewall add rule name=""Gamepad Server (UDP)"" dir=i
17   n action=allow program=""{app}\{#ExeName}"" protocol=UDP profile=any enable=yes"; \
18   StatusMsg: "Adding firewall rule..."; Flags: runhidden
19 Filename: "{sys}\netsh.exe"; Parameters: "advfirewall firewall add rule name=""Gamepad Server (TCP)"" dir=i
20   n action=allow program=""{app}\{#ExeName}"" protocol=TCP profile=any enable=yes"; \
21   StatusMsg: "Adding firewall rule..."; Flags: runhidden
22 ; 3) Launch the server after install. runasoriginaluser → starts NON-elevated (no
23   lingering admin). No skipifsilent → also relaunches after the in-app silent update.
24 Filename: "{app}\{#ExeName}"; Description: "Launch {#AppName} now"; Flags: nowait postinstall runasoriginal
25   user

```

The `[Run]` section is the install-time choreography: install the ViGEmBus MSI silently (`/qn /norestart`, x64 or x86 chosen by `Is64BitInstallMode`, and skipped entirely if `VigemInstalled()` at line 97 finds the driver's registry service), delete any stale firewall rules, add fresh program-scoped UDP and TCP allow rules for the installed exe path, then launch the server *non-elevated* via `runasoriginaluser`. The `[Setup]` section (lines 17-53) fixes `AppId` to a never-changing GUID so upgrades stack cleanly, sets `AppMutex=RemoteGamepadServerSingleton` (the same name the app uses, so Setup asks you to close a running copy), and uses `CloseApplications=force` with `RestartApplications=no` so the one-click updater can replace files without double-launching. `AppVersion` and `VersionInfoVersion` are both 1.1.16, matching `APP_VERSION`.

The driver/ folder

DEFINITION

AOA (Android Open Accessory) — A USB mode where the PC talks to the phone over raw USB bulk endpoints — no IP stack, no adb — for the lowest wired latency (~1-2 ms). GamepadServer already contains the full implementation in `aoa_transport.py`.

The `driver/` folder is *optional* extra setup for that AOA path only. Windows must bind the WinUSB driver to the GamepadOS USB accessory device (VID `18D1`, PID `2D00` for accessory or `2D01` for accessory+adb) before the server can open it. `install-driver.bat` launches Zadig to do that one manual bind, and `README.txt` is candid that you probably do not need it: Wi-Fi and USB-tethering work with no driver at all, and tethering already gives ~2.5-4 ms — AOA is a small extra gain for the cost of the bind (which also disables adb/MTP on that device until removed). It exists for the enthusiast chasing the absolute lowest wired latency; everyone else can ignore the folder entirely.

CHAPTER 8

GRX — The Encrypted Input Layer



Why you encrypt controller input at all

Before this chapter touches a single line of crypto, you need to understand what problem it solves, because the answer is not obvious. A gamepad is not a bank. Nobody is stealing money by watching you press A. So why bother?

Start with what your system used to do. Every 20-byte input frame your phone sent to the PC ended with a 32-bit number called an **authToken**. The PC compared it against a value it computed at pairing time (**expected_hash**). If they matched, the frame drove the virtual pad. If not, it was dropped. That was the entire security model.

DEFINITION

authToken — The old 32-bit trailing field on each input frame that the PC checked to decide whether a packet was "trusted." It is now vestigial — sent as 0 — because GRX authenticates every frame cryptographically instead.

That model has a fatal flaw, and the protocol doc names it in its first sentence: the token is **cleartext** and **replayable**.

DEFINITION

cleartext — Data sent with no encryption — anyone who can see the network traffic reads it directly. Also called plaintext.

DEFINITION

replay attack — An attacker records a legitimate packet off the wire and re-sends ("replays") it later to make something happen again, without ever needing to understand or forge its contents.

Here is the concrete attack. You and a stranger are on the same coffee-shop Wi-Fi. The stranger runs a packet sniffer, sees one of your input frames fly by, copies the 32-bit token out of it, and now they can forge input that your PC accepts as yours — jerking your aim, mashing buttons mid-match. The token never changes, so capturing it once is capturing it forever.

DEFINITION

threat model — The precise statement of *who* you are defending against and *what* they can do, so you can judge whether a design actually helps. Security claims are meaningless without one.

GRX writes its threat model down explicitly.

apps/docs/GRX_PROTOCOL.md · lines 12-15

Markdown

```

1  ## Threat model
2  On-path attacker on the same LAN or USB host. Must guarantee: confidentiality,
3  integrity, replay rejection, and MITM/downgrade resistance. Loss-tolerant: a
4  dropped sample is superseded by the next; never retransmit.

```

The attacker sits *on the path* — same LAN, or the USB host your phone is tethered to — and can read, drop, modify, and inject packets. Against that attacker GRX must guarantee four things: **confidentiality** (they can't read your input), **integrity** (they can't modify it undetected), **replay rejection** (they can't re-send old packets), and **MITM/downgrade resistance** (they can't impersonate either end or force you back to the weak cleartext path). The last line is the design's escape hatch: gamepad input is *loss-tolerant*. If one packet is dropped, the next one supersedes it 1–2 ms later, so GRX never retransmits — which, as you will see, makes the anti-replay design far simpler than TCP's.

DEFINITION

GRX — GamepadOS Realtime eXchange — the authenticated-encryption layer for the phone↔PC input link. "AEAD over the existing UDP transport, ~0.3 μs per packet with hardware AES."

Symmetric, asymmetric, and the X25519 key exchange

Encryption comes in two families, and GRX uses both, each for the job it is good at.

DEFINITION

symmetric encryption — One shared secret key both encrypts and decrypts. Fast, but both sides must already possess the same key — which raises the question of how they got it.

DEFINITION

asymmetric (public-key) encryption — A mathematically linked *pair* of keys: a private key you keep secret and a public key you hand out. Something done with one can only be undone with the other. Slower, but it lets two strangers agree on a secret over an open wire.

IN PLAIN TERMS

Symmetric encryption is a padlock where the same brass key locks and unlocks — great, once both of you have a copy. The hard part is mailing a copy to your friend without a thief on the route copying it. Asymmetric crypto is the trick that lets you and your friend *shout numbers across a crowded room* and end up both knowing a secret that nobody else in the room can figure out, even though they heard every word.

That shouting trick has a name.

DEFINITION

Diffie-Hellman key exchange — A method where each party mixes their own private number with the other party's public number and — thanks to the math — both arrive at the *same* shared secret, while an eavesdropper who saw only the two public numbers cannot compute it.

DEFINITION

elliptic curve — A particular kind of mathematical structure (points on a curve you can "add" together) that makes Diffie-Hellman fast and secure with small keys. X25519 is a specific, widely trusted elliptic curve tuned for exactly this.

DEFINITION

X25519 — The elliptic-curve Diffie-Hellman function GRX uses. You feed it your 32-byte private key and their 32-byte public key; it returns a 32-byte shared secret. Both sides compute the identical secret.

GRX generates a **fresh** X25519 keypair for every single connection, not once per pairing. The protocol doc is emphatic about why: reusing a key while resetting the packet counter to zero is "a total GCM break" (you will see exactly why in the next section).

DEFINITION

ephemeral key — A keypair created for one session and thrown away when it ends. Ephemeral X25519 keys give you *forward secrecy* — recording today's traffic is useless even if a key leaks tomorrow, because tomorrow's session uses a brand-new key.

AES-GCM, nonces, and why reuse is fatal

Once both sides share a secret, they switch to fast symmetric encryption for the actual input stream. GRX uses AES-128-GCM.

DEFINITION

AES — The Advanced Encryption Standard — the standard symmetric cipher, implemented in hardware on essentially every modern CPU and phone (ARMv8), which is why GRX costs almost nothing. The "128" is the key size in bits (16 bytes).

DEFINITION

AEAD — Authenticated Encryption with Associated Data. One primitive that does two jobs at once: it *encrypts* (confidentiality) and it *authenticates* (integrity), producing a tamper-proof tag. If even one bit of the ciphertext is flipped, decryption fails loudly instead of returning garbage.

DEFINITION

GCM — Galois/Counter Mode — the specific AEAD mode of AES that GRX uses. It turns AES into a stream cipher and computes a 128-bit authentication tag over the ciphertext.

DEFINITION

authentication tag — A 16-byte fingerprint GCM appends to the ciphertext. The receiver recomputes it; if it doesn't match, the packet was forged or corrupted and is dropped. GRX uses the full 128-bit tag — "no truncation, costs nothing at 20 B."

GCM's speed comes from a dangerous source: internally it generates a keystream and XORs it with your data. That keystream is derived from the key **and** a per-packet number called the nonce.

DEFINITION

nonce — "Number used once." A value that must be unique for every packet encrypted under a given key. In GCM it is 96 bits (12 bytes).

WATCH OUT

Reusing a (key, nonce) pair in GCM is catastrophic — not "slightly weaker," but a *total break*. If an attacker sees two packets encrypted with the same key and nonce, XORing their ciphertexts cancels the keystream and leaks the XOR of the two plaintexts; worse, it lets the attacker recover GCM's internal authentication key and forge arbitrary packets. This single rule is why GRX is built the way it is.

IN PLAIN TERMS

Think of the keystream as a one-time pad — a ribbon of random numbers you add to your message to scramble it. It is unbreakable *as long as you never reuse a stretch of ribbon*. The nonce is the ribbon's position marker. Use the same marker twice and you've laid the same ribbon over two different messages; subtract one scrambled message from the other and the ribbon vanishes, leaving your two secrets stacked in plain sight.

GRX guarantees uniqueness structurally. The nonce is built, never random:

apps/pc-server/grx_crypto.py · lines 123-128

Python

```
1 def _nonce(direction, counter):
2     return struct.pack('<IQ', direction, counter & 0xFFFFFFFFFFFFFFF)
3
4
5 def _aad(counter):
6     return struct.pack('<BQ', VERSION, counter & 0xFFFFFFFFFFFFFFF)
```

`_nonce` packs a 32-bit `direction` followed by a 64-bit `counter`, little-endian (`<IQ`), giving exactly 96 bits. `direction` is 1 for client→server input and 2 for server→client rumble, and — critically — **each direction has its own key**, so the two never collide even at the same counter value. The `counter` is strictly monotonic and never resets (next section). Fresh ephemeral key per connection + never-reset counter = the (key, nonce) pair is provably unique for the entire life of a session. `_aad` builds the Associated Data.

DEFINITION

AAD (Associated Data) — Bytes fed into the AEAD tag computation but *not* encrypted — they ride in the clear yet are protected from tampering. GRX's AAD is `version || counter`, which cryptographically binds each packet to its protocol version (anti-downgrade) and its sequence number.

DEFINITION

little-endian — A byte ordering where the least-significant byte comes first in memory. `struct.pack('<...')` in Python and `ByteOrder.LITTLE_ENDIAN` in Kotlin both select it — they must agree exactly or the two ends compute different nonces and every packet fails to decrypt.

Turning a shared secret into keys: HKDF and the PSK

The X25519 shared secret is 32 raw bytes with slightly uneven statistical properties — you never use it directly as a key. You run it through a key-derivation function.

DEFINITION

KDF (key-derivation function) — A function that takes some input keying material and stretches/refines it into one or more cryptographically clean keys of the exact length you need.

DEFINITION

HKDF — HMAC-based KDF. It has two phases — *extract* (condense the input into a uniform pseudorandom key) and *expand* (stretch that into as many named subkeys as you want). GRX uses HKDF-SHA256.

DEFINITION

HMAC — A keyed hash (here HMAC-SHA256) that proves both integrity and authenticity: only someone with the key can produce the correct tag over a message. HKDF is built from it, and GRX also uses it directly for handshake confirmation.

DEFINITION

PSK (pre-shared key) — A secret both ends already hold before the handshake starts. GRX's PSK authenticates the whole key agreement — an attacker without it derives different keys and fails confirmation, which is what kills man-in-the-middle attacks.

Where does the PSK come from? You did **not** add anything to the QR code. GRX bootstraps on the pairing `key` that was already the third CSV field of your pairing QR (`ip,port,key`). The server derives the 32-byte PSK from it:

apps/pc-server/server.py · lines 1286-1295

Python

```

1 def _grx_psk_from_key(key_str):
2     """32-byte GRX PSK derived from the shared pairing key (HKDF-SHA256). Both
3     ends already hold `key` (it's in the QR), so this needs no new pairing data."""
4     from cryptography.hazmat.primitives.kdf.hkdf import HKDF
5     from cryptography.hazmat.primitives import hashes
6     try:
7         ikm = bytes.fromhex(key_str)
8     except (ValueError, TypeError):
9         ikm = (key_str or "").encode()
10    return HKDF(algorithm=hashes.SHA256(), length=32, salt=b"", info=b"grx psk v1").derive(ikm)

```

The pairing key string is first parsed as hexadecimal (`bytes.fromhex`); if it isn't valid hex, it falls back to raw UTF-8 bytes. Those bytes are the *input keying material* (`ikm`). HKDF-SHA256 with an empty salt and the label `info=b"grx psk v1"` stretches them to exactly 32 bytes. The label matters: it *domain-separates* this key so the same pairing secret used elsewhere can never produce the same bytes. The Android side must compute the identical value, and it does:

apps/android-client/app/src/main/java/com/gamepad/client/GrxCrypto.kt · lines 54-58

Kotlin

```

1 fun pskFromPairingKey(key: String): ByteArray {
2     val ikm = parseHexOrNull(key) ?: key.toByteArray(Charsets.UTF_8)
3     return Hkdf.computeHkdf(HMAC, ikm, ByteArray(0),
4         "grx psk v1".toByteArray(Charsets.US_ASCII), 32)
5 }

```

This is a line-for-line mirror of the Python: hex-parse-or-UTF-8 the same way, then `Hkdf.computeHkdf("HmacSHA256", ikm, emptySalt, "grx psk v1", 32)`. `parseHexOrNull` returns null on any non-hex character, triggering the identical UTF-8 fallback. Because HKDF is a fixed standard, the same inputs give byte-identical outputs on both platforms — which is the entire point. Alongside the PSK, both ends use a fixed identity string, `GRX_LTID = "gamepados-grx-v1"`, bound into the handshake so a client from a different app can't be mistaken for yours.

The GRX handshake, message by message



Figure 8.1 — The 1-RTT GRX handshake.

Now the pieces combine into a one-round-trip handshake that runs fresh on every connection. First, here is the design intent as the protocol doc states it:

```

apps/docs/GRX_PROTOCOL.md · lines 30-38
Markdown
1 1. exchange ephemeral public keys (32 B each), in the clear
2 2. transcript_hash = SHA256(
3     len-prefixed: [version||cipher_id] , client_eph_pub , server_eph_pub , long_term_id )
4 3. shared = X25519(my_eph_priv, their_eph_pub) # 32 B
5 4. master = HKDF-SHA256(ikm = shared || psk, salt = transcript_hash, info="grx master v1", 32)
6 5. k_c2s = HKDF-Expand(master, "grx c2s v1", 16) # client→server key (AES-128)
7   k_s2c = HKDF-Expand(master, "grx s2c v1", 16) # server→client key
8   k_conf = HKDF-Expand(master, "grx confirm v1", 32)
9 6. each side sends confirm = HMAC-SHA256(k_conf, transcript_hash || role) # role: "C"/"S"
  
```

Steps 1–3 are the Diffie-Hellman: swap 32-byte public keys in the clear, compute the shared secret. Step 2 hashes a *transcript* of everything that was said. Steps 4–6 turn the secret into keys and prove both sides got the same ones.

WATCH OUT

The code contradicts this doc, and per the project rule the code wins. Steps 4–5 describe deriving a single **master** key and then **HKDF-Expand**-ing subkeys from it. The shipped implementation does **not** do this. It derives each subkey with an *independent* full HKDF (**ikm** = **shared** || **psk**, **salt** = **transcript_hash**, per-key **info** label) — there is no master key. The reason is interoperability: Android's Tink library exposes only **computeHkdf** (extract-then-expand together) and has no standalone **HKDF-Expand**, so a master-then-expand scheme would make the two languages diverge. Treat the doc's step 4/5 as stale pseudocode; the transcript hash (step 2), confirm MAC (step 6), and every constant are accurate.

DEFINITION

transcript hash — A SHA-256 hash over every value exchanged in the handshake (version, cipher id, both public keys, the identity). Binding the keys to it means any tampering with the handshake — swapped keys, a downgraded version — produces different keys on each side, so confirmation fails.

apps/pc-server/grx_crypto.py · lines 38-44

Python

```

1 def transcript_hash(client_pub, server_pub, long_term_id):
2     h = hashlib.sha256()
3     h.update(_lp(struct.pack('<BB', VERSION, CIPHER_ID)))
4     h.update(_lp(client_pub))
5     h.update(_lp(server_pub))
6     h.update(_lp(long_term_id))
7     return h.digest()

```

Each field is fed through **_lp** — length-prefixed, a 4-byte little-endian length before the bytes — before hashing. That prevents *canonicalization ambiguity*: without length prefixes, **AB || C** and **A || BC** would hash identically and an attacker could shuffle field boundaries. **VERSION** is 1 and **CIPHER_ID** is 1 (AES-128-GCM), packed as two bytes. The order — version/cipher, client pub, server pub, long_term_id — is fixed and both sides must build it identically.

The transcript and shared secret then feed key derivation:

apps/pc-server/grx_crypto.py · lines 47-65

Python

```

1 def derive_keys(shared, psk, th):
2     """ECDH shared + pairing PSK + transcript -> per-direction keys + confirm key.
3
4     Each subkey is a single HKDF-SHA256(ikm = shared||psk, salt = transcript,
5     info = label) — one extract+expand per key. This exactly matches Android's
6     Tink Hkdf.computeHkdf("HMACSHA256", ikm, salt, info, L), so both ends derive
7     identical bytes. (Do NOT switch to a master-then-expand scheme: Tink has no
8     standalone HKDF-Expand, so the two languages would diverge.)
9     """
10    ikm = shared + psk
11
12    def hk(label, n):
13        return HKDF(algorithm=hashes.SHA256(), length=n, salt=th, info=label).derive(ikm)

```

apps/pc-server/grx_crypto.py · (cont.)

Python

```

14
15     return {
16         'c2s': hk(b'grx c2s v1', KEY_LEN),
17         's2c': hk(b'grx s2c v1', KEY_LEN),
18         'confirm': hk(b'grx confirm v1', 32),
19     }

```

The input keying material is **shared + psk** concatenated — so *both* the Diffie-Hellman secret *and* the pairing PSK must be correct to derive the right keys. The transcript hash is the salt. Three labeled subkeys come out: **c2s** (16 bytes, the AES-128 key for client→server input), **s2c** (16 bytes, server→client), and **confirm** (32 bytes, for the handshake proof). An attacker who runs a valid X25519 exchange but lacks the PSK produces a different **ikm**, hence different keys, hence a confirm that won't verify. That is the whole MITM defense in one line.

DEFINITION

MITM (man-in-the-middle) — An attacker who sits between the two parties, relaying and possibly altering messages while impersonating each end to the other. The PSK-bound confirm MAC defeats it: the attacker can't forge a valid confirmation without the PSK.

The wire messages themselves are defined by the session layer. Here is the server consuming the client's opening message:

apps/pc-server/grx_session.py · lines 59-71

Python

```

1     def handle_hello(self, frame):
2         if not frame or frame[0] != T_HELLO:
3             raise HandshakeError('expected CLIENT_HELLO')
4         client_pub = frame[1:33]
5         ltid, _ = _take_lp(frame, 33)
6         if ltid != self.long_term_id:
7             raise HandshakeError('unknown long_term_id') # not the paired device
8         self._eph = X25519PrivateKey.generate() # fresh per session
9         self._keys = gx.handshake(self._eph, client_pub, self.psk, self.long_term_id,
10                                  am_client=False)
11         server_pub = self._eph.public_key().public_bytes_raw()
12         server_confirm = gx.confirm_tag(self._keys['confirm'], self._keys['transcript'], ROLE_SERVER)
13         return struct.pack('<B', T_SHELLLO) + server_pub + server_confirm

```

A **CLIENT_HELLO** frame (**T_HELLO = 0xE1**) is: type byte, then the client's 32-byte ephemeral public key (**frame[1:33]**), then a length-prefixed **long_term_id**. The server checks the identity, generates its *own* fresh ephemeral keypair, runs the full handshake to derive the keys, and replies with **SERVER_HELLO** (**T_SHELLLO = 0xE2**): its public key plus a **server_confirm** — an HMAC over the transcript tagged with role "S". The client verifies that HMAC (proving the server has the right PSK), then sends back **CLIENT_CONFIRM** (**T_CONFIRM = 0xE3**) with its own "C"-tagged HMAC, which the server verifies in **handle_confirm**. Only when **both** confirms verify does either side set **established = True** and accept input. One round trip; mutual authentication; no key material ever revealed to an eavesdropper.

The sealed packet and the replay window

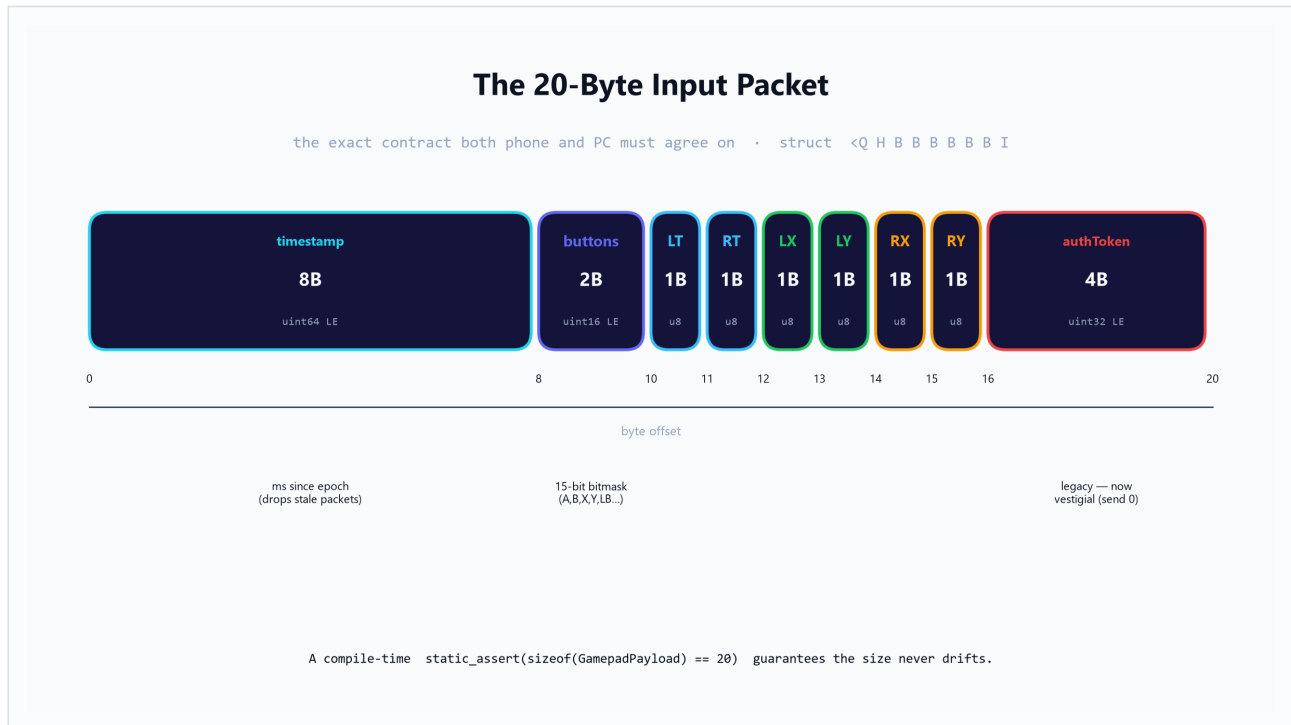


Figure 8.2 — The 20-byte plaintext GRX seals into a 41-byte frame.

Handshake done, every 20-byte input frame is now sealed into a 41-byte wire frame. The math: $\text{HEADER_LEN}(5) + \text{PLAINTEXT_LEN}(20) + \text{TAG_LEN}(16) = \text{WIRE_LEN}(41)$, where the 5-byte header is `version(1) + counter_low32(4)`.

apps/pc-server/grx_crypto.py · lines 131-143

Python

```

1 class GrxSender:
2     def __init__(self, key, direction):
3         self.aead = AESGCM(key)
4         self.direction = direction
5         self.counter = 0
6
7     def seal(self, plaintext):
8         if len(plaintext) != PLAINTEXT_LEN:
9             raise ValueError('GRX plaintext must be %d bytes' % PLAINTEXT_LEN)
10        self.counter += 1 # starts at 1, never resets
11        c = self.counter
12        ct = self.aead.encrypt(_nonce(self.direction, c), plaintext, _aad(c))
13        return struct.pack('<BI', VERSION, c & 0xFFFFFFFF) + ct

```

The counter starts at 0, is incremented to 1 *before* the first packet, and only ever climbs — never resets. `aead.encrypt` takes the freshly built nonce (`direction || counter`), the 20-byte plaintext, and the AAD (`version || counter`); it returns ciphertext with the 16-byte tag appended. The wire frame is `VERSION`, then the **low 32 bits** of the counter (`c & 0xFFFFFFFF`), then the 41-minus-5 bytes of ciphertext+tag. Only 32 bits of the 64-bit counter travel — the receiver reconstructs the top 32 from context, saving 4 bytes per packet without weakening the 96-bit nonce.

The Kotlin sender is the byte-for-byte mirror, with one performance subtlety worth studying:

```

1  class Sender(key: ByteArray, private val direction: Int) {
2      private val secret = SecretKeySpec(key, "AES")
3      private var counter = 0L
4      // Reuse one Cipher instance across seals: Cipher.getInstance does a JCE
5      // provider lookup (allocations + service scan) EVERY call, and this runs on
6      // the hot send path. GCM requires a fresh IV per encryption, which we already
7      // guarantee (counter is strictly monotonic → a distinct nonce every init), so
8      // re-init()ing a cached Cipher is safe and byte-identical on the wire.
9      private val cipher = Cipher.getInstance("AES/GCM/NoPadding")
10     fun seal(plaintext: ByteArray): ByteArray {
11         require(plaintext.size == PLAINTEXT_LEN)
12         counter += 1
13         cipher.init(Cipher.ENCRYPT_MODE, secret, GCMParameterSpec(TAG_LEN * 8, nonce(direction, counter
14     )))
15         cipher.updateAAD(aad(counter))
16         val body = cipher.doFinal(plaintext) // ciphertext||tag (16-byte tag appended)
17         return ByteBuffer.allocate(WIRE_LEN).order(ByteOrder.LITTLE_ENDIAN)
18             .put(VERSION).putInt(counter.toInt()).put(body).array()
19     }
20 }

```

Same logic — increment counter, `init` with a `GCMParameterSpec` carrying the tag length in *bits* (`TAG_LEN * 8 = 128`) and the nonce, `updateAAD`, `doFinal` — but note it caches one `Cipher` object and re-`init`s it per packet. The comment explains why that is safe: GCM's cardinal sin is reusing a nonce, and the monotonic counter already guarantees a fresh nonce on every `init`, so reusing the *cipher object* changes nothing on the wire while avoiding a costly JCE provider lookup on the hot send path. `putInt(counter.toInt())` writes the low 32 bits, exactly matching Python's `c & 0xFFFFFFFF`.

Receiving is where replay defense lives, and the order of operations is load-bearing:

```

1  def open(self, frame):
2      """Returns the 20-byte plaintext, or None to DROP (old/dup/forged/garbage)."""
3      if len(frame) != WIRE_LEN:
4          return None
5      ver, low32 = struct.unpack('<BI', frame[:HEADER_LEN])
6      if ver != VERSION:
7          return None
8      counter = self.win.reconstruct(low32)
9      if not self.win.check(counter):
10         return None # 2. early dup/old reject
11     try: # 3. verify tag FIRST
12         pt = self.aead.decrypt(_nonce(self.direction, counter),
13                                frame[HEADER_LEN:], _aad(counter))
14     except Exception:
15         return None # auth fail -> drop, window UNTOUCHED
16     self.win.commit(counter) # 4. advance ONLY on auth success
17     return pt

```

DEFINITION

sliding replay window — A bitmask tracking the highest authenticated counter seen plus which recent counters already arrived, so out-of-order-but-fresh packets are accepted while duplicates and stale ones are rejected. GRX's c2s window is 1024 packets wide (~1–2 s at 500–1000 Hz), giving reorder tolerance.

The steps run in a precise sequence. First **reconstruct** rebuilds the full 64-bit counter from the 32-bit wire value using the window's high-water mark. Then **check** does a *pre-authentication* reject of anything already seen or older than the window. Then — and this is the invariant that matters — the GCM tag is verified **before** the window is touched. Only if **decrypt** succeeds does **commit** advance the window.

WATCH OUT

If you advanced the window before verifying the tag, an attacker could inject a forged packet with a giant counter, slide your window past all your real packets, and cause an input blackout — a denial-of-service with no valid key. GRX's self-test asserts exactly this: after a forged high-counter packet, `rcv3.win.high == 0` (window untouched) and the next legit packet still opens. Verify first, commit second.

Three implementations, one wire — and where GRX actually stands

Three codebases implement this identical byte contract. The Python reference (`grx_crypto.py` + `grx_session.py`) uses the PyCA `cryptography` library and is the source of truth — running `python grx_crypto.py` executes a 6-part self-test (handshake, seal/open, replay, reorder, tamper/DoS, MITM) with no device attached. The Kotlin client (`GrxCrypto.kt` + `GrxClient.kt`) mirrors it byte-for-byte using Google's Tink for X25519 and HKDF (Tink works back to API 19, unlike `java.security` X25519 which needs API 31+) and `javax.crypto` for AES-GCM (hardware-accelerated on ARMv8). A third implementation, the iOS CryptoKit port, exists and speaks the same wire — its details belong to the iOS chapter.

DEFINITION

Tink — Google's cross-language crypto library. GRX depends on `com.google.crypto.tink:tink-android:1.13.0` specifically because it offers the exact `computeHkdf` primitive that keeps Kotlin and Python deriving identical keys.

Interoperability is not automatic; it is *engineered* by pinning every choice both ends must agree on: little-endian byte order everywhere, length-prefixed transcript fields, the exact HKDF labels (`grx c2s v1`, `grx s2c v1`, `grx confirm v1`, `grx psk v1`), the fixed `long_term_id`, the one-HKDF-per-key scheme, and full 128-bit tags. Change any one on a single side and every packet silently fails to decrypt.

On lifecycle: there is **no mid-session rekeying**. A session's keys live and die with its ephemeral X25519 keypair. If the 64-bit counter were ever exhausted, or any state were lost (a process restart), the

rule is a **new handshake, never a counter reset** — because a reset counter with a reused key is the total GCM break from the nonce section. The server keeps one `GrxServerSession` per client IP, evicts stale ones when a phone goes idle (an idle watchdog retires dead sessions in ~3 s), and the Android side builds a *new* `GrxClient` on every reconnect so a stale sender can never seal with an old key.

One subtlety that trips people up: routing handshake frames apart from data frames by first byte alone is fragile, and the code carries the scar tissue.

apps/pc-server/grx_session.py · lines 137-148

Python

```
1 # Minimum sane length per handshake type: HELLO = 1+pub(32)+lp(ltid) ≥ 34,
2 # SERVER_HELLO = 1+32+32 = 65, CONFIRM = 1+32 = 33. A legacy cleartext INPUT
3 # frame is exactly 20 bytes and starts with a little-endian timestamp whose low
4 # byte sweeps 0-255, so first-byte matching alone misrouted ~1/128 input frames
5 # into the handshake path. Pre-hardening, a fake CONFIRM (20 B unpacked as 37 B)
6 # raised struct.error and KILLED the server's UDP thread — port stayed bound but
7 # silent until the server was restarted.
8 _HS_MIN_LEN = {T_HELLO: 34, T_SHELLO: 65, T_CONFIRM: 33}
9
10 def is_handshake(frame):
11     return (bool(frame) and frame[0] in _HS_MIN_LEN
12             and len(frame) >= _HS_MIN_LEN[frame[0]])
```

A cleartext input frame is 20 bytes and starts with a timestamp whose low byte cycles 0–255, so roughly 1 in 128 real inputs happened to start with a handshake type byte and got misrouted — and a 20-byte frame misread as a 37-byte `CONFIRM` threw `struct.error` and killed the UDP thread, leaving the port bound but dead. The fix: `is_handshake` requires both the right first byte **and** a minimum length per type. Data frames start with `VERSION = 0x01`, which can never collide with `0xE1/0xE2/0xE3`. This is the kind of byte-level detail that separates a protocol that works in a test from one that survives real traffic.

Finally, the most important status fact — and a place where documents and code disagree.

WATCH OUT

`GRX_INTEGRATION.md` and `GRX_ANDROID_WIRING.md` are written as *future* work ("what remains is wiring," "ready-to-paste C++ bridge," "flip `GRX_ENABLED=true`"). The live code says otherwise, and the code wins: `MainActivity.kt` has `private val GRX_ENABLED = true`, the C++ JNI bridge (`nativeGrxSendRaw`, `grxSeal`, `onGrxControl`) is present and cached in `gamepad-engine.cpp`, and `server.py`'s `run_udp_loop` routes the handshake and decrypts input. GRX is **shipped and live** on every non-USB connection, not pending. Two things remain genuinely open: `GRX_REQUIRED` is still `False` on the server, so the legacy cleartext path is still accepted during migration (the downgrade hole isn't closed until it flips to `True`), and the server→client (`s2c`) rumble direction is still cleartext in `v1` — the `s2c` key is derived but the server does not yet encrypt its replies. Confidentiality of *your input* is protected today; sealing the reverse channel and hard-disabling cleartext are the last two steps.

PART IV

**Performance & Alternate
Transports**

CHAPTER 9

Latency Engineering — Chasing Milliseconds

9



Figure 9.1 — A rough millisecond budget for one input over Wi-Fi.

The Finger-to-Game Journey

When you press a button on your phone and a character jumps on the PC, that "instant" reaction is not instant at all. It is a relay race. Your touch has to be *detected*, turned into data, handed across several pieces of software, flung through the air as radio waves, caught by the PC, translated into a fake controller, and finally *noticed* by the game. Every runner in that relay takes a little time. Add them all up and you get **latency**.

DEFINITION

Latency — the total time between a cause and its visible effect — here, between your finger touching the glass and the game reacting. Measured in milliseconds (ms); 1 ms is one-thousandth of a second.

DEFINITION

Jitter — how *inconsistent* that latency is from one press to the next. A steady 20 ms feels better than a latency that bounces between 8 ms and 60 ms, because your brain can adapt to a constant delay but not a random one.

IN PLAIN TERMS

Imagine mailing a postcard. Latency is how many days it takes to arrive. Jitter is whether it *always* takes three days or sometimes three and sometimes ten. A game controller that is late but *predictably* late is playable; one that is erratic feels broken even if its average is lower.

Here is the full relay, in order, for your system:

1. **Touch sampling** — the screen's digitizer physically detects your finger.
2. **JavaScript (App.tsx)** — the React UI turns that touch into a 20-byte packet.
3. **The JSI bridge (sendGamepadPacket)** — JavaScript hands the bytes to native C++.
4. **The native TX thread (gamepad-engine.cpp)** — C++ writes the bytes to a network socket.
5. **Wi-Fi (or USB)** — the packet crosses the air to the router and on to the PC.
6. **The PC receive loop (server.py)** — Python reads the packet and decodes it.
7. **ViGEm / XInput** — the decoded input is written to a *virtual* Xbox controller.
8. **Game poll** — the game reads that virtual controller on its own schedule.

Your own docs give honest targets for the whole chain:

Setup	Latency
USB tethering (wired-quality)	~6–12 ms
Tuned 5 GHz Wi-Fi (optimized)	~10–18 ms
Typical 5 GHz home Wi-Fi	~20–35 ms
2.4 GHz / congested	50–120 ms+

For reference, `LOW_LATENCY_GUIDE.md` notes a wired Xbox pad is roughly 4–8 ms and Bluetooth is 15–25 ms — so a tuned Wi-Fi link on this system is genuinely competitive with a real wireless controller. What actually crosses the wire is this fixed 20-byte contract:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 51-65 C++

1  #pragma pack(push, 1)
2  struct GamepadPayload {
3      uint64_t timestamp; // 8 bytes
4      uint16_t buttons; // 2 bytes
5      uint8_t leftTrigger; // 1 byte
6      uint8_t rightTrigger; // 1 byte
7      uint8_t leftStickX; // 1 byte
8      uint8_t leftStickY; // 1 byte
9      uint8_t rightStickX; // 1 byte
10     uint8_t rightStickY; // 1 byte
11     uint32_t authToken; // 4 bytes
12 };
13 #pragma pack(pop)
14
15 static_assert(sizeof(GamepadPayload) == 20, "FATAL: Payload struct is not exactly 20 bytes!");
```

DEFINITION

`#pragma pack(push, 1)` — an instruction telling the C++ compiler *not* to insert invisible padding bytes between fields for alignment. Without it the compiler might silently make this struct 24 bytes; with it, the layout is exactly what you see.

This block matters for latency because **small is fast**. Twenty bytes fits inside a single network packet with room to spare — there is nothing to split, reassemble, or wait on. The `timestamp` field at offset 0 is the star of this chapter: it is stamped when the packet leaves and echoed back by the PC, which is how you measure real round-trip time later. The `static_assert` is a compile-time tripwire — if anyone ever edits the struct and the size drifts off 20, the build fails instead of shipping a phone and PC that silently disagree about where each field lives.

Where the Milliseconds Hide

Now walk the relay and see who is slow. Some hops you can fix in code; some are set by physics and hardware.

Touch sampling is the biggest phone-side cost, and it is not software.

DEFINITION

Touch sampling rate — how many times per second the touchscreen checks for your finger, measured in Hz (times per second). A 60 Hz digitizer only looks every ~16 ms, so on average it adds ~8 ms just *noticing* you — and up to a full 16 ms in the worst case.

Your `LOW_LATENCY_GUIDE.md` calls this "the single biggest phone-side cost." A 240 Hz gaming-phone digitizer cuts that detection delay to ~4 ms. Note the guide's warning that **touch sampling rate is not the same as display refresh rate** — a phone can have a 120 Hz screen but only 120 Hz touch, and the number that matters for input is the touch rate. No amount of network tuning can beat a slow digitizer; it sets the *floor*.

JavaScript and the bridge. Once the touch is detected, `App.tsx` builds the packet in `sendGamepadTelemetry` and hands it across the **JSI bridge**.

DEFINITION

JSI / bridge — the "JavaScript Interface" — a thin doorway that lets JavaScript running in the WebView call a C++ function directly. Here that function is `window.sendGamepadPacket(buffer)`.

DEFINITION

NDK / native — the Native Development Kit — the toolchain that lets you write parts of an Android app in C++ instead of Java/Kotlin, so you can control memory and threads precisely and avoid garbage-collection pauses.

Wi-Fi is the biggest variable. The radio hop is where jitter lives: a congested 2.4 GHz band, a power-saving radio that falls asleep between packets, or a busy access point can each add tens of milliseconds without warning. This is why the guide's #1 tip is "use 5 GHz, not 2.4 GHz."

The PC and the game. Python decodes the packet and writes it to a virtual controller through **ViGEm**.

DEFINITION

ViGEm — a Windows driver that lets a normal program *pretend* to be a plugged-in game controller. Your `server.py` uses it (via the `vgamepad` library) to create a controller that any game reads as real hardware.

DEFINITION

XInput — Microsoft's standard API through which PC games read Xbox controllers. Games *poll* it — they ask "what's the controller doing?" once per frame — so at 60 fps the game only looks every ~16 ms.

NOTE

`ARCHITECTURE.md` describes the virtual pad as a "DualShock 4," but the shipping code in `server.py` actually creates `vg.VX360Gamepad()` and drives `XUSB_BUTTON` values — an **Xbox 360 / XInput** pad, not a DS4. Where code and an older doc disagree, the code wins: your games see an Xbox controller.

The Native Send Loop: From Fixed Poll to Event-Driven

This is the heart of the 2026-06-21 latency work, and it overturns something the older docs still claim. Both `LOW_LATENCY_GUIDE.md` and `ARCHITECTURE.md` describe a **fixed 1000 Hz send thread** that blasts a packet every millisecond, pinned to the phone's big cores. The code no longer works that way, and the change was deliberate.

The old design had two costs. First, holding the pad still meant sending ~1000 identical packets every second — pure heat and battery drain for zero new information. Second, a *fixed* tick adds latency: if input arrives just after a tick fires, it waits out the whole interval before going. The rewrite made the loop **event-driven** instead.

DEFINITION

Condition variable — a thread-coordination tool that lets a thread *sleep* until another thread signals "wake up, something changed." It avoids busy-waiting (spinning the CPU checking a flag) — the sleeping thread uses zero CPU until poked.

apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 589-595

C++

```

1      {
2          std::unique_lock<std::mutex> lock(payloadMutex);
3          if (!payloadDirty && isRunning) {
4              payloadCv.wait_for(lock, std::chrono::nanoseconds(RX_POLL_NS),
5                               [] { return payloadDirty || !isRunning.load(); });
6          }
7      }

```


This is the "latency → zero" core. The TX thread sleeps on `payloadCv` until either new input marks `payloadDirty` or `RX_POLL_NS` (defined as `2000000L` — 2 ms) elapses. A fresh touch wakes it in **microseconds**, not on the next tick. The 2 ms timeout is only a fallback: it keeps incoming ACKs draining, keeps redundancy resends firing ~2 ms apart, and keeps the idle keep-alive going while you hold the pad steady. The `[] { ... }` lambda is a guard against a "lost wakeup" — if input arrives in the split second between the thread snapshotting state and going to sleep, the predicate catches it so the thread doesn't miss the signal.

Who fires that signal? The bridge function, the instant JavaScript hands over a packet:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 611-629 C++

1 extern "C" JNIEXPORT void JNICALL
2 Java_com_gamepad_client_MainActivity_injectNativePayload(JNIEnv* env, jobject this, jbyteArray data) {
3     jsize len = env->GetArrayLength(data);
4     if (len == sizeof(GamepadPayload)) {
5         // FIX 5: single-copy JNI. GetByteArrayRegion copies the 20 bytes straight
6         // into currentPayload (no Get/Release pin pair, no intermediate buffer +
7         // extra memcpy). Done under payloadMutex so it can't tear against the TX
8         // thread's read. The 20-byte length guard above is preserved.
9         {
10            std::lock_guard<std::mutex> lock(payloadMutex);
11            env->GetByteArrayRegion(data, 0, sizeof(GamepadPayload), reinterpret_cast<jbyte*>(&currentPaylo
ad));
12            payloadDirty = true;
13        }
14        // Wake the TX thread NOW so this input transmits immediately rather than on
15        // the next poll tick. Notify after releasing the lock so the woken thread
16        // doesn't immediately block on a mutex we still hold.
17        payloadCv.notify_one();
18    }
19 }
```

DEFINITION

JNI — the Java Native Interface — the formal rules for how Java/Kotlin code calls C++ functions and passes data across the boundary.

DEFINITION

Mutex — short for "mutual exclusion" — a lock that guarantees only one thread touches shared data at a time, so a half-written packet is never read mid-update.

Two speed wins hide here. **FIX 5** copies the 20 bytes straight into `currentPayload` with one `GetByteArrayRegion` call, skipping the older two-step "pin the array, memcpy it, release it" dance. And the final `payloadCv.notify_one()` is what wakes the sleeping TX thread *now* — the notify happens *after* releasing the lock so the woken thread doesn't instantly stall on a mutex the notifier still holds. Together, a press travels from JavaScript to the socket without ever waiting out a poll interval.

Once awake, the thread decides *whether* to actually transmit. It does not send on every wake — it sends on **change**, on a **keep-alive**, or on **redundancy**:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 270-290 C++
1      uint8_t curInput[8];
2      memcpy(curInput, &networkPayload.buttons, 8);
3      struct timespec now;
4      clock_gettime(CLOCK_MONOTONIC, &now);
5      uint64_t rawTimestamp = (uint64_t)now.tv_sec * 1000000000LL + now.tv_nsec;
6      bool changed = (memcmp(curInput, lastSentInput, 8) != 0);
7      // Adaptive keep-alive: ~30Hz normally (the thermal/CPU win noted above), but
8      // ~60Hz while rumble is active. The PC only emits fresh RMB in reply to an
9      // inbound frame, so when the pad is held steady during rumble a 30Hz uplink
10     // caps rumble-state updates at ~33ms; 60Hz halves that. Reverts to 30Hz the
11     // moment rumble clears (rumbleLeft/Right are the last values the PC sent us).
12     bool rumbleActive = (rumbleLeft.load(std::memory_order_relaxed) != 0) ||
13                         (rumbleRight.load(std::memory_order_relaxed) != 0);
14     uint64_t heartbeatNs = rumbleActive ? 16000000ULL : 33000000ULL;
15     bool heartbeat = (rawTimestamp - lastSendNs) >= heartbeatNs; // adaptive keep-alive
16     // FIX 1: a fresh change arms CHANGED_PACKET_REDUNDANCY back-to-back sends.
17     // While that counter is positive we keep re-sending (redundant) even when
18     // the input hasn't changed again, so a single dropped datagram recovers
19     // fast. The counter is (re)armed below once we confirm a successful send.
20     bool redundant = (redundancyRemaining > 0);
21     bool doSend = changed || heartbeat || redundant;
```

The 8 contiguous bytes from `buttons` through `rightStickY` are your actual input; `changed` is a `memcmp` against what was last sent. `heartbeat` fires a keep-alive every 33000000 ns (33 ms, ~30 Hz) normally, tightening to 16000000 ns (16 ms, ~60 Hz) only while rumble is active so motor updates aren't stale. `redundant` is **FIX 1**: a genuine change arms `CHANGED_PACKET_REDUNDANCY` (defined as 3) back-to-back resends so a single dropped datagram recovers within a couple of 2 ms polls instead of waiting up to 33 ms for the next change. Real input still goes out within one 2 ms poll, so responsiveness is unchanged — but a held pad now sends ~30 packets/second instead of ~1000.

WATCH OUT

The code comment at the top of `txThreadLoop` is explicit: CPU-affinity pinning to the big cores was **removed** because it kept them spun up 1000×/second, which was "the main source of high CPU + phone heat" — and it says *do NOT re-add it*. So `LOW_LATENCY_GUIDE.md`'s claim of a "1000 Hz thread pinned to performance cores" is now historical. The thread is light, mostly sleeps, and lets the scheduler place it. The code wins.

EXPERT DETAIL

On the JavaScript side the same shift happened. `ARCHITECTURE.md` still describes `useInterval(sendGamepadTelemetry, 1000 / hz)` at 60–1000 Hz, but `App.tsx` now sends *synchronously* the instant a button/stick ref changes, backed only by a fixed `setInterval(..., 16)` ~60 Hz heartbeat (line 1030) that re-sends current state so dropped packets recover and held analog stays smooth. The `PollingHz` dropdown (60/120/500/1000) still exists in the tuning dialog, but the real "1000 Hz" now describes how *responsively* the native thread can wake, not a literal packet count.

Socket-Level Tricks: TOS, connect(), Non-Blocking Sends

Getting the packet *out* fast is its own craft. The socket is tuned the moment it is created.

DEFINITION

UDP — User Datagram Protocol — the "fire and forget" way to send data over a network. You throw a **datagram** (a self-contained packet) at an address and move on; there is no waiting for confirmation. Perfect for real-time input, where a *late* packet is worse than a *lost* one.

DEFINITION

TOS / DSCP — Type Of Service — a priority label stamped in a packet's header. **DSCP-EF** (Expedited Forwarding) is the "urgent, deliver first" class that QoS-aware routers honor.

apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 135-149

C++

```
1 static void applyLowLatencyTos(int sock) {
2     if (sock < 0) return;
3     int tos = 0xB8; // DSCP EF (Expedited Forwarding) << 2 | ECN 0
4     setsockopt(sock, IPPROTO_IP, IP_TOS, &tos, sizeof(tos));
5     // Android/Linux Wi-Fi WMM access-category selection is driven by skb->priority
6     // (SO_PRIORITY), which many Wi-Fi HALs honour even when they ignore IP_TOS/DSCP.
7     // prio 6 (TC_PRIO_INTERACTIVE) maps to WMM AC_VO — the same high-priority queue the
8     // WIFI_MODE_FULL_LOW_LATENCY lock targets, so they compound and cut uplink queuing
9     // jitter on busy APs. Best-effort: prio <= 6 is allowed unprivileged; failure is
10    // harmless (older kernels may not define SO_PRIORITY).
11    #ifdef SO_PRIORITY
12        int prio = 6;
13        setsockopt(sock, SOL_SOCKET, SO_PRIORITY, &prio, sizeof(prio));
14    #endif
15 }
```

Two priority signals fire together. `IP_TOS = 0xB8` tags the packet DSCP-EF so a **WMM**-capable router (Wi-Fi's built-in QoS) prioritizes it over bulk traffic like downloads. But the comment reveals a deeper trick: many Android Wi-Fi chips *ignore* the IP header's DSCP and instead read `SO_PRIORITY` (the kernel's internal packet priority). Setting `prio = 6` (`TC_PRIO_INTERACTIVE`) maps to the WMM voice queue — the same high-priority lane the `WIFI_MODE_FULL_LOW_LATENCY` radio lock targets — so the two compound and cut uplink queuing jitter on a busy access point. Both are best-effort: `prio <= 6` is allowed without special permissions, and failure is harmless.

Once the PC's real address is known, the hot path gets one more optimization:

apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 338-348

C++

```
1         } else if (socketConnected) {
2             // FIX 2: socket is connect()ed to the locked unicast peer, so the
3             // kernel route is cached — send() skips the per-call route lookup
4             // and address copy that sendto() repeats every time.
5             sent = send(udpSocket, outBuf, outLen, MSG_DONTWAIT);
6         } else {
7             // Broadcast discovery (or not yet locked on): keep the explicit
```

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · (cont.) C++
8          // destination so the datagram still reaches the broadcast address.
9          sent = sendto(udpSocket, outBuf, outLen, MSG_DONTWAIT,
10                      (struct sockaddr*)&serverAddr, sizeof(serverAddr));
11      }
```

DEFINITION

connect() on a UDP socket — even though UDP is connectionless, calling `connect()` tells the kernel "I only talk to this one peer." It then caches the network route once, so each send skips a fresh route lookup.

DEFINITION

Unicast vs broadcast — unicast is a packet aimed at one specific machine; broadcast is shouted to everyone on the network. The app broadcasts only during discovery, then locks onto the PC's unicast address.

DEFINITION

MSG_DONTWAIT — the flag that makes a send **non-blocking** — if the kernel's outgoing buffer is momentarily full, the call returns immediately with an error instead of freezing the thread until space frees up.

This is **FIX 2**. During discovery the app broadcasts with `sendto()`, which repeats a route lookup and copies the destination address *every call*. As soon as the PC's ACK reveals its real IP, the socket `connect()`s to it and the hot path switches to `send()` — no per-packet route lookup, no address copy. And `MSG_DONTWAIT` guarantees this latency-critical loop never blocks: a full buffer is a survivable hiccup that the keep-alive and redundancy resends recover, not a stall.

The PC Side: Draining to Newest, Priority, Timer Resolution

The PC has its own three-part latency story: never block, never fall behind, and never let Windows nap.

When the socket is bound, it is tuned symmetrically to the phone:

```
apps/pc-server/server.py · lines 665-685 Python
1      # Low-latency socket tuning:
2      # - Large RX buffer so a burst of 1000Hz packets is never dropped
3      #   while the thread is briefly busy (we still drain to newest).
4      # - Modest TX buffer so ACK sends never block on a full kernel buffer.
5      # - IP_TOS DSCP-EF to match the client's QoS tagging symmetrically.
6      try:
7          sock.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 1 << 20) # 1 MB
8      except OSError:
9          pass
10     try:
11         sock.setsockopt(socket.SOL_SOCKET, socket.SO_SNDBUF, 1 << 20) # 1 MB ACK headroom
```

```
apps/pc-server/server.py · (cont.) Python
12         except OSError:
13             pass
14         try:
15             # Match the phone's outbound tagging (DSCP-EF, Expedited Forwarding)
16             # so our ACKs land in the same WMM/QoS class as inbound packets.
17             IPTOS_DSCP_EF = 0xB8
18             sock.setsockopt(socket.IPPROTO_IP, socket.IP_TOS, IPTOS_DSCP_EF)
19         except OSError:
20             pass
21         sock.bind(('0.0.0.0', port))
```

DEFINITION

Receive buffer (SO_RCVBUF) — a holding area in the kernel where incoming packets queue until your program reads them. If it overflows, packets are silently dropped.

A `1 << 20` (1 MB) receive buffer means even a burst of packets arriving while the thread is briefly busy is never dropped — you keep them all and pick the newest. The matching 1 MB send buffer gives ACK replies headroom so they never block, and `IP_TOS = 0xB8` mirrors the phone's DSCP-EF tag so *return* traffic rides the same priority lane. The default bind port here is `7777` (from `bind_server_socket(base_port=7777)`), the same value shared in the pairing QR.

The most important PC trick is what happens with those buffered packets:

```
apps/pc-server/server.py · lines 1430-1441 Python
1         # Drain all waiting datagrams. Handshake frames are handled inline (they
2         # need an immediate reply); input frames keep the NEWEST per source IP.
3         latest = {}          # ip -> (cleartext 20B, addr)
4         latest_grx = {}     # ip -> (encrypted 41B, addr)
5         while True:
6             try:
7                 data, addr = sock.recvfrom(1024)
8                 except BlockingIOError:
9                     break
10                except (ConnectionResetError, OSError):
11                    break
12                ip = addr[0]
```

This loop reads *every* waiting datagram in one go. Because it stores each into a dictionary keyed by source IP (`latest[ip] = (data, addr)` at line 1474), a later packet **overwrites** an earlier one — so when the drain finishes, only the *newest* input per phone survives to be applied. This is the anti-lag rule from [ARCHITECTURE.md](#): if the PC ever falls behind, it skips straight to the truth instead of replaying a backlog of stale positions. An old stick position is worthless; only "where is the stick *now*" matters.

Two more Windows-specific defenses run at startup. `boost_process_priority()` sets `HIGH_PRIORITY_CLASS (0x00000080)` so the scheduler doesn't delay the receive thread under load. And the timer resolution is sharpened:

apps/pc-server/server.py · lines 702-716

Python

```

1  def boost_timer_resolution():
2      """Raise the Windows system timer resolution to 1ms so selectors.select()
3      timeouts and any sleeps aren't quantized to the default ~15.6ms tick. This
4      tightens the worst-case latency of the UDP receive loop. Best-effort and
5      Windows-only; the matching timeEndPeriod(1) is registered via atexit so the
6      resolution is restored on any exit path. Ignored if it fails."""
7      if sys.platform != 'win32':
8          return
9      try:
10         import ctypes, atexit
11         winmm = ctypes.WinDLL('winmm')
12         winmm.timeBeginPeriod(1)
13         atexit.register(lambda: winmm.timeEndPeriod(1))
14     except Exception:
15         pass

```

DEFINITION

Timer resolution — how finely the operating system can measure short sleeps and timeouts. Windows defaults to a coarse ~15.6 ms tick, so a "wait 2 ms" can actually wait ~15 ms.

`timeBeginPeriod(1)` forces a 1 ms tick, so the receive loop's `selectors.select(timeout=0.2)` and any internal sleeps aren't rounded up to ~15.6 ms — this alone can shave double-digit milliseconds off the worst case. The `atexit` registration of `timeEndPeriod(1)` restores the system default on exit, since a raised timer resolution costs a little battery system-wide and should not linger after the server closes.

NOTE

The USB-debugging WebSocket path — the one wired transport that rides TCP — disables **Nagle's algorithm** with `raw.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)`. DEF:: Nagle's algorithm :: a default TCP behavior that *delays* tiny packets to bundle them into bigger ones for efficiency. For 20-byte input frames that batching is pure latency, so it is switched off. UDP has no such delay, which is one more reason input rides UDP.

Measuring Latency Honestly

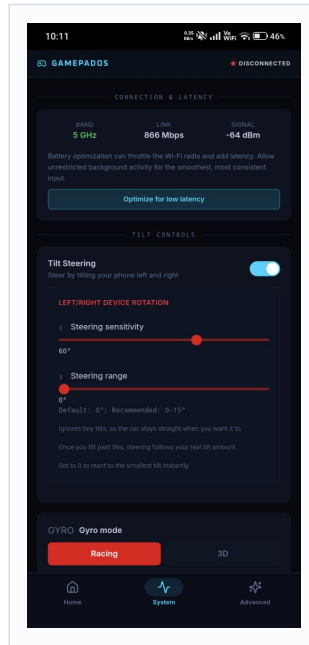


Figure 9.2 — The in-app connection & latency panel.

You cannot improve what you measure wrong. The old code measured the gap between two adjacent C++ lines and called it "latency" — a meaningless few microseconds. The honest number is a true **round trip**, and the `timestamp` field is what makes it possible.

DEFINITION

Round-trip time (RTT) — the time for a packet to travel to the PC *and* for its acknowledgment to travel back. It captures the whole network path in one number.

Every input packet carries the send-time you stamped at offset 0. The PC echoes it straight back:

```
apps/pc-server/server.py · lines 1372-1380 Python
1      # Echo send-time so the phone can compute true RTT. Sent AFTER apply_inputs so
2      # the pad IOCTL (what the game actually sees) isn't delayed by this sendto()
3      # syscall — the RTT figure is unaffected by the few microseconds of reorder.
4      # (Reordered/stale frames return above and are not ACK'd; the next in-order
5      # frame ACKs, so unicast lock / liveness is unaffected.)
6      try:
7          sock.sendto(b"ACK" + struct.pack("<Q", timestamp), addr)
8      except Exception:
9          pass
```

The reply is the literal bytes `b"ACK"` followed by the 8-byte timestamp the phone originally sent. Crucially, the ACK is sent *after* `apply_inputs()` — the driver write the game actually sees is never delayed by this reply syscall. Back on the phone, the native RX drain compares the echoed timestamp against the monotonic clock now, converts to milliseconds, and smooths it. That ACK is also the *only* trustworthy "the PC is really there" signal: a UDP `sendto()` succeeds locally even when nothing is listening, so a rising send count means nothing — the link is alive only if an ACK arrived recently (`getMsSinceLastAck`).

EXPERT DETAIL

The phone smooths RTT with an **exponential moving average** — $\text{smoothed} = \text{prev} * 0.8f + \text{rttMs} * 0.2f$ — and clamps anything outside $0.0f \dots 1000.0f$ as a sanity guard before publishing it via `getNativeLatencyMs`. DEF:: Exponential moving average (EMA) :: a running average that weights recent samples more than old ones, so the HUD number is responsive but doesn't flicker on a single outlier. Be clear-eyed about what this measures: it is the *network round trip only*. It does **not** include the touch-sampling delay (the phone can't see its own digitizer latency), the game's poll interval, or your display's own lag. So the honest reading is "single-digit to low-teens ms is excellent," exactly as `LOW_LATENCY_GUIDE.md` says — with the understanding that a couple more milliseconds of touch and poll delay sit invisibly on either end.

Expert Trade-offs — Chasing the Last Millisecond

Every optimization here is a *trade*, and the 2026-06-21 work is really a set of rebalanced trades.

Packet rate vs battery and heat. The single biggest lesson is that the old fixed 1000 Hz send was a bad deal: it burned CPU and generated phone heat to re-send information that hadn't changed. The event-driven loop with a ~30 Hz keep-alive sends real input just as fast (within one 2 ms poll) while cutting idle traffic ~30×. Thermal throttling is itself a latency enemy — a hot SoC downclocks and adds scheduler jitter — so cutting heat *improves* worst-case latency. This is why the comment forbids re-adding core pinning.

Reliability vs speed. UDP drops packets; input can't afford to *wait* for retransmission the way TCP does.

DEFINITION

Head-of-line blocking — TCP's rule that packets must arrive in order, so one lost packet stalls *everything* behind it until it's resent. Fatal for real-time input.

The system splits the difference with **FIX 1**: instead of TCP's guaranteed-but-slow delivery, it sends a changed packet 3 times back-to-back. The PC dedups by content and orders by timestamp, so duplicates are harmless no-ops — but a single dropped datagram now recovers in ~4 ms instead of waiting up to 33 ms. You buy reliability with a tiny, bounded burst of bandwidth rather than with latency.

Best-effort everything. Notice that `SCHED_FIFO`, the `nice(-19)` fallback, `SO_PRIORITY`, `IP_TOS`, `HIGH_PRIORITY_CLASS`, and `timeBeginPeriod` all fail silently.

DEFINITION

`SCHED_FIFO` — a Linux real-time scheduling policy that lets a thread run ahead of normal threads. On stock Android it almost always needs a permission the app doesn't have, so the code tries it, then falls back to the strongest normal-priority "nice" value it can get.

Every knob is a *best-effort* request the OS may refuse. The design principle is that no single optimization is load-bearing — the link works if all of them fail, and each one that *does* land shaves a

little more jitter. That is the right posture for latency work: stack many cheap, independent, graceful wins rather than betting on one fragile trick.

When physics wins, change transports. The ultimate trade is admitting Wi-Fi is variable. USB tethering routes the same UDP path over the cable for wired-quality latency with no app changes, and the AOA (Android Open Accessory) transport goes further — raw USB bulk frames with no IP stack at all, which `server.py` calls "the lowest-latency wired path (~1–2 ms)." Both use the exact same event-driven send loop and 20-byte payload; only the transport underneath changes. The fastest millisecond, in the end, is the one you route around the router entirely.

CHAPTER 10

USB Mode — The AOA Transport

10

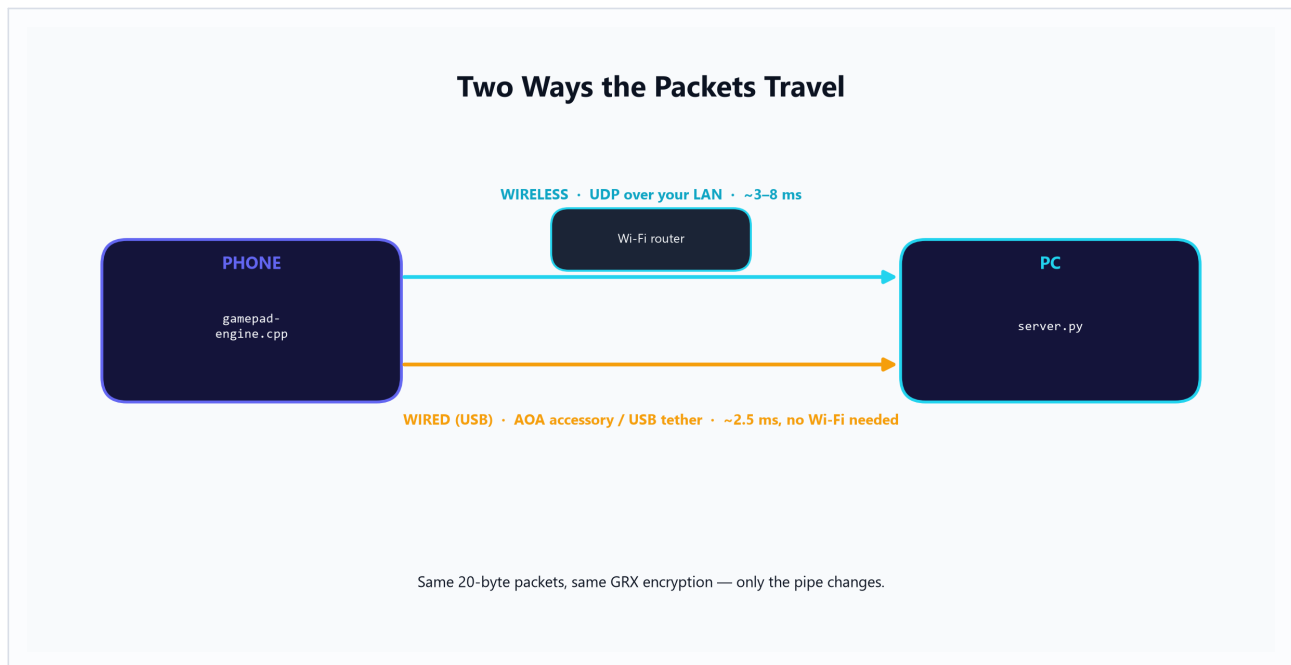


Figure 10.1 — USB is the second pipe; same packets, no Wi-Fi.

What AOA Actually Is

Everywhere else in this book, your phone reaches your PC over the network: it forms an input packet and throws it across Wi-Fi (or a USB-tethering link that still *looks* like a network). This chapter is about the one transport that throws away the network entirely and talks over the USB cable as raw hardware. It is called **AOA**, and in your codebase it lives in `apps/pc-server/aoa_transport.py`.

DEFINITION

AOA (Android Open Accessory) — A standard, built into Android since 2011, that lets a USB *host* device (here, your PC) turn a plugged-in Android phone into a USB *accessory* it can exchange raw data with — no adb, no IP address, no Wi-Fi involved.

To understand why this is even a puzzle, you have to know one strange fact about USB. In USB there is always exactly one boss and one follower.

DEFINITION

USB host — The side of a USB link that supplies power and controls the conversation — normally your PC. It asks; devices answer. A phone is *not* a host by default.

DEFINITION

USB accessory / device — The follower side that only speaks when the host addresses it. Your phone is normally a USB device to a PC (that is how file transfer and adb work), but a plain phone-as-device does not expose a channel your gamepad server can freely read and write.

IN PLAIN TERMS

Think of USB as a classroom where only the teacher (host) may call on people. Students (devices) can only speak when called. Your PC is the teacher and your phone is a student. AOA is a special note you pass the student that says "when I call on you, you and I get our own private two-way walkie-talkie channel." After the phone agrees, the two of you have a dedicated pipe nobody else in the room can use.

That private walkie-talkie is built from two **bulk endpoints**.

DEFINITION

Bulk endpoint — One of the four USB "transfer types," designed for moving arbitrary chunks of data reliably (the USB hardware retries on error) but without a guaranteed time slot. An endpoint is one-directional, so AOA uses a pair: an **IN** endpoint (phone → PC) and an **OUT** endpoint (PC → phone).

DEFINITION

Enumeration — The process where the host detects a freshly attached USB device, reads its identity descriptors, and assigns it an address. AOA works by making the phone *re-enumerate* — drop off the bus and reappear wearing a different identity — so the host can bind to its accessory interface.

The whole point, stated at the top of the file, is latency:

```
apps/pc-server/aoa_transport.py · lines 1-7 Python
1  """
2  AOA (Android Open Accessory) direct-USB transport — Milestone 1 proof-of-concept.
3
4  GOAL: lowest-possible WIRED latency (~1-2 ms) by talking to the phone directly over
5  USB bulk endpoints — no adb, no tethering, no IP stack. The PC is the USB *host*;
6  the phone becomes a USB *accessory* with two raw bulk endpoints (IN = phone->PC
7  input packets, OUT = PC->phone ACK/rumble).
```

This docstring is your ground truth for the *shape* of the system: PC is host, phone is accessory, IN carries your button-and-stick packets toward the PC, OUT carries the server's acknowledgements and rumble commands back. The target — roughly 1 to 2 milliseconds round trip — is why the feature exists at all; it beats every network path you have.

The Handshake: Talking a Phone Into Accessory Mode

A phone will not become an accessory on its own. Your PC has to *ask* it, using three special USB messages called **control transfers**.

DEFINITION

Control transfer — A small, structured USB message sent on the device's "control" channel (endpoint 0), used for setup and commands rather than bulk data. Each one carries a request type, a request number (`bRequest`), two 16-bit parameters (`wValue`, `wIndex`), and an optional data payload.

AOA defines exactly three request numbers, and your file lists them as constants:

```
apps/pc-server/aoa_transport.py · lines 80-95 Python
1 # — AOA control-request opcodes (USB vendor requests on the device) —————
2 AOA_GET_PROTOCOL = 51      # IN  : returns supported AOA protocol version (uint16 LE)
3 AOA_SEND_STRING = 52      # OUT : send one identifying string (by index)
4 AOA_START_ACCESSORY = 53   # OUT : tell the phone to re-enumerate in accessory mode
5
6 # bmRequestType values: vendor request, recipient = device.
7 _REQ_IN = usb.util.CTRL_IN | usb.util.CTRL_TYPE_VENDOR | usb.util.CTRL_RECIPIENT_DEVICE if usb else 0xC0
8 _REQ_OUT = usb.util.CTRL_OUT | usb.util.CTRL_TYPE_VENDOR | usb.util.CTRL_RECIPIENT_DEVICE if usb else 0x40
9
10 # String indices sent via AOA_SEND_STRING (order matters; all six are required).
11 AOA_STR_MANUFACTURER = 0
12 AOA_STR_MODEL = 1
13 AOA_STR_DESCRIPTION = 2
14 AOA_STR_VERSION = 3
15 AOA_STR_URI = 4
16 AOA_STR_SERIAL = 5
```

The three opcodes are 51, 52, 53. `_REQ_IN` and `_REQ_OUT` are the *direction+type* byte (`bmRequestType`) for each transfer: `0xC0` means "reading from the device, vendor-specific request, aimed at the whole device," and `0x40` is the same but writing to the device. The clever bit is the `if usb else` fallback — if PyUSB failed to import, `usb` is `None`, so the constants fall back to the literal `0xC0/0x40` and the module still loads instead of crashing at import time. The six string indices 0–5 are the fields of the accessory's "business card," and the comment warns that all six are required and order matters.

DEFINITION

Vendor request — A USB control transfer whose meaning is defined by the vendor rather than the USB standard. AOA piggybacks on this category so any phone can recognise the requests without a custom driver.

Step one of the handshake asks the phone whether it even speaks AOA:

```
apps/pc-server/aoa_transport.py · lines 185-197 Python
1 def get_protocol(dev) -> int:
2     """AOA GET_PROTOCOL (51) → protocol version (0 means not supported)."""
3     data = dev.ctrl_transfer(_REQ_IN, AOA_GET_PROTOCOL, 0, 0, 2)
4     return struct.unpack("<H", bytes(data))[0]
5
6
7 def send_identity_strings(dev) -> None:
8     """AOA SEND_STRING (52) ×6 — manufacturer, model, description, version, uri, serial."""
9     for index in range(6):
10         payload = ACCESSORY[index].encode("utf-8") + b"\x00"
11         sent = dev.ctrl_transfer(_REQ_OUT, AOA_SEND_STRING, 0, index, payload)
12         if sent != len(payload):
13             raise AoaError(f"SEND_STRING index {index} short write ({sent}/{len(payload)})")
```

`get_protocol` sends request 51 and reads back 2 bytes, then unpacks them as a little-endian unsigned 16-bit integer ("`<H`"). A result of 0 means "I don't do AOA"; 1 or 2 means yes. `send_identity_strings` then loops the six indices, and for each one grabs the matching value from the `ACCESSORY` dictionary, UTF-8 encodes it,

and appends a `\x00` null terminator (AOA strings are C-style null-terminated). Each is sent as request 52 with `wIndex=index`, so the phone knows which field it is receiving. The short-write check guards against a partial transfer.

Those six strings are not arbitrary — they are the phone's identity for your app, and they are declared once here as the single source of truth:

```
apps/pc-server/aoa_transport.py · lines 100-112 Python
1  ACCESSORY = {
2      AOA_STR_MANUFACTURER: "GamepadOS",
3      AOA_STR_MODEL: "GamepadController",
4      AOA_STR_DESCRIPTION: "GamepadOS low-latency USB controller",
5      AOA_STR_VERSION: "1.0",
6      AOA_STR_URI: "https://gamepad.space",
7      AOA_STR_SERIAL: "GP-AOA-0001",
8  }
9
10 # After START_ACCESSORY the phone re-appears under Google's accessory VID with one
11 # of these PIDs (accessory, accessory+adb, and the two audio-capable variants).
12 AOA_VID = 0x18D1
13 AOA_PIDS = (0x2D00, 0x2D01, 0x2D04, 0x2D05)
```

The manufacturer is "GamepadOS", the model "GamepadController", the version "1.0". Remember these three exactly — the Android side has to declare the identical strings or the phone will refuse to hand the connection to your app. `AOA_VID = 0x18D1` is Google's USB vendor ID, and after the handshake the phone reappears wearing one of four product IDs: `0x2D00` (accessory), `0x2D01` (accessory + adb), and `0x2D04/0x2D05` (the audio-capable variants). Your server accepts any of the four.

Re-Enumeration and Opening the Pipes

The third opcode is the trigger. When the PC sends `START_ACCESSORY` (53), the phone literally disconnects itself from USB and reconnects as a different device.

```
apps/pc-server/aoa_transport.py · lines 200-216 Python
1  def start_accessory(dev) -> None:
2      """AOA START_ACCESSORY (53) — phone drops off the bus and re-enumerates."""
3      dev.ctrl_transfer(_REQ_OUT, AOA_START_ACCESSORY, 0, 0, b"")
4
5
6  def wait_for_accessory(timeout=REENUM_TIMEOUT_S):
7      """Poll until the phone re-appears as 18D1:2D0x, or time out."""
8      deadline_polls = int(timeout / REENUM_POLL_S)
9      for _ in range(deadline_polls):
10         for pid in AOA_PIDS:
11             dev = usb.core.find(idVendor=AOA_VID, idProduct=pid, backend=_backend())
12             if dev is not None:
13                 print(f"[reenum] accessory present: {AOA_VID:04x}:{pid:04x}")
14                 return dev
15         time.sleep(REENUM_POLL_S)
16     raise AoaError("Phone did not re-enumerate in accessory mode after START_ACCESSORY "
17                  "(check the on-phone 'Use app with this USB accessory?' prompt / M2 not built yet).")
```

`start_accessory` fires request 53 with an empty body. After that the original phone device is *gone* from the bus, so `wait_for_accessory` polls: every `REENUM_POLL_S` (0.10 s) it scans for Google's VID `0x18D1` paired with each accessory PID, for up to `REENUM_TIMEOUT_S` (8.0 s) total. The moment the accessory node appears it returns that fresh device handle. If nothing shows up in eight seconds it raises — usually because the phone is showing a "Use app with this USB accessory?" prompt that nobody tapped.

Once the accessory device is present, the server has to claim its interface and locate the two bulk endpoints:

```
apps/pc-server/aoa_transport.py · lines 228-247 Python
1  dev.set_configuration()
2  cfg = dev.get_active_configuration()
3  intf = cfg[(0, 0)]
4
5  ep_in = usb.util.find_descriptor(
6      intf, custom_match=lambda e:
7          usb.util.endpoint_direction(e.bEndpointAddress) == usb.util.ENDPOINT_IN
8          and usb.util.endpoint_type(e.bmAttributes) == usb.util.ENDPOINT_TYPE_BULK)
9  ep_out = usb.util.find_descriptor(
10     intf, custom_match=lambda e:
11         usb.util.endpoint_direction(e.bEndpointAddress) == usb.util.ENDPOINT_OUT
12         and usb.util.endpoint_type(e.bmAttributes) == usb.util.ENDPOINT_TYPE_BULK)
13
14  if ep_in is None or ep_out is None:
15     raise AoaError("Accessory interface is missing a bulk IN/OUT endpoint pair.")
16
17  usb.util.claim_interface(dev, intf.bInterfaceNumber)
18  print(f"[open] claimed interface {intf.bInterfaceNumber}; "
19        f"bulk IN=0x{ep_in.bEndpointAddress:02x} OUT=0x{ep_out.bEndpointAddress:02x} "
20        f"maxpkt={ep_in.wMaxPacketSize}")
```

`set_configuration` activates the device's default configuration, then `cfg[(0, 0)]` selects interface number 0, alternate setting 0 — the accessory interface. The two `find_descriptor` calls walk that interface's endpoint list looking for one that is *bulk* and *IN* and one that is *bulk* and *OUT*. `claim_interface` locks the interface to this process so nothing else can touch it. On a real phone the printed result is `bulk IN=0x81 OUT=0x01 maxpkt=512` — endpoint address `0x81` for IN, `0x01` for OUT, with a 512-byte maximum packet size (your actual packets are only 20 bytes, so that ceiling is never a constraint).

EXPERT DETAIL

On Windows there is no kernel driver to detach from a WinUSB device, but the code still guards `is_kernel_driver_active(0)` / `detach_kernel_driver(0)` in a `try` so the same file works unchanged on Linux, where a kernel `usbfs` driver *would* need detaching. The Linux build ships the identical `aoa_transport.py` under `apps/GamepadServer-linux/`.

The Same 20 Bytes, a Different Pipe

Here is the elegant part: nothing about your *packet* changes. The exact same 20-byte input frame that flies over UDP also flies over USB — only the pipe is different. The wire contract is declared identically

on both sides.

apps/pc-server/aoa_transport.py · lines 114-122

Python

```
1 # Shared wire contract — IDENTICAL to server.py. Do NOT diverge.
2 PACKET_SIZE = 20
3 ACK_MAGIC = b"ACK"
4 RMB_MAGIC = b"RMB"
5
6 # Re-enumeration after START_ACCESSORY: the old device drops and the accessory
7 # re-appears. Poll for it.
8 REENUM_TIMEOUT_S = 8.0
9 REENUM_POLL_S = 0.10
```

`PACKET_SIZE = 20` matches `server.py`'s `PAYLOAD_SIZE`, which is `struct.calcsize('<Q H B B B B B I')` — an 8-byte timestamp, a 2-byte button bitfield, six single bytes for triggers and stick axes, and a 4-byte trailer, totalling 20. `ACK_MAGIC` is the three ASCII bytes `ACK` and `RMB_MAGIC` is `RMB`; these are the reply framings the phone already understands from the network transports. The comment is a standing order: this contract must never diverge from `server.py`, because both ends `struct.unpack` the same format.

The `AoaTransport` class turns the endpoints into three simple verbs:

apps/pc-server/aoa_transport.py · lines 276-292

Python

```
1 def recv(self, timeout_ms=200):
2     """Read one input packet. Returns bytes (len PACKET_SIZE) or None on timeout."""
3     try:
4         data = self.ep_in.read(self.ep_in.wMaxPacketSize, timeout=timeout_ms)
5     except usb.core.USBError as e:
6         if e.errno in (110, None) or "timeout" in str(e).lower():
7             return None
8         raise
9     return bytes(data)
10
11 def send_ack(self, timestamp):
12     self.ep_out.write(ACK_MAGIC + struct.pack("<Q", timestamp), timeout=200)
13
14 def send_rumble(self, large, small):
15     """Push force-feedback to the phone: "RMB" + large + small (same framing as
16     the UDP/WS transports)."""
17     self.ep_out.write(RMB_MAGIC + bytes([large & 0xFF, small & 0xFF]), timeout=200)
```

`recv` reads up to one max-packet's worth from the IN endpoint, waiting at most `timeout_ms` (default 200 ms). USB signals "nothing arrived in time" by raising a `USBError` with `errno 110` (that is the POSIX `ETIMEDOUT`), so the code catches exactly that and returns `None` rather than crashing — any *other* USB error re-raises as a genuine fault. `send_ack` writes the literal `ACK` followed by the phone's own timestamp packed as a little-endian 8-byte integer ("`<Q`"), so the phone can subtract and compute true round-trip latency. `send_rumble` writes `RMB` plus the two motor strengths as raw bytes, masked to 0–255. Same framing, same meaning as Wi-Fi — the phone's receive code does not care which pipe delivered it.

The Android Side: Answering the Attach

For any of this to work, the phone has to (a) advertise that it can be an accessory, (b) recognise *your* accessory by its identity strings, and (c) open its side of the pipe. All three exist in your Android project. The manifest opts in and declares an intent-filter:

```
apps/android-client/app/src/main/AndroidManifest.xml · lines 40-47 XML
1      <!-- AOA: the system launches/resumes us when our accessory (the PC
2           running GamepadServer) attaches and matches accessory_filter.xml. -->
3      <intent-filter>
4          <action android:name="android.hardware.usb.action.USB_ACCESSORY_ATTACHED" />
5      </intent-filter>
6      <meta-data
7          android:name="android.hardware.usb.action.USB_ACCESSORY_ATTACHED"
8          android:resource="@xml/accessory_filter" />
```

DEFINITION

Intent-filter — A manifest declaration telling Android "launch or resume this activity when an event of this kind happens." Here the event is `USB_ACCESSORY_ATTACHED` — an accessory just plugged in.

The intent-filter says MainActivity should wake up when *any* accessory attaches, and the `meta-data` line points at `@xml/accessory_filter` to narrow that to accessories matching a specific identity. That filter file exists and must mirror the PC's strings exactly:

```
apps/android-client/app/src/main/res/xml/accessory_filter.xml · lines 7-10 XML
1      <usb-accessory
2          manufacturer="GamepadOS"
3          model="GamepadController"
4          version="1.0" />
```

These three values — `manufacturer="GamepadOS"`, `model="GamepadController"`, `version="1.0"` — are byte-for-byte the manufacturer, model, and version from the PC's `ACCESSORY` dict. If even one character differed, Android would refuse to route the accessory to your app and the phone would show a generic "no app for this accessory" toast. When they match, Android launches MainActivity, hands it a `UsbAccessory`, and after the user approves the "Use app with this USB accessory?" prompt, the app calls `usbManager.openAccessory(accessory)`, gets a `ParcelFileDescriptor`, and does `pfd.detachFd()` to pull out a raw integer file descriptor. That fd goes straight to native code via `initAccessoryNative(fd)`, and `connectionType` is set to "wired".

DEFINITION

File descriptor (fd) — A small integer the operating system uses as a handle to an open I/O channel. Reading or writing the fd reads or writes the underlying USB accessory stream.

The native C++ engine then treats the fd exactly like a socket, except simpler:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 334-337
```

```
C++
```

```
1         if (accessoryMode) {
2             // AOA: write the 20-byte frame straight to the accessory fd (raw USB
3             // bulk → PC). No addressing/auth handshake — USB is point-to-point.
4             sent = write(accessoryFd, &networkPayload, sizeof(GamepadPayload));
```

When `accessoryMode` is true, the same 1000 Hz transmit thread that would normally `sendto()` a UDP socket instead calls plain POSIX `write()` on `accessoryFd`, pushing the 20-byte `GamepadPayload` straight down the USB OUT endpoint. There is no destination address, no discovery broadcast, no auth handshake — USB is point-to-point, so the packet can only go to the one PC on the other end of the cable. The fd is set non-blocking (`O_NONBLOCK`), and a `write` failing with `EAGAIN/EWOULDBLOCK` just means the USB buffer is briefly full and is ignored; any other error means the cable was pulled, so native closes the fd and lets Kotlin tear down. On the receive side the same thread `read()`s the fd and recognises the `RMB` and `ACK` magic bytes to drive rumble and latency — the mirror image of `send_ack/send_rumble` on the PC.

Wiring It Into the Server, Setup, and Real Status

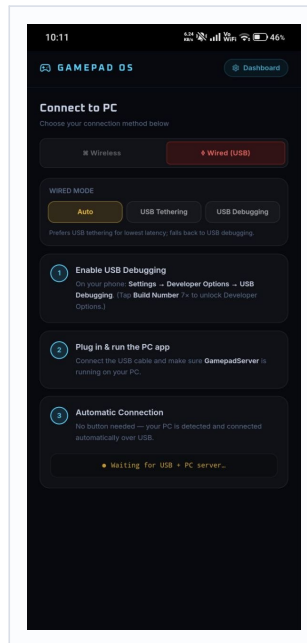


Figure 10.2 — The wired-USB connection screen.

On the PC, AOA is not a separate program — it is a background thread inside your normal `GamepadServer`, started at boot and coexisting with UDP and WebSocket:

```
apps/pc-server/server.py · lines 1244-1256
```

```
Python
```

```
1         try:
2             while True:
3                 pkt = transport.recv(timeout_ms=200)
4                 if pkt is not None and len(pkt) >= PAYLOAD_SIZE:
5                     ts, buttons, lt, rt, ls_x, ls_y, rs_x, rs_y, _auth = struct.unpack(
```

```

apps/pc-server/server.py · (cont.)
Python
6         PAYLOAD_FORMAT, bytes(pkt[:PAYLOAD_SIZE]))
7     if sess is not None:
8         apply_inputs(sess, buttons, lt, rt, ls_x, ls_y, rs_x, rs_y)
9         sess.last_seen = time.perf_counter()
10        telemetry.last_packet_time = time.perf_counter()
11        with telemetry.lock:
12            telemetry.packet_count += 1
13        transport.send_ack(ts)

```

This is `start_aoa_bridge`, launched as a daemon thread in `main()`. It calls `AoaTransport.recv`, and when a full 20-byte packet arrives it unpacks it with the shared `PAYLOAD_FORMAT` and feeds it into the very same `apply_inputs()` that the Wi-Fi and WebSocket loops use — so from ViGEm's point of view an AOA button press is indistinguishable from a Wi-Fi one. It bumps the telemetry counters and sends an `ACK` back. The session is registered under the key `"aoa"`, and the telemetry label becomes `"USB-AOA (Wired)"`. Crucially, the whole bridge is wrapped so that if `pyusb`, `libusb`, or a bound accessory is missing, it prints a note and the server keeps running on UDP/WS — AOA never hard-breaks anything.

The one manual setup step is the Windows driver. PyUSB can only open the accessory if **WinUSB** is bound to it.

DEFINITION

WinUSB — A generic Microsoft driver that lets a user-space program (via `libusb`) talk to a USB device directly, without a device-specific kernel driver. AOA on Windows needs it bound to the `18D1:2D00/2D01` accessory device.

WATCH OUT

Binding WinUSB to the phone's accessory interface disables adb and MTP file transfer for that device until you remove the driver (Device Manager → the device → Uninstall, tick "delete the driver software", then replug). This is documented in `driver/README.txt` and is the price of the lowest latency.

WATCH OUT

Use a real USB **data** cable. Many cheap cables carry power only, and with a charge-only cable the handshake simply never gets a reply — a silent failure that looks like a code bug but is not.

The setup path today is `driver/install-driver.bat`, which launches Zadig to bind WinUSB by hand; the "zero-friction" signed-INF installer is still on the roadmap, not shipped. Here is how the wired options actually stack up, per `driver/README.txt`:

Transport	Typical latency	Setup cost
AOA (this chapter)	~1–2 ms	WinUSB driver bind (one-time)
USB tethering (UDP)	~2.5–4 ms	none
USB debug (WebSocket)	~5 ms	adb / debugging on

NOTE

`driver/README.txt` is blunt that "you probably don't need this": USB tethering already gives ~2.5–4 ms with zero driver hassle, so AOA is a small extra gain for a real setup cost. It is the enthusiast's option, not the default.

Finally, the honest status. The three source files you started from — `aoa_transport.py`, `AOA_REQUIREMENTS.md`, and `SETUP_AOA.md` — all describe AOA as "Milestone 1 proof-of-concept," with M2 (Android handler), M3 (server integration), M4 (driver packaging) and M5 (on-device testing) listed as *not yet built*. **The code contradicts those docs, and the code wins.** M2 exists: `accessory_filter.xml`, the manifest intent-filter, `handleAccessoryIntent/openAccessoryAndStart`, and the native `initAccessoryNative(fd)` fd path are all present. M3 exists: `start_aoa_bridge` is a live daemon thread in `server.py` feeding `apply_inputs()`. M4 exists in Zadig form: the `driver/` folder ships `install-driver.bat` and a README. So AOA has *graduated* from experiment to a wired-in, coexisting third transport — the one gap remaining is the polished, signed-driver install that would make it zero-friction. The comment inside `wait_for_accessory` still says "M2 not built yet," but that line is stale relative to the shipping code around it.

PART V

**Shipping, the Platform, Ports &
Lessons**

CHAPTER 11

Releases, Installers, and the Updater

11

Why Versioning Is the Hardest Easy Problem

Shipping a new build sounds trivial: compile, upload, done. In practice it is the single most dangerous routine operation in this whole project, because **three separate programs on two different machines all carry a version number, and they must agree**. When they disagree, users get one of two silent failures: an update that never appears, or — far worse — an update that installs, relaunches, and *immediately offers itself again*, forever. You have already lived through the second one (the July 2026 incident, covered in the war-stories chapter). This chapter is the antidote: understand exactly where every version number lives and how the updater decides "is there something newer?"

DEFINITION

Version number — a label that says which build of a program this is. GamepadOS uses **semantic-style** names like **1.3.0** (human-readable) and, on Android, an additional integer **versionCode** like **22** that only ever counts up.

DEFINITION

The updater — code inside a *running* build that phones a server, asks "what is the latest version?", compares the answer to its own version, and — if the server's is higher — downloads and installs it.

IN PLAIN TERMS

Think of the version number as the date on a carton of milk. The updater is you, standing at the fridge, comparing the carton in your hand to the one the shop advertises as "freshest". If your carton's date is somehow *later* than the shop's advertised date, you'd be confused forever about whether to buy more. That confusion, in software, is an infinite update loop — and it is caused entirely by dates that don't line up.

The Two Products, and Their Version Fields

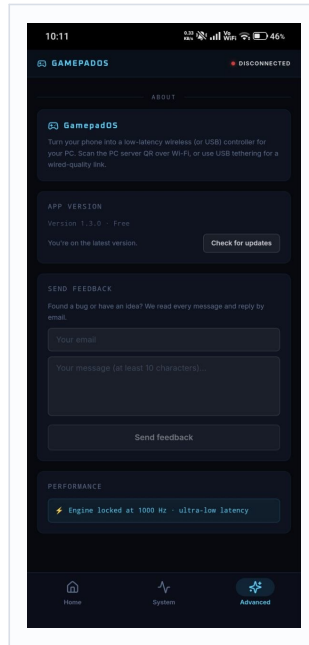


Figure 11.1 — The in-app version + update check.

GamepadOS ships **two** installable things, each with its own versioning:

Product	Human version	Machine field	Where it lives
Android app (APK)	<code>versionName = "1.3.0"</code>	<code>versionCode = 22</code>	<code>build.gradle.kts</code>
PC server (EXE)	<code>APP_VERSION = "1.1.16"</code>	(same string, compared numerically)	<code>server.py</code> + installer <code>.iss</code>

The currently shipped set is **Android 1.3.0 / versionCode 22** and **PC server 1.1.16**. Here is the Android side, straight from the build file:

```
apps/android-client/app/build.gradle.kts · lines 19-35 Kotlin

1  defaultConfig {
2      applicationId = "com.gamepad.client"
3      minSdk = 24
4      targetSdk = 34
5      versionCode = 22
6      versionName = "1.3.0"
7
8      ndk {
9          // Broad device coverage (universal APK for sideload + in-app updater):
10         // arm64-v8a — all modern phones/tablets (primary)
11         // armeabi-v7a — older / budget 32-bit ARM phones
12         // x86_64 — Android emulators, Chromebooks, x86 tablets
13         // (32-bit x86 omitted: effectively dead, only adds size.)
14         abiFilters.add("arm64-v8a")
15         abiFilters.add("armeabi-v7a")
16         abiFilters.add("x86_64")
17     }
```

Two numbers matter here. `versionName` is the friendly string a user sees ("Version 1.3.0"). `versionCode` is the integer Android and the updater actually compare — **it must increase by at least one every single release**, or the update simply won't be offered. The `abiFilters` block bundles three CPU architectures into one universal APK so the same file installs on modern phones, old budget phones, and emulators alike. That matters for the updater because it hands one APK to every device, whatever chip it runs.

DEFINITION

ABI (Application Binary Interface) — the CPU "dialect" a compiled binary speaks. `arm64-v8a` is modern 64-bit phones; `armeabi-v7a` is old 32-bit ARM; `x86_64` is emulators and Chromebooks. Bundling all three is why one APK runs everywhere.

DEFINITION

APK — Android Package — the single file that is an installed Android app, the way `.exe` is a Windows program. The updater downloads a new APK and hands it to Android's package installer.

The PC Server's Version — and the Rule Written Beside It

On the PC side the version is one constant near the top of `server.py`, and the comment beside it is the most important sentence in the release process:

```
apps/pc-server/server.py · lines 128-140 Python
1 # RELEASE RULE (RELEASE.md): APP_VERSION here, AppVersion + VersionInfoVersion in
2 # installer/GamepadServer.iss, and the version registered in the admin Releases
3 # panel must ALL match the build being shipped. A skew between the exe's baked
4 # version and the manifest's advertised version makes the in-app updater loop
5 # forever (install → relaunch → "update available" again).
6 APP_VERSION = "1.1.16"
7 # Public manifest served by the backend ({"pc": {"version","url","notes"}, ...}).
8 # Overridable via env var so a backend move never needs a code change + reship.
9 # NOTE 2026-07-02: the previous hardcoded host (admin.gamepad.space) never served
10 # this endpoint — set GAMEPAD_UPDATE_URL or fix the default to the live backend.
11 UPDATE_MANIFEST_URL = os.environ.get(
12     "GAMEPAD_UPDATE_URL",
13     "https://supportportal.gamepad.space/api/version")
```

`APP_VERSION` is the number the *running* server compares against what the website reports. `UPDATE_MANIFEST_URL` is where it looks — and notice it is read from an **environment variable** with a fallback default. That indirection exists because of a real scar: an earlier default pointed at `admin.gamepad.space`, a host that never actually served the version endpoint, so update checks silently timed out. Making the URL overridable means a backend move never again requires editing and re-shipping the EXE.

DEFINITION

Environment variable — a named value the operating system hands to a program when it starts (here, `GAMEPAD_UPDATE_URL`). It lets you change behavior without recompiling — you set the value outside the code.

DEFINITION

Manifest — a small machine-readable document (here JSON) that advertises the latest available versions and where to download them. The server fetches it and compares.

WATCH OUT

The version lives in **three** places for the PC build: `APP_VERSION` in `server.py`, plus `AppVersion` and `VersionInfoVersion` in the installer's `GamepadServer.iss`. `RELEASE.md` calls these "the must-match group". If the EXE is stamped with a lower version than the manifest advertises, every fresh install thinks it is already out of date — the exact loop the comment warns about.

How the Server Checks for an Update

The check itself is one carefully defensive function. It fetches the manifest, reads the `pc` section, and compares versions *numerically* (so that `1.2.10` correctly beats `1.2.9`, which a naive string compare would get wrong):

apps/pc-server/server.py · lines 142-151

Python

```
1 def _parse_version(v):
2     """'1.2.10' -> (1, 2, 10); non-numeric parts -> 0. Lets us compare versions
3     numerically (so 1.2.10 > 1.2.9) instead of as strings."""
4     parts = []
5     for p in str(v or "").strip().split("."):
6         try:
7             parts.append(int(p))
8         except ValueError:
9             parts.append(0)
10    return tuple(parts) or (0,)
```

This turns `"1.2.10"` into the tuple `(1, 2, 10)`. Python compares tuples element by element, so `(1, 2, 10) > (1, 2, 9)` is `True` — the numeric comparison a string would botch (`"1.2.10" < "1.2.9"` is `True` as text, which would *hide* the newer build). The `try/except` means a weird part like `"1.2.beta"` degrades to `0` instead of crashing.

Now the actual network fetch. Read the docstring first — it promises the function **never raises** and always returns a dictionary describing what happened:

apps/pc-server/server.py · lines 164-186

Python

```

1 def check_for_update(timeout=10, retries=1):
2     """Fetch the manifest and compare to APP_VERSION. Returns
3     {available, latest, url, notes, error, kind} and NEVER raises. `kind` is one of
4     None | 'offline' | 'server' | 'timeout' | 'parse' so the UI can show a specific
5     message. Retries once (the backend cold-starts on the free tier, so the first
6     hit after idle can be slow). Pure stdlib → works inside the frozen exe."""
7     import json, urllib.request, urllib.error, socket as _socket
8     last = None
9     for attempt in range(retries + 1):
10         try:
11             req = urllib.request.Request(
12                 UPDATE_MANIFEST_URL,
13                 headers={"User-Agent": "GamepadServer/%s" % APP_VERSION})
14             with urllib.request.urlopen(req, timeout=timeout) as resp:
15                 data = json.loads(resp.read().decode("utf-8"))
16                 pc = data.get("pc", {}) or {}
17                 latest = str(pc.get("version", "")).strip()
18                 url = pc.get("url") or ""
19                 notes = pc.get("notes") or ""
20                 available = bool(latest) and (_parse_version(latest) > _parse_version(APP_VERSION))
21             return {"available": available, "latest": latest, "url": url,
22                     "notes": notes, "sha256": str(pc.get("sha256") or ""),
23                     "error": None, "kind": None}

```

The one line that decides everything is `available = bool(latest) and (_parse_version(latest) > _parse_version(APP_VERSION))` — "there is an update *only* if the manifest names a version and that version is numerically greater than mine." Everything else is robustness: a `User-Agent` header identifying the build, a `timeout`, and — crucially — a `retries` loop, because the website backend sleeps on the free hosting tier and the first request after idle can be slow to wake ("cold-start"). It retries once rather than reporting a false "no update".

DEFINITION

Cold start — when a server that was idle (asleep to save money on free hosting) takes extra seconds to wake up and answer the first request. The single retry hides this from the user.

EXPERT DETAIL

The function is written entirely against Python's standard library (`urllib`, `json`, `socket`, `hashlib`) with local `imports` inside the function. That is deliberate: the server ships as a **frozen** PyInstaller EXE, and pulling in a third-party HTTP library like `requests` would bloat the bundle and risk import failures inside the frozen environment. Stdlib-only guarantees the updater works in the packaged build exactly as it does in a plain `python server.py` run.

Downloading Safely — Never Execute Garbage

Once an update is available, the server streams the installer to disk. This is where a naive implementation gets people hacked or bricked. The real one verifies both a checksum and the file's magic header before it will ever run what it downloaded:

apps/pc-server/server.py · lines 236-250

Python

```

1      if done == 0:
2          return (False, "downloaded 0 bytes")
3      if first[:2] != b"MZ":
4          return (False, "not a valid Windows installer")
5      if expected_sha256 and h.hexdigest().lower() != str(expected_sha256).lower():
6          try:
7              os.remove(dest_path)
8          except Exception:
9              pass
10         return (False, "checksum mismatch – download corrupt")
11     return (True, None)
12 except (urllib.error.URLError, _socket.timeout) as e:
13     return (False, "download failed (%s)" % getattr(e, "reason", e))
14 except Exception as e:
15     return (False, "download failed (%s)" % e)

```

Three gates protect the user. `done == 0` catches an empty download. `first[:2] != b"MZ"` checks the first two bytes — every Windows executable begins with the letters **MZ**, so if the "installer" is actually an HTML error page (which starts with `<`), it is rejected before it can run. And the SHA-256 checksum, compared against the hash the manifest supplied, guarantees the bytes on disk are exactly the bytes the release was published with; a mismatch deletes the file. Only if all three pass does it return `(True, None)`.

DEFINITION

SHA-256 checksum — a 64-character fingerprint computed from a file's contents. Change one byte and the fingerprint changes completely. Comparing the downloaded file's fingerprint to the published one proves it arrived intact and unmodified.

DEFINITION

Magic header ("MZ") — the fixed first bytes that identify a file type. **MZ** (the initials of a Microsoft engineer, Mark Zbikowski) marks every Windows `.exe`. Checking it is a cheap "is this even an executable?" test.

After a verified download, `launch_installer_elevated` starts the Inno Setup installer with silent flags (`/SILENT /SUPPRESSMSGBOXES /NORESTART /FORCECLOSEAPPLICATIONS`) via a Windows UAC elevation prompt. The installer closes the running server, installs the new build in place, and relaunches it — so from the user's side, "Download & install update" is one click and a brief progress bar.

DEFINITION

Elevation / UAC — Windows' "this program wants admin rights" prompt. Installing to **Program Files** needs it, so the updater launches the installer with `runas`, triggering the familiar consent dialog.

The Website Side — Where "Latest" Is Decided

The server asks a URL "what's newest?" — that URL is the website backend's `/api/version` endpoint. It does not hardcode a version; it reads whichever release row is currently marked **active** in the database

and hashes the actual file on disk:

```
website/backend/server.js · lines 1069-1090 JavaScript

1  try {
2    const pcRel = await prisma.release.findFirst({ where: { asset: 'pc', isActive: true } });
3    const appRel = await prisma.release.findFirst({ where: { asset: 'app', isActive: true } });
4
5    const pcFile = path.join(downloadsDir, pcRel ? pcRel.filename : 'GamepadServer-Setup.exe');
6    const pc = pcRel ? {
7      version: pcRel.version,
8      url: `${PUBLIC_BASE}/api/download/pc?update=1`,
9      sha256: fileSha256(pcFile),
10     notes: pcRel.notes || ''
11   } : { version: '1.1.4', url: `${PUBLIC_BASE}/api/download/pc?update=1`, sha256: fileSha256(pcFile), notes: 'Default' };
12
13   const appFile = path.join(downloadsDir, appRel ? appRel.filename : 'GamepadOS.apk');
14   const android = appRel ? {
15     version: appRel.version,
16     versionCode: appRel.versionCode || 1,
17     url: `${PUBLIC_BASE}/api/download/app?update=1`,
18     sha256: fileSha256(appFile),
19     notes: appRel.notes || ''
20   } : { version: '1.1.8', versionCode: 10, url: `${PUBLIC_BASE}/api/download/app?update=1`, sha256: fileSha256(appFile), notes: 'Default' };
21
22   res.json({ pc, android });
```

This is the heart of the "one source of truth" design. There is no version string typed into the API — it queries `release` rows where `isActive: true`, reads the version and filename from that row, and computes the SHA-256 by hashing the file itself (`fileSha256(pcFile)`). That means the hash the PC updater verifies against can never drift from the file it will actually download; the backend recomputes it live. The download URL carries `?update=1` so the server can tell an in-app update apart from a fresh website download (this feeds the "updates" counter, separate from "unique installs").

DEFINITION

Endpoint — a single URL on the backend that answers one kind of request. `/api/version` answers "what are the latest versions?"; `/api/download/app` answers "give me the app file".

DEFINITION

Active release — the one row per product (`pc` / `app`) in the database flagged `isActive: true`. Activating a new release flips the old one off, so exactly one is ever "latest".

Publishing a Release — the Admin Ritual

How does a `release` row become active? Through the admin portal's **Releases** panel, backed by these endpoints (from `server.js`):

- `GET /api/admin/releases` — lists known releases *and* scans the downloads folder for any `.apk/.exe` not yet registered (shown as "Unregistered").

- `POST /api/admin/releases/register-activate` — registers a file as a release and activates it in one step (owner-only, gated by a secret `RELEASE_KEY`).
- `POST /api/admin/releases/:id/activate` — flips an existing release to active.

The whole process is codified in `RELEASE.md`, which reads like a pre-flight checklist because it is one. Its Android section is blunt about the two failure modes:

```
RELEASE.md · lines 68-72 Markdown
1  ### 1. Bump both fields
2
3  `apps/android-client/app/build.gradle.kts`:
4  - `versionCode = 11` ← **increment the integer every release** (Android compares this; the in-app updater
   only offers an update when the server's versionCode is higher)
5  - `versionName = "1.1.9"` ← human-readable
```

The rule is mechanical: bump `versionCode` every time, or the updater stays silent. There is a second, equally unforgiving rule for Android — the new APK **must be signed with the same release keystore** as every prior build, because Android's package installer refuses to "update" an app to a differently-signed APK (it looks like a different app). Same key = seamless update; wrong key = "App not installed". That signing config lives in the build file:

```
apps/android-client/app/build.gradle.kts · lines 44-51 Kotlin
1  signingConfigs {
2      create("release") {
3          storeFile = file("release.keystore")
4          storePassword = localProperties.getProperty("keystore.password")
5          keyAlias = localProperties.getProperty("key.alias")
6          keyPassword = localProperties.getProperty("key.password")
7      }
8  }
```

The passwords are read from `local.properties` (a file kept out of the shipped code), and the key material lives in `release.keystore`. Every release APK is signed with this exact key so the updater's hand-off to the system installer always succeeds.

DEFINITION

Signing / keystore — a cryptographic key that stamps an APK as "from this developer". Android trusts an update only if it carries the *same* stamp as the installed app. Losing this keystore would mean no existing user could ever update again.

DEFINITION

RELEASE_KEY — a secret string set in the backend's environment. The activate endpoints refuse to run without it, so a stolen admin session alone cannot push a malicious "update" to every user.

One Manual Publish Script, and the Archive

Most activation happens through the portal UI, but there is also a tiny standalone script that does the same database moves in code — useful for scripted or emergency publishes. It shows the pattern plainly: deactivate the old, create the new as active.

```
website/backend/update_releases.js · lines 4-25 JavaScript

1  async function main() {
2    // Deactivate old App releases
3    await prisma.release.updateMany({
4      where: { asset: 'app' },
5      data: { isActive: false }
6    });
7
8    // Create new App release
9    await prisma.release.create({
10     data: {
11       asset: 'app',
12       version: '1.2.5',
13       versionCode: 17,
14       filename: 'GamepadOS-1.2.5.apk',
15       notes: 'Critical fix: restored connection mutex that was accidentally broken in 1.2.4, causing all tr
16       ansports (QR, USB debug, USB tether) to fail.',
17       isActive: true
18     }
19   });
20   // PC stays at 1.1.13
21   console.log('Successfully updated App release to 1.2.5 (PC stays at 1.1.13)');
22 }
```

First it sets every existing `app` release to `isActive: false`, then it creates the new row already active. That "deactivate all, then activate one" order guarantees exactly one active release at any instant — the same invariant `/api/version` relies on. (This script is a historical snapshot pinned at 1.2.5; the live active versions today are 1.3.0 / 1.1.16, set through the portal.)

Finally, every shipped build is preserved. The `releases-archive/` folder keeps a full rollback point per version — for the current set, `v1.3.0-android/` (APK plus the exact `src/` that built it) and `v1.1.16-pc/` (installer plus its `src/`). The archived files are byte-identical to what the downloads folder serves, so you can always diff a regression against a known-good build or roll forward from a clean base.

NOTE

The design choice throughout is **"go forward, never roll back"**. When a bad build escapes, the fix ships as a *higher* version so that even users stuck on the broken build get offered the repair through the normal updater — a rollback would leave them stranded on the bad version with nothing newer to pull.

The Whole Loop, End to End

Putting it together, the life of an update is:

1. You bump `versionCode/versionName` (Android) or `APP_VERSION` + the two `.iss` fields (PC), rebuild, and sign.

2. You copy the built file into `website/backend/downloads/` with a unique name.
3. In the admin **Releases** panel you **Register & Activate** it (owner-only, `RELEASE_KEY`), which flips the previous release off and this one on.
4. `/api/version` now reports the new version and its live-computed SHA-256.
5. A running old build's `check_for_update()` fetches the manifest, sees a higher version, and offers it.
6. On confirm, `download_update()` streams the file, verifies the `MZ` header and SHA-256, and `launch_installer_elevated()` (PC) or Android's package installer (app) installs and relaunches.

Every step exists to keep one promise: **the version the user is told about, the file they download, and the fingerprint they verify are always the same real build** — because the drift between those three is precisely what caused the update loop this pipeline was rebuilt to prevent.

CHAPTER 12

The Website — Downloads, Feedback, Tickets, Admin

12

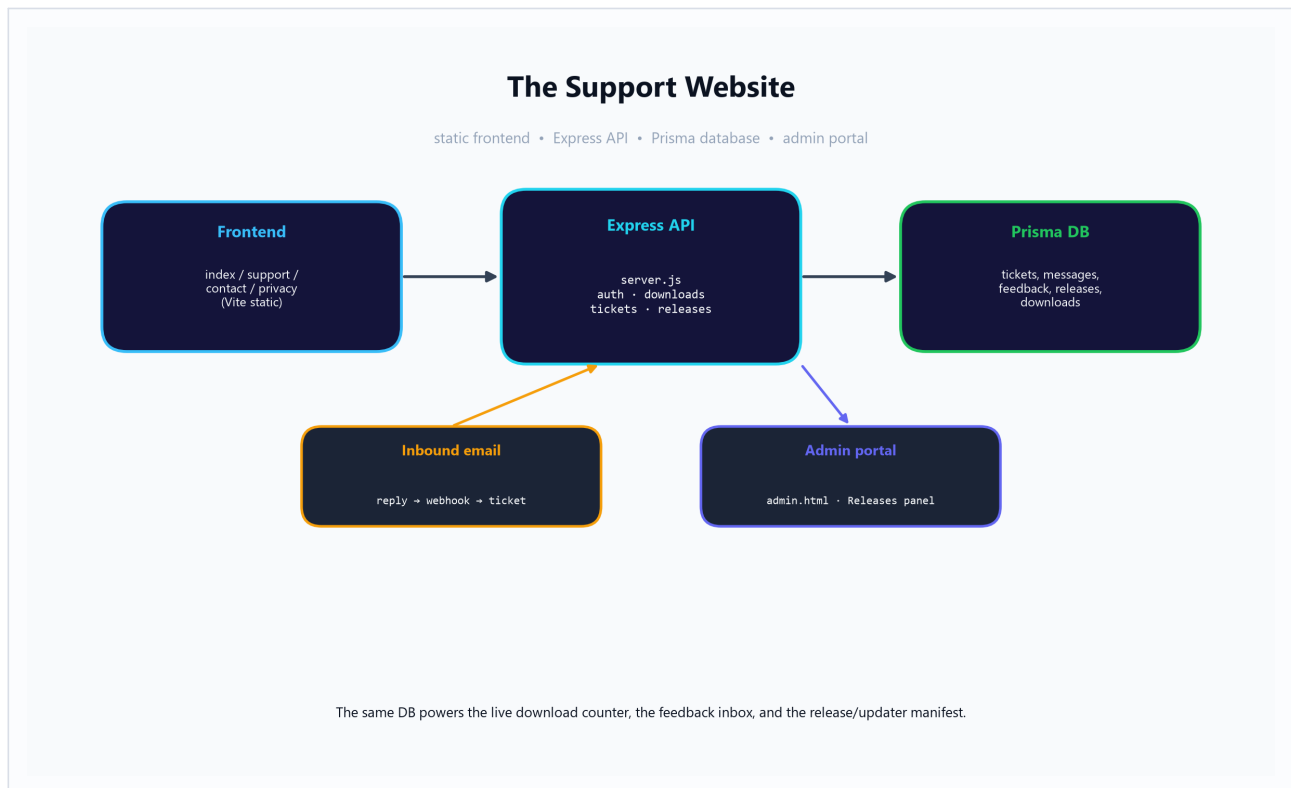


Figure 12.1 — Static frontend, Express API, Prisma DB, admin portal.

What the Website Is For

The controller app and PC server are the product. The website is everything *around* the product: where people download it, where they send feedback, where their support emails become tickets you can answer, where you publish new releases, and where the live download counter ticks up. It is a completely separate program from the game-controller stack — different language, different machine, always online — and it lives at `F:/hlooo/website`.

DEFINITION

Backend — a program that runs on an always-on server (not the user's device) and answers requests over the internet. The website's backend is a single Node.js/Express program, `website/backend/server.js`.

DEFINITION

Frontend — the pages a visitor actually sees in their browser — HTML, CSS, and a little JavaScript. Here it is a set of static pages (`index.html`, `support.html`, `contact.html`, `privacy.html`) built with Vite and served separately from the backend.

IN PLAIN TERMS

Picture a restaurant. The **frontend** is the dining room — menus, tables, the pretty stuff customers see. The **backend** is the kitchen — where orders are actually cooked, ingredients stored, and the till kept. Customers never walk into the kitchen; they send orders in (requests) and get plates back (responses). The website works the same way: the browser sends requests, the backend cooks and replies.

The whole thing is three layers: a **static frontend**, an **Express API** (the backend), and a **Prisma-managed database**. This chapter walks each one, then the two subsystems that make it more than a brochure: the support/ticketing system with its inbound-email webhook, and the admin portal.

How an Express Server Works

DEFINITION

Express — a small framework for building web servers in JavaScript. You tell it "when a request comes in for this URL and method, run this function" — and it handles the networking underneath.

DEFINITION

Route — one rule of the form "**METHOD** /path → run this handler". `app.get('/api/version', handler)` means "when someone does a GET request to /api/version, run handler".

DEFINITION

Middleware — a function that runs *before* your route handler, in a chain. Each one can inspect or modify the request, then either pass control onward (`next()`) or stop and respond. Authentication, CORS, and JSON-parsing are all middleware.

The program starts by creating the app and wiring up the middleware chain. The order matters — each `app.use(...)` adds a link that every request passes through:

website/backend/server.js · lines 14-16

JavaScript

```
1 const app = express();
2 app.set('trust proxy', 1); // Railway/Vercel sit behind a proxy – needed for secure cookies
3 const PORT = process.env.PORT || 3000;
```

`express()` builds the app. `trust proxy` tells Express it sits *behind* a hosting proxy (Railway/Vercel), so it should read the real visitor IP and honor secure-cookie rules from the proxy's headers instead of seeing everything as coming from the proxy itself. `PORT` is read from the environment (the host assigns one) with a local default of 3000.

DEFINITION

Proxy — an intermediary server the host puts in front of your app to handle HTTPS and routing. `trust proxy` makes Express believe the proxy's "here's the real client IP/protocol" headers.

CORS — Who Is Allowed to Call the API

A browser will refuse to let page A call an API on domain B unless domain B explicitly allows it. That policy is **CORS**, and getting it wrong is a classic silent breakage. The backend splits its rules in two: public endpoints anyone may call, and admin endpoints locked to the known site origins.

website/backend/server.js · lines 117-134

JavaScript

```
1 // Public client endpoints (App/PC feedback + downloads) must be callable from ANY
2 // origin — the Android WebView and the PC server aren't in the website allowlist.
3 // No credentials/cookies are used on these, so reflecting any origin is safe; the
4 // admin API stays cookie-gated. Registered BEFORE the allowlist CORS so their
5 // preflight always succeeds.
6 const publicCors = cors({ origin: true });
7 app.use('/api/support/ticket', publicCors);
8 app.use('/api/download', publicCors);
9
10 app.use(cors({
11   origin: (origin, cb) => {
12     // No Origin (curl, server-to-server, same-origin) → allow. Localhost dev → allow.
13     if (!origin || ALLOWED_ORIGINS.has(origin) || /^http:\/\/(localhost|127\.0\.0\.1):\d+$/ .test(origin)) {
14       return cb(null, true);
15     }
16     return cb(null, false);
17   }
18 }));
```

DEFINITION

CORS (Cross-Origin Resource Sharing) — the browser rule that a web page may only call an API on a different domain if that API says "I allow this origin". The backend answers that question per request.

The reasoning is precise. The Android app's WebView and the PC server both need to POST feedback and pull downloads, but they aren't websites with a fixed origin, so `/api/support/ticket` and `/api/download` get `publicCors` (allow anyone) — safe because those endpoints use no cookies, so there's nothing to steal. Every *other* route falls under the allowlist CORS that only accepts origins listed in `FRONTEND_URL` (plus localhost for development). The admin API stays behind that wall *and* behind a login cookie.

Right after CORS, the JSON body parser is raised to a 2 MB limit — because inbound email payloads carry full HTML message bodies that blow past Express's default 100 KB.

website/backend/server.js · lines 135-136

JavaScript

```
1 // 2mb: inbound email payloads (full HTML bodies) easily exceed Express's 100kb default.
2 app.use(express.json({ limit: '2mb' }));
```

The Database, Through Prisma

DEFINITION

Database — a program that stores structured data durably and lets you query it. Here it is PostgreSQL, a robust relational database. Every ticket, message, admin account, download, and release is a row in it.

DEFINITION

Prisma — a toolkit that lets the JavaScript code describe tables as models in one schema file and then read/write them with method calls (`prisma.ticket.create(...)`) instead of raw SQL. The schema doubles as documentation.

The whole data model lives in `schema.prisma`. The datasource block picks PostgreSQL and reads its connection string from the environment:

website/backend/prisma/schema.prisma · lines 5-8

Prisma schema

```
1 datasource db {
2   provider = "postgresql"
3   url      = env("DATABASE_URL")
4 }
```

The most important model is the support ticket. Notice how much lives on one row — status, priority, tags, the assigned admin, CSAT rating, and the email-threading fields that let replies find their way home:

website/backend/prisma/schema.prisma · lines 10-32

Prisma schema

```
1 model Ticket {
2   id      String  @id @default(uuid())
3   name    String
4   email   String
5   subject String
6   message String
7   status  String  @default("open")
8   reply   String? // latest admin reply (compatibility; full thread lives in messages)
9   repliedAt DateTime? // when the admin last replied
10  hasNewReply Boolean @default(false) // user replied by email and admin hasn't opened it yet
11  lastEmailMessageId String? // Message-ID of the user's last email, so our replies thread
    in their inbox
12  assigneeId String? // which admin owns this ticket (null = unassigned)
13  assignee  Admin? @relation("AssignedTickets", fields: [assigneeId], references: [id], onDelete: SetNull
    )
14  priority  String  @default("normal") // low | normal | high | urgent
15  tags      String  @default("") // comma-separated, lowercased (cross-DB friendly)
16  source    String  @default("web") // 'web' | 'pc' | 'mobile' — origin of the ticket/feedback
17  firstResponseAt DateTime? // first admin reply — drives the response-time stat
18  resolvedAt DateTime? // last time it moved to resolved — drives resolved-this-week
19  csatRating String? // 'good' or 'bad'
20  csatFeedback String?
21  createdAt DateTime @default(now())
22  messages TicketMessage[]
23 }
```

DEFINITION

Model — one table described in the Prisma schema. Each field is a column; `@id` marks the primary key; `@default(uuid())` auto-generates a unique id; `?` means the field may be empty.

DEFINITION

Relation — a link between models. `messages TicketMessage[]` means one ticket owns many messages; the matching `TicketMessage` model points back with `ticketId`. That's how a ticket's whole conversation is stored.

The `source` field ("web" / "pc" / "mobile") is what lets the admin portal badge each ticket with where it came from — a bug report typed into the PC server looks different from one sent through the website. And `lastEmailMessageId` is the key to email threading: storing the user's last Message-ID lets outgoing replies carry the right *In-Reply-To* header so they land in the same conversation in the user's inbox.

The schema's other models round out the platform: `TicketMessage` (one row per message, either direction), `Admin` + `AdminSession` + `AuditLog` (the multi-admin system), `Download` (one row per unique downloader per day), `Release` (the version rows from the updater chapter), `CannedResponse` (saved reply templates), and `Setting` (a tiny key/value store).

Authentication — How Admin Login Works

Admin accounts are protected by real password hashing, not stored passwords. The crypto lives in `auth.js` and uses **scrypt**, a deliberately slow hash designed to resist brute-force cracking:

```
website/backend/auth.js · lines 10-26 JavaScript

1 function hashPassword(password) {
2   const salt = crypto.randomBytes(16).toString('hex');
3   const hash = crypto.scryptSync(String(password), salt, SCRYPT_KEYLEN).toString('hex');
4   return `scrypt:${salt}:${hash}`;
5 }
6
7 function verifyPassword(password, stored) {
8   try {
9     const [scheme, salt, hash] = String(stored || '').split(':');
10    if (scheme !== 'scrypt' || !salt || !hash) return false;
11    const candidate = crypto.scryptSync(String(password), salt, SCRYPT_KEYLEN);
12    const expected = Buffer.from(hash, 'hex');
13    return candidate.length === expected.length && crypto.timingSafeEqual(candidate, expected);
14  } catch {
15    return false;
16  }
17 }
```

DEFINITION

Password hash — a one-way scramble of a password. The database stores the hash, never the password. To check a login you hash the attempt and compare — you can verify without ever knowing the real password.

DEFINITION

Salt — random bytes mixed into each password before hashing, unique per account. It means two users with the same password get different hashes, defeating precomputed "rainbow table" attacks.

`hashPassword` generates a fresh 16-byte salt, runs `scrypt`, and stores `scrypt:<salt>:<hash>` as one string. `verifyPassword` splits that apart, re-hashes the attempt with the *same* salt, and compares — but crucially with `crypto.timingSafeEqual`, not `===`.

DEFINITION

Constant-time compare — comparing two secrets in a way that always takes the same amount of time, whether they match at byte 1 or byte 60. A normal `===` bails out early on the first mismatch, and an attacker can measure that timing to guess a secret byte by byte. `timingSafeEqual` closes that leak.

Once logged in, the browser holds an opaque session token in an `HttpOnly` cookie (the token is a random 64-hex string from `newSessionToken()`), and every admin request is gated by the `requireAdmin` middleware:

```
website/backend/server.js · lines 199-219 JavaScript
1  async function requireAdmin(req, res, next) {
2    try {
3      const admin = await getSessionAdmin(req);
4      if (admin) { req.admin = admin; return next(); }
5      // Script/tool fallback: "Bearer <ADMIN_PASSWORD env>" acts as an owner.
6      const adminPass = process.env.ADMIN_PASSWORD;
7      if (adminPass && req.headers['authorization'] === `Bearer ${adminPass}`) {
8        req.admin = { id: null, name: 'API token', role: 'owner', active: true };
9        return next();
10     }
11     return res.status(401).json({ success: false, error: 'Not signed in.' });
12   } catch (e) {
13     console.error('Auth error:', e);
14     return res.status(500).json({ success: false, error: 'Internal server error.' });
15   }
16 }
17
18 function requireOwner(req, res, next) {
19   if (req.admin && req.admin.role === 'owner') return next();
20   return res.status(403).json({ success: false, error: 'Owner access required.' });
21 }
```

`requireAdmin` looks up the session from the cookie; if valid it attaches the acting admin as `req.admin` and calls `next()` to let the route run. If not, it responds `401 Not signed in`. `requireOwner` is a second, stricter gate — some actions (managing the team, activating releases) require the **owner** role, not just any admin. Routes stack them: `app.post('/api/admin/team', requireAdmin, requireOwner, ...)` means "must be signed in *and* must be an owner".

DEFINITION

HttpOnly cookie — a cookie the browser will not let JavaScript read. Storing the session token this way means a cross-site script can't steal it.

DEFINITION

Role — an account's permission level. Here it is **owner** (full control) or **agent** (can handle tickets but not manage the team or publish releases).

The Ticketing System — from Form to Inbox

When someone submits the support form, the request hits `/api/support/ticket`. First it is validated with a **Zod** schema, then auto-assigned, saved, broadcast to open dashboards, and acknowledged by email:

```
website/backend/server.js · lines 533-554 JavaScript

1  app.post('/api/support/ticket', async (req, res) => {
2    try {
3      const parseResult = ticketSchema.safeParse(req.body);
4      if (!parseResult.success) {
5        const errorMsg = parseResult.error.errors.map(err => err.message).join(' ');
6        return res.status(400).json({ success: false, error: errorMsg });
7      }
8
9      const { name, email, subject, message, source } = parseResult.data;
10
11     // Auto-route on arrival (business → least-loaded owner; else least-loaded admin).
12     const assignee = await pickAssignee(subject);
13
14     const newTicket = await prisma.ticket.create({
15       data: { name, email, subject, message, source, status: 'open', assigneeId: assignee ? assignee.id : null },
16     });
17     if (assignee) {
18       audit(null, 'auto-assign', newTicket.id, `→ ${assignee.name}${subject === 'business' ? ' (business → owner)' : ' (least-loaded)'} `);
19     }
20
21     // Push the new ticket to every open dashboard instantly (source → badge).
22     sseBroadcast('ticket:new', { id: newTicket.id, name, subject, source });
```

DEFINITION

Zod / validation — a library that checks incoming data against a shape you declare (name at least 2 chars, valid email, subject from a fixed list, message at least 10 chars). Bad input is rejected with a clear message *before* it ever touches the database.

The flow is: validate → `pickAssignee` (auto-route: "business" inquiries go to the least-loaded owner, everything else to the least-loaded admin, so nothing sits unowned) → create the row → `audit(...)` the assignment → `sseBroadcast` so every admin's open dashboard shows the new ticket instantly. After this it fires two emails — a "new ticket" alert to the admin inbox and a confirmation to the user whose subject carries a `[#ref]` and whose Reply-To is a per-ticket address. That Reply-To is the hook for the

whole inbound-email trick below.

DEFINITION

SSE (Server-Sent Events) — a one-way live stream from server to browser. `sseBroadcast` pushes "a new ticket arrived" to every open admin dashboard so the list updates without a refresh. A heartbeat ping every 25 s keeps the proxy from closing the idle stream.

The Inbound-Email Webhook — Replies Become Messages

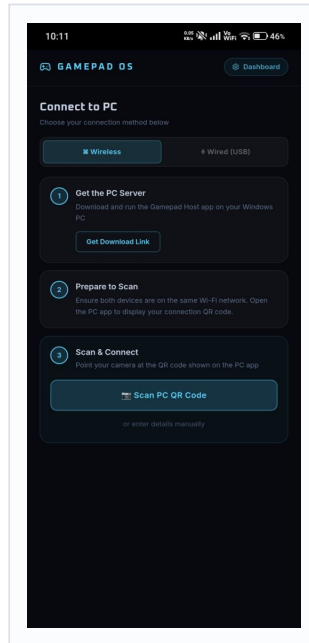


Figure 12.2 — The wireless connect screen users reach from the site's download.

This is the platform's cleverest piece. When you reply to a user from the portal, the email's Reply-To is `ticket+<id>@<reply-domain>`. When the user hits "reply" in their own mail app, their message goes to *that* address — which the mail provider (Brevo) receives and POSTs to the backend as JSON. The webhook turns it back into a ticket message.

website/backend/server.js · lines 763-784

JavaScript

```
1 app.post('/api/email/inbound', async (req, res) => {
2   if (!INBOUND_TOKEN) {
3     return res.status(503).json({ success: false, error: 'Inbound email is not configured (INBOUND_WEBHOOK_TOKEN unset).' });
4   }
5   if ((req.query.token || '') !== INBOUND_TOKEN) {
6     return res.status(401).json({ success: false, error: 'Invalid token.' });
7   }
8   const items = Array.isArray(req.body && req.body.items) ? req.body.items : [];
9   // Acknowledge Brevo INSTANTLY (sub-second), THEN save in the background. Brevo's
10  // inbound webhook gives the backend only a few seconds to respond; awaiting the
11  // DB writes here meant a cold start / redeploy / slow query could exceed that
12  // window and Brevo logged `webhookFailed`, silently dropping the reply. Replying
```

```

13 // first removes timing from the equation entirely – the save can take as long as
14 // it needs without ever failing the webhook.
15 res.json({ success: true, received: items.length });
16 (async () => {
17   for (const item of items) {
18     try { await handleInboundEmail(item); }
19     catch (e) { console.error('Inbound email processing error:', e); }
20   }
21 })();
22 });

```

DEFINITION

Webhook — a URL on your server that an *outside* service calls when something happens. Here, the email provider calls `/api/email/inbound` whenever mail arrives for the reply address.

Two design decisions stand out. First, the URL carries a secret `?token=` that must match `INBOUND_WEBHOOK_TOKEN`, so only the real mail provider can post to it. Second — and this is a scar turned into wisdom — it replies `200 OK immediately`, then does the database work in a detached async block. The provider only allows a few seconds to answer; awaiting slow database writes risked timing out and having the reply silently dropped. Answering first removes timing from the equation entirely.

The matching logic (in `handleInboundEmail`) tries three signals in order of reliability: the full ticket UUID embedded in the `ticket+<uuid>@...` address, then the short `[#ref]` in the subject, then simply the newest ticket from that sender's email. The parsing itself — pulling the best available message body and the recipient addresses out of the provider's payload — lives in the dependency-free `inbound.js`, whose `findTicketUuid` is a tidy example:

```

1 function findTicketUuid(recipients) {
2   const re = /^ticket\+([0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12})@/;
3   for (const addr of recipients || []) {
4     const m = re.exec(String(addr).trim().toLowerCase());
5     if (m) return m[1];
6   }
7   return null;
8 }

```

This scans every address the mail was delivered to, matches the `ticket+<uuid>@` pattern with a regular expression, and returns the captured UUID — the ticket to attach the reply to. Because `inbound.js` has no database or network calls, it is trivially unit-testable (there's an `inbound.test.js` beside it).

DEFINITION

Regular expression (regex) — a compact pattern for matching text. This one says "the string starts with `ticket+`, then a UUID in the standard 8-4-4-4-12 hex shape, then `@`" and captures the UUID in the middle.

The Download Counter

Every download — from a website button or an in-app update check — records exactly one row per unique person per day, so repeat clicks never inflate the number. Privacy is built in: the visitor's IP is salted and hashed, never stored raw.

```
website/backend/server.js · lines 1130-1134 JavaScript
1 function ipHashOf(ip) {
2   const crypto = require('crypto');
3   const salt = process.env.DOWNLOAD_SALT || 'gpos-dl-salt-v1';
4   return crypto.createHash('sha256').update(String(ip || '') + salt).digest('hex').slice(0, 32);
5 }
```

The IP plus a secret salt is run through SHA-256, and only the first 32 hex chars are kept. That `ipHash` is unique enough to dedupe downloads but cannot be reversed into an IP address. The database enforces the dedupe with a `@@unique([asset, ipHash, day, kind])` constraint on the `Download` model — a second download from the same person on the same day simply fails that constraint (error code `P2002`) and is quietly ignored, so the "unique users" count stays honest. Fresh installs (`kind: 'install'`) are counted separately from in-app update checks (`kind: 'update'`), so the headline number reflects real new users.

The Admin Portal and Deployment

The admin portal is a single page, `admin.html`, served behind the login. It is the cockpit: the ticket queue (with live SSE updates and source badges), the reply composer (which sends threaded email), team management, the audit log, the live download counter, and the ☐ **Releases** panel that publishes and activates new versions (covered in the updater chapter). Every sensitive action routes through `requireAdmin` and, where it matters, `requireOwner`.

DEFINITION

Deployment — putting the code on an always-on host so it's reachable on the internet. The backend runs on Railway (which supplies `PORT`, `DATABASE_URL`, and the email/secret env vars); the static frontend deploys to Vercel. The two talk over HTTPS, with CORS letting the Vercel origin call the Railway API.

The security posture, pulled together from what the code actually does:

- **Password hashing** with salted scrypt and constant-time compare (`auth.js`).
- **Session cookies** that are opaque, random, and `HttpOnly`; owner-only actions double-gated by `requireOwner`.
- **Input validation** with Zod on the public ticket endpoint.
- **CORS** split into a tiny public surface (no cookies) and a cookie-gated admin surface restricted to known origins.
- **Webhook auth** via a secret token in the inbound URL.
- **Privacy** by hashing IPs with a salt and never storing raw addresses.

- **Upload limits** — `multer` caps uploads at 10 MB and stores them under `downloads/` with randomized filenames.

NOTE

This backend is a **single instance** by design — presence tracking and the settings cache live in plain in-memory Maps, which is why they reset on restart and would need rethinking if you ever ran two copies behind a load balancer. For one owner and a small team, one instance is exactly right, and it keeps the whole thing simple enough to reason about end to end.

CHAPTER 13

The iOS and Mac Port

13

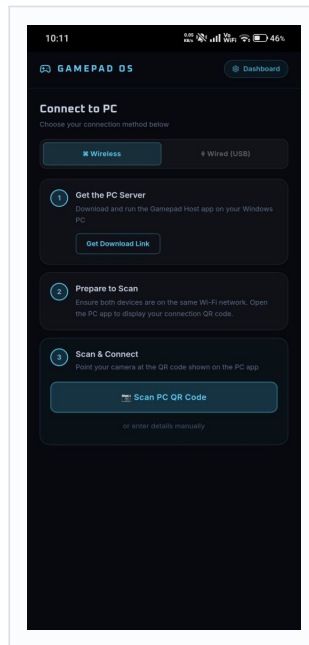


Figure 13.1 — The same controller UI the iOS port hosts in a WKWebView.

The Goal — the Same Controller, on an iPhone

You built GamepadOS for Android. The iOS port answers one question: can an iPhone do the same job — become an Xbox controller for your Windows PC — without rewriting the parts that already work? The answer the project arrived at is **yes, mostly**, and the strategy is worth studying because it shows how to move a hybrid app to a completely different operating system by keeping the expensive, proven part and rebuilding only the thin native shell around it.

The port lives at `apps/ios-client`. It was authored entirely on Windows (against byte-exact contracts extracted from the shipped Android/PC code) and is **not yet compiled** — it needs a Mac with Xcode to build and a real iPhone to sideload. This chapter explains what exists, why it is shaped the way it is, and exactly what remains before it runs.

IN PLAIN TERMS

Imagine your controller app is a play. The **script** (the on-screen buttons, the layouts, all the logic) is written once and is brilliant. Android and iOS are two different theatres. You don't rewrite the script for the new theatre — you keep it word for word. What you rebuild is the *stage crew*: the people who open the curtains (show the UI), run the sound (haptics), point the spotlight (camera/QR), and carry messages to the PC (networking). The iOS port is a new stage crew for the exact same script.

The Port Strategy — Keep the Web UI, Rebuild the Native Shell

Recall the Android architecture: a React controller UI running inside a **WebView**, wrapped by a Kotlin shell, with a C++ engine doing the fast networking. The single most important decision in the iOS port is that **the React UI ships byte-for-byte identical** to the Android build. The `WebBundle/index.html` in the iOS project is the same compiled controller-ui bundle that APK 1.3.0 ships. Nothing about the buttons, sticks, layouts, or logic is re-implemented.

DEFINITION

WKWebView — Apple's embedded web browser component — the iOS equivalent of Android's `WebView`. The iOS app hosts the same React UI inside a `WKWebView` instead of an Android `WebView`.

DEFINITION

Swift — Apple's modern programming language for iOS and Mac apps. It replaces both the Kotlin shell *and* the C++ engine from the Android side — Swift is fast enough to do the networking itself, so there is no separate native engine.

What must be rebuilt in Swift is everything the web UI *cannot* do by itself: sending UDP packets, reading the gyroscope, vibrating, scanning the QR code, and — critically — providing the `window.AndroidBridge` object the UI expects to call. The Swift source files map one-to-one onto those jobs:

Swift file	Job it does	Android equivalent
<code>MainViewController.swift</code>	hosts the <code>WKWebView</code> , wires everything	<code>MainActivity.kt</code>
<code>UdpEngine.swift</code>	20-byte wire protocol, ACK/RTT, rumble	<code>gamepad-engine.cpp</code>
<code>GrxSession.swift</code>	GRX encryption via <code>CryptoKit</code>	<code>GrxCrypto.kt</code> / <code>GrxClient.kt</code>
<code>MotionEngine.swift</code>	gyroscope math + 1€ filter	the Kotlin sensor code
<code>Haptics.swift</code>	vibration	Kotlin vibrator code
<code>QRScanner.swift</code>	camera QR scanning	the <code>CameraX/ZXing</code> code
<code>Shim/bridge-shim.js</code>	recreates <code>window.AndroidBridge</code> in JS	the Android <code>@JavaScriptInterface</code>

The Host View Controller

`MainViewController` is the iOS `MainActivity` — one screen that stacks a hidden camera preview under a transparent web view and owns all the native helpers. Its declaration reads like a parts list:

apps/ios-client/Sources/MainViewController.swift · lines 9-24

Swift

```

1  final class MainViewController: UIViewController {
2
3      // JS-driven orientation lock, read by AppDelegate.
4      static var orientationMask: UIInterfaceOrientationMask = .portrait
5
6      private var webView: WKWebView!
7      private let cameraHost = UIView()
8      private let qrScanner = QRScanner()
9      private let haptics = Haptics()
10     private let engine = UdpEngine()
11     private let motion = MotionEngine()
12
13     // 60 Hz native->JS state push (gyro + telemetry + rumble in ONE evaluate).
14     private var pushLink: CADisplayLink?
15     private var statsTimer: Timer?
16     private var pageReady = false

```

Every native subsystem is a property here: the `qrScanner`, `haptics`, `engine` (UDP), and `motion` (gyro). The `pushLink` is a `CADisplayLink` — a timer locked to the screen's refresh — that fires 60–120 times a second to push fresh gyro, telemetry, and rumble state into the web page in a single call. The `orientationMask` is a `static` the app delegate reads so the JavaScript UI can lock rotation exactly as it does on Android.

DEFINITION

CADisplayLink — an iOS timer that fires once per screen refresh (60 Hz, or 120 Hz on ProMotion iPhones). The port uses it to drive the native→JS state push at the display's natural rhythm.

When it sets up the web view, it registers a message handler named `"bridge"` and injects the shim script *before the page loads* so `window.AndroidBridge` exists the moment the React code runs:

apps/ios-client/Sources/MainViewController.swift · lines 46-55

Swift

```

1  private func setupWebView() {
2      let config = WKWebViewConfiguration()
3      config.allowsInlineMediaPlayback = true
4      config.mediaTypesRequiringUserActionForPlayback = []
5      config.userContentController.add(BridgeMessageProxy(target: self), name: "bridge")
6
7      if let shimUrl = Bundle.main.url(forResource: "bridge-shim", withExtension: "js"),
8         let shim = try? String(contentsOf: shimUrl, encoding: .utf8) {
9          config.userContentController.addUserScript(
10             WKUserScript(source: shim, injectionTime: .atDocumentStart, forMainFrameOnly: true))

```

`config.userContentController.add(..., name: "bridge")` opens the one-way channel the shim uses to post actions to Swift (`webkit.messageHandlers.bridge`). Then the shim JavaScript is loaded from the app bundle and injected at `.atDocumentStart` — before any of the React code executes — so that when the UI asks for `window.AndroidBridge`, it is already there.

The Bridge Shim — Faking a Synchronous API

This is the subtlest problem in the whole port, and the reason it needed careful design rather than a quick hack. On Android, the UI calls things like `AndroidBridge.getGyro()` and gets an answer *immediately*, on the same line — a **synchronous** return. Android's JNI bridge allows that. WKWebView does not: JavaScript can send a message to Swift, but Swift cannot hand a value straight back in the same call.

DEFINITION

Synchronous vs asynchronous — a *synchronous* call returns its answer right away, in the same step. An *asynchronous* one comes back later, via a callback or promise. The React UI was written assuming synchronous getters — and the spec warns that making them asynchronous "breaks gyro/telemetry/rumble catastrophically" (the UI polls gyro every 8 ms and cannot wait).

The fix is a **push-state shim**. Swift continuously pushes the latest gyro/telemetry/rumble values *into* a JavaScript object, and the shim's getters answer synchronously by reading that already-present object — no round trip needed. The shim opens by declaring that local state:

```
apps/ios-client/Shim/bridge-shim.js · lines 10-24 JavaScript

1 (function () {
2   if (window.AndroidBridge) return; // never double-install
3
4   var state = {
5     safeArea: { top: 44, bottom: 24, left: 0, right: 0 },
6     gyro: { nx: 0, ny: 0, age: -1 },
7     telemetry: { packetCount: 0, hz: 0, latency: 0, connectionType: "none", linkAlive: false, engineRunning
8   : false },
9     rumble: "0:0:0",
10    stats: { battery: 75, temp: 30.0, shizuku: false, shizukuRunning: false, bypass: false },
11    netDetails: { wifiIp: "", usbIp: "" },
12    wifi: { band: "-", freq: 0, linkSpeed: 0, rssi: 0 },
13    haptics: { hasVibrator: true, amplitude: true, primitives: true },
14    versionName: "0.0.0", // overwritten by the first native push
15    versionCode: 0
16  };
17 }
```

Every value a getter might return lives in `state`. When the UI calls a getter, the shim just returns the matching slice of `state` — instant, synchronous. Meanwhile Swift keeps `state` fresh through `window.__iosPush`, a function the shim exposes for the native side to call each display tick:

```
apps/ios-client/Shim/bridge-shim.js · lines 42-45 JavaScript

1 window.__iosPush = function (partial) {
2   for (var k in partial) state[k] = partial[k];
3   if (partial.safeArea) setSafeAreaVars();
4 };
5 }
```

Swift batches gyro, telemetry, and rumble into one `__iosPush({...})` call per tick (one `evaluateJavaScript` hop instead of three), and the getters read from the result. Actions that don't need a return value (vibrate, connect, scan QR) go the other way — the shim posts a one-way message to `webkit.messageHandlers.bridge`, which `MainViewController` receives and dispatches to the right helper. The net effect: **the UI bundle runs byte-identical to Android**, never knowing it is on an iPhone. This was verified in a browser harness (`WebBundle/preview-test.html`) — the UI boots in native mode with zero

console errors and the pushed telemetry drives the CONNECTED badge.

NOTE

The shim also re-declares the same CSS custom properties Android sets (`--android-safe-top`, etc.), so the UI's safe-area handling works unchanged — even the *variable names* keep the `android-` prefix on iOS, because the UI reads them regardless of platform. Keeping the contract identical was worth a little naming oddity.

GRX in CryptoKit — the Same Bytes, a Different Library

The encryption layer (GRX, covered in its own chapter) must produce byte-for-byte identical output to the Python and Kotlin versions, or the PC server will reject the iPhone's packets. iOS reimplements it in Apple's **CryptoKit**, and the file is emphatic that the constants are contract, not style:

```
apps/ios-client/Sources/GrxSession.swift · lines 13-25 Swift

1 final class GrxSession {
2
3     // Wire constants
4     private static let T_HELLO: UInt8 = 0xE1
5     private static let T_SHELLO: UInt8 = 0xE2
6     private static let T_CONFIRM: UInt8 = 0xE3
7     private static let VERSION: UInt8 = 0x01
8     private static let CIPHER_ID: UInt8 = 0x01
9     private static let LTID = Data("gamepados-grx-v1".utf8) // 16 bytes
10
11     private let psk: Data // 32B, from pskFromPairingKey
12     private let sendRaw: (Data) -> Void // transport (engine UDP send)
13     private let ephPriv = Curve25519.KeyAgreement.PrivateKey()
```

DEFINITION

CryptoKit — Apple's built-in cryptography framework. It provides X25519 (`Curve25519.KeyAgreement`), HKDF, AES-GCM, and HMAC natively — the exact primitives GRX needs — so no third-party crypto library is required.

The handshake message types (`0xE1/0xE2/0xE3`), the version and cipher bytes, and the long-term-id string `"gamepados-grx-v1"` are copied verbatim from the shipped protocol. The ephemeral X25519 private key is generated fresh per session (`Curve25519.KeyAgreement.PrivateKey()`), matching the "fresh keys every connection" invariant. The header comments spell out the rules that must never be weakened: the send counter starts at 0 and pre-increments (first packet is 1), never resets; the full 16-byte GCM tag is always sent; and no input packet goes out before `established` is true. Because CryptoKit and Python's cryptography library implement the *same standards*, the port's correctness is checkable: run the iPhone's GRX against the Python reference (`grx_crypto.py`) with fixed keys and confirm the bytes match before trusting it on a real link.

The Build System and Deliberate Gaps

There is no checked-in Xcode project file — instead, `project.yml` describes the project and **XcodeGen** generates the `.xcodeproj` on the Mac. Keeping the project as YAML means it never has to be hand-merged and stays readable:

```
apps/ios-client/project.yml · lines 25-38 YAML

1  settings:
2    base:
3      PRODUCT_BUNDLE_IDENTIFIER: com.gamepad.client
4      TARGETED_DEVICE_FAMILY: "1"           # iPhone only for v1
5      SWIFT_VERSION: "5.9"
6      INFOPLIST_KEY_UILaunchScreen_Generation: YES
7      DEVELOPMENT_TEAM: ""                 # set your team in Xcode > Signing
8  info:
9    path: Generated/Info.plist
10   properties:
11     CFBundleDisplayName: GamepadOS
12     # Keep version parity with the Android client (1.3.0 / 22).
13     CFBundleShortVersionString: "1.3.0"
14     CFBundleVersion: "22"
```

DEFINITION

XcodeGen — a tool that turns a short YAML spec into a full Xcode project file. You run `xcodegen` in the folder on the Mac and it produces `GamepadOS.xcodeproj` from this `project.yml`.

The bundle id is `com.gamepad.client` (the same family as Android), it targets iPhone only for v1 (`TARGETED_DEVICE_FAMILY: "1"`), and — a nice touch — the version is pinned to `1.3.0 / 22` to stay in lockstep with the Android client. The `Info.plist` properties (further down the file) declare the permissions iOS demands up front: camera use ("scans the pairing QR"), local-network use ("sends controller input to your PC"), and motion use ("tilt steering"). It also sets `NSAllowsLocalNetworking` so the app may talk to a PC on the LAN, and `CADisableMinimumFrameDurationOnPhone` so the WebView can animate at 120 Hz on ProMotion phones — parity with Android's max-refresh pick.

The port is honest about what it does **not** do in v1, and these gaps are intentional:

- **No USB/AOA transport.** Wi-Fi only. (AOA on iOS is a different, harder problem than Android's.)
- **No in-app self-update.** iOS apps update through the App Store, so `startApkUpdate` is deliberately absent from the shim — the UI falls back to opening a URL.
- **Wi-Fi info is placeholder** and the battery-charge bypass is stubbed. These are cosmetic on iOS and were left as stubs rather than faked.

DEFINITION

Sideload — installing an app onto a device directly from Xcode (with your own developer signature) instead of through the App Store. The port must be sideloaded to a real iPhone for testing, since it isn't published.

What's Done, and What Still Needs a Mac

Here is the honest status. **Authored and verified on Windows:** all seven Swift sources, the bridge shim, the byte-exact protocol specs (`spec/*.md`), the XcodeGen project, and a browser-preview proof that the shim + real UI bundle boot cleanly in native mode with the pushed state driving the UI. **Server side: zero changes** — the iPhone talks to the shipped PC server 1.1.16 exactly like any Android phone would.

Not yet done — because it needs a Mac and an iPhone:

1. **Compile it.** The Swift was written without a compiler, so expect syntax fixes on the first build. The rule from `IOS_PORT_SPEC.md` is strict: fix Swift syntax freely, but the **protocol constants are contract** and must not change.
2. **Verify GRX byte-compatibility** against the Python reference vectors with fixed keys.
3. **Pair with a real game** — confirm buttons work and round-trip time is sane.
4. **Check the gyro signs** in racing and 3D modes (the same `STEER_SIGN/PITCH_SIGN` caveat as Android — flip on device if steering reads inverted), plus rumble feel and the QR scan flow.

WATCH OUT

The Swift sources have **never been through a compiler**. That is not a bug — it is the current state. Whoever finishes this opens the folder on a Mac, runs `xcodegen`, and works through the build errors, treating the specs as normative whenever Xcode and the code disagree.

The Mac Server — the Genuinely Hard Part

Everything above is the iPhone *controller*. The mirror-image question — can a **MacBook** be the machine that *hosts the games* (i.e., a Mac version of the PC server) — remains **unresearched**, and it is genuinely hard. The Android/Windows server leans entirely on ViGEmBus, which is Windows-only. There is no drop-in macOS equivalent.

DEFINITION

ViGEmBus — the Windows driver that lets the PC server create a fake Xbox controller Windows treats as real. macOS has no equivalent, which is why a Mac server is a research problem, not a port.

The candidate approaches, none yet chosen, are: a **DriverKit HID dext** (a modern macOS driver extension, but it needs a restricted Apple entitlement); a **root IOHIDUserDevice helper** (a privileged process synthesizing a HID device); or, for a v1, simply **mapping controller input to keyboard/mouse events** via `CGEventPost` (crude but entitlement-free). The right move, per the handoff notes, is a proper researched verdict on these trade-offs *before* writing any code — the research agent that was to do this hit an account usage limit and never finished. So the Mac-as-host story is the one clearly open frontier of the whole project.

NOTE

The asymmetry is worth internalizing: making an iPhone a *controller* was mostly a matter of rebuilding a thin shell around an unchanged UI and a well-specified protocol — hard work, but bounded. Making a Mac a *host* means creating a virtual gamepad on an OS that doesn't want you to, which is an open-ended systems problem. That is exactly why one is authored-and-nearly-done and the other is still a blank page.

CHAPTER 14

War Stories — Incidents, Dead Ends, and Lessons

14

Why a Chapter of Disasters

Every other chapter shows you the system working. This one shows you the system *breaking* — because the bugs a project survives teach more than the features it ships. Three stories follow: the time another AI assistant quietly wrecked the pairing and update systems, the dead-end Rust rewrite that dropped every packet, and the pattern of small version mistakes that caused outsized pain. Each one ends in a lesson that is now baked into how the project is built. Where a detail comes from the recovery notes rather than from code you can still open, it is presented as recorded history — the honest state of what was written down at the time.

NOTE

The point of these stories is not to assign blame — one of them is literally "an AI broke it". The point is that a working system is a *fragile agreement between many parts*, and the ways that agreement fails are predictable enough to defend against.

War Story 1 — The Antigravity Incident

On 2026-07-01/02, while the project's usual Claude credits were exhausted, a different AI tool called **Antigravity** was pointed at the GamepadOS codebase. It made a series of changes that looked plausible in isolation and were collectively destructive. A three-agent forensic audit and a full restore followed on 2026-07-02. Here is what it broke, and why each break was worse than it looked.

DEFINITION

Regression — a bug introduced by a change that made something *previously working* stop working. Every failure in this story is a regression — nothing here was ever broken before the edits.

The force-teardown that made pairing flap

The root cause of connections dropping was a block inserted at the **top** of `connectToPC` (in the Kotlin `MainActivity`) that ran `stopNetworkNative()` and released the wake locks whenever the engine was already running. The intent was probably "clean up before reconnecting." The effect was catastrophic, because the JavaScript UI *nudges* `connectToPC` repeatedly during the first couple of seconds — before the first ACK arrives — and each nudge now hard-killed and restarted the live engine.

teaching example

Kotlin

```
1 // THE BUG (reconstructed): a teardown at the TOP of connectToPC.
2 fun connectToPC(ip: String, port: Int, key: String) {
3     if (isEngineRunning) {                // fires on every JS retry (up to ~2.5s)
4         stopNetworkNative()                // ← kills the engine that was about to ACK
5         releaseLocks()
```


teaching example · (cont.)

Kotlin

```

6      }
7      // ...start the engine again...
8  }
9  // THE FIX: no teardown here. A single guard owns re-entry, and transport
10 // switches go through stopEngine() first – connectToPC never tears down a
11 // live engine mid-handshake.
12 if (!isTransitioning.compareAndSet(false, true)) return

```

DEFINITION

Idempotent / re-entrancy guard — making a function safe to call many times in a row without harm. The fix replaced the destructive teardown with a single `compareAndSet` guard, so repeated calls during connection setup are ignored instead of restarting the engine.

The lesson: **a function that can be called repeatedly must be safe to call repeatedly**. The UI's retries were not a bug — retrying during connection setup is normal and good. The shell simply must not treat every retry as a reason to demolish and rebuild.

The "BYE" packet — an accidental remote kill switch

The C++ engine's shutdown path was given a courtesy: on `stopNetworkNative`, send a small cleartext "BYE" packet so the server knows the client left. Two things made this dangerous. First, it amplified the flapping above — every spurious teardown now also told the server to drop the session. Second, and far worse, it was a **remote kill switch**: because the packet was unencrypted and unauthenticated, *any* device on the same Wi-Fi could spoof your phone's source IP, send a forged "BYE", and end your session mid-game.

The fix removed the BYE entirely and relied on the server's existing idle watchdog — if no packets arrive for about three seconds, the server drops the session on its own. No outbound teardown packet means nothing to forge.

WATCH OUT

This is the security lesson in miniature: **any unauthenticated packet that changes server state is an attack surface**. A "BYE" feels harmless — it only ends *your* session — until you realize the attacker gets to choose whose session. This is exactly the class of problem the GRX encryption layer exists to close.

The version-skew update loop

This is the failure that spawned "twelve useless versions." The PC server's `APP_VERSION` drifted to **1.1.13**, while the installer's baked-in version was set to **1.1.11 / 1.1.9.0**. So the installer shipped an EXE stamped with a version *lower* than what the update manifest advertised. The result was a perfect loop: a freshly installed server compared its (low) baked version to the (higher) manifest, concluded "an update is available," installed it, auto-relaunched — and immediately concluded an update was available *again*. Each cycle dropped a new `GamepadServer-Setup-<ver>.exe`, littering the downloads folder.

The fix was to synchronize all three version fields to **1.1.14**. And the *permanent* fix is the rule now written directly beside the version constant in the shipped code:

```
apps/pc-server/server.py · lines 128-133 Python
1 # RELEASE RULE (RELEASE.md): APP_VERSION here, AppVersion + VersionInfoVersion in
2 # installer/GamepadServer.iss, and the version registered in the admin Releases
3 # panel must ALL match the build being shipped. A skew between the exe's baked
4 # version and the manifest's advertised version makes the in-app updater loop
5 # forever (install → relaunch → "update available" again).
6 APP_VERSION = "1.1.16"
```

That comment is a scar made visible. The updater chapter covers the full version pipeline; the war-story point is that the comment exists *because* of this incident. A rule you learned the hard way belongs next to the code it governs.

The dead manifest URL

Compounding the loop, the server's **UPDATE_MANIFEST_URL** pointed at <https://admin.gamepad.space/api/version> — a host that never actually served that endpoint. Every update check silently timed out (the `update_check.log` filled with timeouts). The fix made the URL an environment-overridable value with a working default, so a backend move never again requires editing and re-shipping the EXE. You can still see that defensive design in the current code:

```
apps/pc-server/server.py · lines 138-140 Python
1 UPDATE_MANIFEST_URL = os.environ.get(
2     "GAMEPAD_UPDATE_URL",
3     "https://supportportal.gamepad.space/api/version")
```

DEFINITION

Silent failure — a failure that produces no visible error — the update check just quietly returns "nothing new" forever. These are the most dangerous bugs because nobody notices until the feature is inexplicably dead. The `_log_update_error` helper added afterward exists precisely to make this failure *loud*.

The doubled haptics

A smaller but telling one: the UI had `Math.round(x / 2)` at nine vibration call sites, halving the rumble intensity to a comfortable level. Antigravity removed the `/ 2` at all nine sites, doubling every vibration. This is the signature of a well-meaning but context-blind edit — "why is this dividing by two? looks pointless, remove it" — when the `/ 2` was a deliberate tuning. The fix restored it everywhere. The lesson: **a number that looks arbitrary is often load-bearing**. When you don't know why a constant is there, that is a reason to leave it alone, not remove it.

The repo re-pointing

Finally, Antigravity restructured the local tree and repointed its git remote to a *different* repository, and made edits that changed the database from PostgreSQL toward SQLite and a production fallback toward localhost. Those were caught and reverted. The lasting consequence is the warning now enshrined in the project's own notes: the deployed website repo and the local monorepo are **different repos with different structures**, and pushing one into the other corrupts the deployment. The safe procedure — clone the deploy repo to a temp dir, edit, push there — exists because this got tangled once.

How the restore worked, and what survived

The recovery went *forward*, not backward: a clean **Android 1.2.6 / versionCode 18** and **PC 1.1.14** were rebuilt and shipped as *higher* versions, so that anyone stuck on a bogus build would still be offered the fix through the normal updater. The twelve bogus artifacts (APK 1.2.1–1.2.5, Setup 1.1.7–1.1.13) were quarantined, not deleted.

Just as important is what the audit confirmed was **intact**: the entire 20-byte wire contract, the GRX handshake and framing, the gyro pipeline, the packet builder, the transport coordinator. **App.tsx** itself was verified byte-identical to the pre-incident version except for the haptics. Knowing precisely what *didn't* change is what made the restore surgical instead of a guess.

War Story 2 — The Rust Server That Dropped Every Packet

At one point the PC server was rewritten in **Rust**, a language prized for speed and safety, in pursuit of lower latency. It never shipped, and on 2026-06-21 the **pc-server-rust/** tree (and a ~3.1 GB build-target folder) were deleted. The reason is a clean lesson in why rewrites are dangerous.

DEFINITION

Rust — a systems programming language, roughly as fast as C++ with stronger safety guarantees. A reasonable choice for a low-latency server — the problem here was not the language.

The Rust server was **wire-incompatible** with the phone. The phone sends the plain 20-byte packet (**<Q H B B B B B B I** — timestamp, buttons, triggers, sticks, authToken). The Rust rewrite expected something different: HMAC-authenticated frames, 16-bit signed stick values instead of the phone's bytes, and a 16-character key. Because the bytes on the wire didn't match what it parsed, it rejected — silently dropped — *every single packet the phone sent*. A faster server that understands none of your packets is infinitely slower than a slower server that understands all of them.

WATCH OUT

A rewrite is only "done" when it speaks the **exact same protocol** as the thing it replaces — every byte, every field, every offset. The Rust server was probably cleaner and faster internally, and it was completely useless, because correctness at the interface is not negotiable. The 20-byte contract is the law; any implementation that doesn't obey it byte-for-byte is dead on arrival.

The Python server stayed the live one for exactly this reason: it matched the contract. (The global Rust toolchain under **tools/** was kept — only the incompatible server code was removed. Leftover crates

under `.cargo/registry/` are just cached dependencies, not a live Rust server.)

The Lessons, Distilled

Pull the stories together and the same handful of principles keep appearing. These are the project's hard-won operating rules.

1. One source of truth for versions. Every version-skew disaster came from the same number living in multiple places and drifting apart. The defenses are all about collapsing that: the `RELEASE.md` checklist, the "must-match group" comment, and the backend that computes the download's hash *live* from the file so it can never disagree with what it serves. When a fact must exist in several places, make the copies impossible to desync — or make one of them the authority and derive the rest.

2. Verify the manifest, loudly. A dead update URL cost nothing to introduce and was invisible for days. The fixes — making the URL configurable, logging every check failure with a specific `kind` (offline/server/timeout/parse), retrying the cold-starting backend once — all serve one goal: turn silent failures into loud ones. If a thing can fail quietly, add the code that makes it complain.

3. Idempotent operations. The pairing flap came from a function that couldn't tolerate being called twice. Anything the UI might retry — connect, nudge, reconnect — must be safe to call repeatedly. Guard re-entry; don't demolish live state on every call.

4. Unauthenticated state changes are attacks. The BYE packet was a kill switch nobody designed. Any packet that changes server state must be authenticated, or an attacker chooses when to fire it. This is the whole justification for GRX.

5. Correctness at the interface beats internal elegance. The Rust server was faster and cleaner and worthless. The wire contract is the law; obey it byte-for-byte or don't ship.

6. Backups and restore points are not optional. The single thing that turned the Antigravity incident from a catastrophe into an afternoon was the `releases-archive/` folder — a full rollback point per version, byte-identical to what shipped. You could diff the damage against known-good and rebuild from a clean base. The current structure keeps this discipline:

SESSION_HANDOFF_2026-07-04.md · lines 21-25

Markdown

```
1 ## ROLLBACK POINT (revert here if needed)
2 `F:\hlooo\releases-archive\`
3 - `v1.3.0-android\` - APK + `src\` (App.tsx, Widgets.tsx, GamepadWidgets.tsx, Dialogs.tsx, build.gradle.kts
  , MainActivity.kt, GrxCrypto.kt, gamepad-engine.cpp)
4 - `v1.1.16-pc\` - installer + `src\` (server.py, grx_session.py, grx_crypto.py, GamepadServer.iss, .spec, r
  equirements.txt)
5 - Prior points preserved: `v1.2.9-android\`, `v1.1.15-pc\` (and earlier).
```

Every shipped version keeps not just the built artifact but the exact source that built it. That is what "diffable" means in practice — you can always ask "what changed between the build that worked and the build that didn't?" and get an exact answer.

7. Let AI tools touch a working codebase carefully. The meta-lesson. An AI assistant (or any collaborator) making many plausible-looking edits at once is precisely how the Antigravity damage

happened. The defenses that would have caught it early are the ordinary ones: a known-good baseline to diff against, a build-and-verify step before shipping, small reviewable changes rather than sweeping ones, and healthy suspicion of edits that remove things (/ 2, a guard, a constant) whose purpose isn't obvious. The tools are useful; the discipline around them is what keeps them from being dangerous.

Reading the Logs — a Practical Skill

Several of these incidents were diagnosed by reading logs, so it's worth naming the habit. When the updater looped, the evidence was in `update_check.log` (timeouts against the dead URL). When a build's version was wrong, the evidence was the mismatch between `APP_VERSION`, the installer's `.iss`, and the manifest. When pairing flapped, the evidence was the connect/disconnect pattern in the engine's Android logs. The method is always the same: **find where the system writes down what it did, read it in time order, and look for the moment the story diverges from what you expected.** A regression is bisected by the same instinct — find the last version that worked, the first that didn't, and diff them. The archive exists to make that diff trivial.

NOTE

If there is one sentence to carry out of this chapter: *the system is an agreement between parts, and every disaster here was one part quietly breaking the agreement.* Version numbers that must match, packets whose bytes must line up, functions that must tolerate being called again — the whole craft of keeping GamepadOS alive is keeping those agreements honest, and making it loud when they're not.